Марсель Салихов

Количественные финансы

Анализ финансовых данных на Python и R

Учебник к курсу «Количественные финансы»
НИУ ВШЭ, факультет мировой экономики и мировой политики

Москва · 2026

Марсель Салихов
Количественные финансы. Анализ финансовых данных на Python и R. – Москва, 2026.

Книга знакомит с количественными методами анализа финансовых данных на примерах российского рынка: доходность и риск, отчётность компаний, модели временных рядов, портфели, облигации, проверка торговых стратегий. Примеры кода приводятся на Python; онлайн-версия содержит также код на R.

Код, данные и онлайн-версия книги: github.com/quantviews/quant-finance-book

© Марсель Салихов, 2026. Текст распространяется на условиях лицензии CC BY-SA 4.0.

Оглавление

Предисловие 1
Как читать эту книгу 2
Обратная связь 2

1 Введение 3
1.1 Для кого эта книга 3
1.2 Как устроена книга 3
1.3 Как запускать код книги 4
1.4 Почему код, а не Excel 6
1.5 Учиться с ИИ-ассистентом 7
1.6 История курса 7
1.7 Если вы изучаете книгу в рамках курса 8
1.8 Об авторе 9

ЧАСТЬ I. ДАННЫЕ И ИНСТРУМЕНТЫ

2 Источники финансовых данных 13
2.1 Какие бывают финансовые данные 14
2.2 Московская биржа и ISS 14
2.3 Данные Банка России 17
2.4 Российская экономическая статистика: rustata 18
2.5 Международные источники 20
2.6 Форматы хранения данных 21
2.7 Данные этой книги 22
Итоги 23
2.8 Упражнения 23
Вопросы для самопроверки 24
Что почитать 25

3 Подготовка данных 27
3.1 Сплиты и консолидации 27
3.2 Переименования: одна компания – несколько тикеров 31
3.3 Дивиденды и скорректированная цена 31
3.4 Ошибка выжившего 33
3.5 Пропуски, остановки торгов и выбросы 34
3.6 Не заглядывать в будущее 36
3.7 Проверка качества данных 37
Итоги 38
3.8 Упражнения 38
Вопросы для самопроверки 39
Что почитать 40

4 Доходность и риск 41
4.1 Цены и доходности 42
4.2 Дивиденды и полная доходность 45
4.3 Волатильность 48
4.4 Распределение доходностей: тяжёлые хвосты 50

4.5 Просадки . . . 52
4.6 Риск и доходность отдельных акций . . . 53
4.7 Доходность с поправкой на риск: коэффициент Шарпа . . . 55
Итоги . . . 57
4.8 Упражнения . . . 57
Вопросы для самопроверки . . . 58
Что почитать . . . 59

ЧАСТЬ II. КОРПОРАТИВНАЯ ОТЧЁТНОСТЬ

5 Большие данные корпоративной отчётности 63
5.1 Откуда берётся отчётность . . . 64
5.2 Формы отчётности . . . 65
5.3 Большие таблицы: Polars . . . 67
5.4 Классификатор ОКВЭД . . . 69
5.5 Качество данных . . . 70
5.6 Финансовые коэффициенты . . . 76
5.7 Связь форм отчётности . . . 80
Итоги . . . 81
5.8 Упражнения . . . 82
Вопросы для самопроверки . . . 83
Что почитать . . . 84

6 Отраслевой анализ 85
6.1 Разложение DuPont . . . 86
6.2 Концентрация отрасли . . . 90
6.3 Лидеры и остальные . . . 92
6.4 Динамика и состав выборки . . . 93
6.5 Структура балансов . . . 95
Итоги . . . 96
6.6 Упражнения . . . 96
Вопросы для самопроверки . . . 98
Что почитать . . . 98

ЧАСТЬ III. ВРЕМЕННЫЕ РЯДЫ

7 Свойства финансовых рядов и модели ARMA 101
7.1 Автокорреляция . . . 102
7.2 Стационарность . . . 108
7.3 Модель AR(1): терм-спред ОФЗ . . . 111
7.4 Модель MA(1): неликвидная акция . . . 115
7.5 Модели ARMA и выбор порядка . . . 117
7.6 Диагностика остатков . . . 119
7.7 Прогноз вне выборки . . . 120
Итоги . . . 124
7.8 Упражнения . . . 124
Вопросы для самопроверки . . . 126
Что почитать . . . 127

8 Волатильность: модели ARCH и GARCH 129
8.1 Кластеры волатильности . . . 130
8.2 Модели ARCH и GARCH . . . 131
8.3 Тяжёлые хвосты и асимметрия . . . 136

8.4 Диагностика модели 141
8.5 Прогноз волатильности 142
Итоги 146
8.6 Упражнения 146
Вопросы для самопроверки 148
Что почитать 149

9 Прогнозирование и машинное обучение 151
9.1 Постановка задачи прогноза 152
9.2 Утечка данных 153
9.3 Валидация во времени 154
9.4 Прогноз доходности: линейная модель и бустинг 158
9.5 Где сигнал есть: прогноз волатильности 159
9.6 От прогноза к стратегии 161
Итоги 162
9.7 Упражнения 162
Вопросы для самопроверки 164
Что почитать 165

ЧАСТЬ IV. РИСК И ПОРТФЕЛИ

10 VaR и Expected Shortfall 169

11 Портфельная теория 171
11.1 Доходность и риск портфеля 172
11.2 Диверсификация 174
11.3 Эффективная граница 176
11.4 Почему оптимальные веса неустойчивы 178
11.5 Проверка вне выборки 181
11.6 Распределение капитала: правило 60/40 182
Итоги 183
11.7 Упражнения 184
Вопросы для самопроверки 185
Что почитать 186

12 CAPM и факторные модели 187

ЧАСТЬ V. ДОЛГОВОЙ РЫНОК

13 Облигации 191
13.1 Чистая и грязная цена 191
13.2 Восстановление денежных потоков 193
13.3 Доходность как решение уравнения 194
13.4 Дюрация, выпуклость и DV01 195
13.5 Сравнение выпусков и бескупонная кривая 197
13.6 Как использовать расчёт для своей позиции 199
Итоги 200
13.7 Упражнения 200
Вопросы для самопроверки 201
Что почитать 201

14 Кредитный риск 203

ЧАСТЬ VI. ТОРГОВЫЕ СТРАТЕГИИ

15 Бэктестинг и переобучение 207
15.1 От гипотезы к торговому правилу 208
15.2 Время в бэктесте 208
15.3 Как меняется результат при исправлении ошибок 211
15.4 Проверка данных и причинности 214
15.5 Когда лучший параметр оказывается случайным 215
15.6 Диагностика переобучения 218
15.7 Почему прогноз ARIMA не равен торговой прибыли 219
15.8 Как оформить результат исследования 221
Итоги 221
15.9 Упражнения 222
Вопросы для самопроверки 223
Что почитать 224

16 Моментум и управление волатильностью 225

17 Микроструктура рынка и как её измеряют 227

Литература 229

Приложения 233

A Основы Python для финансиста 235
A.1 Где запускать код 235
A.2 Значения, переменные и типы 237
A.3 Контейнеры 239
A.4 Условия и циклы 241
A.5 Функции 243
A.6 Ошибки и как их читать 245
A.7 Модули и пакеты 246
A.8 NumPy: вычисления над массивами 247
A.9 pandas: таблицы с данными 248
A.10 matplotlib: графики 252
A.11 Как учиться дальше 253
Итоги 254
A.12 Упражнения 254
Вопросы для самопроверки 255
Что почитать 256

B Справочник по данным 257

C Проекты, экзамен и оценивание 259
C.1 Итоговая оценка 259
C.2 Общие правила для проектов 260
C.3 Проект 1. Фундаментальный анализ отрасли 262
C.4 Проект 2. Модель и стратегия для акции 264
C.5 Работа на семинарах 266
C.6 Посещаемость 266
C.7 Экзамен 267

Предисловие

Эта книга выросла из курса «Количественные финансы», который я читаю студентам факультета мировой экономики и мировой политики НИУ ВШЭ с 2016 г. За эти годы курс несколько раз существенно менялся: переходил с R на Python, получал новые темы и данные, менял форму контроля. Неизменной оставалась задача – научить работать с реальными финансовыми данными так, чтобы результату можно было доверять. Книга собирает материалы курса в один связный текст для читателей, интересующихся количественными методами в финансах.

Хороших учебников по количественным финансам немало, но почти все они построены на данных американского рынка и на английском языке. Российский рынок устроен иначе, и эти различия существенны для анализа. Безрисковая ставка здесь высокая и меняется резко: в 2024 г. вложения на денежном рынке по ставке RUONIA принесли инвестору в десять с лишним раз больше, чем индекс акций с учётом дивидендов. Компании платят высокие дивиденды, и без них доходность рынка акций занижается в разы. История рынка включает остановки торгов, делистинги, смену юрисдикции эмитентов. Методы в книге общие, но примеры, кейсы и ловушки в данных – исключительно российские.

Книга опирается на три принципа.

Реальные данные. Все примеры построены на котировках Московской биржи, дивидендах, ставках Банка России и отчётности российских компаний. Полный датасет для примеров и графиков прилагается к книге, поэтому каждый расчёт можно повторить.

Воспроизводимость. Каждое число в тексте вычисляется кодом при сборке книги, а не переносится вручную из другой программы. Если обновить данные и пересобрать книгу, все выводы пересчитаются.

Понимание, а не генерация кода. ИИ-ассистенты уже пишут код для большинства задач этой книги. Поэтому акцент сделан на том, чего ассистент за вас не сделает: поставить вопрос, проверить результат и объяснить, что он означает. Упражнения построены так же – об этом подробнее в введении.

Как читать эту книгу

Книга существует в двух вариантах. **Онлайн-версия** содержит код на Python и, в ключевых примерах, на R – во вкладках. Там же на полях страницы размещены короткие заметки и ссылки на литературу. **Печатная версия** (PDF) содержит только код на Python, заметки в ней набраны в тексте мелким шрифтом.

Код в книге можно запускать: он работает на данных, которые прилагаются к книге, и даёт те же результаты, что приведены в тексте. Рекомендуем не просто читать примеры, а запускать их и менять – например, подставлять другой тикер или период.

В тексте встречаются врезки нескольких видов.

> **В этой главе**
> В начале каждой главы: её цели, используемые данные и то, что нужно знать заранее.

> **Сначала предскажите**
> Вопрос, на который стоит ответить *до* запуска кода. Сравнение своей догадки с результатом – один из самых быстрых способов понять метод.

> **Типичная ошибка**
> Ошибка, которую часто допускают на практике, и способ её избежать.

> **Особенность данных**
> Ловушка в данных: пропуски, остановки торгов, корпоративные события. Такие вещи редко описаны в учебниках, но постоянно встречаются в работе.

В конце каждой главы есть итоги, упражнения, вопросы для самопроверки с ответами и список литературы.

Обратная связь

Книга пишется и обновляется вместе с курсом. Если вы нашли ошибку, неточность или место, которое трудно понять, напишите об этом в разделе Issues репозитория книги на GitHub. Там же лежат исходные тексты, код и данные. Либо свяжитесь со мной по электронной почте.

Текст книги распространяется на условиях лицензии CC BY-SA 4.0: его можно копировать и переделывать с указанием автора и на тех же условиях.

Марсель Салихов
Москва, 2026

1 Введение

Количественные финансы – это применение статистики, эконометрики и программирования к финансовым данным. Цель – отвечать на практические вопросы с помощью измерений и проверяемых расчётов. Какую доходность приносил рынок акций и сколько риска за неё приходилось нести? Чем отличается финансовое положение двух компаний одной отрасли? Работает ли торговая стратегия после учёта издержек или её результат – следствие подгонки под историю?

Эта книга учит решать такие задачи на реальных данных, в первую очередь российских: котировках Московской биржи, дивидендах, ставках Банка России, бухгалтерской отчётности юридических лиц. Каждый метод показан в коде, который можно запустить и повторить: все расчёты полностью воспроизводимы.

1.1 Для кого эта книга

Книга написана как учебник к курсу «Количественные финансы» для студентов бакалавриата Факультета мировой экономики и мировой политики НИУ ВШЭ. Она подойдёт и тем, кто изучает предмет самостоятельно: аналитикам, начинающим исследователям, частным инвесторам, которые хотят проверять идеи на данных.

Предполагается, что читатель знаком с основами:

- **статистики и эконометрики** – среднее арифметическое, дисперсия, корреляция, линейная регрессия, проверка гипотез;
- **финансов** – процентная ставка, дисконтирование, акции и облигации;
- **программирования** – переменные, циклы, функции. Основы Python кратко повторяются в приложении А.

1.2 Как устроена книга

Книга состоит из шести частей, которые идут в порядке курса ВШЭ. Часть I нужна для всех остальных. Часть IV опирается на модели волатильности из части III, остальные части почти независимы.

Таблица 1.1. Структура книги

Часть	О чём	Основные методы
I. Данные и инструменты	Источники финансовых данных, их подготовка, доходность и риск	MOEX ISS, корпоративные действия, волатильность, просадки, коэффициент Шарпа
II. Корпоративная отчётность	Анализ отчётности сотен тысяч российских компаний	RFSD, Polars, финансовые коэффициенты, DuPont
III. Временные ряды	Модели доходности и волатильности, прогнозирование	ARMA, GARCH, машинное обучение с валидацией во времени
IV. Риск и портфели	Измерение риска и построение портфелей	VaR и ES, Марковиц, CAPM, факторные модели
V. Долговой рынок	Облигации и кредитный риск	доходность к погашению, дюрация, кривая ОФЗ, спреды
VI. Торговые стратегии	Проверка инвестиционных идей на истории	бэктест, издержки, переобучение, моментум

Каждая глава построена одинаково:

- в начале – цели главы, используемые данные и что нужно знать заранее;
- в тексте – разобранные примеры на данных, врезки **«Сначала предскажите»** (оцените результат до запуска кода) и **«Типичная ошибка»**;
- в конце – итоги, упражнения, вопросы для самопроверки и литература.

Упражнения бывают четырёх типов:

- **А. Понимание** – решаются без компьютера и проверяют, понятна ли идея.
- **Б. Практика** – выполняются на данных своей акции или отрасли, поэтому готового ответа у них нет.
- **В. Работа с ИИ-ассистентом** – не получить ответ от ассистента, а проверить его: найти ошибку, неявное допущение, неподтверждённое утверждение.
- **Г. Мини-исследование** – небольшая самостоятельная задача с устной защитой результата.

1.3 Как запускать код книги

Основной язык книги – **Python**. Для ключевых примеров в онлайн-версии есть вкладка с тем же расчётом на **R**. Печатная версия содержит только Python-код.

Весь код из книги можно запустить у себя и получить те же результаты. Для этого нужны три вещи: код, данные и небольшой вспомогательный модуль `qf`. Всё это лежит в открытом репозитории книги: https://github.com/quantviews/quant-finance-book.

1.3.1 На своём компьютере

Нужен Python 3.11 или новее. Скачайте репозиторий и установите пакеты:

```
git clone https://github.com/quantviews/quant-finance-book.git
cd quant-finance-book
pip install -r requirements.txt
```

Запускайте Jupyter (или другой редактор) **из папки книги** – тогда Python найдёт модуль `qf` и данные в папке `data/`. Если ноутбук лежит в другом месте, добавьте путь к книге в начало ноутбука:

```
import sys
sys.path.append("путь/к/quant-finance-book")
```

1.3.2 В Google Colab

В Colab ничего не нужно устанавливать на компьютер. В первой ячейке ноутбука скачайте репозиторий и перейдите в его папку:

```
!git clone -q https://github.com/quantviews/quant-finance-book.git
%cd quant-finance-book
!pip install -q -r requirements.txt
```

После этого код из книги работает без изменений. При перезапуске среды Colab первую ячейку нужно выполнить снова.

1.3.3 Модуль `qf`

`qf` – папка с кодом в корне книги, которая избавляет от повторения одних и тех же строк в каждой главе. В ней три части:

Таблица 1.2. Модуль `qf`

Модуль	Что делает	Пример
`qf.data`	загружает данные книги из папки `data/`	`data.load_prices(["SBER", "GAZP"])`
`qf.plot`	задаёт единый стиль графиков и подпись источника	`plot.source(ax, "Московская биржа")`
`qf.secrets`	читает токены API из файла `.env`	`secrets.get("RUSTATA_TOKEN")`

Основные функции загрузки данных:

- `load_stocks()` – дневные данные акций: цены, объёмы, цена с поправкой на дивиденды, капитализация;
- `load_prices()` – цены нескольких акций в «широком» формате: даты по строкам, тикеры по столбцам;
- `load_index()` – индексы Мосбиржи; `load_dividends()` – дивиденды;

- `load_key_rate()`, `load_ruonia()`, `load_zcyc()` – ставки и кривая доходности ОФЗ.

Все они принимают тикеры и период: `data.load_stocks("SBER", start="2020-01-01")`. Модуль – обычный код на Python: его можно открыть и прочитать, а при желании – заменить своими функциями. Для R те же функции собраны в файле `R/qf.R` (`qf_load_stocks()` и т. д.); подключите его командой `source("R/qf.R")`.

```
from qf import data

sber = data.load_stocks("SBER", start="2026-01-01")
sber[["date", "close", "adj_close", "market_cap"]].tail(3)
#>           date   close  adj_close    market_cap
#> 185 2026-09-25  276.35     276.35  5.965553e+12
#> 186 2026-09-28  272.45     272.45  5.881364e+12
#> 187 2026-09-29  274.65     274.65  5.928855e+12
```

Вариант кода на R – в онлайн-версии книги.

1.3.4 Данные

Все примеры работают на отдельном датасете, который прилагается к книге (папка `data/`). В снимке – дневные котировки около ста акций Московской биржи с поправкой на дивиденды и сплиты, индексы, дивиденды, ключевая ставка, RUONIA, кривая доходности ОФЗ и выборка бухгалтерской отчётности российских компаний.

Отдельным архивом доступна полная история всего рынка акций Московской биржи с 1997 г. Поскольку снимок зафиксирован, результаты в книге полностью воспроизводятся, а все числа в тексте рассчитываются при сборке книги из тех же данных, а не переносятся вручную. Состав данных и источники описаны в главе «Источники финансовых данных» и в приложении В.

1.4 Почему код, а не Excel

Excel остаётся стандартом финансовой отрасли, для быстрой проверки идеи или разбора отчётности одной компании он удобен и полезен. Но у него есть ограничения, существенные для количественного анализа:

- **объём данных** – с сотнями тысяч строк работать неудобно, а с миллионами просто невозможно;
- **ошибки** – в большой таблице с формулами ошибку трудно найти. Известные примеры – ошибка в таблице Reinhart и Rogoff (Reinhart и Rogoff 2010 г.), обнаруженная при попытке повторить их расчёты (Herndon и др. 2014 г.), и потеря данных о заражениях COVID-19 в Великобритании в 2020 г.;
- **статистика** – кроме базовых функций, статистических методов почти нет;
- **воспроизводимость** – по файлу Excel трудно восстановить, что и в каком порядке было сделано.

Анализ в виде кода лишён этих недостатков: его можно перезапустить на новых данных, проверить построчно и передать коллеге. Это основа **воспроизводимого исследования** (reproducible research).

Python и R – бесплатные языки с огромным набором библиотек. Python – язык общего назначения и стандарт в машинном обучении. R создавался для статистики и статистиками, и многие эконометрические методы появляются в нём раньше. Для задач этой книги подходят оба. Мы выбрали Python как основной: он более распространён и чаще может понадобиться в работе. Python также является отраслевым стандартом для методов машинного обучения.

1.5 Учиться с ИИ-ассистентом

ИИ-ассистенты уже умеют писать код для большинства задач этой книги. Это меняет то, чему стоит учиться. Написать функцию, которая считает волатильность, ассистент может быстрее студента. Но решить, *какую* волатильность считать, заметить, что в данных пропущен месяц торгов, и понять, почему результат бэктеста слишком хорош, – это остаётся задачей аналитика.

Поэтому книга делает акцент на трёх навыках:

1. **Постановка задачи** – сформулировать вопрос так, чтобы на него можно было ответить данными.
2. **Проверка** – сверить результат с простым расчётом, со здравым смыслом, с другим источником.
3. **Интерпретация** – объяснить, что означают числа и где границы вывода.

Пользоваться ассистентом при выполнении упражнений можно. Но результат, который вы не можете объяснить и проверить, не засчитывается.

1.6 История курса

Я читаю курс «Количественные финансы» на факультете мировой экономики и мировой политики НИУ ВШЭ с 2016/17 учебного года. Раньше также читал курс по выбору «Финансовые рынки и институты» для второго курса, который давал общее представление о финансовой системе: денежно-кредитная политика, банки, рынки ценных бумаг и деривативов, пенсионная система. «Количественные финансы» также являются курсом по выбору, но уже с более серьёзными требованиями к умению программировать. Со временем остались только «Количественные финансы» – это курс для четвёртого курса бакалавриата с упором на работу с данными и построение моделей.

Содержание и инструменты курса менялись:

- **2016–2021** – курс на R. Темы: финансовые данные, модели временных рядов, CAPM, риск и волатильность, портфельная теория, процентные ставки и опционы. Тогда различия между R и Python в популярности были не так

сильны, как сейчас. Поэтому R казался естественным выбором, тем более что мои собственные знания Python были в то время очень ограниченны.
- **2021/22** – семинары на R и Python. Добавлена работа с корпоративной отчётностью юрлиц с помощью кода.
- **с 2024/25** – курс полностью на Python. Перешёл на Python, так как его популярность стала очевидной. В качестве языка программирования на будущей работе аналитика более вероятно, что придется работать с Python, чем с любым другим языком.
- **2025/26** – форма контроля стала проектной. Это требования ВШЭ, с одной стороны, а с другой – использование ИИ становится всё более повсеместным. В курс вошли открытые данные отчётности российских компаний (RFSD) и прогнозирование с помощью машинного обучения.

Материалы прошлых лет доступны на GitHub: github.com/quantviews. Эта книга собирает их в единый учебник и дополняет новыми главами.

1.7 Если вы изучаете книгу в рамках курса

Курс состоит из лекций и семинаров. На семинарах разбираются примеры и упражнения из соответствующих глав. Итоговая оценка складывается из пяти частей:

Таблица 1.3. Структура оценки в курсе

Компонент	Вес
Проект 1. Фундаментальный анализ отрасли	20%
Проект 2. Модель и стратегия для акции	25%
Работа на семинарах	10%
Письменный экзамен	35%
Посещаемость	10%

Проект 1 – сравнительный анализ компаний одной отрасли по данным бухгалтерской отчётности (RFSD): за счёт чего лидеры отрасли эффективнее остальных. **Проект 2** – модель временного ряда и торговая стратегия для назначенной акции Московской биржи с честной проверкой: предварительной фиксацией решений, сравнением с бенчмарками, учётом издержек и проверкой на данных, которые появились уже после сдачи работы. Работы сдаются через Google-формы, проект 1 дополнительно проходит взаимную проверку. Часть студентов по каждому проекту вызывается на короткую устную защиту – случайно или по результатам проверки. Пользоваться ИИ-ассистентами можно, но к работе прикладывается журнал запросов, а баллы засчитываются только за то, что студент может объяснить.

Работа на семинарах – короткие письменные задания в начале семинара, на бумаге и без компьютера: предсказать результат кода, объяснить график, найти

ошибку. Ответы проверяют сами студенты по критериям, которые показывает преподаватель.

Экзамен проверяет понимание, а не умение генерировать код. Он пишется в аудитории без гаджетов и состоит из четырёх частей: короткий тест, интерпретация готового результата (графика, таблицы оценок модели, расчёта риска), поиск логической или концептуальной ошибки во фрагменте кода и кейс, в котором нужно применить понятия курса и дать рекомендацию.

Подробное описание проектов, критерии оценки и примеры экзаменационных заданий – в приложении C.

1.8 Об авторе

Марсель Салихов – президент Института энергетики и финансов, директор Центра экономической экспертизы ИГМУ НИУ ВШЭ. Преподаёт в НИУ ВШЭ с 2008 г. Более 20 лет работает в консалтинге и прикладных исследованиях. Основные области – российская макроэкономика, финансовые и нефтегазовые рынки. Пишет на R с 2012 г., на Python – с 2017 г.

- Личный сайт
- Профиль на сайте НИУ ВШЭ
- Институт энергетики и финансов
- Материалы и проекты на GitHub

ЧАСТЬ I

Данные и инструменты

2 Источники финансовых данных

Сколько заработал акционер Сбербанка в 2024 г.? Вопрос кажется простым: достаточно взять цену акции в начале и в конце года. Откройте любой сайт с котировками – получится рост на 3,2%. Но в июле 2024 г. Сбербанк выплатил крупные дивиденды, и с их учётом доходность акционера составила 15,3% – почти в пять раз больше. Обе цифры «правильные», но отвечают на разные вопросы, и только одна из них отвечает на заданный.

Этот пример показывает главное правило работы с финансовыми данными: прежде чем считать, нужно понять, откуда взялись числа и что именно они измеряют. Данные – не нейтральный материал, а результат чьих-то решений: какую цену считать ценой закрытия, учитывать ли дивиденды, что делать с днями без торгов, как склеить историю переименованной компании. В анализе данных есть выражение «мусор на входе – мусор на выходе» (garbage in, garbage out): никакая модель не исправит ошибку, заложенную в исходных данных.

В этой главе мы разберёмся, какие данные нужны для количественного анализа и где их брать, а в следующей – как их подготовить.

В этой главе

- Разбираемся, какие бывают финансовые данные и чем они различаются.
- Загружаем котировки напрямую с Московской биржи через её программный интерфейс ISS.
- Знакомимся с данными Банка России: ключевой ставкой и ставкой RUONIA.
- Узнаём, как получить российскую макроэкономическую статистику через rustata.
- Обсуждаем международные источники и их ограничения для российского рынка.
- Сравниваем форматы хранения данных: CSV и Parquet.
- Знакомимся с данными, которые прилагаются к книге.

Данные: запросы к ISS Московской биржи; `stocks.parquet`, `key_rate.parquet`, `ruonia.parquet`. **Нужно знать:** основы Python и pandas (приложение А).

2.1 Какие бывают финансовые данные

Финансовые данные удобно разделить на четыре группы.

Рыночные данные – всё, что порождают торги: цены сделок, объёмы, котировки на покупку и продажу. Их главное свойство – высокая частота и точная привязка ко времени. Одна и та же информация может храниться с разной степенью детализации: от журнала всех заявок с точностью до микросекунды до одной цены закрытия в день.

Фундаментальные данные описывают сами компании: выручку, прибыль, активы, долг, число акций, дивиденды. Они выходят редко – раз в квартал или раз в год – и с задержкой: годовую отчётность за 2025 г. компании публикуют весной 2026 г.

Макроэкономические данные и ставки: ключевая ставка, инфляция, курс рубля, ставки денежного рынка. Для финансового анализа они служат ориентирами: безрисковая ставка нужна для расчёта премии за риск, инфляция – для перехода к реальной доходности.

Альтернативные данные – всё остальное: новости, поисковые запросы, спутниковые снимки, данные о платежах. Мы почти не будем их использовать, но важно знать, что граница между «основными» и «альтернативными» данными постоянно сдвигается.

Помимо содержания, данные различаются **частотой** (тиковые, минутные, дневные, месячные) и **моментом доступности**. Последнее особенно важно: в исторических данных число стоит под той датой, к которой оно относится, а не под той, когда о нём узнали. Прибыль за 2025 г. относится к 2025 г., но инвестор узнал о ней только весной 2026 г. Модель, которая «видит» её уже в декабре 2025 г., заглядывает в будущее. К этой проблеме мы вернёмся в следующей главе.

2.2 Московская биржа и ISS

2.2.1 Как устроены торги

Московская биржа объединяет несколько рынков: фондовый (акции, облигации, фонды), валютный, денежный (РЕПО) и срочный (фьючерсы и опционы). Внутри каждого рынка торги идут в разных **режимах торгов** (boards). Одна и та же акция может одновременно торговаться в нескольких режимах: в основном, в режиме неполных лотов, в режиме крупных пакетов, в режиме РЕПО.

Для анализа почти всегда нужен **основной режим** – для акций это `TQBR`. В нём проходит большая часть оборота, и именно по нему рассчитываются индексы. Смешивание режимов – частая ошибка: цены в режиме неполных лотов или крупных пакетов могут заметно отличаться от основных.

2.2.2 Программный интерфейс ISS

Биржа бесплатно раскрывает данные через **ISS** (Informational & Statistical Server) – программный интерфейс, к которому можно обращаться по обычным веб-адресам. Текущие котировки без подписки отдаются с задержкой, а история торгов доступна полностью. Ответ приходит в формате JSON, XML или CSV.

Начнём с карточки ценной бумаги. Адрес строится из базового `https://iss.moex.com/iss/` и пути к нужному ресурсу.

```
import requests

ISS = "https://iss.moex.com/iss"

card = requests.get(f"{ISS}/securities/SBER.json", params={"iss.meta": "off"}, timeout=30).json()
boards = pd.DataFrame(card["boards"]["data"], columns=card["boards"]["columns"])
boards[["boardid", "title", "is_primary"]].head(5)
#>   boardid                                      title  is_primary
#> 0    TQBR                  T+: Акции и ДР - безадрес.           1
#> 1    EQBR  Основной режим: A1-Акции и паи - безадрес.           0
#> 2    SMAL       T+: Неполные лоты (акции) - безадрес.           0
#> 3    TQDP          Крупные пакеты - Акции - безадрес.           0
#> 4    EQDP          Крупные пакеты - Акции - безадрес.           0
```

Ответ ISS устроен одинаково для всех запросов: это набор таблиц, у каждой – список столбцов `columns` и строки `data`. Из них легко собрать `DataFrame`. Параметр `iss.meta=off` убирает из ответа описание типов столбцов, которое нам не нужно. В столбце `is_primary` отмечен основной режим торгов – `TQBR`.

Теперь загрузим историю торгов Сбербанка в основном режиме за июль 2024 г.:

```
url = f"{ISS}/history/engines/stock/markets/shares/boards/TQBR/securities/SBER.json"
params = {"from": "2024-07-01", "till": "2024-07-31", "iss.meta": "off"}
resp = requests.get(url, params=params, timeout=30).json()

hist = pd.DataFrame(resp["history"]["data"], columns=resp["history"]["columns"])
hist[["TRADEDATE", "OPEN", "LOW", "HIGH", "CLOSE", "VOLUME", "VALUE"]].tail()
#>      TRADEDATE    OPEN     LOW    HIGH   CLOSE    VOLUME         VALUE
#> 18  2024-07-25  295.69  294.20  297.30  296.37  22868740  6.768217e+09
#> 19  2024-07-26  296.40  292.67  300.00  293.30  84909830  2.514714e+10
#> 20  2024-07-29  293.00  286.00  293.00  286.58  52445850  1.513426e+10
#> 21  2024-07-30  286.50  284.32  291.99  290.00  42572480  1.229057e+10
#> 22  2024-07-31  290.47  288.01  291.20  289.30  25974520  7.524944e+09
```

Вариант кода на R – в онлайн-версии книги.

Путь запроса читается как адрес в иерархии биржи: движок `stock` (фондовый рынок) → рынок `shares` (акции) → режим `TQBR` → бумага `SBER`. Столбец `VALUE` – оборот в рублях, `VOLUME` – число акций.

2.2.3 Постраничная выдача

Попробуем загрузить историю за весь 2023 г. и посмотрим, сколько строк придёт.

```
params = {"from": "2023-01-01", "till": "2023-12-31", "iss.meta": "off"}
resp = requests.get(url, params=params, timeout=30).json()
print(len(resp["history"]["data"]), "строк")
#> 100 строк
resp["history.cursor"]
#> {'columns': ['INDEX', 'TOTAL', 'PAGESIZE'], 'data': [[0, 254, 100]]}
```

В 2023 г. было около 250 торговых дней, а пришло только 100 строк. ISS отдаёт историю **страницами по 100 строк**, и об этом сообщает служебная таблица `history.cursor`: `TOTAL` – сколько строк всего, `PAGESIZE` – размер страницы, `INDEX` – номер первой строки текущей страницы. Чтобы получить всё, нужно запрашивать страницы по очереди, увеличивая параметр `start`:

```
def iss_history(ticker, start, end, board="TQBR"):
    """История торгов акцией на Мосбирже с учётом постраничной выдачи ISS."""
    url = (f"{ISS}/history/engines/stock/markets/shares/boards/{board}"
           f"/securities/{ticker}.json")
    pages, first = [], 0
    while True:
        params = {"from": start, "till": end, "start": first, "iss.meta": "off"}
        resp = requests.get(url, params=params, timeout=30).json()
        page = pd.DataFrame(resp["history"]["data"], columns=resp["history"]["columns"])
        pages.append(page)
        total = resp["history.cursor"]["data"][0][1]
        first += len(page)
        if len(page) == 0 or first >= total:
            break
    return pd.concat(pages, ignore_index=True)

sber_2023 = iss_history("SBER", "2023-01-01", "2023-12-31")
len(sber_2023)
#> 254
```

Теперь пришли все 254 торговых дня. Функцию загрузки полезно проверить сравнением с независимым источником – например, с данными книги. Цены закрытия совпадают во всех днях, кроме одного: 16.06.2023 в данных книги стоит 245,00 руб., а ISS сегодня отдаёт 243,87 руб. Вероятно, запись в данных книги за этот день получена из другой выгрузки: у неё немного другой объём торгов. Единичные расхождения между источниками – обычное дело: биржа иногда уточняет исторические данные, а разные поставщики по-разному обрабатывают нестандартные дни. Важно, чтобы таких расхождений было мало и чтобы вы о них знали.

> **Типичная ошибка**
> Загрузить из ISS только первую страницу и не заметить этого. Код работает, ошибок не выдаёт, данные выглядят правдоподобно – но это лишь первые 100 дней периода. Проверяйте число строк: для дневных данных за год их должно быть около 250.

Для регулярной работы с ISS удобно пользоваться готовыми библиотеками, которые берут постраничную выдачу на себя, например `apimoex`. Данные для этой книги собраны пакетом `moexutils`, который дополнительно учитывает сплиты, переименования и дивиденды (глава о подготовке данных).

2.3 Данные Банка России

Банк России публикует ставки, которые служат ориентиром для всего финансового рынка. Для нас важны две.

Ключевая ставка – главный инструмент денежно-кредитной политики: ставка, по которой Банк России предоставляет банкам ликвидность и принимает от них депозиты на неделю. Её изменения задают уровень всех остальных ставок в экономике.

RUONIA (Ruble OverNight Index Average) – средневзвешенная ставка однодневных рублёвых межбанковских кредитов. Это рыночная ставка, которая держится вблизи ключевой, и лучший ориентир для доходности «денег без риска» на один день: именно её используют фонды денежного рынка и как безрисковую ставку в расчётах (глава «Доходность и риск»).

```
kr = data.load_key_rate().set_index("date")["rate"].loc["2013-09-13":]
ruonia = data.load_ruonia(start="2013-09-13").set_index("date")["rate"]

fig, ax = plt.subplots()
ax.plot(ruonia.index, ruonia, color="#9CA3AF", lw=0.8, label="RUONIA")
ax.step(kr.index, kr, where="post", color="#23415F", lw=1.4, label="Ключевая ставка")
ax.set_ylabel("% годовых")
ax.legend(loc="upper left")
plot.source(ax, "Банк России")
plt.show()
```



Рисунок 2.1. Ключевая ставка Банка России и ставка RUONIA, % годовых

На графике видны три эпизода резкого ужесточения денежно-кредитной политики. В декабре 2014 г. ставку подняли до 17% на фоне обвала рубля. 28 февраля 2022 г. – сразу до 20%. В 2023–2024 гг. ставку повышали постепенно, пока она не достигла 21% (28.10.2024). Минимум – 4,25% – пришёлся на период пандемии (27.07.2020). Для российского инвестора это означает, что «безрисковая» альтернатива вложениям в акции в разные годы давала от 4 до 21% годовых – разброс, немыслимый для развитых рынков.

> **Особенность данных: ключевая ставка до 2013 г.**
> Ключевая ставка введена Банком России 13 сентября 2013 г. В файле `key_rate.parquet` для более ранних дат приведена ставка рефинансирования, которая до этого выполняла похожую роль ориентира, но на практике была менее тесно связана с рыночными ставками. Поэтому на графике ряд начинается с 13.09.2013, а для расчётов безрисковой доходности за ранние годы лучше использовать RUONIA (с 2010 г.) или доходности коротких ОФЗ.

Кроме ставок, на сайте Банка России публикуются курсы валют, статистика банковского сектора, данные платёжного баланса и многое другое. Кривую бескупонной доходности ОФЗ рассчитывает Московская биржа; её история входит в данные книги (`zcyc.parquet`) и понадобится в главе об облигациях.

2.4 Российская экономическая статистика: rustata

Для многих задач финансового анализа нужны макроэкономические данные: инфляция, чтобы перейти от номинальной доходности к реальной; промышленное производство, чтобы понять состояние отраслей; региональная статистика,

бюджет, цены отдельных товаров. Основной источник таких данных в России – Росстат, но работать с ними напрямую неудобно. Данные разбросаны по системе ЕМИСС (fedstat.ru) и десяткам Excel-бюллетеней, показатели обозначены номерами вроде «31074», а один и тот же ряд в разных выгрузках называется по-разному. Кроме того, статистика **пересматривается**: первая оценка показателя через несколько месяцев может быть уточнена, и в свежей выгрузке вы увидите уже исправленное значение, а не то, что было известно в момент публикации.

Эти проблемы решает проект **rustata** (https://rustata.ru), который разрабатывает автор книги. Он собирает ряды Росстата, Банка России и Федерального казначейства, приводит их к единым справочникам территорий и единиц измерения и хранит **историю публикаций**. На сайте можно найти ряд поиском («инфляция», «цены на бензин», «промышленное производство»), посмотреть его на графике, сравнить регионы и скачать данные в CSV. Для программного доступа есть API.

Доступ к API – по токену, который выдаётся в личном кабинете после бесплатной регистрации (вход по ссылке из письма). Бесплатного тарифа достаточно для учебных задач: объём ограничен числом значений в месяц, а не числом запросов.

Токен – это пароль, поэтому его нельзя писать прямо в коде: код попадает в репозиторий, в отчёт, в чат с коллегами. Секреты принято хранить в файле `.env`, который не попадает в систему контроля версий. В корне книги лежит образец `.env.example`: скопируйте его в `.env` и впишите свой токен в строку `RUSTATA_TOKEN=`. Функция `secrets.get()` из модуля книги читает значение сначала из переменных окружения, затем из `.env`.

Загрузим годовую инфляцию в России – индекс потребительских цен на все товары и услуги к тому же месяцу прошлого года:

```
import io
import requests
from qf import secrets

API = "https://rustata.ru/api/v1"
headers = {"Authorization": f"Bearer {secrets.get('RUSTATA_TOKEN')}"}
dataset = "fedstat/indicator_31074"          # индексы потребительских цен (ЕМИСС 31074)

# Справочник кодов товаров и услуг набора: код 1 – «Все товары и услуги»
codes = requests.get(f"{API}/reference/codes", headers=headers,
                     params={"dataset": dataset, "format": "csv"}, timeout=60)
codes = pd.read_csv(io.StringIO(codes.text), sep=";")

resp = requests.get(f"{API}/data", headers=headers, timeout=60, params={
    "dataset": dataset, "geo": "country", "class": "1", "measure": "YOY",
    "start": "2015-01", "format": "csv", "labels": 1,
})
cpi = pd.read_csv(io.StringIO(resp.text), sep=";", parse_dates=["period", "as_of"])
inflation = cpi.set_index("period")["value"] / 100 - 1     # 109,92 → 9,92%
```

Параметры запроса `/data` задают **срез набора**: территории (`geo` – страна, федеральные округа, регионы или конкретные коды), продукты (`class`), вид показателя (`measure`) и период (`start`, `end`). Индексы цен Росстат публикует в нескольких видах: к предыдущему месяцу (`MOM`), к тому же месяцу прошлого года (`YOY`), к декабрю прошлого года (`DEC`). Значение 109,92 означает рост цен на 9,92%. Если не указать `class` и `measure`, запрос вернёт все товары во всех видах – сотни рядов. Один запрос может вернуть ряд сразу по всем регионам (`geo=region`) – это удобно для региональных сравнений. Полное описание методов и примеры на Python, R и Julia – на странице https://rustata.ru/api.

Главная особенность rustata для исследователя – **история публикаций**. У каждого значения хранится не только период, к которому оно относится, но и дата, когда оно стало известно. Поэтому можно запросить ряд в том виде, в каком он был опубликован на заданную дату (параметр `as_of` у метода `/series/{ключ}/observations`), и построить модель только на той информации, которая действительно была доступна. Это прямой ответ на проблему заглядывания в будущее, о которой пойдёт речь в следующей главе.

2.5 Международные источники

Для данных по зарубежным рынкам и макроэкономике есть несколько удобных бесплатных источников. Код в этом разделе обращается к внешним сервисам и приведён для справки: при сборке книги он не выполняется.

FRED (Federal Reserve Economic Data) – база Федерального резервного банка Сент-Луиса с сотнями тысяч макроэкономических и финансовых рядов: ставки ФРС, инфляция, доходности казначейских облигаций. Ряды загружаются по коду:

```
import pandas_datareader.data as pdr

us = pdr.DataReader(["FEDFUNDS", "CPIAUCSL"], "fred", start="2000-01-01")
```

SEC EDGAR – раскрытие американских публичных компаний. Через программный интерфейс `companyfacts` можно получить все показатели из отчётности компании в машиночитаемом виде.

Yahoo Finance (через библиотеку `yfinance`) – самый популярный бесплатный источник котировок в учебниках и курсах. Он удобен для американских акций, но у него есть серьёзные ограничения: нет гарантий качества и полноты, бывают ошибки в корректировках на дивиденды, сервис ограничивает частоту запросов. Для российских бумаг международные агрегаторы после 2022 г. стали ненадёжным источником: котировки могут не обновляться или содержать пропуски. Проверяйте дату последнего наблюдения и сравнивайте данные с биржей.

> **Сначала предскажите**
> Вы загрузили из международного агрегатора дневные цены акции за 10 лет и посчитали доходность по столбцу `Close`. Какие две проблемы из этой и предыдущей глав могли исказить результат? Подумайте, прежде чем читать дальше.

Ответ: во-первых, `Close` может быть как скорректированной, так и нескорректированной ценой – это зависит от источника и настроек загрузки (в `yfinance` за это отвечает параметр `auto_adjust`). Во-вторых, если в истории были сплиты или консолидации, нескорректированный ряд содержит разрывы.

Для проверки российских эмитентов полезны и нерыночные источники: центр раскрытия корпоративной информации (e-disclosure.ru), картотека арбитражных дел (kad.arbitr.ru), государственный информационный ресурс бухгалтерской отчётности ФНС (bo.nalog.ru). Отчётность компаний из последнего источника собрана в открытый набор данных RFSD, с которым мы будем работать в части II.

2.6 Форматы хранения данных

Загруженные данные нужно где-то хранить. Самый распространённый формат – **CSV**: текстовый файл, в котором значения разделены запятыми. Его можно открыть в любой программе, включая Excel и блокнот. Но у CSV есть недостатки: он занимает много места, медленно читается и не хранит типы данных – дата при чтении может превратиться в строку, а код ценной бумаги из цифр – в число.

Формат **Parquet** разработан для аналитических данных. Он хранит данные по столбцам, сжимает их и сохраняет типы. Сравним оба формата на данных книги – дневных котировках около ста акций за двадцать с лишним лет.

```
import time
from pathlib import Path

stocks = data.load_stocks()
tmp = Path("_tmp_formats"); tmp.mkdir(exist_ok=True)

stocks.to_csv(tmp / "stocks.csv", index=False)
stocks.to_parquet(tmp / "stocks.parquet", index=False)

def read_time(fn, path, n=3):
    t0 = time.perf_counter()
    for _ in range(n):
        fn(path)
    return (time.perf_counter() - t0) / n

formats = pd.DataFrame({
    "размер, МБ": [(tmp / f).stat().st_size / 1e6 for f in ("stocks.csv", "stocks.parquet")],
    "чтение, с": [read_time(pd.read_csv, tmp / "stocks.csv"),
```

```
                read_time(pd.read_parquet, tmp / "stocks.parquet")],
}, index=["CSV", "Parquet"])
formats.round(2)
#>          размер, МБ  чтение, с
#> CSV           35.69       0.20
#> Parquet       14.91       0.03
```

Таблица из 311 631 строки в формате CSV занимает в 2 раза больше места и читается в 8 раз дольше, чем в Parquet. Кроме того, Parquet позволяет читать только нужные столбцы и строки, не загружая файл целиком, – это особенно важно для больших наборов данных, например отчётности сотен тысяч компаний в части II. Поэтому данные книги хранятся в Parquet, а CSV мы используем для небольших справочников и обмена с другими программами.

Для работы с большими таблицами наряду с pandas мы будем использовать библиотеку **Polars**. Она быстрее pandas на больших данных и умеет выполнять запросы «лениво»: сначала строит план вычислений, а затем читает из файла только то, что нужно для результата.

2.7 Данные этой книги

Все примеры книги работают на снимке данных, который прилагается к ней (папка `data/`). Он собран из данных Московской биржи, Банка России и истории дивидендов и зафиксирован на определённую дату, поэтому результаты в тексте воспроизводятся точно.

Таблица 2.1. Данные книги

Файл	Содержание	Функция загрузки
`stocks.parquet`	дневные данные около 100 акций с 2003 г., с поправкой на сплиты и дивиденды	`data.load_stocks()`, `data.load_prices()`
`stocks_raw.parquet`	«сырые» цены ISS по бумагам со сплитами и переименованиями	`data.load_stocks_raw()`
`indexes.parquet`	индексы Мосбиржи: акций, облигаций, отраслевые	`data.load_index()`
`dividends.parquet`	история дивидендов	`data.load_dividends()`
`key_rate.parquet`, `ruonia.parquet`	ключевая ставка и RUONIA	`data.load_key_rate()`, `data.load_ruonia()`
`zcyc.parquet`	кривая бескупонной доходности ОФЗ	`data.load_zcyc()`
`sectors.csv`, `splits.csv`, `renames.csv`, `delisted.csv`	справочники	`data.load_reference()`

Отдельным архивом доступна полная история торгов всеми акциями Московской биржи с 1997 г. – около 2,5 тыс. бумаг во всех режимах торгов. Её загружает

функция `data.load_shares_full()`. Подробное описание таблиц и полей – в приложении B.

Функции загрузки возвращают таблицы в «длинном» формате: одна строка – одна бумага в один день. Для расчётов часто удобнее «широкий» формат, где даты идут по строкам, а бумаги – по столбцам:

```
prices = data.load_prices(["SBER", "GAZP", "LKOH"], start="2026-09-01")
prices.tail(3)
#> ticker       GAZP    LKOH    SBER
#> date
#> 2026-09-25  98.78  5374.0  276.35
#> 2026-09-28  96.39  5299.5  272.45
#> 2026-09-29  97.58  5369.0  274.65
```

Итоги

- Финансовые данные бывают рыночными, фундаментальными, макроэкономическими и альтернативными; они различаются частотой и моментом доступности.
- Московская биржа бесплатно раскрывает историю торгов через ISS. Для акций нужен основной режим торгов `TQBR`; историю ISS отдаёт страницами по 100 строк.
- Банк России публикует ключевую ставку и RUONIA – ориентиры безрисковой доходности. Ключевая ставка существует с сентября 2013 г.
- Статистику Росстата, Банка России и Казначейства с историей публикаций удобно получать через rustata: сайт, CSV и API.
- Международные агрегаторы удобны, но для российских бумаг ненадёжны; данные нужно сверять с биржей.
- Для аналитических данных Parquet компактнее и быстрее CSV и сохраняет типы.
- Прежде чем считать, выясните, что именно измеряет каждое число: цена или полная доходность, какой режим торгов, какая дата.

2.8 Упражнения

2.8.1 A. Понимание

1. Почему для анализа акций используют режим торгов `TQBR`, а не все сделки с бумагой? Приведите пример, когда это может сильно исказить результат.
2. Компания опубликовала годовую отчётность за 2025 г. 28 марта 2026 г. С какой даты эти данные можно использовать в торговой стратегии? Что будет, если привязать их к 31 декабря 2025 г.?
3. Почему в 2024 г. RUONIA была бóльшую часть времени близка к ключевой ставке? Что означало бы устойчивое расхождение между ними?

2.8.2 Б. Практика на своей акции

4. Загрузите из ISS историю торгов своей акцией за последние три года с помощью функции `iss_history()`. Сравните цены закрытия с `data.load_stocks()`. Совпадают ли они? Если нет – в какие даты и почему?
5. Запросите карточку своей бумаги (`/securities/{тикер}.json`). В каких режимах она торгуется? Какой у неё уровень листинга?
6. Посчитайте доходность своей акции за последний полный год по `close` и по `adj_close`. Объясните разницу.
7. Зарегистрируйтесь на rustata.ru, получите токен и загрузите месячный индекс потребительских цен по России. Посчитайте годовую инфляцию и **реальную ключевую ставку** – разницу между ключевой ставкой и инфляцией за предыдущие 12 месяцев. В какие периоды реальная ставка была отрицательной?

2.8.3 В. Работа с ИИ-ассистентом

8. Попросите ИИ-ассистента написать функцию загрузки дневной истории акции с Московской биржи через ISS за 5 лет. Проверьте: учитывает ли функция постраничную выдачу? Какой режим торгов она использует? Что произойдёт, если бумага торговалась в нескольких режимах? Исправьте найденные проблемы.
9. Спросите ассистента, какой была ключевая ставка Банка России в 2012 г. Сверьте ответ с разделом 2.3. Какую ошибку легко допустить в этом вопросе?

2.8.4 Г. Мини-исследование

10. С помощью ISS найдите пять самых торгуемых акций Мосбиржи по обороту за последний месяц и сравните их с пятью самыми торгуемыми акциями пять лет назад. Какие компании ушли из списка и почему? Подготовьте короткий устный рассказ.

Вопросы для самопроверки

1. Что такое режим торгов и какой режим является основным для акций на Мосбирже?
2. Как узнать, что ISS вернул не все строки запроса?
3. Чем RUONIA отличается от ключевой ставки?
4. Назовите три преимущества формата Parquet перед CSV.
5. Что означает «момент доступности» данных и почему он важен для моделей?

Ответы

1. Режим торгов – отдельная «площадка» внутри рынка со своими правилами (размер лота, расчёты, участники). Основной режим для акций – `TQBR`.

2. По служебной таблице `history.cursor`: если `TOTAL` больше числа полученных строк, нужно запрашивать следующие страницы.
3. Ключевая ставка устанавливается Банком России; RUONIA – рыночная ставка однодневных межбанковских кредитов, которая колеблется вблизи ключевой.
4. Меньший размер, более быстрое чтение, сохранение типов данных (а также чтение только нужных столбцов).
5. Это дата, когда информация стала известна участникам рынка. Если модель использует данные раньше этой даты, она заглядывает в будущее, и её результаты на истории будут завышены.

Что почитать

- Описание API rustata: https://rustata.ru/api.
- Документация ISS Московской биржи: https://iss.moex.com/iss/reference/ – полный список ресурсов программного интерфейса.
- Scheuch и др. (2024 г.), гл. 3 – организация финансовых данных в проекте и работа с базой данных.

3 Подготовка данных

08.07.2024 акция ВТБ закрылась по цене 0,0199 руб., а на следующий торговый день, 15.07.2024, – по 92,95 руб. Если посчитать доходность по этим двум ценам, получится рост на 466 282% за один день. Разумеется, никто столько не заработал: ВТБ провёл **консолидацию акций** – каждые 5000 старых акций превратились в одну новую, и цена одной акции выросла в 5000 раз. Стоимость вложений акционеров при этом не изменилась.

Такие события происходят регулярно, и каждое из них – ловушка для аналитика. Модель, обученная на неочищенных данных, увидит в этот день «сигнал» чудовищной силы; средняя доходность рынка, посчитанная по таким данным, окажется бессмысленной. В предыдущей главе мы научились загружать данные. В этой – разберёмся, что с ними нужно сделать, прежде чем считать.

> **В этой главе**
>
> - Находим и исправляем сплиты и консолидации акций.
> - Склеиваем историю компаний, сменивших тикер.
> - Строим цену с поправкой на дивиденды своими руками и сверяем её с данными книги.
> - Измеряем ошибку выжившего на российском рынке.
> - Разбираемся с пропусками, остановками торгов и выбросами.
> - Учимся не заглядывать в будущее и проверять качество данных.
>
> **Данные:** `stocks.parquet`, `stocks_raw.parquet`, `dividends.parquet`, `splits.csv`, `renames.csv`, `delisted.csv`.
> **Нужно знать:** доходность (глава «Доходность и риск»), загрузка данных (глава «Источники финансовых данных»).

3.1 Сплиты и консолидации

Сплит (дробление) – это деление каждой акции на несколько: при сплите 1:10 акционер вместо одной акции получает десять, а цена одной акции падает примерно в десять раз. **Консолидация** (обратный сплит) – обратная операция: несколько акций объединяются в одну. Ни то, ни другое не меняет стоимость бизнеса и долю акционера: меняется только «размер купюры».

Зачем это делают? Слишком дорогую акцию трудно купить небольшому инвестору: акция «Транснефти» перед сплитом 2024 г. стоила около 160 тыс. руб. Слишком дешёвая акция неудобна для торговли и восприятия: у ВТБ цена опустилась до двух копеек, и котировки записывались с шестью знаками после запятой.

3.1.1 Как сплит выглядит в данных

В файле `stocks_raw.parquet` лежат цены в том виде, в каком их отдаёт ISS, – без поправок. Посмотрим на дни вокруг сплита «Норникеля» в апреле 2024 г. и консолидации ВТБ в июле 2024 г.

```
raw = data.load_stocks_raw(["GMKN", "VTBR"]).set_index("date")

def around(ticker, date, n=3):
    s = raw[raw["ticker"] == ticker]["close"]
    i = s.index.searchsorted(pd.Timestamp(date))
    return s.iloc[i - n:i + n]

pd.concat({"GMKN": around("GMKN", "2024-04-04"), "VTBR": around("VTBR", "2024-07-15")}, axis=1)
#>                 GMKN       VTBR
#> date
#> 2024-03-28  15208.00        NaN
#> 2024-03-29  15110.00        NaN
#> 2024-04-01  15054.00        NaN
#> 2024-04-08    152.96        NaN
#> 2024-04-09    164.00        NaN
#> 2024-04-10    163.30        NaN
#> 2024-07-04       NaN   0.020505
#> 2024-07-05       NaN   0.020270
#> 2024-07-08       NaN   0.019930
#> 2024-07-15       NaN  92.950000
#> 2024-07-16       NaN  95.320000
#> 2024-07-17       NaN  93.720000
```

Видны сразу две особенности. Во-первых, цена меняется в 100 или 5000 раз за один «день». Во-вторых, «день» на самом деле длиннее: между последней ценой до события и первой после него есть пропуск в несколько торговых дней – на время технической процедуры торги бумагой приостанавливаются.

3.1.2 Как найти сплиты автоматически

Реестр корпоративных событий удобен, но его нужно вести, и он может быть неполным. Поэтому полезно уметь находить подозрительные дни в самих данных. Простое правило: цена акции почти никогда не меняется за день более чем в три раза. Если это произошло – скорее всего, это сплит, консолидация или ошибка в данных.

```
raw_all = data.load_stocks_raw()
jumps = []
for ticker, g in raw_all.groupby("ticker"):
    close = g.set_index("date")["close"]
    ratio = close / close.shift(1)
    for date, x in ratio[(ratio > 3) | (ratio < 1 / 3)].items():
        jumps.append({"ticker": ticker, "date": date.date(), "изменение цены, раз": round(x, 4)})

pd.DataFrame(jumps)
#>   ticker        date  изменение цены, раз
#> 0   GMKN  2024-04-08               0.0102
#> 1   IRAO  2015-01-20             107.3174
#> 2   PHOR  2012-03-26               0.0921
#> 3      T  2026-04-17               0.1021
#> 4  TRNFP  2024-02-21               0.0095
#> 5   VTBR  2024-07-15            4663.8234
```

Сравним найденное с реестром сплитов, который используется для корректировки данных книги:

```
data.load_reference("splits")
#>   ticker        date     ratio  kind
#> 0   PHOR  2012-03-26   10.0000  auto
#> 1   IRAO  2015-01-20    0.0100  auto
#> 2      T  2026-04-17   10.0000  auto
#> 3  TRNFP  2024-02-21  100.0000  auto
#> 4   GMKN  2024-04-04  100.0000  auto
#> 5   VTBR  2024-07-15    0.0002  auto
#> 6   PLZL  2025-03-27   10.0000  auto
```

Простое правило нашло 6 событий из 6, которые есть в реестре для бумаг этого файла. Столбец `ratio` в реестре показывает, сколько новых акций получает владелец одной старой: 100 у «Норникеля», 0,0002 (то есть 1/5000) у ВТБ. Даты в реестре и в данных иногда различаются на несколько дней: в реестре записана дата события, а в данных – первый день торгов после перерыва.

3.1.3 Как скорректировать цены

Чтобы ряд цен стал сопоставимым, все цены **до** сплита делят на коэффициент `ratio`. Цены после сплита не меняются: так история приводится к текущему «размеру купюры».

```
def adjust_splits(raw, splits):
    """Приводит цены к текущей базе с учётом сплитов и консолидаций."""
    out = raw.copy()
    for _, s in splits.iterrows():
        before = (out["ticker"] == s["ticker"]) & (out["date"] < pd.Timestamp(s["date"]))
        out.loc[before, ["open", "high", "low", "close"]] /= s["ratio"]
    return out
```

```
fixed = adjust_splits(data.load_stocks_raw(["VTBR"]), data.load_reference("splits"))
book = data.load_stocks("VTBR")
check = fixed.merge(book[["date", "close"]], on="date", suffixes=("_fixed", "_book"))
(check["close_fixed"] / check["close_book"] - 1).abs().max()
#> 2.220446049250313e-16
```

Скорректированные цены совпадают с ценами в данных книги с точностью до ошибок округления. После корректировки объёмы торгов (в штуках) нужно, наоборот, умножить на `ratio`: иначе число проторгованных акций до события будет несопоставимо с числом после.

```
vt_raw = raw_all[raw_all["ticker"] == "VTBR"].set_index("date")["close"].loc["2022":]
vt_fix = fixed.set_index("date")["close"].loc["2022":]

fig, ax = plt.subplots()
ax.plot(vt_raw.index, vt_raw, color="#B8412E", lw=1, label="сырые данные ISS")
ax.plot(vt_fix.index, vt_fix, color="#34699A", lw=1.2, label="с поправкой на консолидацию")
ax.set_yscale("log")
ax.set_ylabel("руб.")
ax.legend(loc="center left")
plot.source(ax, "Московская биржа, расчёты автора")
plt.show()
```

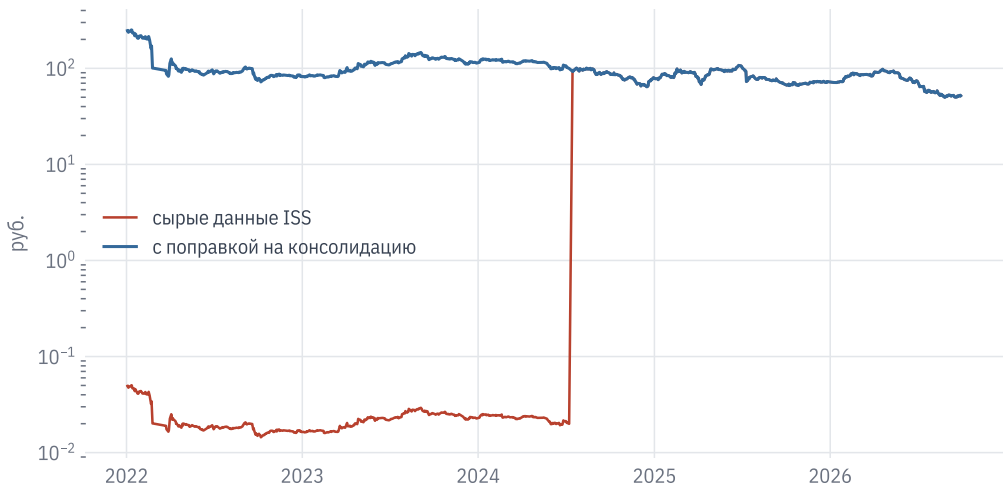


Рисунок 3.1. Цена акции ВТБ: сырые данные ISS и с поправкой на консолидацию (логарифмическая шкала)

Вариант кода на R – в онлайн-версии книги.

3.2 Переименования: одна компания – несколько тикеров

Иногда бумага компании получает новый тикер: после реорганизации, смены эмитента или просто по решению биржи. Для аналитика это означает, что история «одной компании» разбита на два ряда. Реестр переименований в данных книги:

```
data.load_reference("renames")
#>     old   new        date
#> 0  MRKH  RSTI  2014-01-06
#> 1  EONR  UPRO  2016-07-01
#> 2  MAIL  VKCO  2021-12-14
#> 3  YNDX  YDEX  2024-07-24
#> 4  HHRU  HEAD  2024-09-26
#> 5  TCSG     T  2024-11-28
```

Самый известный пример – «Яндекс». Последний день торгов бумагами YNDX – 14.06.2024, первый день торгов YDEX – 24.07.2024: между ними 40 дней без торгов. За это время прошла корпоративная реструктуризация: акции нидерландской компании обменивались на акции новой российской компании.

Если загрузить данные только по текущему тикеру YDEX, история начнётся в июле 2024 г. Если склеить два ряда, получится история с 2014 г. – но это уже решение аналитика: формально это бумаги разных эмитентов, и доходность «через разрыв» – допущение. В данных книги такие истории склеены (функция `load_stocks` возвращает их под новым тикером), а исходные ряды доступны в `stocks_raw.parquet`.

> **Типичная ошибка**
> Искать историю компании только по текущему тикеру. Для «Яндекса», VK (бывший MAIL), «Т-Технологий» (бывший TCSG) и других компаний вы потеряете годы истории – и, что хуже, не заметите этого, если не проверите дату первого наблюдения.

3.3 Дивиденды и скорректированная цена

В главе «Доходность и риск» мы использовали столбец `adj_close` – цену с поправкой на дивиденды. Теперь построим её сами. Это полезно по двум причинам: так понятнее, что именно лежит в этом столбце, и так можно проверить данные, полученные из стороннего источника.

3.3.1 Экс-дивидендная дата

Дивиденды получает тот, кто был акционером на **дату закрытия реестра** (дату отсечки). Но между сделкой и её расчётом проходит время: с 31 июля 2023 г.

сделки с акциями на Мосбирже рассчитываются в режиме T+1 (на следующий рабочий день), а до этого – в режиме T+2. Поэтому последний день, когда можно купить акцию и получить дивиденд, наступает раньше даты отсечки. Первый день, когда акция торгуется уже без дивиденда, называется **экс-дивидендной датой**:

- в режиме T+1 это сама дата закрытия реестра (или последний торговый день перед ней, если она выпала на выходной);
- в режиме T+2 – торговый день перед датой закрытия реестра.

3.3.2 Алгоритм корректировки

Пусть D – дивиденд на акцию, а P – цена закрытия накануне экс-дивидендной даты. Все цены **до** экс-даты умножаются на коэффициент

$$k = 1 - \frac{D}{P}. \tag{3.1}$$

Если дивидендов было несколько, коэффициенты перемножаются. Смысл корректировки: доходность в день отсечки должна учитывать и падение цены, и полученный дивиденд. Проверим, что формула это обеспечивает. Скорректированная цена накануне равна $kP = P - D$, цена в день отсечки P_1 не меняется, поэтому доходность по скорректированным ценам равна $P_1/(P - D) - 1$. Полная доходность акционера за этот день равна $(P_1 + D)/P - 1$. Эти две величины не совпадают точно, но при обычных размерах дивидендов разница составляет сотые доли процента. Зато ряд скорректированных цен непрерывен, и доходность за любой период считается как отношение двух цен.

```
def adjust_dividends(close, dividends, t1_date="2023-07-31"):
    """Цена с поправкой на дивиденды (корректировка «назад»)."""
    factor = pd.Series(1.0, index=close.index)
    for _, d in dividends.iterrows():
        rec = d["closing_date"]
        if rec ⩾ pd.Timestamp(t1_date):
              ex = close.index.searchsorted(rec, side="right") - 1    # последний торговый день
⩽ отсечки
        else:
            ex = close.index.searchsorted(rec, side="left") - 1    # торговый день перед отсечкой
        if ex ⩽ 0 or ex ⩾ len(close):
            continue                                                # отсечка вне периода данных
        prev_close = close.iloc[ex - 1]
        factor.iloc[:ex] *= 1 - d["dividend_value"] / prev_close
    return close * factor

sber = data.load_stocks("SBER").set_index("date")
my_adj = adjust_dividends(sber["close"], data.load_dividends("SBER"))
(my_adj / sber["adj_close"] - 1).abs().max()
#> 6.661338147750939e-16
```

Наша корректировка по 28 дивидендам Сбербанка совпала с adj_close из данных книги до ошибок округления.

> **Особенность данных: корректировка «назад» меняет прошлое**
> При корректировке назад прошлые цены зависят от *будущих* дивидендов. Скорректированная цена Сбербанка на первую дату в данных книги, 21.11.2011, составляет 41% от фактической цены того дня, и после каждой новой выплаты это число будет уменьшаться. Для доходностей это не важно: отношение соседних цен не меняется. Но если стратегия использует *уровень* скорректированной цены – например, правило «покупать акции дешевле 100 руб.» или дивидендную доходность по `adj_close`, – она незаметно использует информацию, которой в прошлом не было.

3.4 Ошибка выжившего

Представьте исследователя, который в 2026 г. хочет узнать среднюю доходность российских акций за последние десять лет. Он загружает список бумаг, которые сейчас торгуются на бирже, скачивает их историю и считает. Проблема в том, что в этом списке нет компаний, которые за десять лет ушли с биржи – обанкротились, были выкуплены, провели делистинг. А среди них непропорционально много неудачников. Результат окажется завышенным. Эта ошибка называется **ошибкой выжившего** (survivorship bias).

В данных книги есть и бумаги, снятые с торгов. Это позволяет измерить величину ошибки.

```
prices = data.load_prices(start="2015-12-30", end="2025-12-31")
last_date = prices.apply(lambda c: c.last_valid_index())
survivor = last_date ⩾ pd.Timestamp("2025-12-01")        # торгуются в конце периода

rows = []
for year in range(2016, 2026):
    start = prices.loc[:f"{year - 1}-12-31"].iloc[-1]       # цена на конец прошлого года
    end = prices.loc[str(year)].apply(lambda c: c.dropna().iloc[-1] if c.notna().any() else np.nan)
    r = (end / start - 1).dropna()                          # у ушедших – доходность до последнего дня
   rows.append({"год": year, "все бумаги": r.mean(), "только выжившие": r[survivor[r.index]].mean(),
                "число бумаг": len(r)})

surv = pd.DataFrame(rows).set_index("год")
surv.round(3)
#>       все бумаги  только выжившие  число бумаг
#> год
#> 2016       0.509            0.598           68
#> 2017       0.046            0.066           68
#> 2018       0.032            0.038           67
#> 2019       0.265            0.276           68
```

```
#> 2020       0.231            0.221           66
#> 2021       0.289            0.342           75
#> 2022      -0.305           -0.276           79
#> 2023       0.680            0.704           75
#> 2024      -0.027           -0.005           79
#> 2025      -0.047           -0.044           75
```

За 2016–2025 гг. средняя годовая доходность (с учётом дивидендов, равные веса) по всем бумагам составила 16,7%, а только по «выжившим» – 19,2%. Ошибка выжившего добавляет около 2,5 процентного пункта в год – и это при том, что в наших данных только 17 ушедших бумаг из примерно ста. В полной истории рынка ушедших компаний гораздо больше, и эффект сильнее.

Важная деталь в коде: для бумаги, ушедшей с биржи в течение года, мы берём доходность до последнего дня торгов. На практике инвестор при делистинге может получить и меньше – например, если выкуп проходит по цене ниже рыночной или бумаги перестают торговаться совсем. Поэтому даже наша оценка ошибки выжившего скорее занижена.

3.5 Пропуски, остановки торгов и выбросы

3.5.1 Календарь торгов

Прежде чем считать доходности, нужно понять, какие даты считать торговыми днями. Естественный календарь – дни, в которые рассчитывался индекс Мосбиржи. Посмотрим на самые длинные перерывы в нём.

```
imoex = data.load_index("IMOEX", start="2005-01-01").set_index("date")["close"]
gaps = imoex.index.to_series().diff().dt.days
gaps[gaps > 5].sort_values(ascending=False).head(5)
#> date
#> 2022-03-24    27.0
#> 2008-01-09    12.0
#> 2009-01-12    12.0
#> 2011-01-11    12.0
#> 2006-01-10    11.0
#> Name: date, dtype: float64
```

Большинство длинных перерывов – новогодние каникулы. Выделяется перерыв весной 2022 г.: торги акциями были остановлены с 28 февраля по 23 марта. Доходность, посчитанная через такой перерыв, – это доходность за месяц, а не за день.

3.5.2 Пропуски у отдельных бумаг

У отдельной акции бывают и собственные пропуски: приостановка торгов перед корпоративным событием, как у ВТБ и «Норникеля» выше, или просто отсут-

ствие сделок. Найдём для каждой бумаги самый длинный перерыв внутри её истории, не считая общих перерывов рынка.

```
stocks = data.load_stocks(start="2015-01-01", columns=["close"])
calendar = imoex.loc["2015":].index

def longest_gap(dates):
    """Самый длинный перерыв в торговых днях биржевого календаря."""
    pos = calendar.searchsorted(dates)
    return int(np.diff(pos).max()) - 1 if len(pos) > 1 else 0

stock_gaps = stocks.groupby("ticker")["date"].apply(longest_gap).sort_values(ascending=False)
stock_gaps.head(8)
#> ticker
#> POLY    39
#> OZON    34
#> HEAD    33
#> YDEX    27
#> VKCO    23
#> T       19
#> MSTT    17
#> MDMG    16
#> Name: date, dtype: int64
```

Перерывы длиной в десятки торговых дней – повод разобраться, что происходило с бумагой. Для большинства расчётов такие интервалы нужно либо исключить, либо явно помнить, что доходность «через перерыв» относится к длинному периоду.

3.5.3 Выбросы

Выброс – наблюдение, которое сильно отличается от остальных. В финансовых данных выбросы бывают двух видов: ошибки (сплит без поправки, неверная цена) и реальные экстремальные события (24 февраля 2022 г.). Отличить одно от другого механически нельзя – нужно смотреть на каждый случай.

```
stocks = data.load_stocks(start="2010-01-01", columns=["adj_close"])
stocks["r"] = stocks.groupby("ticker")["adj_close"].pct_change(fill_method=None)
extreme = stocks[stocks["r"].abs() > 0.3].sort_values("r")
extreme[["date", "ticker", "r"]].round(3).head(10)
#>              date ticker      r
#> 170539 2022-07-12   POGR -0.770
#> 125349 2022-02-24  MTLRP -0.495
#> 121467 2022-02-24   MTLR -0.469
#> 25401  2022-02-24  BANEP -0.459
#> 180591 2022-02-24   RASP -0.447
#> 119382 2013-11-13   MTLR -0.414
#> 272188 2022-02-24   VTBR -0.413
#> 153319 2022-02-24   OGKB -0.408
```

```
#> 14616  2011-12-02   ALRS -0.400
#> 220866 2022-02-24   SMLT -0.398
```

Дневных изменений больше 30% по модулю в данных с 2010 г. – 87, из них 36 приходятся на февраль и март 2022 г. Это реальные события: обвал 24 февраля и первые дни после возобновления торгов. Удалять их нельзя – они и есть риск, который мы хотим измерить. А вот сплит без поправки в этом списке был бы ошибкой, которую нужно исправить.

> **Типичная ошибка**
> «Очистить» данные, удалив все доходности больше заданного порога. Так из данных исчезают кризисы, и модель риска начинает считать рынок гораздо спокойнее, чем он есть.

3.6 Не заглядывать в будущее

Заглядывание в будущее (look-ahead bias) – использование в расчётах на дату t информации, которая стала известна позже. Это самая коварная ошибка подготовки данных: код работает, результаты выглядят прекрасно, и только при реальном применении выясняется, что стратегия не работает.

Типичные источники заглядывания в будущее:

- **Отчётность по дате периода, а не по дате публикации.** Годовая отчётность за 2025 г. выходит весной 2026 г. Правило: сдвигать фундаментальные данные на дату публикации или хотя бы на консервативный срок (например, отчётность за год использовать с июля следующего года).
- **Пересмотры макроэкономической статистики.** Инфляция, ВВП и другие показатели после первой публикации уточняются. В свежей выгрузке вы видите исправленные значения, которых в прошлом не знал никто. Для исторических расчётов нужны ряды в том виде, в каком они публиковались на каждую дату, – например, из rustata (раздел 2.4).
- **Текущий состав индекса.** Отбор «акций из индекса Мосбиржи» по сегодняшнему списку в прошлом – это одновременно ошибка выжившего и заглядывание в будущее: в индекс чаще попадают компании, которые выросли.
- **Уровни скорректированных цен.** Как мы видели в разделе 3.3, они зависят от будущих дивидендов.
- **Параметры, подобранные на всей выборке.** Если порог, окно или набор бумаг выбраны по результатам за весь период, а затем «проверены» на этом же периоде, проверка ничего не говорит о будущем. Этой проблеме посвящены главы о машинном обучении и бэктестинге.

- **Цена исполнения.** Сигнал, рассчитанный по цене закрытия дня t, нельзя исполнить по той же цене закрытия: в лучшем случае – по цене следующего дня.

3.7 Проверка качества данных

Подготовку данных полезно завершать формальной проверкой. Ниже – функция, которая проверяет таблицу котировок на типичные проблемы. Её стоит запускать каждый раз, когда вы получаете данные из нового источника.

```
def check_prices(df):
    """Проверка таблицы дневных котировок (date, ticker, open, high, low, close, volume)."""
    issues = {
        "дубликаты (date, ticker)": df.duplicated(["date", "ticker"]).sum(),
        "пропуски в close": df["close"].isna().sum(),
        "неположительные цены": (df[["open", "high", "low", "close"]] <= 0).any(axis=1).sum(),
        "high < low": (df["high"] < df["low"]).sum(),
        "close вне [low, high]": ((df["close"] > df["high"] * 1.0001)
                                  | (df["close"] < df["low"] * 0.9999)).sum(),
        "нулевой объём": (df["volume"] == 0).sum(),
    }
    r = df.sort_values(["ticker", "date"]).groupby("ticker")["close"].pct_change(fill_method=None)
    issues["|доходность| > 50%"] = (r.abs() > 0.5).sum()
    return pd.Series(issues, name="число строк")

check_prices(data.load_stocks())
#> дубликаты (date, ticker)     0
#> пропуски в close             0
#> неположительные цены         0
#> high < low                   0
#> close вне [low, high]        0
#> нулевой объём                1
#> |доходность| > 50%          18
#> Name: число строк, dtype: int64
```

В данных книги проверка не находит ни дубликатов, ни пропусков, ни неположительных цен, ни противоречий между high, low и close – эти данные уже прошли очистку. Но остаются 18 дневных изменений цены больше 50% по модулю. Среди них есть реальные события – например, резкий рост рынка 19 сентября 2008 г. после нескольких дней приостановки торгов или движения декабря 2014 г. – и случаи, которые требуют проверки по новостям компании. Такой список – обычный результат проверки: машина находит подозрительные места, а объяснить их должен аналитик.

На «сырых» данных ISS та же проверка находит 8 дневных изменений больше 50% по модулю. Почти все они – сплиты и консолидации из раздела 3.1.

Итоги

- Сплиты и консолидации создают в сырых ценах разрывы в десятки и тысячи раз. Их можно найти по данным и исправить, разделив цены до события на коэффициент сплита.
- История компании может быть разбита на несколько тикеров; склейка – осознанное решение аналитика.
- Цена с поправкой на дивиденды строится умножением прошлых цен на $1 - D/P$ начиная с экс-дивидендной даты; с 31.07.2023 экс-дата совпадает с датой закрытия реестра.
- Корректировка «назад» меняет уровни прошлых цен – уровни скорректированных цен нельзя использовать как сигналы.
- Ошибка выжившего на российском рынке в 2016–2025 гг. завышала среднюю доходность примерно на 2–3 процентных пункта в год.
- Пропуски и выбросы нужно не удалять механически, а объяснять: одни – ошибки, другие – реальные события.
- Главное правило: на дату t можно использовать только то, что было известно на дату t.

3.8 Упражнения

3.8.1 А. Понимание

1. Акция стоила 3000 руб., компания провела сплит 1:10. Какой будет цена после сплита? Как изменится стоимость пакета из 5 акций? Какое значение `ratio` нужно записать в реестр?
2. Дата закрытия реестра – пятница, 18 июля 2025 г. Какая экс-дивидендная дата? А если бы то же событие было в 2022 г.?
3. Почему удаление всех дневных доходностей больше 20% по модулю искажает оценку риска? Когда такое удаление было бы оправдано?

3.8.2 Б. Практика на своей акции

4. Проверьте, были ли у вашей акции сплиты, консолидации или смена тикера. Найдите самый длинный перерыв в торгах внутри её истории и выясните его причину.
5. Постройте для своей акции цену с поправкой на дивиденды функцией `adjust_dividends()` и сравните с `adj_close`. Если есть расхождения – в какие даты и почему?
6. Запустите `check_prices()` на данных своей акции. Разберите каждую найденную проблему.

3.8.3 В. Работа с ИИ-ассистентом

7. Попросите ИИ-ассистента написать функцию корректировки цен на дивиденды. Проверьте на данных Сбербанка: как ассистент определяет экс-дивидендную дату? Учитывает ли переход с T+2 на T+1? Какую цену берёт в знаменателе D/P? Сравните результат с `adj_close`.
8. Спросите ассистента, как правильно посчитать среднюю доходность российских акций за 10 лет. Упомянул ли он ошибку выжившего? Предложите ему проверить свой код на данных книги и оцените, что изменилось.

3.8.4 Г. Мини-исследование

9. Используя полную историю рынка (`data.load_shares_full()`), найдите все бумаги, которые торговались в режиме TQBR в начале 2015 г., и определите, какая доля из них торгуется сегодня. Как изменится оценка ошибки выжившего, если посчитать её на этом, более полном наборе? Подготовьте короткий устный рассказ.
10. Функция `check_prices()` находит в данных книги дневные изменения цены больше 50% по модулю. Выберите пять из них и для каждого выясните по новостям и раскрытию компании, реальное ли это событие или ошибка в данных. Какие признаки помогают отличить одно от другого?

Вопросы для самопроверки

1. Чем сплит отличается от консолидации и как каждый из них выглядит в сырых данных?
2. Какая экс-дивидендная дата при расчётах T+1 и при расчётах T+2?
3. Почему уровни цен, скорректированных «назад», нельзя использовать как торговые сигналы?
4. Что такое ошибка выжившего и почему она завышает доходность?
5. Приведите три примера заглядывания в будущее.

Ответы

1. При сплите одна акция делится на несколько, цена падает; при консолидации несколько акций объединяются в одну, цена растёт. В сырых данных – скачок цены в число раз, равное коэффициенту, обычно после перерыва в торгах.
2. T+1 – дата закрытия реестра (или последний торговый день перед ней); T+2 – торговый день перед датой закрытия реестра.
3. Они зависят от дивидендов, выплаченных позже, то есть содержат информацию из будущего.
4. Это отбор только тех компаний, которые существуют сегодня. Ушедшие с рынка компании в среднем были неудачными, поэтому их исключение завышает среднюю доходность.

5. Использование отчётности до даты её публикации; отбор бумаг по текущему составу индекса; подбор параметров стратегии на всей выборке; исполнение сделки по цене, по которой рассчитан сигнал.

Что почитать

- Scheuch и др. (2024 г.), гл. 3–4 – подготовка и хранение финансовых данных.
- Campbell и др. (1997 г.), гл. 1 – корректировка цен и расчёт доходностей.

4 Доходность и риск

Представьте инвестора, который 20.10.2021 вложил деньги в индекс Московской биржи с учётом дивидендов. Через год, 26.09.2022, от вложений осталось меньше половины: индекс опустился на 53,5% от точки входа. Вернуться к исходной сумме удалось только 17.02.2025, спустя 40 месяцев. Хорошее это было вложение или плохое? Какую доходность оно принесло и сколько риска за неё пришлось нести? Что было бы, если бы инвестор вложился на год раньше или на год позже?

Чтобы ответить на такие вопросы, нужен общий язык: способ измерять доходность и риск так, чтобы результаты можно было сравнивать между активами, периодами и стратегиями. Этот язык и есть предмет главы. Мы будем опираться на данные Московской биржи за два десятилетия. За это время российский рынок пережил кризис 2008 г., девальвацию 2014 г., пандемию 2020 г. и шок 2022 г., поэтому примеров того, как ведут себя доходности в спокойные и в кризисные годы, будет достаточно.

В этой главе

- Считаем простую и логарифмическую доходность и выясняем, какую из них для чего использовать.
- Разбираем, почему доходность российского рынка без дивидендов занижена в разы и почему средняя доходность бывает обманчивой.
- Измеряем риск: волатильность, тяжёлые хвосты, просадки.
- Сравниваем активы по доходности с поправкой на риск с учётом высокой безрисковой ставки в России.

Данные: `stocks.parquet`, `indexes.parquet`, `ruonia.parquet`, `key_rate.parquet` (см. данные книги).
Нужно знать: работа с pandas, загрузка данных (глава о финансовых данных).

В примерах главы используются следующие библиотеки и модули книги: `data` загружает учебные данные, `plot` задаёт единый стиль графиков.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from scipy import stats
from qf import data, plot
```

4.1 Цены и доходности

4.1.1 Почему не цены

Акция Сбербанка в сентябре 2026 г. стоит около 275 руб., акция ВТБ – несколько десятков рублей. Значит ли это, что Сбербанк «дороже»? Нет: цена одной акции зависит от того, на сколько акций разделён капитал компании, и сама по себе ничего не говорит ни о стоимости бизнеса, ни о привлекательности вложения. Инвестора интересует другое – на сколько процентов вырастут или упадут его деньги.

Поэтому почти весь количественный анализ в финансах работает не с ценами, а с **доходностями** – относительными изменениями стоимости вложения. У доходностей есть и статистическое преимущество. Цены со временем растут и «уходят» в сторону, поэтому их среднее и разброс зависят от выбранного периода. Доходности колеблются вокруг сравнительно устойчивого уровня, и их свойства можно изучать и сравнивать между периодами.

4.1.2 Простая доходность

Пусть P_t – цена актива на закрытие дня t. **Простая доходность** за день – это относительное изменение цены:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{P_t}{P_{t-1}} - 1. \tag{4.1}$$

Если акция стоила 100 руб. и подорожала до 103 руб., простая доходность равна 3%. Именно её инвестор видит в отчёте брокера. Ей удобно пользоваться для портфелей: доходность портфеля равна взвешенной сумме доходностей входящих в него активов, где веса – доли вложенных денег.

Но у простой доходности есть неудобное свойство: она несимметрична. Падение на 50% не компенсируется ростом на 50% – после такой пары движений от 100 руб. останется 75. Чтобы вернуться к исходному уровню после падения на 50%, нужен рост на 100%.

Российский рынок дал наглядный пример этой асимметрии. В 2008 г. индекс полной доходности MCFTR упал на 66,6%, а в 2009 г. вырос на 126,1%. Кажется, что рост с лихвой перекрыл падение. На деле за два года инвестор потерял 24,5%: чтобы отыграть падение 2008 г., индексу нужно было вырасти на 199%.

4.1.3 Логарифмическая доходность

Эту асимметрию устраняет **логарифмическая (непрерывно начисляемая) доходность**:

$$r_t = \ln \frac{P_t}{P_{t-1}} = \ln(1 + R_t). \tag{4.2}$$

Её главное свойство – **аддитивность во времени**. Доходность за несколько дней равна сумме дневных лог-доходностей, потому что логарифм произведения равен сумме логарифмов:

$$\ln \frac{P_T}{P_0} = \ln \frac{P_1}{P_0} + \ln \frac{P_2}{P_1} + \ldots + \ln \frac{P_T}{P_{T-1}} = \sum_{t=1}^{T} r_t.$$

Падение с 100 до 50 руб. – это лог-доходность $\ln 0,5 \approx -0,693$, а рост с 50 до 100 руб. – ровно $+0,693$. Сумма равна нулю, как и должно быть. Поэтому модели временных рядов, которые мы будем строить в следующих главах, почти всегда работают с лог-доходностями.

При небольших изменениях цены две доходности почти совпадают: $\ln(1 + R) \approx R$, когда R мало. Для дневных доходностей разница обычно видна лишь в четвёртом знаке после запятой. Но для крупных движений она существенна: падение на 33% – это лог-доходность около $-0,40$.

Таблица 4.1. Простая и логарифмическая доходность

Свойство	Простая R_t	Логарифмическая r_t
Доходность за несколько периодов	$\prod(1 + R_t) - 1$	$\sum r_t$ – простая сумма
Доходность портфеля	$\sum w_i R_{i,t}$ – взвешенная сумма	не складывается по активам
Нижняя граница	-100%	$-\infty$
Где применять	портфели, отчётность, интерпретация	модели временных рядов, агрегирование во времени

Из таблицы следует правило, которое пригодится во всей книге: **складываем во времени – логарифмические, складываем по активам – простые**. Перейти от одной доходности к другой можно по формулам $r = \ln(1 + R)$ и $R = e^r - 1$.

Посчитаем обе доходности для акций Сбербанка. Метод `pct_change()` возвращает простую доходность, а разность логарифмов цен – логарифмическую.

```
sber = data.load_stocks("SBER", start="2024-01-01").set_index("date")
sber["R"] = sber["adj_close"].pct_change()            # простая доходность
sber["r"] = np.log(sber["adj_close"]).diff()          # логарифмическая
sber[["adj_close", "R", "r"]].tail()
#>             adj_close         R         r
#> date
```

```
#> 2026-09-23     280.74 -0.000036 -0.000036
#> 2026-09-24     279.33 -0.005022 -0.005035
#> 2026-09-25     276.35 -0.010668 -0.010726
#> 2026-09-28     272.45 -0.014113 -0.014213
#> 2026-09-29     274.65  0.008075  0.008042
```

Первая строка любого ряда доходностей пустая (`NaN`): для первого дня нет предыдущей цены. В обычные дни столбцы `R` и `r` различаются лишь в четвёртом знаке.

4.1.4 Какую цену брать: `close` или `adj_close`

В коде выше мы взяли не цену закрытия `close`, а столбец `adj_close`. Почему?

Допустим, вы купили акцию за 317 руб. накануне дивидендной отсечки. На следующий день цена упала до 296 руб., но на ваш счёт придёт дивиденд 33 руб. Потеряли вы или заработали? Если смотреть только на цену – потеряли 7%. Если учесть дивиденд – заработали: $296 + 33 = 329$ руб. против вложенных 317.

> С 31 июля 2023 г. сделки с акциями на Мосбирже рассчитываются в режиме T+1, поэтому экс-дивидендная дата совпадает с датой закрытия реестра. До этого, при расчётах T+2, она наступала на торговый день раньше.

Именно это происходит в день, когда акция начинает торговаться без права на дивиденд (**экс-дивидендная дата**): цена падает примерно на размер выплаты. Для акционера это не убыток – деньги просто переходят из цены акции на его счёт. Но доходность, посчитанная по `close`, покажет в этот день провал.

Поэтому для доходностей используют **скорректированную цену** `adj_close`. Все цены до экс-даты умножаются на коэффициент $1 - D/P$, где D – дивиденд, а P – цена закрытия накануне. После такой корректировки доходность в день отсечки отражает полный результат акционера: изменение цены плюс дивиденд. Аналогично цены корректируются на сплиты. Подробно корректировка разобрана в главе о подготовке данных.

> **Сначала предскажите**
>
> 11 июля 2024 г. – экс-дивидендная дата Сбербанка: дивиденд 33,30 руб. на акцию при цене около 317 руб. накануне. Какой будет доходность за этот день по `close` и по `adj_close`? Запишите оценку, потом запустите код.

```
ex = data.load_stocks("SBER", start="2024-07-09", end="2024-07-12").set_index("date")
ex["R_close"] = ex["close"].pct_change()
ex["R_adj"] = ex["adj_close"].pct_change()
ex[["close", "adj_close", "R_close", "R_adj"]].round(4)
#>              close  adj_close  R_close   R_adj
#> date
#> 2024-07-09  319.99   221.4367      NaN     NaN
#> 2024-07-10  316.68   219.1461  -0.0103 -0.0103
```

```
#> 2024-07-11  295.87   228.8050  -0.0657  0.0441
#> 2024-07-12  292.21   225.9746  -0.0124 -0.0124
```

По `close` получается −6,6% – будто акция резко упала. По `adj_close` доходность 4,4%: цена снизилась меньше, чем на размер дивиденда, и с учётом выплаты акционер в тот день заработал.

Скорректированная цена устраняет *технический* эффект выплаты, но не отменяет реальных новостей о дивидендах. Хороший контрпример – Газпром. 30 июня 2022 г. годовое собрание акционеров отказалось от выплаты дивидендов за 2021 г., которые рынок уже заложил в цену. В этот день акции упали на 30,5%, и `adj_close` показывает то же падение: дивиденда не было, корректировать нечего. Инвесторы действительно потеряли эти деньги.

4.2 Дивиденды и полная доходность

Для российского рынка вопрос о дивидендах особенно важен. Многие крупные компании, прежде всего с государственным участием, платят высокие дивиденды, и значительная часть дохода акционера приходит в виде выплат, а не роста цены.

Посмотрим, как это отражается на индексах. Индекс Мосбиржи **IMOEX** – ценовой: он показывает только изменение цен входящих в него акций. Индекс **MCFTR** состоит из тех же акций, но это индекс *полной доходности*: в нём дивиденды реинвестируются, то есть как будто сразу направляются на покупку акций. Для сравнения возьмём индекс полной доходности гособлигаций **RGBITR**.

```
ix = (data.load_index(["IMOEX", "MCFTR", "RGBITR"], start="2010-01-01")
        .pivot(index="date", columns="ticker", values="close"))

years = (ix.index[-1] - ix.index[0]).days / 365.25
cagr = (ix.iloc[-1] / ix.iloc[0]) ** (1 / years) - 1   # среднегодовой темп роста
cagr.round(4)
#> ticker
#> IMOEX     0.0271
#> MCFTR     0.0886
#> RGBITR    0.0724
#> dtype: float64
```

Чтобы сравнивать рост за разные сроки, мы пересчитали его в **среднегодовой темп роста** (CAGR, compound annual growth rate). Это такая постоянная годовая доходность, которая за тот же срок дала бы тот же итоговый результат:

$$\mathrm{CAGR} = \left(\frac{P_T}{P_0}\right)^{1/T} - 1, \tag{4.3}$$

где T – срок в годах. CAGR не означает, что каждый год доходность была такой: реальные годовые доходности сильно колебались вокруг неё.

```
growth100 = 100 * ix / ix.iloc[0]
ax = growth100.plot(logy=True, rot=0)
ax.minorticks_off()                                    # без промежуточных делений
ax.set_yticks([100, 150, 200, 300, 400], labels=["100", "150", "200", "300", "400"])
ax.set_xlabel("")
ax.set_ylabel("руб.")
ax.legend(["IMOEX (цены)", "MCFTR (с дивидендами)", "RGBITR (ОФЗ)"])
plot.source(ax, "Московская биржа")
plt.show()
```

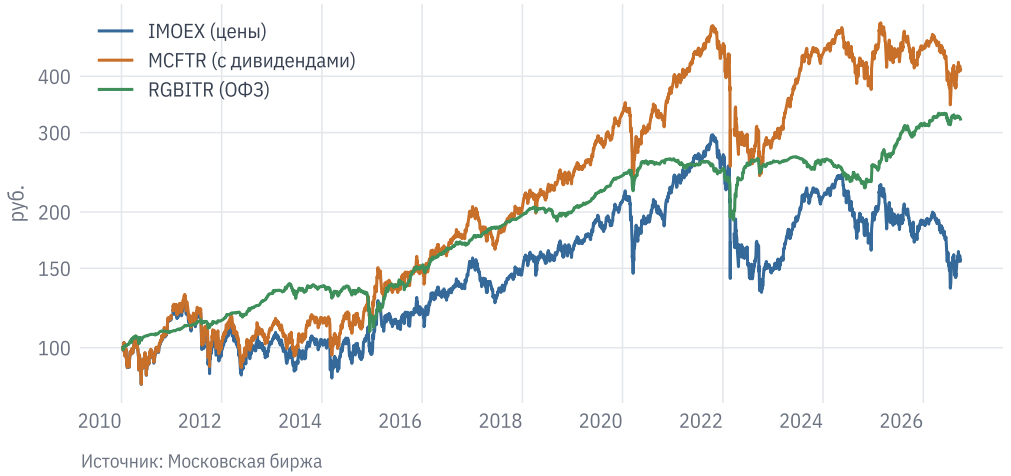


Рисунок 4.1. Рост 100 рублей, вложенных в индексы MOEX в начале 2010 г. (логарифмическая шкала)

Вариант кода на R – в онлайн-версии книги.

Обратите внимание на логарифмическую шкалу по вертикали. На ней одинаковые *относительные* изменения выглядят одинаково: рост со 100 до 200 руб. занимает столько же места, сколько рост с 200 до 400. Для длинных рядов цен это почти всегда правильный выбор – на обычной шкале ранние годы сжимаются в линию у оси.

С 11.01.2010 по 29.09.2026 ценовой индекс IMOEX рос в среднем на 2,7% в год, а индекс полной доходности MCFTR – на 8,9%. Разница в 6,1 процентного пункта в год – это вклад реинвестированных дивидендов. За весь период она накопилась в большой разрыв: 100 руб. в MCFTR превратились в 413 руб., а в IMOEX – лишь в 156 руб. Большую часть дохода российского инвестора в акции за эти годы составили дивиденды. Заметьте и то, что индекс гособлигаций RGBITR рос в среднем на 7,2% в год при гораздо меньших колебаниях.

Типичная ошибка

Сравнивать стратегию, которая получает дивиденды, с ценовым индексом IMOEX. Такое сравнение завышает превосходство стратегии на несколько процентных пунктов в год. Бенчмарком для российских акций служит индекс полной доходности MCFTR или MCFTRR – вариант с учётом налога на дивиденды для российских инвесторов.

4.2.1 Средняя доходность: арифметическая и геометрическая

Когда говорят о «средней доходности», часто имеют в виду разные вещи. Посчитаем годовые доходности MCFTR и сравним два вида среднего.

```
annual = mcftr_all.resample("YE").last().pct_change().loc["2004":"2025"]
arith = annual.mean()                                    # арифметическое среднее
geom = (1 + annual).prod() ** (1 / len(annual)) - 1      # геометрическое = CAGR
pd.Series({"арифметическое": arith, "геометрическое": geom,
           "волатильность": annual.std()}).round(4)
#> арифметическое    0.1993
#> геометрическое    0.1283
#> волатильность     0.4014
#> dtype: float64
```

За 22 полных года с 2004 по 2025 г. арифметическое среднее годовых доходностей составило 19,9%, а геометрическое (фактический среднегодовой рост) – только 12,8%. Разница возникает из-за колебаний: как мы видели на примере 2008–2009 гг., потери и приросты одинаковой величины не компенсируют друг друга. Чем выше волатильность, тем больше разрыв. Приближённо

$$\bar{R}_{\text{геом}} \approx \bar{R}_{\text{арифм}} - \frac{\sigma^2}{2}, \tag{4.4}$$

что даёт 11,9%. Приближение грубое, потому что годовые доходности российского рынка очень велики по модулю, но направление и порядок эффекта оно передаёт верно. Эту разницу называют *потерями от волатильности* (volatility drag).

Какое среднее правильное? Оба, но для разных вопросов. Геометрическое отвечает на вопрос «как в среднем росли деньги инвестора за прошлые годы». Арифметическое – лучшая оценка ожидаемой доходности *одного* следующего года. Путать их опасно: в рекламе инвестиционных продуктов иногда приводят арифметическое среднее, которое выглядит привлекательнее.

4.2.2 Доходность в какой валюте

Доходность всегда измеряется в какой-то валюте, и для российского инвестора это различие бывает решающим. У индекса Мосбиржи есть «двойник» – индекс **RTSI**, который рассчитывается по тем же акциям, но в долларах.

Хорошо это видно на событиях декабря 2014 г. В ночь на 16 декабря Банк России повысил ключевую ставку с 10,5% до 17%, пытаясь остановить обвал рубля. В этот день IMOEX *вырос* на 1,4%, а RTSI *упал* на 12,4%. Цены акций в рублях почти не изменились, но сами рубли резко подешевели, и для инвестора, считающего в долларах, это был один из худших дней десятилетия. Индекс гособлигаций RGBITR в тот же день потерял 8,7%: при резком росте ставок цены облигаций падают. Эту связь мы разберём в главе об облигациях.

4.3 Волатильность

Доходность – только половина картины. Два актива с одинаковой средней доходностью могут вести себя совершенно по-разному: один растёт ровно, другой то взлетает, то падает. Второй рискованнее: у него выше шанс оказаться в плохом положении в тот момент, когда деньги понадобятся. Главная мера такого риска в финансах – **волатильность**, стандартное отклонение доходностей:

$$\sigma = \sqrt{\frac{1}{n-1}\sum_{t=1}^{n}(r_t - \bar{r})^2}. \tag{4.5}$$

Волатильность показывает, насколько сильно доходности обычно отклоняются от своего среднего. Если бы доходности были распределены нормально, примерно в двух случаях из трёх доходность попадала бы в интервал $\bar{r} \pm \sigma$ и в 95% случаев – в интервал $\bar{r} \pm 2\sigma$.

> Число торговых дней на Мосбирже в разные годы немного различается, поэтому для годового пересчёта используют условное значение 252 – так же, как на большинстве мировых рынков.

Волатильность считают по дневным данным, но приводят в годовом выражении, чтобы её можно было сопоставлять с годовой доходностью. Дневную волатильность умножают на $\sqrt{252}$ – примерно столько торговых дней в году:

$$\sigma_{\text{год}} = \sigma_{\text{день}}\sqrt{252}. \tag{4.6}$$

Откуда корень? Если дневные доходности независимы, дисперсия их суммы равна сумме дисперсий. За 252 дня дисперсия вырастает в 252 раза, а стандартное отклонение – в $\sqrt{252}$ раз. Если доходности зависимы – например, за ростом чаще следует рост, – правило даёт лишь приближение. Для месячных данных множитель равен $\sqrt{12}$, для недельных – $\sqrt{52}$.

```
r = np.log(ix).diff().dropna()
vol = r.std() * np.sqrt(252)
vol.round(3)
#> ticker
#> IMOEX     0.237
#> MCFTR     0.237
```

```
#> RGBITR    0.076
#> dtype: float64
```

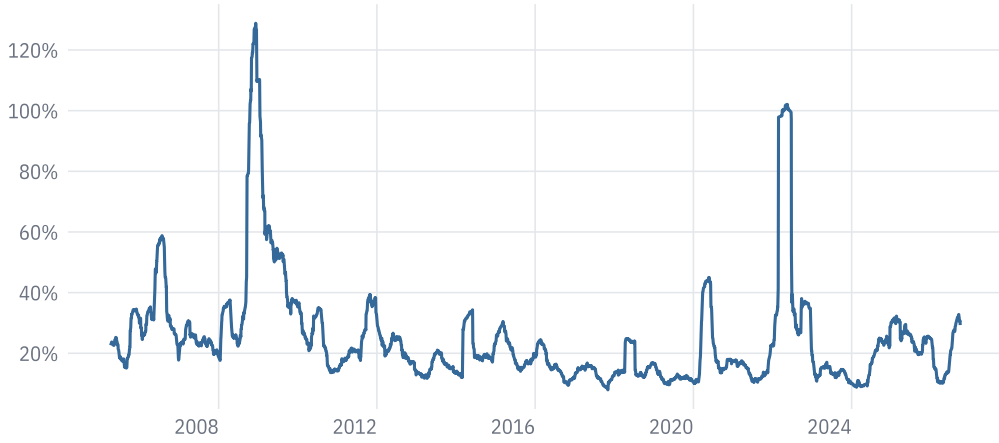
Годовая волатильность индекса акций MCFTR с 2010 г. – 23,7%, индекса гособлигаций RGBITR – 7,6%. На языке инвестора это значит: при волатильности около 24% год, в котором индекс акций отклонится от своей средней доходности на 20–25 процентных пунктов в любую сторону, – обычное дело, а не исключение.

4.3.1 Волатильность меняется во времени

Одно число за полтора десятилетия скрывает важную деталь: риск не постоянен. Посчитаем волатильность в скользящем окне из 63 торговых дней – примерно квартала – и посмотрим, как она менялась.

```
imoex = data.load_index("IMOEX", start="2005-01-01").set_index("date")["close"]
r_imoex = np.log(imoex).diff().dropna()
roll_vol = r_imoex.rolling(63).std() * np.sqrt(252)

ax = roll_vol.plot(color="#34699A", rot=0)
ax.set_xlabel("")
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:.0%}"))
plot.source(ax, "Московская биржа, расчёты автора")
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 4.2. Годовая волатильность IMOEX в скользящем окне 63 дня

Вариант кода на R – в онлайн-версии книги.

На графике хорошо видны **кластеры волатильности**: долгие спокойные периоды сменяются короткими бурными, и большие движения цен следуют за большими, а малые – за малыми. Пики легко связать с событиями: осень 2008

г., декабрь 2014 г., март 2020 г., февраль 2022 г. Вот пять худших дней IMOEX с 2005 г.:

```
worst = (np.exp(r_imoex) - 1).nsmallest(5)
worst.to_frame("доходность").round(3)
#>             доходность
#> date
#> 2022-02-24      -0.333
#> 2008-10-06      -0.187
#> 2008-09-16      -0.175
#> 2008-10-08      -0.144
#> 2008-10-24      -0.142
```

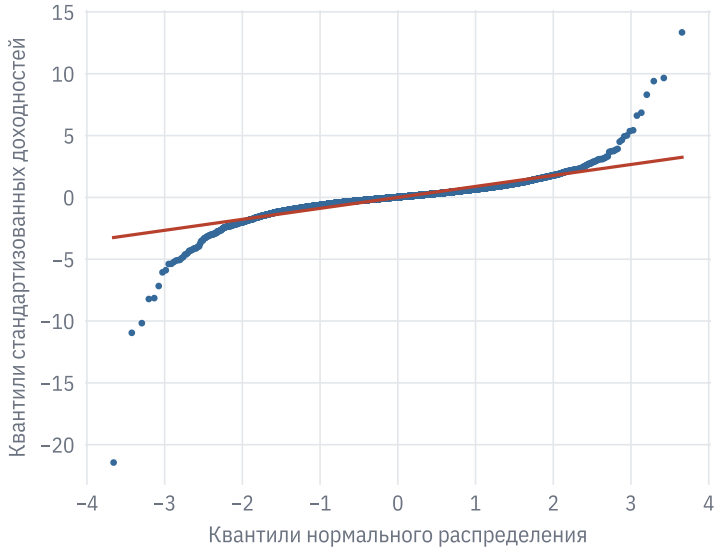
Из пяти худших дней 4 пришлись на осень 2008 г., когда биржа несколько раз приостанавливала торги из-за обвала котировок. Кластеризация волатильности – одно из самых устойчивых эмпирических свойств финансовых рядов: она наблюдается на всех рынках и во все периоды (Cont 2001 г.). Практический вывод: волатильность, посчитанная за спокойный год, плохо описывает риск в кризис. В главе о волатильности мы построим модели GARCH, которые учитывают, что сегодняшний риск зависит от вчерашних движений.

4.4 Распределение доходностей: тяжёлые хвосты

Во многих финансовых моделях – от классической теории портфеля до формулы Блэка – Шоулза – предполагается, что доходности распределены нормально. Это удобно: нормальное распределение полностью описывается средним и дисперсией, и для него хорошо известны вероятности любых отклонений. Проверим, насколько это допущение соответствует данным.

Самый наглядный способ – **QQ-график** (quantile-quantile plot). По горизонтали на нём откладываются квантили нормального распределения, по вертикали – квантили наших доходностей, приведённых к нулевому среднему и единичному стандартному отклонению. Если данные нормальны, точки ложатся на прямую.

```
fig, ax = plt.subplots(figsize=(4.2, 3.2))
stats.probplot(z, dist="norm", plot=ax)
ax.get_lines()[0].set(marker=".", markersize=3, color="#34699A")
ax.get_lines()[1].set(color="#B8412E")
ax.set_title("")
ax.set_xlabel("Квантили нормального распределения")
ax.set_ylabel("Квантили стандартизованных доходностей")
plot.source(ax, "Московская биржа, расчёты автора", offset=32)
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 4.3. Квантили дневных доходностей IMOEX против квантилей нормального распределения

В центре точки действительно лежат на прямой: обычные дни неплохо описываются нормальным распределением. Но на концах они резко уходят от неё – вниз слева и вверх справа. Это значит, что крайние доходности встречаются гораздо чаще, чем предсказывает нормальное распределение. Такие распределения называют распределениями с **тяжёлыми хвостами**.

Насколько чаще? При нормальном распределении движение сильнее 4 стандартных отклонений – редчайшее событие: оно должно случаться в среднем раз в 63 года. Из 5 418 торговых дней IMOEX с 2005 г. таких движений было 39, а нормальное распределение ожидает около 0,3. Худший день – 24.02.2022, когда индекс упал на 33,3%, – это отклонение от среднего на 21 стандартное отклонение. С точки зрения нормального распределения такое событие практически невозможно, а на реальном рынке оно произошло.

Мера тяжести хвостов – **коэффициент эксцесса** (kurtosis). У нормального распределения избыточный эксцесс равен нулю, а у дневных доходностей IMOEX он составляет 58,6. Модели, которые опираются на нормальность, систематически недооценивают вероятность больших потерь. К этому мы вернёмся в главе о VaR и Expected Shortfall.

Особенность данных: пропуск в 2022 г.

После 25 февраля 2022 г. торги акциями на Мосбирже были остановлены и возобновились только 24 марта. Здесь подстерегают две ошибки.

Во-первых, `diff()` посчитает изменение цены между 25 февраля и 24 марта как доходность *одного дня*, хотя на самом деле это доходность за месяц.

Во-вторых, облигации начали торговаться раньше акций, с 21 марта. Если в одной таблице есть и индекс акций, и индекс облигаций, то в дни 21–23 марта у индекса акций будут пропуски. Метод `pct_change()` в pandas 2.x по умолчанию заполняет пропуски предыдущим значением и вернёт в эти дни *нулевую* доходность, будто рынок работал, а цены не менялись. Чтобы этого не случилось, передавайте `pct_change(fill_method=None)` или считайте доходности через `np.log(prices).diff()`.

Перед любыми расчётами проверяйте разрывы в датах: `prices.index.to_series().diff().dt.days.max()`.

4.5 Просадки

Волатильность – удобная мера, но у неё есть недостаток: она одинаково «наказывает» и неожиданный рост, и неожиданное падение. Инвестора же рост не беспокоит. Его волнует другое: сколько он может потерять, если купит в неудачный момент, и сколько времени придётся ждать восстановления. На эти вопросы отвечает **просадка** (drawdown) – падение стоимости от предыдущего максимума:

$$DD_t = \frac{P_t}{\max_{s \le t} P_s} - 1. \tag{4.7}$$

Просадка всегда неположительна: она равна нулю в дни новых максимумов и отрицательна в остальное время. Наихудшее значение за период – **максимальная просадка** (maximum drawdown) – показывает результат самого неудачливого инвестора, купившего на пике.

```
mcftr = ix["MCFTR"]
dd = mcftr / mcftr.cummax() - 1

ax = dd.plot(color="#B8412E", rot=0)
ax.fill_between(dd.index, dd, 0, color="#B8412E", alpha=0.15)
ax.set_xlabel("")
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:.0%}"))
plot.source(ax, "Московская биржа, расчёты автора")
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 4.4. Просадки индекса полной доходности MCFTR

График показывает, что индекс большую часть времени – 94% торговых дней – находился *ниже* своего предыдущего максимума. Для рынка акций это нормальное состояние, а не исключение. Максимальная просадка MCFTR с 2010 г. – 53,5% (26.09.2022). Это тот самый эпизод из начала главы: инвестор, купивший индекс на пике осени 2021 г., потерял больше половины вложений даже с учётом дивидендов и ждал восстановления больше трёх лет. Кризис 2008 г. был ещё глубже: тогда просадка MCFTR достигла 73,6%. На последнюю дату в данных индекс находится на 21,4% ниже своего максимума.

Максимальная просадка – популярная мера в управлении активами, но она определяется одним-единственным эпизодом. Две стратегии с одинаковой максимальной просадкой могут сильно различаться тем, как часто и надолго они уходят в минус. Поэтому её дополняют длительностью просадок и их средней глубиной.

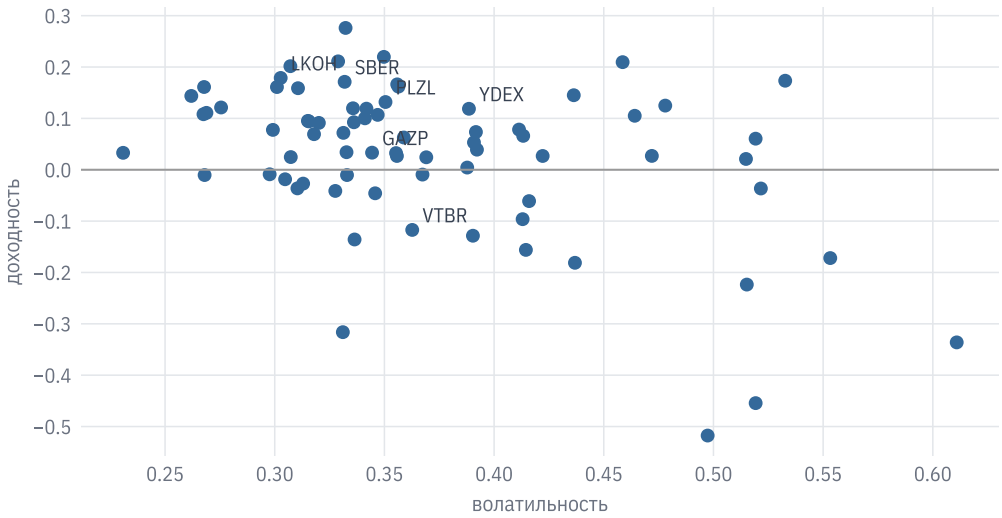
4.6 Риск и доходность отдельных акций

До сих пор мы смотрели на индексы. Теперь сопоставим среднюю доходность и волатильность отдельных акций из набора данных книги за последние 10 лет. Возьмём только бумаги, у которых есть хотя бы пять лет истории, чтобы оценки не были слишком шумными.

```
prices = data.load_prices(start="2016-01-01")
rets = np.log(prices).diff()
enough = rets.count() ⩾ 252 * 5                    # не менее 5 лет истории
summary = pd.DataFrame({
    "доходность": rets.loc[:, enough].mean() * 252,
    "волатильность": rets.loc[:, enough].std() * np.sqrt(252),
```

```
})

ax = summary.plot.scatter(x="волатильность", y="доходность", color="#34699A")
for t in ["SBER", "GAZP", "LKOH", "YDEX", "VTBR", "PLZL"]:
    if t in summary.index:
        ax.annotate(t, summary.loc[t, ["волатильность", "доходность"]],
                    xytext=(4, 3), textcoords="offset points", fontsize=8)
ax.axhline(0, color="#999999", lw=0.8)
plot.source(ax, "Московская биржа, расчёты автора", offset=32)
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 4.5. Годовая доходность и волатильность акций MOEX (лог-доходности, акции с историей не менее 5 лет)

Из учебников по финансам известен принцип: чем выше риск, тем выше должна быть ожидаемая доходность, иначе инвестор не стал бы брать риск на себя. На графике мы видим обратное. Корреляция между волатильностью и средней доходностью по 74 акциям равна −0,45: более волатильные акции в среднем приносили *меньше*. У 23 бумаг средняя лог-доходность за период отрицательна, и среди них много самых волатильных.

Означает ли это, что теория неверна? Не обязательно, и у результата есть три объяснения. Первое – механическое. Мы усредняли логарифмические доходности, а средняя лог-доходность примерно на $\sigma^2/2$ меньше средней простой доходности (вспомните формулу (4.4)). Для акции с волатильностью 50% эта поправка составляет 12,5 процентного пункта в год, так что волатильные бумаги «наказываются» уже самим способом расчёта.

Второе – статистическое. Средняя доходность за 10 лет – очень неточная оценка ожидаемой доходности: при волатильности 30–40% в год её стандартная ошибка составляет порядка 10 процентных пунктов.

Третье – содержательное. Теория говорит, что рынок вознаграждает не любую волатильность, а только ту её часть, которую нельзя убрать диверсификацией. Значительная часть колебаний отдельной акции связана с новостями самой компании и гасится в портфеле. Эту идею формализует модель CAPM, которую мы разберём в главе о портфелях.

> **Ошибка выжившего**
> В наборе данных книги есть и бумаги, которые уже не торгуются на бирже, – 25 компаний. Если бы мы взяли только акции, которые торгуются сегодня, то незаметно исключили бы часть неудачников и завысили бы среднюю доходность рынка. Эта ошибка называется **ошибкой выжившего** (survivorship bias). Подробнее о ней – в главе о подготовке данных.

4.7 Доходность с поправкой на риск: коэффициент Шарпа

Допустим, одна стратегия принесла 15% годовых при волатильности 30%, а другая – 12% при волатильности 10%. Какая лучше? Ответ зависит от того, что можно было получить без риска. Если депозит приносил 11%, первая стратегия дала 4 процентных пункта сверх него, но за очень большой риск, а вторая – 1 пункт за небольшой.

Эту логику формализует **коэффициент Шарпа** (Sharpe 1966 г.) – избыточная доходность сверх безрисковой ставки в расчёте на единицу риска:

$$SR = \frac{\mathbb{E}[R - R_f]}{\sigma(R - R_f)}. \tag{4.8}$$

Числитель показывает, сколько инвестор получает за то, что берёт на себя риск, знаменатель – сколько этого риска. Коэффициент Шарпа не зависит от кредитного плеча: если удвоить вложения в рискованный актив за счёт заёмных денег по безрисковой ставке, и избыточная доходность, и волатильность вырастут вдвое, а их отношение не изменится.

> **RUONIA** (Ruble OverNight Index Average) – средневзвешенная ставка по однодневным рублёвым межбанковским кредитам. Ставку публикует Банк России; её история с 2010 г. есть в данных книги (`ruonia.parquet`).

В России выбор безрисковой ставки особенно важен, потому что она высокая и сильно менялась. В качестве R_f возьмём ставку **RUONIA** – ставку однодневных межбанковских кредитов, которую рассчитывает Банк России. Её доходность

близка к доходности фондов денежного рынка, доступных любому частному инвестору.

```
ruonia = data.load_ruonia(start="2011-01-01").set_index("date")["rate"]

# Дневная безрисковая доходность: ставка RUONIA начисляется за календарные дни
# до следующей даты, в которую есть цена индекса.
px = ix.loc["2011-01-01":, ["MCFTR", "RGBITR"]].dropna()
rf_rate = ruonia.reindex(px.index, method="ffill") / 100
days = px.index.to_series().diff().dt.days
rf = (1 + rf_rate.shift(1) * days / 365) - 1

excess = px.pct_change().sub(rf, axis=0).dropna()
sharpe = excess.mean() / excess.std() * np.sqrt(252)
sharpe.round(2)
#> ticker
#> MCFTR     0.05
#> RGBITR   -0.29
#> dtype: float64
```

Разберём код. Ставка RUONIA публикуется в процентах годовых, поэтому мы делим её на 100 и на 365 и умножаем на число календарных дней между соседними торговыми датами: деньги на денежном рынке приносят доход и в выходные. Сдвиг `shift(1)` означает, что доход за период определяется ставкой, известной в его начале. Затем из дневной доходности индекса вычитается безрисковая доходность, а отношение среднего к стандартному отклонению переводится в годовое выражение умножением на $\sqrt{252}$.

С 2011 г. безрисковая ставка в среднем давала около 9,9% в год. На этом фоне коэффициент Шарпа индекса акций MCFTR составил всего 0,05, а у индекса ОФЗ RGBITR он и вовсе отрицательный: −0,29. Для сравнения: для американского рынка акций за длинные периоды этот коэффициент составляет около 0,3–0,4. Российский рынок акций за эти годы почти не вознаграждал инвестора за риск сверх того, что давал денежный рынок, а длинные ОФЗ в среднем ему проигрывали: ставки несколько раз резко повышались, и цены облигаций падали.

Особенно наглядно это проявилось в 2024 г. Банк России повышал ключевую ставку, пока она не достигла 21% (28.10.2024). Индекс MCFTR за 2024 г. принёс 1,6%, а вложения по ставке RUONIA – 18,9%. Инвестор, который весь год держал деньги в фонде денежного рынка, заработал намного больше владельца акций и не пережил ни одного дня падения стоимости.

Коэффициент Шарпа – самая распространённая мера доходности с поправкой на риск, и мы будем использовать его при сравнении портфелей и стратегий. Но он наследует слабости волатильности: не различает рост и падение и плохо отражает риск редких крупных потерь. Поэтому его дополняют просадками и мерами хвостового риска.

Итоги

- Анализ ведётся не в ценах, а в доходностях. Для агрегирования во времени удобны логарифмические доходности, для портфелей – простые.
- Доходность считают по цене с поправкой на дивиденды и сплиты. Бенчмарк для российских акций – индекс полной доходности MCFTR, а не ценовой IMOEX: разница составляет несколько процентных пунктов в год.
- Арифметическое среднее доходностей выше фактического среднегодового роста; разрыв тем больше, чем выше волатильность.
- Доходность зависит от валюты, в которой она измеряется.
- Волатильность меняется во времени и образует кластеры. Распределение доходностей имеет тяжёлые хвосты: крайние движения случаются намного чаще, чем предсказывает нормальное распределение.
- Просадка показывает риск так, как его ощущает инвестор: глубину и длительность потерь.
- Коэффициент Шарпа сравнивает активы по доходности сверх безрисковой ставки. В России эта ставка высокая, и премия акций за риск в последние 15 лет была невелика.

4.8 Упражнения

Упражнения разделены на четыре типа. Задания **А** проверяют понимание и решаются без компьютера. Задания **Б** выполняются на данных вашей акции (тикер назначает преподаватель), поэтому готового ответа у них нет. Задания **В** учат работать с ИИ-ассистентом: не получать ответ, а проверять его. Задание **Г** – небольшое исследование, результаты которого нужно защитить устно.

4.8.1 А. Понимание

1. Акция за день выросла на 10%, а на следующий упала на 10%. Какова доходность за два дня? Чему равна сумма двух логарифмических доходностей и почему она не равна нулю?
2. Портфель на 60% состоит из акций с дневной доходностью +2% и на 40% – из облигаций с доходностью −1%. Посчитайте доходность портфеля. Можно ли так же взвесить логарифмические доходности?
3. Дневная волатильность акции – 2%. Оцените годовую. При каком свойстве доходностей эта оценка будет занижена?
4. Фонд рекламирует «среднюю доходность 20% годовых за 5 лет». Годовые доходности фонда: +60%, −30%, +40%, −20%, +50%. Посчитайте арифметическое и геометрическое среднее. Какое из них использовал фонд?
5. Почему коэффициент Шарпа одного и того же индекса будет разным для инвестора из США и из России? А для российского инвестора в 2019 и в 2024 г.?

4.8.2 Б. Практика на своей акции

6. Для своего тикера посчитайте CAGR по `close` и по `adj_close` за последние 10 лет (или за всю историю, если она короче). Объясните разницу. Какую долю полной доходности дали дивиденды?
7. Постройте скользящую 63-дневную волатильность своей акции и IMOEX на одном графике. Когда волатильность акции была выше рыночной и с какими событиями в компании или отрасли это связано?
8. Найдите пять худших дней для вашей акции. Для каждого выясните: это реальное падение (какая новость?), дивидендная отсечка, сплит или разрыв в данных?
9. Найдите максимальную просадку вашей акции, дату пика, дату дна и дату восстановления (если оно было). Сколько месяцев длилась просадка?

4.8.3 В. Работа с ИИ-ассистентом

10. Попросите ИИ-ассистента написать функцию годовой волатильности по *месячным* ценам. Проверьте множитель годового пересчёта. Затем попросите посчитать CAGR по дневным лог-доходностям. Какую формулу он предложил? Совпадает ли результат с формулой (4.3)? Запишите, где ассистент ошибся или сделал неявное допущение.
11. Ассистент сгенерировал код ниже для расчёта коэффициента Шарпа акции. Найдите в нём как минимум три ошибки (подсказка: вспомните разделы 4.1, 4.3 и 4.7).

```
px = data.load_stocks("GAZP").set_index("date")["close"]
ret = px.pct_change()
sharpe = (ret.mean() - 0.16) / ret.std() * np.sqrt(365)
```

12. Спросите ассистента, какой была доходность индекса Мосбиржи в 2008, 2014 и 2022 г. Сверьте ответы с данными книги. Какой индекс (IMOEX или MCFTR) и какую валюту подразумевал ассистент? Какие утверждения вы не смогли подтвердить?

4.8.4 Г. Мини-исследование

13. Верно ли, что после дней с очень большими падениями IMOEX (хуже −4%) следующая неделя в среднем тоже неудачная? Сформулируйте гипотезу, проверьте её на данных, оцените, сколько у вас наблюдений и насколько вывод надёжен. Подготовьте короткий (3–5 минут) устный рассказ о результате.

Вопросы для самопроверки

1. Дневная лог-доходность за 5 дней: 0,01; −0,02; 0,005; 0,01; −0,005. Чему равна лог-доходность за неделю?

2. Как перевести дневную волатильность в годовую и на каком допущении основано это правило?
3. Почему для расчёта доходности акции используют `adj_close`, а не `close`? В каком случае падение цены после дивидендных новостей *не* исчезает после корректировки?
4. Какой индекс – IMOEX или MCFTR – корректный бенчмарк для стратегии, получающей дивиденды?
5. Что такое тяжёлые хвосты и как их увидеть на QQ-графике?
6. Почему максимальная просадка не может быть единственной мерой риска стратегии?

Ответы

1. $0,01 - 0,02 + 0,005 + 0,01 - 0,005 = 0$: лог-доходности складываются.
2. Умножить на $\sqrt{252}$; допущение – дневные доходности независимы и одинаково распределены.
3. `close` падает в день дивидендной отсечки и прыгает при сплите; `adj_close` учитывает дивиденды и сплиты. Если падение вызвано *отменой* ожидаемого дивиденда (как у Газпрома 30.06.2022), корректировать нечего – это реальная потеря.
4. MCFTR – индекс полной доходности.
5. Крайние доходности встречаются чаще, чем при нормальном распределении; на QQ-графике точки на концах уходят от прямой.
6. Она определяется одним худшим эпизодом и ничего не говорит о частоте и длительности остальных просадок.

Что почитать

- Campbell и др. (1997 г.), гл. 1 – классическое введение в доходности и их статистические свойства.
- Cont (2001 г.) – «стилизованные факты» о финансовых доходностях: тяжёлые хвосты, кластеры волатильности и др.
- Scheuch и др. (2024 г.) – открытый учебник с кодом на Python; разделы о доходностях и данных.

ЧАСТЬ II

Корпоративная отчётность

5 Большие данные корпоративной отчётности

Выручка ПАО «Магнит» (ИНН 2309085638) по бухгалтерской отчётности за 2023 г. – 0,56 млрд руб. Не триллионы, как можно было бы ожидать от одной из крупнейших розничных сетей страны, а меньше миллиарда. При этом чистая прибыль той же компании – 98,3 млрд руб., в сотню с лишним раз больше выручки. Ошибка в данных? Нет. ПАО «Магнит» – холдинговая компания: магазины сети принадлежат его дочерней компании АО «Тандер» (ИНН 2310031475), выручка которой за тот же год составила 2,31 трлн руб. Прибыль холдинга почти целиком состоит из дивидендов дочерних компаний: 97,6 млрд руб. доходов от участия в других организациях. А в 2025 г., когда «Тандер» получил убыток (−22,5 млрд руб.), доходы от участия у холдинга равны 0, и его чистая прибыль сократилась до 5,4 млрд руб.

Этот пример показывает главное, что нужно знать о бухгалтерской отчётности российских компаний: она описывает **отдельное юридическое лицо**, а не группу компаний. Чтобы проанализировать бизнес, нужно понимать, какое юрлицо его ведёт, как устроены формы отчётности и какие ловушки в них встречаются. Зато в обмен на эту сложность мы получаем уникальный источник: отчётность **всех** российских организаций – 2,0 млн отчётов только за 2025 г. Открытые данные такого охвата о частных, а не только о публичных компаниях есть лишь в немногих странах.

В этой главе

- Узнаём, откуда берётся отчётность российских компаний и что такое набор данных RFSD.
- Разбираемся в формах отчётности: баланс, отчёт о финансовых результатах, отчёт о движении денежных средств.
- Учимся работать с большими таблицами в Polars: ленивые вычисления, фильтры, группировки.
- Отбираем компании отрасли по классификатору ОКВЭД.
- Проверяем качество отчётности и считаем финансовые коэффициенты.

Данные: `rfsd_sample.parquet` (выборка отчётности), `rfsd_summary.parquet`, `rfsd_cases.parquet`, `okved.csv`, `rfsd_lines.csv`. **Нужно знать:** основы бухгалтерского учёта (что такое активы, обязательства, выручка, прибыль), работа с pandas.

5.1 Откуда берётся отчётность

Российские организации обязаны ежегодно сдавать бухгалтерскую отчётность в налоговую службу. Исключения – банки и некоторые другие финансовые организации (они отчитываются перед Банком России), а также организации государственного сектора. Отчётность составляется по российским стандартам бухгалтерского учёта (**РСБУ**) на стандартных формах, где у каждой строки есть свой код. С 2019 г. эта отчётность собирается в государственном информационном ресурсе бухгалтерской отчётности (ГИР БО) и открыто публикуется на сайте ФНС.

Скачать с сайта ФНС отчётность одной компании просто, а отчётность трёх миллионов компаний за полтора десятилетия – очень трудоёмко: данные нужно собрать, разобрать, привести к единому виду и очистить от ошибок. Есть коммерческие компании, которые продают такие наборы данных, но они стоят дорого и не всегда прозрачны. Также такие провайдеры (к примеру, СПАРК Интерфакс или ФИРА) не дают возможности работать с данными напрямую в Python или R, а только через их интерфейсы.

Эту работу проделали исследователи Европейского университета в Санкт-Петербурге и опубликовали результат как открытый набор данных **RFSD** (Russian Financial Statements Database) (Bondarkov и др. 2025 г.). Набор распространяется по лицензии CC BY 4.0 и доступен на платформе Hugging Face.

```
s = summary[summary["year"] ⩾ 2012]
fig, ax = plt.subplots()
ax.bar(s["year"], s["filed"] / 1e6, color="#34699A", label="поданные отчёты")
ax.plot(s["year"], s["firms"] / 1e6, color="#C8702A", marker="o", label="все организации в наборе")
ax.set_ylabel("млн")
ax.legend(loc="upper right")
plot.source(ax, "RFSD")
plt.show()
```

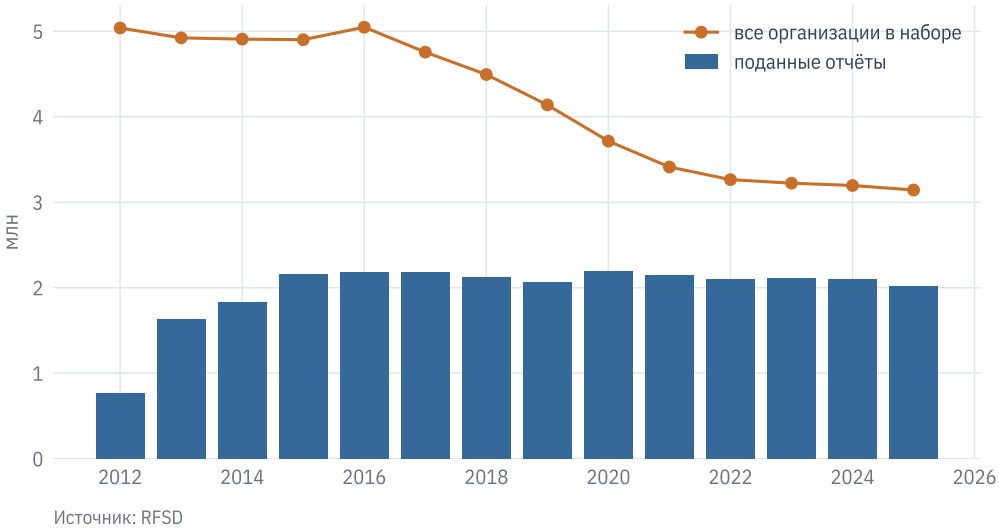


Рисунок 5.1. Число отчётов в RFSD по годам

В 2025 г. в наборе 3,1 млн организаций, из них отчётность подали 2,0 млн, а положительную выручку показали только 1,5 млн. Остальные – компании, освобождённые от сдачи отчётности, «спящие» юрлица без деятельности, фирмы в процессе ликвидации. Для анализа почти всегда нужны только поданные отчёты – в данных они отмечены флагом `filed = 1`.

Полный набор RFSD занимает около 8 ГБ: один год – это таблица примерно из 3 млн строк и более чем 200 столбцов. Для примеров этой главы мы используем выборку: компании пяти отраслей, выручка которых хотя бы в одном году 2018–2025 гг. превышала 1 млрд руб. Обратите внимание на «хотя бы в одном году»: так в выборку попадают и компании, которые потом сократились или ушли с рынка, и мы избегаем ошибки выжившего (глава «Подготовка данных»).

```
rfsd = data.load_rfsd()
rfsd.groupby(rfsd["okved"].str[:2])["inn"].nunique()
#> okved
#> 10    2472
#> 41    5350
#> 47    2996
#> 49    1849
#> 62    1377
#> Name: inn, dtype: int64
```

5.2 Формы отчётности

Бухгалтерская отчётность состоит из нескольких форм. Для анализа важны три.

Бухгалтерский баланс – снимок на конец года: что у компании есть (активы) и за счёт чего это финансируется (пассивы: капитал и обязательства). Активы всегда равны пассивам. **Отчёт о финансовых результатах** показывает, как из выручки через вычет расходов получается чистая прибыль. **Отчёт о движении денежных средств** показывает реальные поступления и платежи денег, разделённые на три вида деятельности: текущую (операционную), инвестиционную и финансовую.

В RFSD каждая строка формы – отдельный столбец с кодом строки: `line_1600` – итог баланса, `line_2110` – выручка. Основные строки:

Таблица 5.1. Основные строки бухгалтерской отчётности

Код	Строка	Форма
1100 / 1200	Внеоборотные / оборотные активы	баланс
1250	Денежные средства и эквиваленты	баланс
1600	Итого активы (баланс)	баланс
1300	Капитал и резервы	баланс
1400 / 1500	Долгосрочные / краткосрочные обязательства	баланс
1410 / 1510	Заёмные средства (долгосрочные / краткосрочные)	баланс
2110	Выручка	финансовые результаты
2120	Себестоимость продаж	финансовые результаты
2200	Прибыль от продаж	финансовые результаты
2330	Проценты к уплате	финансовые результаты
2300	Прибыль до налогообложения	финансовые результаты
2400	Чистая прибыль	финансовые результаты
4100	Сальдо денежных потоков от текущих операций	движение денежных средств
4500	Остаток денежных средств на конец периода	движение денежных средств

Полный справочник строк с названиями – в файле `rfsd_lines.csv`:

```
lines = data.load_rfsd_lines()
lines[lines["code"].isin(["line_2110",    "line_2120",    "line_2330",    "line_2400"])][["code",
"name_ru", "sign"]]
#>          code                  name_ru                        sign
#> 40  line_2110                  Выручка              положительная
#> 41  line_2120     Себестоимость продаж  в скобках (отрицательная)
#> 48  line_2330        Проценты к уплате  в скобках (отрицательная)
#> 60  line_2400  Чистая прибыль (убыток)                любой знак
```

Два соглашения нужно запомнить сразу.

Единицы измерения – тысячи рублей. Выручка «Магнита» из введения записана в данных как 558 670 – это 558 670 тыс. руб., или 0,56 млрд руб.

Расходы хранятся со знаком минус. В бумажных формах некоторые строки пишутся в скобках: себестоимость, коммерческие расходы, проценты к уплате. В RFSD такие строки хранятся отрицательными числами. Поэтому валовая прибыль – это `line_2110 + line_2120`, а не разность.

> **Типичная ошибка**
> Использовать формулы, написанные для другой версии данных. В ранних версиях RFSD знаки строк «в скобках» были непоследовательны; после их исправления изменилась, в частности, чистая прибыль у 6–25% отчётов (в зависимости от года). Код, в котором прибыль до вычета процентов считается как `line_2300 + line_2330`, давал правильный ответ на старых данных и неправильный – на новых: теперь проценты отрицательны, и EBIT равен `line_2300 - line_2330`. Всегда проверяйте, какую версию данных вы используете, и знаки ключевых строк.

5.3 Большие таблицы: Polars

С таблицей в сотню тысяч строк pandas справляется легко. С таблицей в 3 млн строк и 200 столбцов – уже с трудом: pandas читает всё в память и выполняет каждую операцию по очереди. Для больших данных удобнее библиотека **Polars**. Она быстрее за счёт параллельных вычислений, но главное её преимущество – **ленивые вычисления**.

В ленивом режиме Polars не выполняет операции сразу, а строит **план запроса**. Когда результат действительно нужен (метод `collect()`), библиотека оптимизирует план: читает из файла только нужные столбцы, применяет фильтры прямо при чтении и не создаёт промежуточных таблиц. Посмотрим на примере: крупнейшие розничные компании в 2025 г.

```
lf = pl.scan_parquet("data/rfsd_sample.parquet")      # ничего не прочитано

top_retail = (
    lf.filter((pl.col("year") == YEAR) & (pl.col("filed") == 1)
              & pl.col("okved").str.starts_with("47"))
      .select("name", "inn", "okved", "line_2110", "line_2400")
      .with_columns((pl.col("line_2110") / 1e6).alias("выручка, млрд"),
                    (pl.col("line_2400") / 1e6).alias("прибыль, млрд"))
      .sort("line_2110", descending=True, nulls_last=True)
      .head(5)
)
top_retail.collect()
#> shape: (5, 7)
```

```
#> ┌──────────────┬────────────┬─────────┬──────────────┬──────────────┬──────────────┬──────────────┐
#> │ name         ┆ inn        ┆ okved   ┆ line_2110    ┆ line_2400    ┆ выручка,     ┆ прибыль,     │
#> │ ---          ┆ ---        ┆ ---     ┆ ---          ┆ ---          ┆ млрд         ┆ млрд         │
#> │ str          ┆ str        ┆ str     ┆ f64          ┆ f64          ┆ ---          ┆ ---          │
#> │              ┆            ┆         ┆              ┆              ┆ f64          ┆ f64          │
#> ╞══════════════╪════════════╪═════════╪══════════════╪══════════════╪══════════════╪══════════════╡
#> │ ООО          ┆ 7825706086 ┆ 47.19   ┆ 3.4408e9     ┆ 2.3904568e7  ┆ 3440.848396  ┆ 23.904568    │
#> │ "АГРОТОРГ"   ┆            ┆         ┆              ┆              ┆              ┆              │
#> │ АО "ТАНДЕР"  ┆ 2310031475 ┆ 47.11   ┆ 3.0500e9     ┆ -2.2477921e7 ┆ 3050.026081  ┆ -22.477921   │
#> │ ООО "РВБ"    ┆ 9714053621 ┆ 47.91.20┆ 1.1871e9     ┆ 5.5322338e7  ┆ 1187.129163  ┆ 55.322338    │
#> │ АО "ТОРГОВЫЙ ┆ 7728029110 ┆ 47.11   ┆ 1.0386e9     ┆ 1.1614455e7  ┆ 1038.647866  ┆ 11.614455    │
#> │ ДОМ "ПЕРЕКРЕ ┆            ┆         ┆              ┆              ┆              ┆              │
#> │ СТОК"        ┆            ┆         ┆              ┆              ┆              ┆              │
#> │ ООО          ┆ 7743931676 ┆ 47.11   ┆ 8.46615845e8 ┆ 2.6097579e7  ┆ 846.615845   ┆ 26.097579    │
#> │ "АЛЬФА-М"    ┆            ┆         ┆              ┆              ┆              ┆              │
#> └──────────────┴────────────┴─────────┴──────────────┴──────────────┴──────────────┴──────────────┘
```

Обратите внимание на `nulls_last=True`: по умолчанию Polars при сортировке по убыванию ставит пропуски в начало, и без этого параметра «крупнейшими» оказались бы компании, не раскрывшие выручку.

До вызова `collect()` в переменной `top_retail` лежит не таблица, а описание запроса. Его можно посмотреть методом `top_retail.explain()`: в плане видно, что фильтр по году и коду выполняется при чтении файла, а из почти полусотни столбцов читаются только нужные для запроса.

Синтаксис Polars построен на **выражениях**: `pl.col("line_2110") / 1e6` – это не вычисление, а описание того, что нужно сделать со столбцом. Выражения передаются в методы `filter` (отбор строк), `select` (отбор и вычисление столбцов), `with_columns` (добавление столбцов), `group_by(...).agg(...)` (группировка). Та же задача в pandas выглядела бы так:

```
df23 = rfsd[(rfsd["year"] == YEAR) & (rfsd["filed"] == 1) & rfsd["okved"].str.startswith("47")]
df23.nlargest(5, "line_2110")[["name", "inn", "okved", "line_2110", "line_2400"]]
#>                                 name         inn  ...     line_2110   line_2400
#> 79898                  ООО "АГРОТОРГ"  7825706086  ...  3.440848e+09  23904568.0
#> 10704                     АО "ТАНДЕР"  2310031475  ...  3.050026e+09 -22477921.0
#> 86316                       ООО "РВБ"  9714053621  ...  1.187129e+09  55322338.0
#> 68492  АО "ТОРГОВЫЙ ДОМ "ПЕРЕКРЕСТОК"  7728029110  ...  1.038648e+09  11614455.0
#> 73520                   ООО "АЛЬФА-М"  7743931676  ...  8.466158e+08  26097579.0
#>
#> [5 rows x 5 columns]
```

На небольшой выборке разницы в скорости не заметно. Но полный RFSD в pandas пришлось бы целиком загрузить в память, а Polars может читать его прямо с сервера, скачивая только нужные столбцы. Так выглядит запрос к полному набору за 2025 г. на Hugging Face (при сборке книги он не выполняется: файл за один год весит около 550 МБ):

```
url = f"hf://datasets/irlspbru/RFSD/RFSD/year={YEAR}/part-0.parquet"

retail = (
    pl.scan_parquet(url)
      .filter((pl.col("filed") == 1) & pl.col("okved").str.starts_with("47"))
      .select("inn", "okved", "region", "line_2110", "line_2400", "line_1600")
      .collect()
)
```

Этим способом вы будете работать с полными данными в проекте по анализу отрасли.

5.4 Классификатор ОКВЭД

Чтобы сравнивать компании, их нужно разделить на отрасли. В России для этого используется **ОКВЭД 2** – общероссийский классификатор видов экономической деятельности. Он иерархический: раздел (буква), класс (две цифры), подкласс, группа, подгруппа, вид.

```
okv = data.load_okved()
okv[okv["code"].isin(["47", "47.1", "47.11", "47.19"])][["code", "section", "name"]]
#>        code section                                                name
#> 1756     47       G  Торговля розничная, кроме торговли автотранспо...
#> 1757   47.1       G  Торговля розничная в неспециализированных мага...
#> 1758  47.11       G  Торговля розничная преимущественно пищевыми пр...
#> 1762  47.19       G  Торговля розничная прочая в неспециализированн...
```

Код `47` – класс «Торговля розничная», `47.11` – розничная торговля преимущественно продуктами в неспециализированных магазинах (так классифицируются продуктовые сети). У каждой компании в RFSD указан **основной** код ОКВЭД. Это важно учитывать: многие компании занимаются несколькими видами деятельности, а основной код иногда меняется.

В нашей выборке у 5,1% компаний основной код ОКВЭД менялся за 2018–2025 гг. Поэтому, отбирая компании отрасли, нужно решить, по какому году брать код, и описать это правило в работе.

> **Сначала предскажите**
>
> В каком из пяти классов нашей выборки – производство пищевых продуктов (10), строительство зданий (41), розничная торговля (47), сухопутный транспорт (49), разработка программного обеспечения (62) – медианная рентабельность продаж выше всего? А где ниже всего? Запишите ответ, прежде чем читать дальше.

5.5 Качество данных

Отчётность составляют сами компании, и ошибок в ней немало. Авторы RFSD проверили данные и пометили проблемные наблюдения флагами:

- `filed` – отчёт действительно подан (а не восстановлен);
- `imputed` – значения восстановлены по отчёту следующего года;
- `outlier` – подозрительный отчёт, помеченный авторами набора после ручной проверки;
- `simplified` – упрощённая форма отчётности малого предприятия: в ней меньше строк.

5.5.1 Сходится ли баланс

Самая простая проверка – равенство активов и пассивов: `line_1600 = line_1700`. Нарушить его в правильно составленной отчётности невозможно.

В полном наборе за 2025 г. баланс не сходится лишь в 0,47% отчётов – авторы RFSD исправили большинство ошибок. Но другая особенность встречается очень часто: у 17% поданных отчётов **капитал неположительный** – обязательства компании больше её активов. Для крупного бизнеса это редкость, а среди мелких компаний такие «фирмы с отрицательным капиталом» – обычное дело. Коэффициенты, в знаменателе которых стоит капитал (ROE, долг/капитал), для таких компаний теряют смысл, и их нужно исключать или рассматривать отдельно.

5.5.2 Скрытая отчётность

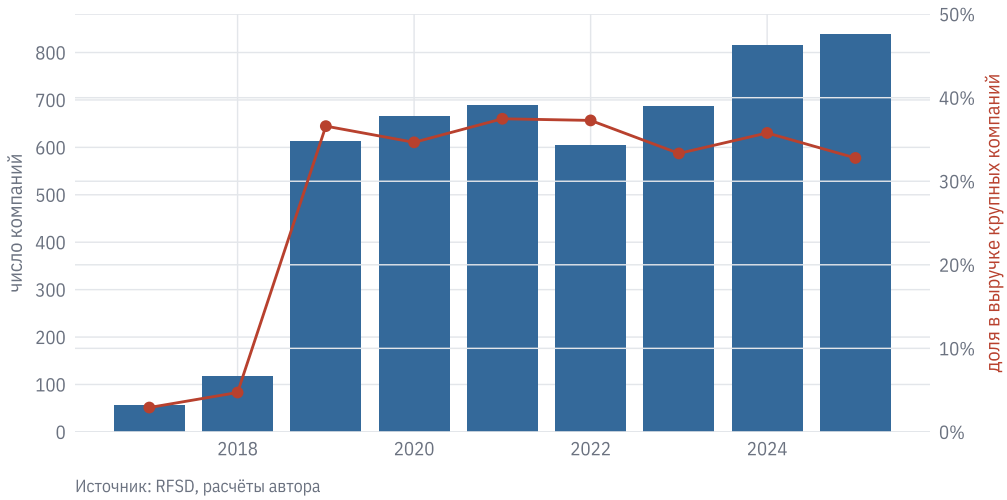
Не все компании публикуют отчётность открыто. Уже в первые годы работы ГИР БО некоторым организациям разрешили ограничивать доступ к своей отчётности – например, работающим в оборонной промышленности или находящимся под иностранными санкциями, – а после 2022 г. круг таких компаний заметно расширился. Отчётность они по-прежнему сдают в налоговую службу, но в ГИР БО она не публикуется. В RFSD у таких компаний пустые строки, а флаг `filed` равен нулю.

```
ozon = cases[cases["inn"] == "7704217370"][["inn", "year", "filed", "line_2110", "line_2400"]]
ozon[ozon["year"] ⩾ 2019]
#>            inn  year  filed    line_2110   line_2400
#> 38  7704217370  2019    0.0          NaN         NaN
#> 39  7704217370  2020    0.0          NaN         NaN
#> 40  7704217370  2021    0.0  177823485.0         0.0
#> 41  7704217370  2022    1.0  261369829.0 -48695018.0
#> 42  7704217370  2023    0.0          NaN         NaN
#> 43  7704217370  2024    0.0          NaN         NaN
#> 44  7704217370  2025    0.0          NaN         NaN
```

ООО «Интернет Решения» (ИНН 7704217370) – основная операционная компания Ozon – за 2022 г. отчётность раскрыла, а за следующие годы – уже нет. И это

не единичный случай. Посчитаем по полному RFSD, сколько крупных компаний – с выручкой не меньше 10 млрд руб. в последнем раскрытом отчёте – в каждом году не раскрыли отчётность, хотя продолжали работать.

```
hid = data.load_rfsd_hidden()
hid["доля по выручке"] = hid["hidden_revenue"] / hid["revenue"]
fig, ax = plt.subplots()
ax.bar(hid["year"], hid["hidden"], color="#34699A")
ax.set_ylabel("число компаний")
ax2 = ax.twinx()
ax2.plot(hid["year"], hid["доля по выручке"], color="#B8412E", marker="o")
ax2.set_ylim(0, 0.5)
ax2.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:.0%}"))
ax2.set_ylabel("доля в выручке крупных компаний", color="#B8412E")
plot.source(ax, "RFSD, расчёты автора")
plt.show()
```



Источник: RFSD, расчёты автора

Рисунок 5.2. Крупные компании, не раскрывшие отчётность: выручка не меньше 10 млрд руб. в последнем раскрытом отчёте

В 2025 г. отчётность не раскрыли 839 из 4 077 крупных компаний – 21%. По выручке это около 33% всех крупных компаний: скрываются прежде всего самые большие. Число таких компаний растёт: 118 в 2018 г., 612 в 2019 г. и 839 в 2025 г.

Резкий скачок в 2019 г. объясняется не только решениями о закрытии отчётности, но и сменой источника. До 2018 г. RFSD опирается на данные Росстата, куда отчётность попадала независимо от публичности, а с 2019 г. – на ГИР БО, где закрытая отчётность не публикуется. Поэтому у многих компаний последний раскрытый год в RFSD – 2018-й.

Вот крупнейшие из них в 2025 г.:

```
top_hidden.head(10).assign(last_revenue=lambda x: (x["last_revenue"] / 1e6).round(0))[
    ["name", "inn", "okved", "last_filed_year", "last_revenue"]]
#>                     name         inn     okved  last_filed_year  last_revenue
#> 0    ПАО "НК "РОСНЕФТЬ"  7706107510  06.10.10             2018        6968.0
#> 1                   None  9707001911     64.19             2023        3746.0
#> 2                   None  7707572492  70.10.20             2018        3130.0
#> 3   ПАО "ГАЗПРОМ НЕФТЬ"  5504036333     46.71             2021        2675.0
#> 4             ОАО "РЖД"  7708503727     49.20             2021        1964.0
#> 5  ПАО "СУРГУТНЕФТЕГАЗ"  8602060555  06.10.10             2018        1525.0
#> 6                   None  7842155505  71.12.20             2022        1180.0
#> 7      ПАО "ТРАНСНЕФТЬ"  7706061801  49.50.10             2018         938.0
#> 8    ПАО АНК "БАШНЕФТЬ"  0274051582  06.10.10             2018         755.0
#> 9                   None  6316031581     46.71             2021         730.0
```

В списке – ПАО «НК «Роснефть» (ИНН 7706107510), ПАО «Газпром нефть» (ИНН 5504036333), ОАО «РЖД» (ИНН 7708503727), ПАО «Сургутнефтегаз» (ИНН 8602060555), ПАО «Транснефть» (ИНН 7706061801): компании, от которых зависят целые отрасли. Выручка в столбце `last_revenue` (млрд руб.) – из последнего раскрытого отчёта, иногда многолетней давности, так что это лишь ориентир размера.

Для отраслевого анализа это означает смещение: из данных выпадают прежде всего крупнейшие компании, в том числе под санкциями. В нефтедобыче, трубопроводном транспорте, металлургии анализ по RFSD описывает скорее «остальную» отрасль, чем отрасль целиком. Выводы о «всей отрасли» нужно делать осторожно и всегда проверять, не пропали ли из данных её лидеры. В следующей главе мы увидим, как исчезновение одной крупной компании меняет показатели концентрации.

5.5.3 Какую часть экономики покрывает RFSD

Насколько велик пробел из-за скрытой отчётности? Ответить помогает сравнение с официальной статистикой. Росстат публикует выручку организаций по видам деятельности, и в неё входят все организации, в том числе закрывшие отчётность от публики: статистическую отчётность они сдают по-прежнему. Ряды Росстата мы возьмём из rustata (набор `showdata/indicator_278140` «Выручка организаций», глава «Источники финансовых данных») и сравним с суммой выручки поданных отчётов в полном RFSD.

```
rs = data.load_rosstat_revenue()                    # тыс. руб., как и в RFSD
rf = data.load_rfsd_revenue_okved()
head = rs[rs["okved"] == "101.АГ"].set_index("year")["revenue"]   # «Всего по обследуемым видам»
coverage = (rf.groupby("year")["revenue"].sum() / head).dropna().loc[2018:]

fig, ax = plt.subplots()
ax.plot(coverage.index, coverage, color="#34699A", marker="o")
ax.set_ylim(0, 1.05)
```

```
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:.0%}"))
plot.source(ax, "RFSD; Росстат (rustata.ru); расчёты автора")
plt.show()
```

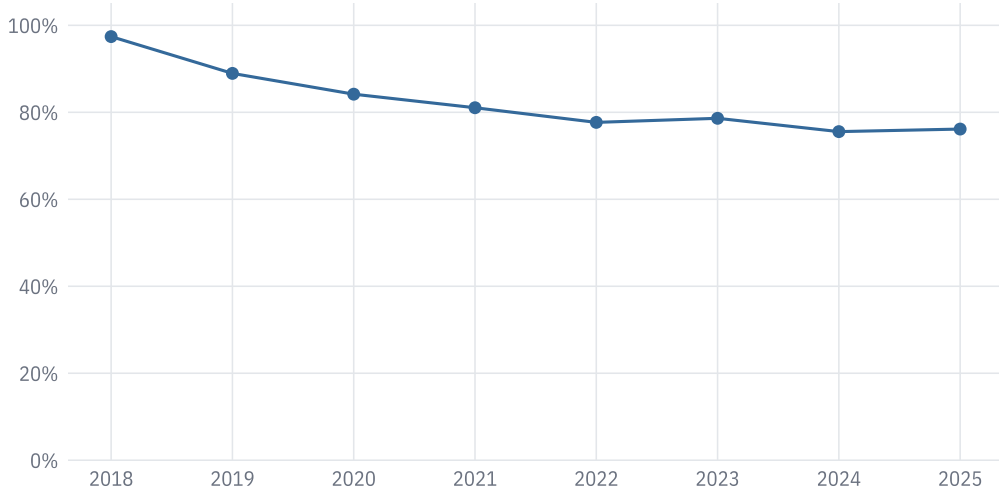


Рисунок 5.3. Выручка в поданных отчётах RFSD в процентах от выручки организаций по данным Росстата

В 2018 г. выручка в отчётах RFSD составляла 97% выручки организаций по данным Росстата, в 2025 г. – 76%. Четверть выручки экономики в открытых данных не видна. Посмотрим, в каких отраслях пробел больше всего (Рисунок 5.4).

```
m = rf.merge(rs, on=["okved", "year"], suffixes=("_rfsd", "_rosstat"))
m["охват"] = m["revenue_rfsd"] / m["revenue_rosstat"]
top20 = m[m["year"] == YEAR].nlargest(20, "revenue_rosstat").sort_values("охват")
labels = top20["okved"] + " " + top20["name"].str.slice(0, 38)

fig, ax = plt.subplots(figsize=(6.5, 5))
ax.barh(labels, top20["охват"], color=np.where(top20["охват"] < 0.5, "#B8412E", "#34699A"))
ax.axvline(1, color="#6B7280", lw=0.8, ls=":")
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"{x:.0%}"))
ax.tick_params(axis="y", labelsize=7)
plot.source(ax, "RFSD; Росстат (rustata.ru); расчёты автора")
plt.show()
```

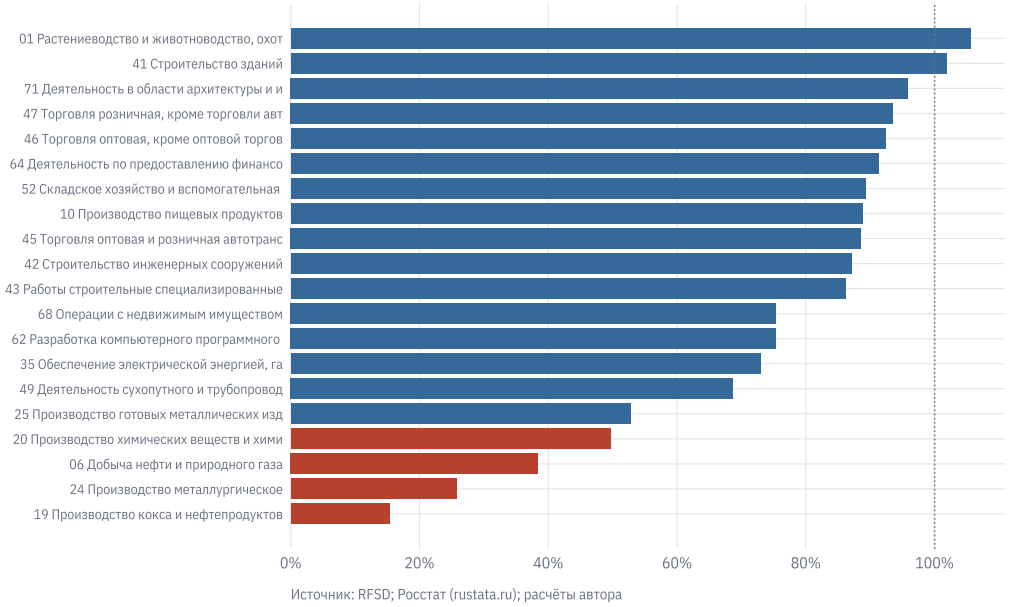


Рисунок 5.4. Охват RFSD по 20 крупнейшим классам ОКВЭД (по выручке по данным Росстата) в 2025 г.

Крупнейшие по выручке классы делятся на две группы. В торговле, строительстве, пищевой промышленности RFSD покрывает почти всю выручку: в розничной торговле 94%, в строительстве зданий 102%. А в производстве нефтепродуктов (класс 19) – 15%, в металлургии (24) – 26%, в добыче нефти и газа (06) – 38%. За пределами двадцатки охват бывает ещё ниже: в производстве прочих транспортных средств, куда входят авиа- и судостроение (класс 30), – 13%, в производстве электроники (26) – 28%. Вероятная причина – в этих отраслях много компаний оборонной промышленности и компаний под санкциями, которые закрыли отчётность. Анализ таких отраслей по RFSD описывает меньшую и нетипичную их часть.

Сравнение приблизительное. Росстат относит организацию к отрасли по данным своего статистического регистра, а в RFSD указан код ОКВЭД из реестра юрлиц, и они не всегда совпадают. Примеры – ПАО «НК «Роснефть» и ПАО «Газпром нефть» (ИНН 5504036333): в RFSD у них коды 06.10.10 (добыча нефти) и 46.71 (оптовая торговля топливом), а в выписках ЕГРЮЛ основным видом деятельности по данным Росстата указано производство нефтепродуктов (19.20.9 и 19.20.1). Поэтому охват по отдельным классам бывает больше 100%: например, для деятельности головных офисов (класс 70) – 158%, по-видимому, потому что многие холдинги записаны в реестре юрлиц под этим кодом, а Росстат относит их к отрасли основного бизнеса. Кроме того, в рядах Росстата есть методологический разрыв между 2017 и 2018 г.: например, выручка по финансовым услугам (класс 64) за 2017 г. в восемь раз больше, чем за 2018 г. Поэтому сравнение начинается с 2018 г.

5.5.4 Отчётность юрлица, а не группы

Пример «Магнита» из начала главы – не исключение. Крупные группы компаний обычно состоят из десятков юрлиц: холдинга, операционных компаний, финансовых и управляющих компаний. Отчётность по РСБУ каждого из них – это часть картины. Отчётность группы в целом (консолидированная, по МСФО) в RFSD не входит. Поэтому, анализируя крупную компанию, сначала выясните, какое юрлицо ведёт основной бизнес.

5.5.5 Где взять названия компаний

В RFSD нет названий компаний – только идентификаторы: ИНН (`inn`) и ОГРН (`ogrn`). Так сделано намеренно: названия меняются, пишутся по-разному и не уникальны, а ИНН однозначно определяет юрлицо на всём периоде. Поэтому связывать данные всегда нужно по ИНН, а название – лишь подпись для читателя.

Столбец `name` в нашей выборке добавлен отдельно: названия найдены для 96% компаний. Если вам понадобятся названия для своих данных, есть несколько способов.

- **Сайт ГИР БО** (bo.nalog.gov.ru). Поиск по ИНН показывает название и отчётность компании. Для нескольких компаний удобно искать вручную. Для сотен компаний можно отправлять запросы из программы, как в примере ниже, но это неофициальный интерфейс сайта: он может измениться, а запросы нужно отправлять не чаще нескольких раз в секунду.
- **Сервис ФНС «Предоставление сведений из ЕГРЮЛ/ЕГРИП»** (egrul.nalog.gov.ru) – выписка из реестра юрлиц по ИНН: полное и краткое название, адрес, дата регистрации и ликвидации. Подходит для проверки отдельных компаний.
- **Коммерческие базы** (СПАРК-Интерфакс, Контур.Фокус) дают названия, связи между компаниями и группы, но платно и обычно через собственный интерфейс.
- **Для публичных компаний** ИНН эмитента указан в раскрытии информации (например, на сайте e-disclosure.ru), и по нему отчётность связывается с тикером на бирже.

```python
import time
import requests

def bo_name(inn: str) -> str | None:
    """Краткое название организации по ИНН из поиска ГИР БО (неофициальный интерфейс)."""
    r = requests.get("https://bo.nalog.gov.ru/advanced-search/organizations/search",
                     params={"query": inn, "page": 0}, timeout=30)
    for org in r.json().get("content", []):
        if org.get("inn", "").replace("<strong>", "").replace("</strong>", "") == inn:
            return org.get("shortName")
```

```
    return None

names = {}
for inn in ["2310031475", "7825706086"]:
    names[inn] = bo_name(inn)
    time.sleep(0.5)                                  # не перегружать сервис
```

У компаний, скрывших отчётность (подраздел «Скрытая отчётность» выше), названия обычно не находятся и в поиске ГИР БО: их там просто нет. Например, ООО «Яндекс» (ИНН 7736207543) раскрыло отчётность за 2024 г., но не за 2025 г., и в поиске ГИР БО компания уже не видна. Для таких компаний название нужно брать из выписки ЕГРЮЛ по ИНН или из раскрытия информации эмитента.

> **Особенность данных: новые формы с 2025 г.**
> С отчётности за 2025 г. в России действуют новые формы. Коды части строк изменились: например, в упрощённой форме дебиторская задолженность отражается по строке 1240 вместо 1230. RFSD сохраняет коды в том виде, в каком они были в формах соответствующего года, поэтому при анализе рядов, включающих 2025 г., нужно сверять коды. В нашей выборке есть 2025 г., и в примерах мы учитываем эту особенность. Кроме того, для последнего года в RFSD нет восстановленных значений (`imputed`): их восстанавливают по отчёту следующего года, которого ещё нет.

5.6 Финансовые коэффициенты

Абсолютные показатели компаний разного размера сравнивать бессмысленно: выручка «Тандера» в тысячи раз больше выручки небольшой сети. Поэтому анализ отчётности строится на **финансовых коэффициентах** – отношениях показателей друг к другу.

Таблица 5.2. Основные финансовые коэффициенты

Коэффициент	Формула в RFSD	Что показывает
Рентабельность продаж (ROS)	`line_2400 / line_2110`	сколько чистой прибыли приносит рубль выручки
Рентабельность активов (ROA)	`line_2400 / line_1600`	сколько прибыли приносит рубль активов
Рентабельность капитала (ROE)	`line_2400 / line_1300`	сколько прибыли приносит рубль капитала собственников
Оборачиваемость активов	`line_2110 / line_1600`	сколько выручки приносит рубль активов
Текущая ликвидность	`line_1200 / line_1500`	хватает ли оборотных активов на краткосрочные обязательства
Долг / капитал	`(line_1410 + line_1510) / line_1300`	долговая нагрузка
Покрытие процентов	`(line_2300 - line_2330) / (-line_2330)`	во сколько раз операционная прибыль превышает проценты

Посчитаем их для всех компаний выборки в 2025 г.:

```
d = data.load_rfsd(years=YEAR)
d = d[d["filed"] == 1].copy()

pos = lambda x: x.where(x > 0)                    # знаменатель должен быть положительным
d["ros"] = d["line_2400"] / pos(d["line_2110"])
d["roa"] = d["line_2400"] / pos(d["line_1600"])
d["roe"] = d["line_2400"] / pos(d["line_1300"])
d["turnover"] = d["line_2110"] / pos(d["line_1600"])
d["current"] = d["line_1200"] / pos(d["line_1500"])
d["ebit"] = d["line_2300"] - d["line_2330"].fillna(0)
d["coverage"] = d["ebit"] / pos(-d["line_2330"])
d["div"] = d["okved"].str[:2]

ratios = ["ros", "roa", "roe", "turnover", "current", "coverage"]
d.groupby("div")[ratios].median().round(3)
#>        ros    roa    roe  turnover  current  coverage
#> div
#> 10   0.017  0.038  0.154     1.950    1.466     2.669
#> 41   0.011  0.011  0.186     1.091    1.081     4.262
#> 47   0.011  0.028  0.183     2.562    1.187     2.089
#> 49   0.008  0.014  0.122     1.454    1.031     1.922
#> 62   0.055  0.084  0.281     1.539    1.532     8.942
```

Медианная рентабельность продаж выше всего в классе 62 (разработка программного обеспечения) – 5,5%, ниже всего в классе 49 (сухопутный транспорт) – 0,8%. Разница во многом объясняется бизнес-моделью: разработчику программ почти не нужны материальные активы и закупки, транспортные компании

несут большие расходы на содержание дорогих основных средств, а строители и розничные сети работают с большими объёмами при небольшой наценке. На уровне рентабельности сказывается и год: в 2025 г. маржа снизилась во всех пяти отраслях (в следующей главе мы посмотрим на её динамику). Зато у розницы высокая оборачиваемость активов: магазин быстро продаёт товар и снова закупает его. Эту связь маржи и оборачиваемости мы разберём в следующей главе с помощью разложения DuPont.

5.6.1 Чего нет в отчётности: амортизация и EBITDA

В аналитике и в новостях о компаниях постоянно встречается показатель **EBITDA** – прибыль до вычета процентов, налогов и амортизации (Earnings Before Interest, Taxes, Depreciation and Amortization). На нём основаны самые популярные мерки долговой нагрузки: отношение чистого долга к EBITDA, ковенанты в кредитных договорах, мультипликатор EV/EBITDA. Естественно захотеть посчитать EBITDA по данным RFSD. Сделать это корректно нельзя.

Причина в устройстве российской отчётности. EBITDA = EBIT + амортизация. EBIT мы посчитали выше, а вот амортизации нет ни в одной из строк RFSD:

- в отчёте о финансовых результатах (форма 2) расходы сгруппированы по функциям – себестоимость, коммерческие, управленческие, – и амортизация «растворена» в них;
- отчёт о движении денежных средств (форма 4) по РСБУ составляется прямым методом: в нём показаны реальные поступления и платежи, а не корректировка прибыли на неденежные расходы, как в косвенном методе МСФО. Поэтому амортизации нет и там;
- амортизация раскрывается в пояснениях к отчётности (строка 5640 «Амортизация» в разделе о затратах на производство), но пояснения в RFSD не входят, а многие компании, сдающие упрощённую отчётность, их не составляют вовсе.

Иногда амортизацию просто пропускают и называют EBITDA то, что на самом деле является EBIT. Для разработчика программ, у которого основные средства составляют в среднем (по медиане) 3% активов, ошибка будет небольшой. Для транспортной компании, у которой на основные средства приходится 39% активов, амортизация – одна из крупнейших статей затрат, и «EBITDA», посчитанная без неё, окажется сильно заниженной. Отношение долга к такой «EBITDA» будет завышено, и компания будет выглядеть более закредитованной, чем при расчёте по её опубликованной отчётности. Хуже того, ошибка разная для разных отраслей, поэтому искажаются и сравнения между ними.

Что делать на практике:

- по данным RFSD использовать показатели, которые считаются честно: EBIT, прибыль от продаж (`line_2200`), чистую прибыль – и называть их своими именами;

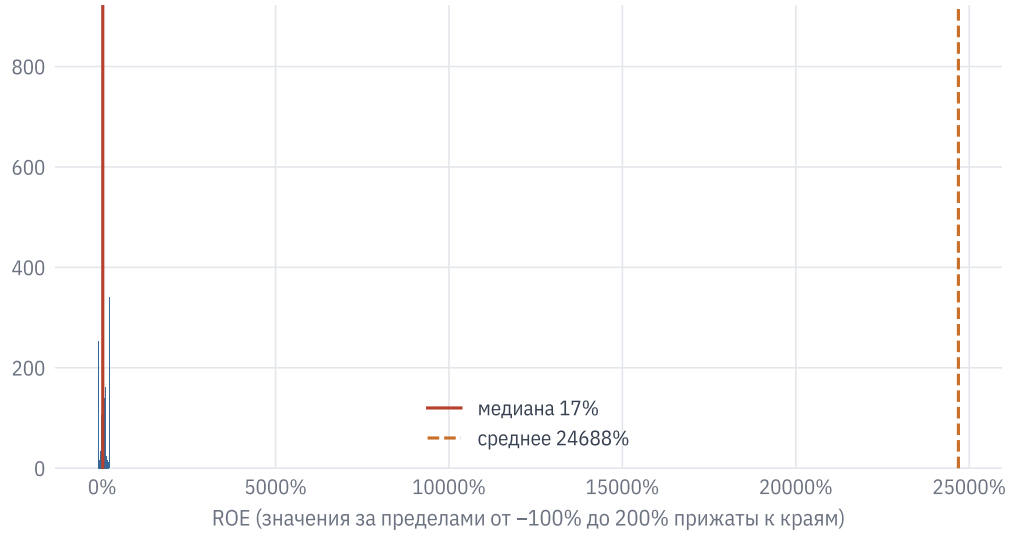
- если нужна долговая нагрузка, считать долг к EBIT и помнить, что пороги, привычные для долга к EBITDA (например, «больше 3–4 – высокая нагрузка»), к нему не применимы: при той же компании отношение к EBIT всегда выше;
- для публичных компаний брать EBITDA из консолидированной отчётности по МСФО или раскрытий эмитента и указывать источник.

> **Типичная ошибка**
> Назвать EBITDA показатель без амортизации. В прошлые годы в учебных ноутбуках этого курса EBITDA по RFSD считалась «упрощённо, как EBIT». Название обещает читателю одно, а число означает другое. Если нужного показателя в данных нет, так и напишите и используйте ближайший корректный показатель с его настоящим названием.

5.6.2 Почему медиана, а не среднее

Распределения финансовых коэффициентов очень неравномерны. Посмотрим на ROE:

```
roe = d["roe"].dropna()
fig, ax = plt.subplots()
ax.hist(roe.clip(-1, 2), bins=80, color="#34699A")
ax.axvline(roe.median(), color="#B8412E", lw=1.2, label=f"медиана {roe.median():.0%}")
ax.axvline(roe.mean(), color="#C8702A", lw=1.2, ls="--", label=f"среднее {roe.mean():.0%}")
ax.set_xlabel("ROE (значения за пределами от -100% до 200% прижаты к краям)")
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"{x:.0%}"))
ax.legend()
plot.source(ax, "RFSD, расчёты автора", offset=32)
plt.show()
```



Источник: RFSD, расчёты автора

Рисунок 5.5. Распределение рентабельности капитала (ROE) компаний выборки в 2025 г.

Один процент компаний с самой низкой ROE имеет рентабельность ниже −434%, один процент с самой высокой – выше 2 589%. Такие значения возникают, когда капитал компании очень мал: небольшая прибыль, делённая на почти нулевой капитал, даёт тысячи процентов. Среднее значение по отрасли оказывается во власти этих крайних случаев, а медиана и квантили – устойчивы. Поэтому для отраслевых сравнений используют медиану и межквартильный размах, а крайние значения либо исключают, либо «прижимают» к заданным квантилям (**винсоризация**).

5.7 Связь форм отчётности

Формы отчётности связаны между собой, и эти связи – ещё одна проверка данных и источник содержательных выводов.

Деньги в двух формах. Остаток денежных средств на конец года в отчёте о движении денежных средств (`line_4500`) должен совпадать с денежными средствами в балансе (`line_1250`).

```
both = d[d["line_4500"].notna() & d["line_1250"].notna()]
match = (both["line_4500"] - both["line_1250"]).abs() ⩽ 5      # допуск на округление
match.mean()
#> 0.9307426789889768
```

Совпадение – в 93% отчётов. Остальные расхождения – это ошибки при заполнении форм или разные подходы к тому, что считать денежными эквивалентами.

Прежде чем строить выводы на денежных потоках, стоит проверить, к каким компаниям относятся расхождения.

Прибыль и деньги. Чистая прибыль – это не деньги. Прибыль отражает доходы и расходы в момент их признания, а денежный поток – в момент оплаты. Компания может показывать прибыль, но не получать денег: например, если покупатели платят с большой отсрочкой или деньги оседают в запасах.

```
flows = d.groupby("div").apply(lambda g: pd.Series({
    "доля с прибылью": (g["line_2400"] > 0).mean(),
    "доля с положительным CFO": (g["line_4100"] > 0).mean(),
}), include_groups=False).round(2)
flows
#>      доля с прибылью  доля с положительным CFO
#> div
#> 10              0.82                      0.64
#> 41              0.78                      0.41
#> 47              0.77                      0.49
#> 49              0.73                      0.56
#> 62              0.81                      0.61
```

Разница особенно велика в строительстве: прибыль показывают 78% компаний, а положительный денежный поток от текущих операций – только 41%. Застройщик годами вкладывает деньги в строительство, а выручку и прибыль признаёт при сдаче объекта. Кроме того, с 2019 г. деньги покупателей жилья поступают не застройщику, а на эскроу-счета в банке и раскрываются только после ввода дома. Сравнение прибыли с денежным потоком – один из главных инструментов проверки «качества прибыли», к нему мы вернёмся при анализе кредитного риска.

Итоги

- Бухгалтерская отчётность (РСБУ) всех российских организаций публикуется ФНС; набор RFSD собирает и очищает её с 2011 г. – около 2 млн отчётов в год.
- Отчётность описывает отдельное юрлицо, а не группу: выручка холдинга может быть в тысячи раз меньше выручки операционной компании.
- В RFSD строки форм – это столбцы с кодами; единицы – тысячи рублей; расходы («строки в скобках») хранятся отрицательными. Формулы зависят от версии данных.
- Для больших таблиц удобен Polars: ленивые вычисления читают только нужные столбцы и строки, в том числе прямо с сервера.
- Компании отрасли отбираются по основному коду ОКВЭД; правило отбора нужно явно описать.
- Качество данных: проверка баланса, флаги RFSD, отрицательный капитал, скрытая отчётность, новые формы с 2025 г.

- Около пятой части крупных компаний, а по выручке – около трети, не раскрывают отчётность; среди них крупнейшие компании страны. Отраслевые выводы по RFSD описывают компании, раскрывшие отчётность.
- В RFSD нет названий компаний: данные связывают по ИНН, названия берут из ГИР БО, ЕГРЮЛ или раскрытия эмитентов.
- Распределения коэффициентов имеют тяжёлые хвосты – для сравнений используют медианы и квантили.
- Формы отчётности связаны: сверка денег в балансе и отчёте о движении денежных средств, сравнение прибыли и денежного потока.

5.8 Упражнения

5.8.1 А. Понимание

1. Почему у холдинговой компании чистая прибыль может быть больше выручки? В какой строке отчёта о финансовых результатах это видно?
2. В RFSD `line_2110 = 500 000`, `line_2120 = -380 000`. Чему равна валовая прибыль в рублях? Что было бы, если бы вы посчитали её как `line_2110 - line_2120`?
3. У компании капитал 1 млн руб., чистая прибыль 3 млн руб. Чему равна ROE? Что эта цифра говорит о компании, а что – нет?
4. Почему для сравнения рентабельности компаний отрасли используют медиану, а не среднее?

5.8.2 Б. Практика на своей отрасли

5. Загрузите из полного RFSD (Hugging Face) отчётность компаний своей отрасли (код ОКВЭД назначает преподаватель) за 2025 г. Сколько компаний подали отчётность? Сколько из них с выручкой больше 100 млн руб.?
6. Для своей отрасли проверьте: у какой доли компаний не сходится баланс, отрицательный капитал, скрытая отчётность (компании, которые отчитывались в 2023 г., но не в 2025 г.)?
7. Посчитайте медианы и квартили коэффициентов из таблицы 5.2 для своей отрасли. Сравните с классами выборки книги.

5.8.3 В. Работа с ИИ-ассистентом

8. Попросите ИИ-ассистента написать функцию расчёта EBIT и коэффициента покрытия процентов по данным RFSD. Проверьте на отчётности «Тандера» из `data.load_rfsd_cases()`: учёл ли ассистент, что проценты к уплате в RFSD отрицательны? Какой результат получился бы без учёта знака?
9. Спросите ассистента, какова выручка «Магнита» за 2023 г. Сравните ответ с данными RFSD по ПАО «Магнит» и АО «Тандер». Какую отчётность – РСБУ или МСФО, какого юрлица – подразумевал ассистент?

5.8.4 Г. Мини-исследование

10. Какая доля компаний вашей отрасли, отчитывавшихся в 2021 г., перестала раскрывать отчётность к 2025 г.? Отличаются ли они (по выручке и рентабельности 2021 г.) от тех, кто продолжил отчитываться? Какие выводы об отрасли по данным 2025 г. из-за этого могут быть смещены? Подготовьте короткий устный рассказ.
11. Сравните охват RFSD со статистикой Росстата для своей отрасли за 2018–2025 гг. (ряды – в rustata, набор `showdata/indicator_278140`, выручка по более детальным кодам ОКВЭД). Как менялся охват? Каких крупных компаний не хватает в RFSD и когда они перестали раскрывать отчётность? Как это влияет на выводы о рентабельности и концентрации отрасли?

Вопросы для самопроверки

1. Чем отчётность по РСБУ отличается от консолидированной отчётности группы по МСФО?
2. В каких единицах записаны денежные показатели в RFSD и с каким знаком хранятся расходы?
3. Что делает метод `collect()` в Polars и почему ленивые вычисления ускоряют работу с большими файлами?
4. Какие проверки качества отчётности вы знаете?
5. Почему прибыль компании может быть положительной, а денежный поток от текущих операций – отрицательным?
6. Почему по данным RFSD нельзя посчитать EBITDA и что использовать вместо неё?

Ответы

1. РСБУ – отчётность отдельного юрлица по российским стандартам; МСФО – консолидированная отчётность группы по международным стандартам, где внутригрупповые обороты исключаются.
2. В тысячах рублей; строки, которые в формах пишутся в скобках (расходы, выбытия), – отрицательными числами.
3. Запускает выполнение построенного плана запроса. Ленивые вычисления позволяют оптимизировать план: читать только нужные столбцы, применять фильтры при чтении, не создавать промежуточных таблиц.
4. Равенство активов и пассивов; суммы разделов; совпадение денег в балансе и отчёте о движении денежных средств; флаги RFSD (`filed`, `imputed`, `outlier`); неположительный капитал.
5. Прибыль признаётся в момент продажи, а деньги поступают при оплате: отсрочка платежа покупателям, рост запасов, специфика отрасли (например, строительство с эскроу-счетами).
6. В RFSD нет амортизации: в форме 2 она входит в себестоимость и другие расходы, отчёт о движении денежных средств по РСБУ составляется прямым

> методом, а пояснения (строка 5640) в базу не входят. Вместо EBITDA используют EBIT или прибыль от продаж под их настоящими названиями, а EBITDA публичных компаний берут из отчётности по МСФО.

Что почитать

- Bondarkov и др. (2025 г.) – описание набора данных RFSD: источники, методы очистки, ограничения.
- Scheuch и др. (2024 г.), раздел о работе с базами данных – организация больших финансовых данных в исследовательском проекте.

6 Отраслевой анализ

В 2025 г. две крупнейшие продуктовые сети страны продали товаров на сопоставимые суммы: выручка ООО «Агроторг» (ИНН 7825706086), которому принадлежат магазины «Пятёрочка», составила 3,44 трлн руб., АО «Тандер» (ИНН 2310031475), которое управляет магазинами «Магнит», – 3,05 трлн руб. «Агроторг» заработал 23,9 млрд руб. чистой прибыли, а «Тандер» получил убыток 22,5 млрд. Рентабельность продаж первого – 0,7%, второго – −0,7%: меньше копейки прибыли или убытка с рубля выручки.

А теперь посмотрим на рентабельность капитала – доход собственников на вложенный ими капитал. У «Тандера» она отрицательна: −16%. А у «Агроторга» – 82%, больше, чем у большинства компаний-разработчиков программ, которые зарабатывают с рубля выручки в разы больше. Как сеть, у которой с рубля выручки остаётся меньше копейки, может приносить собственникам 82% в год? Ответ даёт разложение DuPont, с которого начинается эта глава. Затем мы научимся описывать отрасль целиком: какова «типичная» компания, насколько сосредоточен рынок, чем лидеры отличаются от остальных и как устроены балансы компаний разных отраслей.

В этой главе

- Раскладываем рентабельность капитала на маржу, оборачиваемость и финансовый рычаг (DuPont).
- Строим «гиперболу ROA» – карту бизнес-моделей отрасли.
- Измеряем концентрацию отрасли: доля крупнейших компаний, индекс Херфиндаля – Хиршмана, кривая Лоренца.
- Сравниваем лидеров отрасли с остальными и проверяем гипотезу статистически.
- Следим за динамикой показателей и не путаем изменения компаний с изменением состава выборки.
- Сравниваем структуру балансов разных отраслей.

Данные: `rfsd_sample.parquet` – выборка RFSD из предыдущей главы. **Нужно знать:** формы отчётности и финансовые коэффициенты (глава «Большие данные корпоративной отчётности»).

6.1 Разложение DuPont

6.1.1 Три источника рентабельности

Рентабельность капитала (ROE) – отношение чистой прибыли к капиталу собственников. Умножим и разделим её на выручку и на активы:

$$\mathrm{ROE} = \frac{\text{Прибыль}}{\text{Капитал}} = \underbrace{\frac{\text{Прибыль}}{\text{Выручка}}}_{\text{маржа (ROS)}} \times \underbrace{\frac{\text{Выручка}}{\text{Активы}}}_{\text{оборачиваемость}} \times \underbrace{\frac{\text{Активы}}{\text{Капитал}}}_{\text{мультипликатор}}. \tag{6.1}$$

Это тождество – **разложение DuPont** (по названию химической компании DuPont, где его стали использовать ещё в начале XX в.). Оно показывает, что акционер может зарабатывать тремя способами:

- **маржой** – получать много прибыли с каждого рубля продаж;
- **оборачиваемостью** – быстро «прокручивать» активы, продавая на каждый рубль активов много товаров;
- **финансовым рычагом** – финансировать активы в большей степени за счёт долга, а не собственного капитала.

Произведение первых двух множителей – рентабельность активов: ROA = ROS × оборачиваемость. Третий множитель показывает, во сколько раз рычаг «увеличивает» ROA до ROE. Рычаг работает в обе стороны: при убытках он так же увеличивает потери капитала, поэтому высокий ROE за счёт мультипликатора – это не только эффективность, но и риск.

6.1.2 Две сети

Применим разложение к «Тандеру» и «Агроторгу» за три последних года.

```
cols = ["ros", "turnover", "multiplier", "roa", "roe"]
dupont = pairs.loc[pairs.index.get_level_values("year") ⩾ YEAR - 2, cols]
dupont.round(3)
#>                    ros  turnover  multiplier    roa    roe
#> inn        year
#> 2310031475 2023  0.028     1.742       7.628  0.049  0.372
#>            2024  0.017     2.001       8.352  0.034  0.287
#>            2025 -0.007     1.847      11.882 -0.014 -0.162
#> 7825706086 2023  0.015     2.286      12.834  0.034  0.439
#>            2024  0.014     2.351      14.961  0.034  0.506
#>            2025  0.007     2.729      43.152  0.019  0.818
```

В 2023 г. обе сети были прибыльны, и разница между ними была умеренной: маржа «Тандера» – 2,8%, «Агроторга» – 1,5%; мультипликатор капитала – 7,6 и 12,8. «Агроторг» зарабатывал с рубля выручки меньше, но за счёт большего рычага его ROE (44%) была выше, чем у «Тандера» (37%).

К 2025 г. маржа обеих сетей снизилась, а у «Тандера» стала отрицательной. Рентабельность активов «Агроторга» – всего 1,9%. Но мультипликатор капитала

вырос до 43: активы компании в 43 раза больше её собственного капитала. Капитал сократился с 83,0 млрд руб. на конец 2024 г. до 29,2 млрд на конец 2025 г., хотя за год компания получила прибыль. Так бывает, когда собственникам распределяют больше, чем компания заработала, например выплачивают дивиденды за прошлые годы. Именно рычаг превращает рентабельность активов в 1,9% в рентабельность капитала в 82%. У «Тандера» тот же механизм работает в обратную сторону: при мультипликаторе 11,9 небольшой убыток относительно выручки превращается в ощутимую потерю капитала.

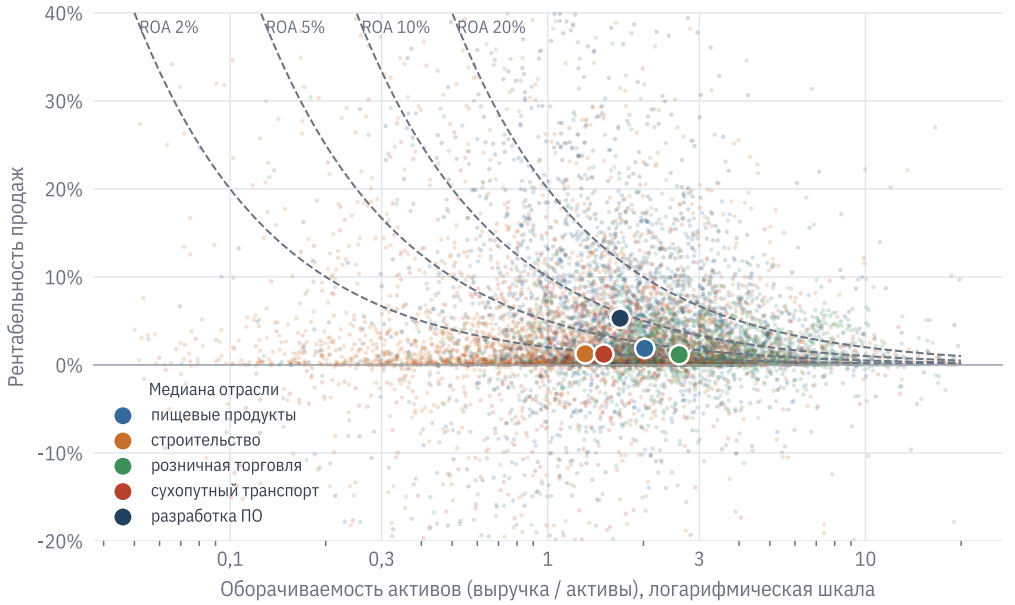
Отсюда важный вывод для анализа: сравнивать компании по одному показателю рентабельности недостаточно. Нужно понимать, из чего он складывается. И помнить, что отчётность юрлица по РСБУ отражает его место внутри группы: сколько капитала оставлено в операционной компании, а сколько выведено в виде дивидендов материнской компании, – это во многом решение собственников, а не результат работы магазинов. Высокая ROE «Агроторга» говорит скорее о финансовой политике группы, чем о выдающейся эффективности магазинов.

6.1.3 Гипербола ROA

Разложение DuPont удобно изобразить на графике. Отложим по горизонтали оборачиваемость активов, по вертикали – маржу. Тогда компании с одинаковой рентабельностью активов лежат на гиперболе ROS = ROA/оборачиваемость.

```
d = r[(r["year"] == YEAR) & r["ros"].between(-0.2, 0.4) & r["turnover"].between(0.05, 20)]
colors = dict(zip(DIV, [plot.BLUE, plot.ORANGE, plot.GREEN, plot.RED, plot.NAVY]))

fig, ax = plt.subplots(figsize=(6.5, 3.8))
for code in DIV:
    g = d[d["div"] == code]
    ax.scatter(g["turnover"], g["ros"], s=3, alpha=0.2, color=colors[code], lw=0)
    ax.scatter(g["turnover"].median(), g["ros"].median(), s=60, color=colors[code],
               edgecolor="white", lw=1, zorder=3, label=DIV[code])
x = np.logspace(np.log10(0.05), np.log10(20), 200)
for roa in (0.02, 0.05, 0.10, 0.20):
    ax.plot(x, roa / x, color="#6B7280", lw=0.8, ls="--")
    ax.annotate(f"ROA {roa:.0%}", (roa / 0.4, 0.4), fontsize=7, color="#6B7280",
                xytext=(2, -8), textcoords="offset points")
ax.set_xscale("log")
ax.set_xticks([0.1, 0.3, 1, 3, 10])
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:g}".replace(".", ",")))
ax.axhline(0, color="#9CA3AF", lw=0.6)
ax.set_ylim(-0.2, 0.4)
ax.set_xlabel("Оборачиваемость активов (выручка / активы), логарифмическая шкала")
ax.set_ylabel("Рентабельность продаж")
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:.0%}"))
ax.legend(title="Медиана отрасли", loc="lower left", fontsize=7, title_fontsize=7)
plot.source(ax, "RFSD, расчёты автора", offset=32)
plt.show()
```



Источник: RFSD, расчёты автора

Рисунок 6.1. Маржа и оборачиваемость активов компаний выборки в 2025 г. Крупные точки – медианы отраслей, пунктир – линии одинаковой ROA

Отрасли занимают на графике разные области. Разработчики программного обеспечения – наверху: медианная маржа 5,5% при умеренной оборачиваемости. Розничная торговля – справа внизу: маржа всего 1,1%, но активы оборачиваются 2,6 раза в год. Одна и та же рентабельность активов может достигаться очень разными бизнес-моделями, и сравнивать маржу розничной сети с маржой ИТ-компании бессмысленно.

```
med.round(3)
#>        ros  turnover  multiplier    roa    roe
#> div
#> 10   0.017     1.950       2.547  0.038  0.154
#> 41   0.011     1.091       7.214  0.011  0.186
#> 47   0.011     2.562       3.812  0.028  0.183
#> 49   0.008     1.454       3.676  0.014  0.122
#> 62   0.055     1.539       2.146  0.084  0.281
```

Вариант кода на R – в онлайн-версии книги.

Таблица показывает ещё одну особенность: в строительстве медианный мультипликатор капитала – 7,2. Застройщики финансируют стройку в основном заёмными средствами, прежде всего проектным кредитованием банков, поэтому даже при очень низкой рентабельности активов (1,1%) их ROE сопоставима с другими отраслями.

6.1.4 Рычаг: ROA падает, ROE – нет

Разложение DuPont подсказывает, что долг «увеличивает» рентабельность капитала. Посмотрим, как это выглядит на всех компаниях выборки: разобьём их на группы по доле обязательств в активах и посчитаем медианы.

```
d = r[r["year"] == YEAR].copy()
d["обязательства / активы"] = (d["line_1400"] + d["line_1500"]) / pos(d["line_1600"])
d = d[d["обязательства / активы"].between(0, 1.5)]
d["группа"] = pd.cut(d["обязательства / активы"], [0, 0.2, 0.4, 0.6, 0.8, 0.9, 1.0, 1.5])
lev = d.groupby("группа", observed=True).agg(
    компаний=("inn", "size"), roa=("roa", "median"), roe=("roe", "median"),
    доля_убыточных=("line_2400", lambda x: (x < 0).mean()))
lev.round(3)
#>             компаний    roa    roe  доля_убыточных
#> группа
#> (0.0, 0.2]       727  0.095  0.108           0.193
#> (0.2, 0.4]       965  0.106  0.157           0.113
#> (0.4, 0.6]      1285  0.080  0.166           0.100
#> (0.6, 0.8]      1857  0.048  0.164           0.113
#> (0.8, 0.9]      1360  0.023  0.160           0.132
#> (0.9, 1.0]      2316  0.007  0.265           0.142
#> (1.0, 1.5]       764 -0.034    NaN           0.733
```

Две медианы ведут себя по-разному. У компаний, у которых обязательства составляют 20–40% активов, медианная ROA – 10,6%, у компаний с долей обязательств 90–100% – 0,7%. Ранговая корреляция доли обязательств и ROA равна −0,47. А медианная ROE не падает вслед за ROA: в группе 20–40% она равна 16%, а в группе 90–100% – 27%.

Как это понимать? Во-первых, связь между долгом и ROA работает в обе стороны. Компания может брать в долг, потому что активно растёт, но чаще высокая доля обязательств – следствие проблем: убытки «съедают» капитал, и компания живёт за счёт кредиторов, в том числе поставщиков и связанных лиц. Прибыльные компании, наоборот, накапливают капитал и меньше нуждаются в займах. Во-вторых, высокая ROE в правой части таблицы во многом механическая: когда капитал мал, даже небольшая прибыль даёт высокое отношение. В-третьих, у 8% компаний капитал отрицательный (последняя группа), ROE для них не определена, и они выпадают из расчёта, хотя 73% из них убыточны. Высокая медианная ROE закредитованных компаний – отчасти результат того, что самые проблемные из них в расчёт не попали.

Вывод для анализа: высокая ROE при большом мультипликаторе капитала – повод посмотреть на баланс внимательнее, а не признак успеха.

> **Почему не EBITDA**
>
> В отраслевых обзорах маржу и долговую нагрузку часто сравнивают по EBITDA. По данным RFSD этого сделать нельзя: в российской отчётности нет строки

> с амортизацией (см. раздел 5.6.1). Для капиталоёмких отраслей – транспорта, энергетики, добычи – разница между EBIT и EBITDA особенно велика, поэтому сравнивать их с малокапиталоёмкими отраслями по EBIT нужно с оговоркой: часть различий в марже объясняется амортизацией, а не эффективностью.

6.2 Концентрация отрасли

Насколько рынок поделен между немногими крупными игроками? Это важно и для инвестора (в концентрированных отраслях лидеры обычно устойчивее и прибыльнее), и для регулятора (высокая концентрация может означать недостаток конкуренции). Три стандартные меры концентрации:

- **доля крупнейших компаний** – например, CR5, доля пяти крупнейших компаний в суммарной выручке;
- **индекс Херфиндаля – Хиршмана (HHI)** – сумма квадратов рыночных долей в процентах: $\text{HHI} = \sum_i s_i^2$. Он равен 10 000 при монополии и близок к нулю на раздробленном рынке. В методике ФАС России рынок с HHI от 1 000 до 2 000 считается умеренно концентрированным, выше 2 000 – высококонцентрированным;
- **коэффициент Джини** и кривая Лоренца – насколько неравномерно выручка распределена между компаниями.

```
trapz = getattr(np, "trapezoid", None) or np.trapz

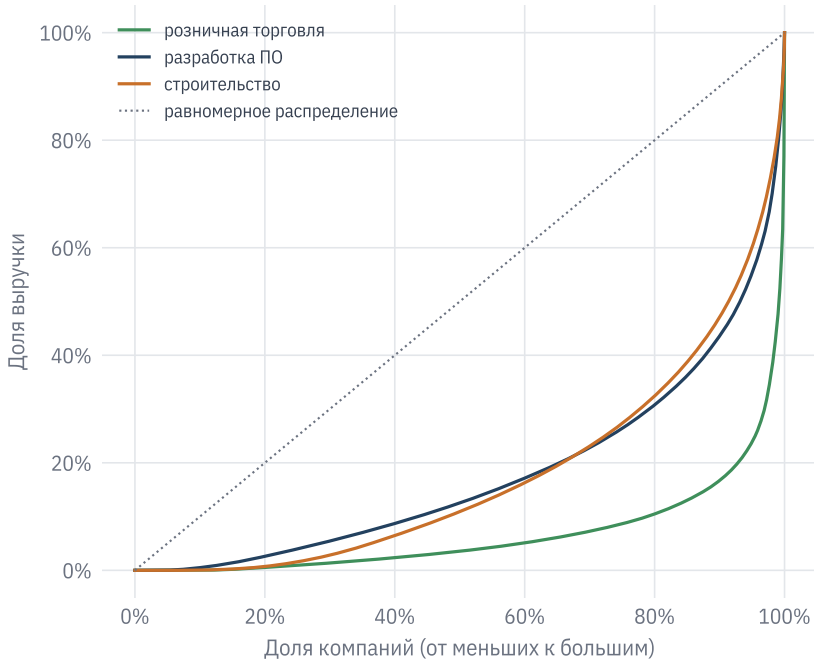
def concentration(g):
    x = np.sort(g["line_2110"].dropna().to_numpy())
    share = x / x.sum()
    lorenz = np.concatenate([[0], np.cumsum(x) / x.sum()])
    gini = 1 - 2 * trapz(lorenz, np.linspace(0, 1, len(lorenz)))
    return pd.Series({"компаний": len(x), "CR5": share[::-1][:5].sum(),
                      "HHI": (share ** 2).sum() * 1e4, "Джини": gini})

conc = r[r["year"] == YEAR].groupby("div").apply(concentration, include_groups=False)
conc.round(3)
#>      компаний    CR5      HHI  Джини
#> div
#> 10     2141.0  0.060   24.158  0.614
#> 41     3687.0  0.060   20.345  0.645
#> 47     2098.0  0.338  338.938  0.866
#> 49     1319.0  0.190  128.548  0.771
#> 62     1102.0  0.129   67.252  0.641
```

Розничная торговля – самая концентрированная из пяти отраслей: на пять крупнейших компаний приходится 34% выручки, HHI – 339. Производство пищевых продуктов и строительство, напротив, раздроблены: CR5 – 6% и 6%.

А вот цифру для разработчиков программ стоит проверить, прежде чем её интерпретировать. В 2024 г. CR5 в классе 62 был 23%, в 2025 г. – 13%. Неужели за год рынок стал почти вдвое менее концентрированным? Нет: крупнейшая компания класса, ООО «Яндекс» (ИНН 7736207543) с выручкой 545 млрд руб. в 2024 г., за 2025 г. отчётность не раскрыла (`filed = 0`), и ещё несколько крупных компаний выпали из данных. Концентрация «снизилась» не из-за конкуренции, а из-за того, что лидеры пропали из выборки.

```
fig, ax = plt.subplots(figsize=(4.8, 4))
for code, color in [("47", "#3F8F5B"), ("62", "#23415F"), ("41", "#C8702A")]:
    x = np.sort(r[(r["year"] == YEAR) & (r["div"] == code)]["line_2110"].dropna().to_numpy())
    lorenz = np.concatenate([[0], np.cumsum(x) / x.sum()])
    ax.plot(np.linspace(0, 1, len(lorenz)), lorenz, color=color, label=DIV[code])
ax.plot([0, 1], [0, 1], color="#6B7280", ls=":", lw=0.8, label="равномерное распределение")
ax.set_xlabel("Доля компаний (от меньших к большим)")
ax.set_ylabel("Доля выручки")
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0%}"))
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0%}"))
ax.legend(fontsize=7)
plot.source(ax, "RFSD, расчёты автора", offset=32)
plt.show()
```



Источник: RFSD, расчёты автора

Рисунок 6.2. Кривые Лоренца для выручки компаний в 2025 г.

Типичная ошибка

Считать концентрацию по неполному списку компаний. Наша выборка содержит только компании с выручкой от 1 млрд руб. хотя бы в одном году, поэтому показатели выше описывают концентрацию **среди крупных компаний**. Если добавить тысячи мелких фирм, доля пяти крупнейших снизится, а коэффициент Джини вырастет – насколько, зависит от отрасли. Кроме того, компании, скрывшие отчётность (глава «Большие данные корпоративной отчётности»), выпадают из расчёта: концентрация получается заниженной, если среди них есть крупные игроки, – как в примере с ИТ-компаниями выше. Прежде чем интерпретировать резкое изменение концентрации, проверьте, не пропали ли из данных лидеры прошлого года. Всегда указывайте, по какому кругу компаний посчитан показатель.

6.3 Лидеры и остальные

Правда ли, что крупнейшие компании отрасли эффективнее остальных? И если да, то за счёт чего – маржи, оборачиваемости или рычага? Это типичный вопрос отраслевого анализа, и отвечать на него нужно так же, как на любой исследовательский вопрос: сформулировать гипотезу, выбрать показатели и проверить гипотезу на данных.

Разделим компании каждой отрасли в 2025 г. на лидеров (10% крупнейших по выручке) и остальных и сравним медианы.

```
d23 = r[r["year"] == YEAR].copy()
d23["лидер"] = d23.groupby("div")["line_2110"].transform(lambda x: x >= x.quantile(0.9))
comp = d23.groupby(["div", "лидер"])[["ros", "turnover", "multiplier", "roa"]].median().round(3)
comp
#>              ros  turnover  multiplier    roa
#> div лидер
#> 10  False  0.017     2.000       2.542  0.036
#>     True   0.028     1.647       2.669  0.058
#> 41  False  0.010     1.055       6.857  0.009
#>     True   0.014     1.364      11.224  0.022
#> 47  False  0.010     2.564       3.691  0.026
#>     True   0.022     2.492       5.256  0.050
#> 49  False  0.008     1.469       3.761  0.014
#>     True   0.012     1.186       2.414  0.017
#> 62  False  0.058     1.498       2.049  0.087
#>     True   0.026     1.726       3.339  0.074
```

В розничной торговле разница заметна: медианная ROA лидеров – 5,0%, остальных – 2,6%. Оборачиваемость активов у лидеров и остальных почти одинакова (2,49 и 2,56), а разница создаётся маржой: 2,2% против 1,0%. Крупные сети, по-видимому, выигрывают на масштабе закупок, логистике и переговорной силе с

поставщиками. Заметьте и ещё одно отличие: мультипликатор капитала у лидеров выше (5,3 против 3,7), то есть они активнее используют заёмные средства.

Не случайна ли эта разница? Распределения рентабельности далеки от нормального, поэтому вместо *t*-теста для сравнения двух групп используем непараметрический **критерий Манна – Уитни**: он проверяет, склонны ли значения одной группы быть больше значений другой, и не чувствителен к выбросам.

```
stats.mannwhitneyu(lead, rest, alternative="two-sided")
#> MannwhitneyuResult(statistic=234456.0, pvalue=7.3151914408634156e-06)
```

p-значение меньше 0,001 при 210 лидерах и 1 941 остальных компаниях: разница статистически значима.

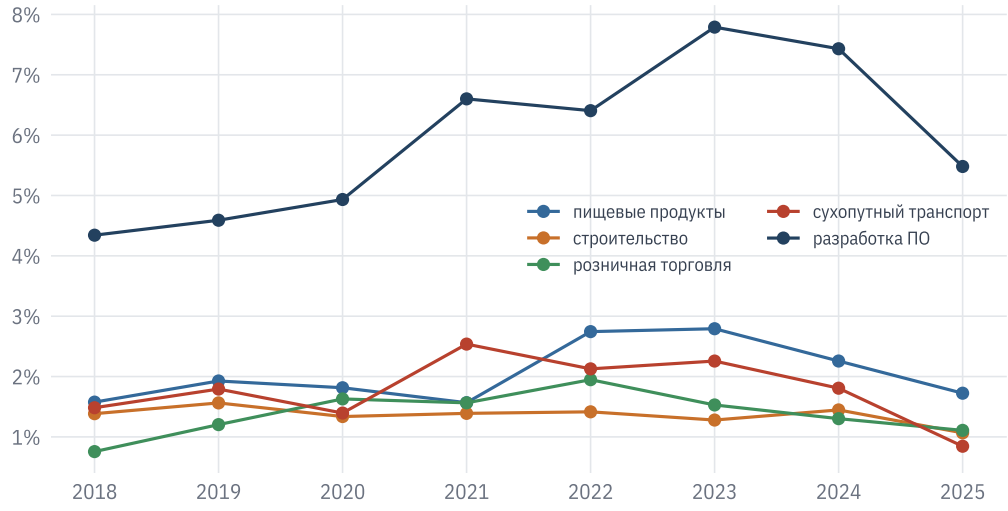
Проверим ту же гипотезу для производства пищевых продуктов. Медианная ROA лидеров – 5,8%, остальных – 3,6%, но p-значение 0,07: на уровне 5% разница незначима. При этом по данным за 2023 г. та же проверка давала p-значение меньше 0,001 (ROA лидеров 11,3% против 5,3%). Вывод «лидеры эффективнее» оказывается неустойчивым: он зависит от отрасли и года. Поэтому хороший отраслевой анализ проверяет вывод на нескольких годах, а не на одном удачном.

Наконец, статистическая значимость не отвечает на вопрос «почему». Лидеры могут быть эффективнее, потому что они крупные (эффект масштаба), а могут быть крупными, потому что эффективнее (успешные компании растут). По данным одного года эти объяснения не различить – для этого нужны данные за много лет и более сложные методы.

6.4 Динамика и состав выборки

Как менялась рентабельность отраслей в 2018–2025 гг.?

```
dyn = r.groupby(["year", "div"])["ros"].median().unstack()
ax = dyn.rename(columns=DIV).plot(marker="o", rot=0)
ax.set_xlabel("")
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:.0%}"))
ax.legend(fontsize=7, ncol=2)
plot.source(ax, "RFSD, расчёты автора")
plt.show()
```



Источник: RFSD, расчёты автора

Рисунок 6.3. Медианная рентабельность продаж по отраслям

Бросаются в глаза две вещи. Первая – рост рентабельности разработчиков программного обеспечения: с 4,3% в 2018 г. до 7,8% в 2023 г. Среди вероятных причин – уход с российского рынка иностранных поставщиков программ после 2022 г. и налоговые льготы для ИТ-компаний. Вторая – снижение рентабельности во всех пяти отраслях в 2025 г. У ИТ-компаний медианная маржа вернулась к 5,5%, а доля убыточных среди крупных компаний выросла: в 2024 г. она была от 14% до 20% в зависимости от отрасли, в 2025 г. – от 17% до 25%. Объяснять эти изменения экономикой – санкциями, спросом, ставками – можно, но только после того, как исключено более простое объяснение: изменение **состава выборки**.

Медиана за каждый год считается по тем компаниям, которые отчитались в этом году. Если в отрасль пришли новые, более рентабельные компании, медиана вырастет, даже если ни одна из прежних компаний не стала эффективнее. Бывает и наоборот: если сильные компании перестали раскрывать отчётность, медиана упадёт. Проверим это на **сбалансированной панели** – только тех ИТ-компаниях, которые отчитывались во все годы периода.

```
it = r[r["div"] == "62"]
years_present = it.groupby("inn")["year"].nunique()
balanced = it[it["inn"].isin(years_present[years_present == it["year"].nunique()].index)]
panel = pd.DataFrame({
    "все компании": it.groupby("year")["ros"].median(),
    "компаний": it.groupby("year")["inn"].nunique(),
    "сбалансированная панель": balanced.groupby("year")["ros"].median(),
    "компаний в панели": balanced.groupby("year")["inn"].nunique(),
})
panel.round(3)
#>       все компании  компаний  сбалансированная панель  компаний в панели
```

```
#> year
#> 2018         0.043       660                  0.045               483
#> 2019         0.046       715                  0.047               483
#> 2020         0.049       795                  0.051               483
#> 2021         0.066       870                  0.063               483
#> 2022         0.064      1094                  0.062               483
#> 2023         0.078      1153                  0.092               483
#> 2024         0.074      1151                  0.080               483
#> 2025         0.055      1124                  0.055               483
```

Состав выборки действительно менялся: в 2018 г. отчитались 660 ИТ-компаний, в 2023 г. – 1 153. Но в сбалансированной панели из 483 компаний медианная рентабельность выросла так же – с 4,5% до 9,2%. Значит, рост не сводится к смене состава: рентабельнее стали сами компании. И снижение в 2025 г. в панели тоже есть – до 5,5%: это изменение внутри компаний, а не результат того, что часть компаний перестала отчитываться. Если бы изменение было видно только по всем компаниям, а в панели – нет, объяснять его пришлось бы сменой состава. Обратите внимание, что сбалансированная панель тоже не свободна от смещения: в неё попадают только компании, которые проработали весь период, то есть «выжившие».

6.5 Структура балансов

Отрасли различаются не только рентабельностью, но и тем, во что вложены их активы. Посмотрим на медианные доли основных статей актива. Здесь пригодится особенность новых форм 2025 г. из предыдущей главы: компании, сдающие упрощённую отчётность, показывают дебиторскую задолженность в строке 1240, а не 1230. Если этого не учесть, у них дебиторка окажется пропущенной. В упрощённой форме строка 1240 включает и финансовые вложения, так что это приближение.

```
d23 = r[r["year"] == YEAR]
# с 2025 г. в упрощённой форме дебиторская задолженность - в строке 1240 (глава 5)
receivables = d23["line_1230"].fillna(d23["line_1240"].where(d23["simplified"] == 1))
profile = pd.DataFrame({
    "осн. средства": d23["line_1150"] / d23["line_1600"],
    "запасы": d23["line_1210"] / d23["line_1600"],
    "дебиторка": receivables / d23["line_1600"],
    "деньги": d23["line_1250"] / d23["line_1600"],
}).groupby(d23["div"]).median().rename(index=DIV)
profile.round(3)
#>                       осн. средства  запасы  дебиторка  деньги
#> div
#> пищевые продукты              0.240   0.208      0.281   0.008
#> строительство                 0.028   0.184      0.473   0.010
#> розничная торговля            0.114   0.356      0.150   0.039
```

```
#> сухопутный транспорт          0.387   0.024      0.331   0.006
#> разработка ПО                 0.029   0.012      0.373   0.068
```

Структура баланса хорошо отражает бизнес-модель. У транспортных компаний основные средства – 39% активов: это подвижной состав, трубопроводы, терминалы. У розничных сетей самая большая статья – запасы (36%), то есть товары на полках и складах. У строителей и разработчиков программ основных средств почти нет, а крупнейшая статья активов – дебиторская задолженность (47% и 37%): неоплаченные заказчиками работы и авансы, выданные подрядчикам и поставщикам.

Такой «профиль отрасли» полезен как ориентир: если структура баланса отдельной компании сильно отличается от типичной для её отрасли, стоит выяснить почему. Возможно, у компании особая бизнес-модель, а возможно, код ОКВЭД не отражает её реальную деятельность или в отчётности есть ошибки.

Итоги

- Разложение DuPont делит рентабельность капитала на маржу, оборачиваемость активов и финансовый рычаг. Одинаковую ROE можно получить очень разными бизнес-моделями, а высокий рычаг означает не только доходность, но и риск. На данных у закредитованных компаний ROA ниже, а высокая ROE во многом механическая.
- EBITDA по данным RFSD не считается: амортизации в отчётности нет. Используйте EBIT и прибыль от продаж под их настоящими названиями.
- «Гипербола ROA» – карта бизнес-моделей: отрасли занимают на ней разные области, и сравнивать маржу компаний разных отраслей нельзя.
- Концентрацию измеряют долей крупнейших компаний (CR5), индексом Херфиндаля – Хиршмана и коэффициентом Джини. Результат зависит от того, по какому кругу компаний он посчитан.
- Сравнение лидеров и остальных – проверка гипотезы: нужны показатели, статистический критерий (для рентабельности – непараметрический) и осторожность в выводах о причинах.
- Изменение медианы во времени может объясняться сменой состава компаний; проверка – сбалансированная панель.
- Структура баланса отражает бизнес-модель отрасли и помогает находить нетипичные компании.

6.6 Упражнения

6.6.1 A. Понимание

1. У компании рентабельность продаж 4%, оборачиваемость активов 2,5, мультипликатор капитала 3. Чему равны ROA и ROE? Как изменится ROE, если

компания возьмёт кредит и мультипликатор вырастет до 5, а процентные расходы снизят маржу до 3%?
2. На рынке пять компаний с долями 40%, 30%, 15%, 10% и 5%. Посчитайте CR3 и HHI. Как изменится HHI, если две крупнейшие компании объединятся?
3. Почему для сравнения рентабельности двух групп компаний лучше использовать критерий Манна – Уитни, а не t-тест?
4. В таблице раздела о рычаге медианная ROE растёт с долей обязательств, а медианная ROA падает. Приведите два объяснения и скажите, какие компании выпадают из расчёта ROE.

6.6.2 Б. Практика на своей отрасли

5. Постройте для своей отрасли (полный RFSD, 2025 г.) гиперболу ROA. Найдите на графике компании с необычной бизнес-моделью и выясните, чем они занимаются.
6. Посчитайте CR5, HHI и коэффициент Джини для своей отрасли за 2018 и 2025 гг. – по всем компаниям и только по компаниям с выручкой от 1 млрд руб. Объясните различия.
7. Проверьте гипотезу «лидеры отрасли эффективнее остальных» для своей отрасли. За счёт какого множителя DuPont возникает разница?

6.6.3 В. Работа с ИИ-ассистентом

8. Попросите ИИ-ассистента объяснить, почему ROE «Агроторга» выше, чем у «Тандера», при меньшей марже. Сравните его объяснение с разложением DuPont на данных. Упомянул ли ассистент, что речь идёт об отчётности отдельных юрлиц внутри групп?
9. Попросите ассистента написать функцию расчёта HHI. Проверьте её на примере из упражнения 2. В каких единицах ассистент выражает доли и какой масштаб HHI получается?
10. Попросите ассистента посчитать EBITDA-маржу вашей отрасли по данным RFSD. Какую строку он использует как амортизацию? Объясните, что на самом деле посчитано, и переделайте расчёт с честным названием показателя.

6.6.4 Г. Мини-исследование

11. Выберите отрасль, пережившую крупное изменение регулирования или рынка (например, переход строительства на эскроу-счета в 2019 г. или уход иностранных компаний в 2022 г.), и проследите по RFSD, как изменились концентрация, рентабельность и структура балансов. Отделите изменения внутри компаний от изменения состава отрасли. Подготовьте короткий устный рассказ.
12. Постройте гистограмму выручки всех компаний (полный RFSD) в диапазоне от 100 до 400 млн руб. за несколько лет. Есть ли скопление компаний чуть ниже порога доходов для упрощённой системы налогообложения

(уточните порог для каждого года)? Как менялось положение скопления при изменении порога и что это говорит о поведении компаний?

Вопросы для самопроверки

1. Запишите разложение DuPont. Какой множитель отвечает за финансовый риск?
2. Что показывает гипербола ROA и почему её удобно строить в логарифмической шкале по оборачиваемости?
3. Как считается индекс Херфиндаля – Хиршмана и в каких пределах он меняется?
4. Почему рост медианной рентабельности отрасли не обязательно означает, что компании стали эффективнее?
5. Какие статьи баланса преобладают у транспортных компаний, розничных сетей и строителей?

Ответы

1. ROE = ROS × оборачиваемость × мультипликатор капитала; финансовый риск отражает мультипликатор (финансовый рычаг).
2. Сочетания маржи и оборачиваемости с одинаковой ROA; логарифмическая шкала растягивает область низкой оборачиваемости, где лежит большинство капиталоёмких компаний.
3. Сумма квадратов рыночных долей в процентах; от почти нуля (раздробленный рынок) до 10 000 (монополия).
4. Из-за изменения состава компаний: могли прийти новые, более рентабельные, или уйти слабые. Проверка – сбалансированная панель.
5. У транспорта – основные средства, у розницы – запасы, у строителей – дебиторская задолженность.

Что почитать

- Bodie и др. (2021 г.), глава об анализе финансовой отчётности – разложение DuPont и финансовые коэффициенты.
- Bondarkov и др. (2025 г.) – описание данных RFSD, в том числе примеры отраслевых распределений.

ЧАСТЬ III

Временные ряды

7 Свойства финансовых рядов и модели ARMA

Если индекс Московской биржи сегодня вырос, стоит ли ждать роста и завтра? Посчитаем корреляцию между дневной доходностью индекса IMOEX и доходностью предыдущего дня за всю историю индекса, с 22.09.1997 по 29.09.2026. Она равна 0,063: небольшая, но положительная и статистически значимая. Кажется, рынок обладает «инерцией», и на ней можно заработать.

Посмотрим, однако, на отдельные годы. В 2022 г. та же корреляция была −0,17, то есть после роста чаще следовало падение. В 2023 г. она стала 0,15, в 2024 г. – 0,16, а в 2025 г. – −0,01. Правило «покупай после роста» в одни годы работало бы, в другие приносило бы убытки, и заранее не было бы известно, какой год наступил. Устойчива ли вообще зависимость доходности от прошлого? Как отличить настоящую закономерность от шума? И как построить прогноз, который не обманет на новых данных?

Эта глава вводит инструменты для ответа на такие вопросы: автокорреляционную функцию, тесты на стационарность и модели ARMA. Модели ARMA – отправная точка анализа временных рядов. Для доходностей акций они обычно показывают, что предсказать почти нечего. Этот отрицательный результат тоже важен: он задаёт планку, которую должна превзойти любая торговая стратегия. Зато для других рядов, например процентных спредов, модели ARMA описывают данные хорошо.

В этой главе

- Измеряем зависимость ряда от своего прошлого: автокорреляционная функция и тест Льюнга – Бокса.
- Разбираем белый шум, случайное блуждание и стационарность; тесты ADF и KPSS.
- Строим модели AR и MA на рядах, где они действительно уместны: термспред ОФЗ и неликвидная акция.

- Выбираем порядок модели ARMA по информационным критериям и проверяем остатки.
- Оцениваем прогноз вне выборки и разбираем типичные ошибки при прогнозировании.

Данные: `indexes.parquet`, `stocks.parquet`, `zcyc.parquet` (см. данные книги). **Нужно знать:** лог-доходности и волатильность (глава «Доходность и риск»), основы статистики: корреляция, проверка гипотез.

В примерах главы используются статистические функции из пакета statsmodels и модули книги.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import acf, pacf, adfuller, kpss
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox
from qf import data, plot
```

7.1 Автокорреляция

7.1.1 Корреляция ряда с собственным прошлым

Пусть r_t – доходность в день t. Автокорреляция порядка k – это корреляция ряда с самим собой, сдвинутым на k периодов:

$$\rho_k = \frac{\mathrm{Cov}(r_t, r_{t-k})}{\mathrm{Var}(r_t)}. \tag{7.1}$$

Число k называют лагом. Набор $\rho_1, \rho_2, \ldots$ как функция от лага – это автокорреляционная функция (ACF, autocorrelation function). По выборке из T наблюдений её оценивают выборочной автокорреляцией $\hat{\rho}_k$: в числителе стоит средняя по выборке сумма произведений отклонений r_t и r_{t-k} от среднего, в знаменателе – выборочная дисперсия.

Даже если истинные автокорреляции равны нулю, выборочные будут отличаться от нуля из-за случайности. Для ряда независимых одинаково распределённых величин $\hat{\rho}_k$ примерно нормальна со средним 0 и стандартным отклонением $1/\sqrt{T}$. Отсюда граница значимости $\pm 1{,}96/\sqrt{T}$: выборочная автокорреляция за её пределами на уровне 5% отличается от нуля. График ACF с этой полосой называют коррелограммой.

Граница $\pm 1{,}96/\sqrt{T}$ быстро сужается с ростом выборки. За 20 лет набирается около 5 тыс. торговых дней, и граница опускается до 0,028. На таких выборках значимой оказывается даже очень слабая зависимость, которая не имеет никако-

го практического значения. К этому различию статистической и экономической значимости мы ещё вернёмся.

7.1.2 Коррелограмма доходностей и их модулей

Построим коррелограммы дневных лог-доходностей IMOEX с 2003 г. – периода, когда рынок акций уже стал достаточно ликвидным. Рядом поставим коррелограмму модулей доходности $|r_t|$: она показывает, связана ли *величина* сегодняшнего движения с величиной вчерашнего, независимо от знака. Функцию для рисования коррелограммы оформим отдельно, она понадобится в главе ещё не раз.

```
def plot_acf_bars(ax, x, nlags=30, kind="acf", title=None):
    """Коррелограмма: столбики ACF (или PACF) и полоса значимости ±1,96/√T."""
    f = acf if kind == "acf" else pacf
    vals = f(x, nlags=nlags)[1:]          # лаг 0 всегда равен 1, его не рисуем
    band = 1.96 / np.sqrt(len(x))         # граница для белого шума
    ax.axhspan(-band, band, color=plot.GREY, alpha=0.25, lw=0)
    ax.bar(np.arange(1, nlags + 1), vals, width=0.6, color=plot.BLUE)
    ax.axhline(0, color=plot.LIGHT, lw=0.6)
    ax.xaxis.set_major_locator(plt.MaxNLocator(integer=True))   # лаги – целые числа
    ax.set_title(title)
    ax.set_xlabel("Лаг")

# Лог-доходности индекса с 2003 г.
imoex = data.load_index("IMOEX", start="2003-01-01").set_index("date")["close"]
r = np.log(imoex).diff().dropna()

fig, axes = plt.subplots(1, 2, figsize=(6.5, 2.8), sharey=True)
plot_acf_bars(axes[0], r, title="Доходность $r_t$")
plot_acf_bars(axes[1], r.abs(), title="Модуль доходности $|r_t|$")
plot.source(axes[0], "Московская биржа, расчёты автора", offset=32)
plt.show()
```

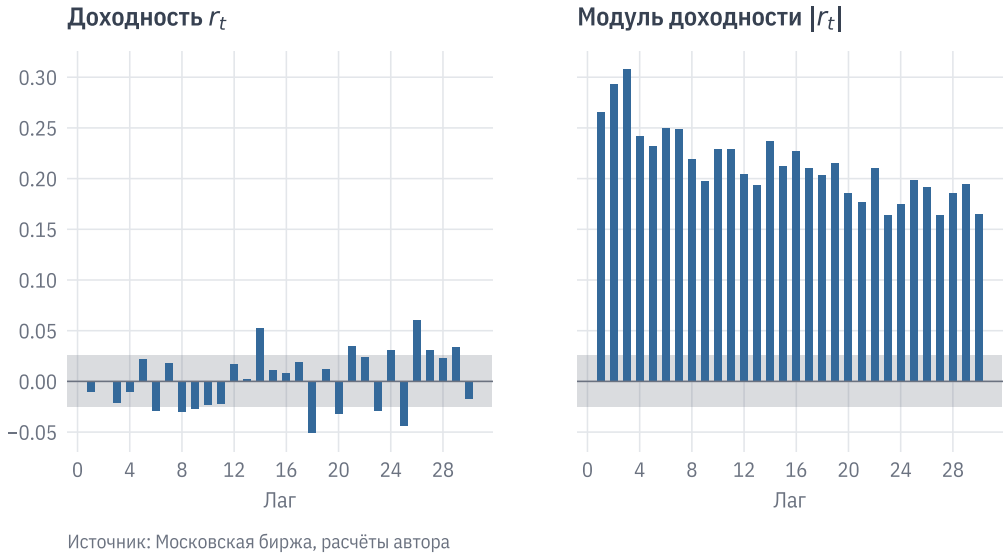


Рисунок 7.1. Автокорреляции дневных лог-доходностей IMOEX и их модулей, 2003–2026 гг. Серая полоса – граница значимости ±1,96/√T

Вариант кода на R – в онлайн-версии книги.

Как читать эти графики? По горизонтали отложен лаг k в торговых днях, по вертикали – коэффициент корреляции между значением ряда в день t и его значением за k дней до этого. Коэффициент корреляции безразмерен и лежит в пределах от −1 до 1, поэтому единицы измерения исходного ряда на графике не видны. На левой панели ряд – сами лог-доходности, на правой – их модули $|r_t|$, то есть размер дневного движения индекса без учёта направления. Модуль измеряется в тех же единицах, что и доходность, – в долях единицы: $|r_t| = 0,02$ означает, что индекс за день изменился примерно на 2% вверх или вниз. В среднем с 2003 г. $|r_t|$ равен 1,2%, а в каждый десятый день превышает 2,5%. Столбик высотой около 0,3 на лаге 1 правой панели означает: если сегодня движение было крупнее обычного, то и завтра оно, скорее всего, будет крупнее обычного, хотя направление предсказать нельзя.

Два графика устроены совсем по-разному. Автокорреляции самих доходностей малы: первая равна −0,011 при границе значимости ±0,025. За границу выходят 13 из 30 лагов, но это следствие большой выборки: наибольшая по модулю автокорреляция равна всего 0,061. Автокорреляции модулей, напротив, велики и убывают очень медленно: 0,27 на первом лаге, 0,23 на пятом и ещё 0,16 на тридцатом, то есть спустя полтора месяца торгов.

Насколько эта картина устойчива? Модули доходности чувствительны к экстремальным дням, а их в выборке немало: 24.02.2022 лог-доходность индекса составила −0,40 (падение на 33%), осенью 2008 г. было несколько дней, когда индекс за день падал или рос на 15–30%. Кроме того, волатильность в кризисные годы в

разы выше, чем в спокойные. Если выборка объединяет такие периоды, модули доходности коррелированы на любых лагах просто потому, что в бурный год они все большие, а в спокойный – все малые. Проверим это:

- без 2008–2009 и 2022 гг. автокорреляция модулей равна 0,21 на первом лаге и 0,10 на тридцатом;
- внутри спокойного периода 2010–2019 гг. – 0,17 и 0,09;
- для рангов модулей, на которые выбросы не влияют, – 0,16 и 0,13.

Автокорреляция модулей на первых лагах остаётся на порядок выше границы значимости при любой проверке: кластеризация волатильности – не артефакт нескольких кризисов. Но «длинный хвост» коррелограммы на лагах 20–30 в заметной мере создают смены режима между годами. Отличать кратковременные всплески волатильности от долгих смен её уровня придётся и в следующей главе, при оценке моделей GARCH.

Это два из «стилизованных фактов» финансовых доходностей (Cont 2001 г.), которые наблюдаются на рынках разных стран и в разные периоды:

1. **Линейная автокорреляция доходностей близка к нулю.** Знак завтрашней доходности почти не зависит от знака сегодняшней.
2. **Модули и квадраты доходностей сильно автокоррелированы.** За днями больших движений следуют дни больших движений, за спокойными днями – спокойные. Это кластеризация волатильности, которую мы видели на графике скользящей волатильности в главе «Доходность и риск».

Отсюда важное различие. Некоррелированность доходностей не означает их независимости. Ряд может быть непредсказуем по знаку и при этом хорошо предсказуем по размаху колебаний. Модели ARMA этой главы описывают только первое – условное среднее ряда. Размах колебаний, то есть условную дисперсию, описывают модели GARCH из следующей главы.

7.1.3 Тест Льюнга – Бокса

Смотреть на коррелограмму лаг за лагом неудобно: при 30 лагах и уровне 5% в среднем полтора столбика выйдут за границу даже у чистого шума. Тест Льюнга – Бокса проверяет сразу гипотезу, что первые m автокорреляций равны нулю:

$$Q(m) = T(T+2)\sum_{k=1}^{m}\frac{\hat{\rho}_k^2}{T-k}. \tag{7.2}$$

Если гипотеза верна, статистика $Q(m)$ имеет распределение χ^2 с m степенями свободы. Большие значения Q и малые p-значения говорят о том, что в ряде есть автокорреляция. Применим тест с $m = 10$ к доходностям IMOEX за несколько периодов.

```
r_full = np.log(data.load_index("IMOEX").set_index("date")["close"]).diff().dropna()
```

```
rows = []
for start in ["1997", "2003", "2010", "2015"]:
    x = r_full.loc[start:]
    lb = acorr_ljungbox(x, lags=[10])                # таблица со статистикой и p-значением
    rows.append({
        "Период": f"{x.index[0].year}–{x.index[-1].year}",
        "Наблюдений": len(x),
        "ACF(1)": acf(x, nlags=1)[1],
        "Граница ±1,96/√T": 1.96 / np.sqrt(len(x)),
        "p-значение Q(10)": lb["lb_pvalue"].iloc[0],
    })
# Числа в русской записи: запятая, типографский минус, пробел между разрядами
ru = lambda v, d=3: f"{v:,.{d}f}".replace(",", " ").replace(".", ",").replace("-", "−")
pval = lambda v: "<0,0001" if v < 1e-4 else ru(v, 4)
pd.DataFrame(rows).style.format(
    {"Наблюдений": lambda v: ru(v, 0), "ACF(1)": ru, "Граница ±1,96/√T": ru,
     "p-значение Q(10)": pval}).hide(axis="index")
```

Таблица 7.1. Первая автокорреляция и тест Льюнга – Бокса (10 лагов) для дневных доходностей IMOEX

Период	Наблюдений	ACF(1)	Граница ±1,96/√T	p-значение Q(10)
1997–2026	7 221	0,063	0,023	<0,0001
2003–2026	5 915	−0,011	0,025	0,0031
2010–2026	4 186	−0,035	0,030	0,0602
2015–2026	2 943	−0,053	0,036	0,0046

За всю историю индекса первая автокорреляция положительна, а тест уверенно отвергает отсутствие автокорреляции. Но почти весь этот эффект создают 1997–2002 гг., когда рынок был узким и цены многих акций индекса отставали от новостей. С 2003 г. первая автокорреляция отрицательна (−0,011), а с 2015 г. равна −0,053. Для периода с 2010 г. p-значение 0,06, и на уровне 5% гипотеза об отсутствии автокорреляции не отвергается.

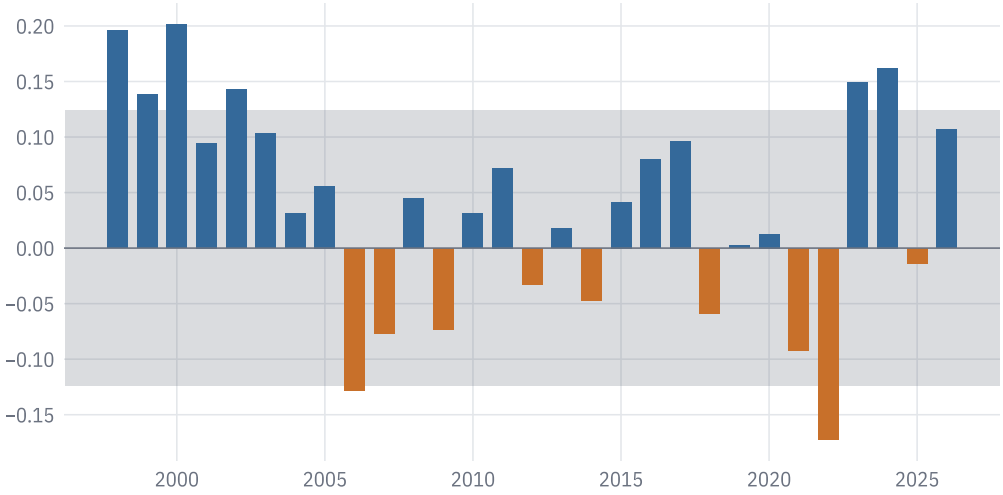
Какой из этого вывод? Автокорреляция доходностей существует, но она мала, меняет знак и зависит от выбранного периода. Малое p-значение на длинной выборке говорит лишь о том, что корреляция, скорее всего, не равна нулю в точности. Оно ничего не говорит о том, достаточно ли она велика, чтобы на ней заработать после комиссий.

7.1.4 Автокорреляция по годам

> **Сначала предскажите**
>
> За всю историю IMOEX первая автокорреляция дневных доходностей положительна. Сколько, по-вашему, лет из трёх десятилетий она была отрицательной? Сколько раз она по модулю превышала 0,1?

```
# Автокорреляция доходности с доходностью предыдущего дня внутри каждого года
by_year = r_full.groupby(r_full.index.year).apply(lambda x: x.autocorr(1)).loc[1998:]

fig, ax = plt.subplots()
band = 1.96 / np.sqrt(250)
ax.axhspan(-band, band, color=plot.GREY, alpha=0.25, lw=0)
ax.bar(by_year.index, by_year, color=np.where(by_year > 0, plot.BLUE, plot.ORANGE), width=0.7)
ax.axhline(0, color=plot.LIGHT, lw=0.6)
plot.source(ax, "Московская биржа, расчёты автора")
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 7.2. Первая автокорреляция дневных доходностей IMOEX по годам. Серая полоса – граница значимости для года из 250 торговых дней; 2026 г. – по конец сентября

Из 29 лет с 1998 г. первая автокорреляция была отрицательной в 9, а по модулю превышала 0,1 в 10. Устойчиво положительной она была только в 1998–2003 гг., на узком рынке тех лет. С 2004 г. отрицательных лет 9 из 23, а самые большие по модулю значения обоих знаков пришлись на последние годы: −0,17 в 2022 г., 0,15 и 0,16 в 2023 и 2024 гг.

Обратите внимание на парадокс. Средняя по годам автокорреляция с 2004 г. положительна (0,009), а посчитанная по всему периоду с 2003 г. – отрицательна (−0,011). Объединённая оценка придаёт больший вес годам с высокой волатильностью, а в бурном 2022 г. автокорреляция была сильно отрицательной. Если вывод о знаке зависимости меняется от способа усреднения, устойчивой закономерности, скорее всего, нет. Это типичная картина для ликвидного рынка: если бы устойчивая автокорреляция существовала, участники быстро бы её использовали и тем самым устранили.

Похожую проверку можно провести на недельных и месячных доходностях. Автокорреляции на более длинных горизонтах иногда оказываются больше, но и наблюдений становится в 5 и в 20 раз меньше, поэтому граница значимости расширяется во столько же корней раз. Классическую проверку гипотезы случайного блуждания через сравнение дисперсий доходностей на разных горизонтах предложили Lo и MacKinlay (1988 г.). Мы вернёмся к этой идее в упражнениях.

7.2 Стационарность

7.2.1 Белый шум и случайное блуждание

Две простейшие модели временного ряда служат точками отсчёта для всего остального.

Белый шум ε_t – последовательность величин с нулевым средним, постоянной дисперсией σ^2 и нулевыми автокорреляциями на всех лагах. Его коррелограмма целиком лежит в пределах полосы значимости. Если к тому же величины независимы и нормальны, говорят о гауссовском белом шуме.

Случайное блуждание – ряд, приращения которого являются белым шумом:

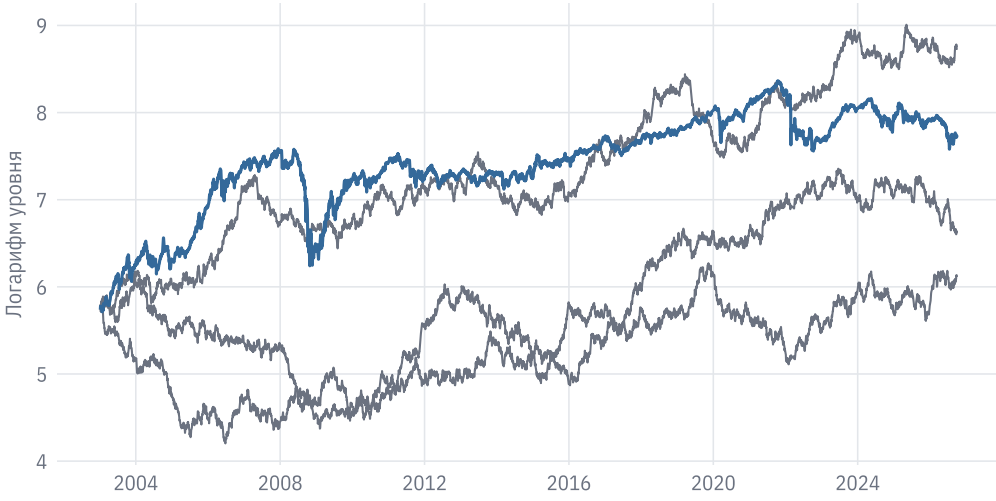
$$p_t = \mu + p_{t-1} + \varepsilon_t. \tag{7.3}$$

Если p_t – логарифм цены, то $p_t - p_{t-1} = r_t$ – лог-доходность, и гипотеза случайного блуждания цены – это гипотеза о том, что доходности являются белым шумом со средним μ. Первый раздел главы показал, что для IMOEX это неплохое приближение, если говорить об автокорреляциях доходностей (но не их модулей).

Случайное блуждание обманчиво: оно выглядит так, будто в нём есть тренды, циклы и уровни поддержки. Сравним логарифм индекса IMOEX с тремя искусственными рядами. Каждый из них – случайное блуждание, шаги которого взяты из нормального распределения с тем же средним и стандартным отклонением, что у дневных доходностей индекса.

```
rng = np.random.default_rng(7)                    # фиксируем зерно для воспроизводимости
log_p = np.log(imoex)

fig, ax = plt.subplots()
for i in range(3):
    # Случайное блуждание: накопленная сумма нормальных шагов, старт с того же уровня
    steps = rng.normal(r.mean(), r.std(), size=len(r))
    walk = log_p.iloc[0] + np.concatenate([[0], steps.cumsum()])
    ax.plot(log_p.index, walk, color=plot.GREY, lw=0.8)
ax.plot(log_p.index, log_p, color=plot.BLUE, lw=1.2)
ax.set_ylabel("Логарифм уровня")
plot.source(ax, "Московская биржа, расчёты автора")
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 7.3. Логарифм IMOEX (синим) и три случайных блуждания с той же средней доходностью и волатильностью

На первый взгляд серые линии мало отличаются от синей. В каждой можно увидеть «бычий рынок», «коррекцию» и «боковик», хотя никакой закономерности в них нет по построению. Это полезная прививка от поиска фигур технического анализа на графике цены.

7.2.2 Что такое стационарность

Ряд называют (слабо) стационарным, если его среднее и дисперсия не меняются во времени, а ковариация между x_t и x_{t-k} зависит только от лага k, но не от момента t. Стационарность нужна, чтобы по одной реализации ряда можно было оценивать его свойства: среднее, ACF, параметры модели. Если свойства ряда меняются, то выборочное среднее за 20 лет описывает не «среднее ряда», а просто среднее за эти 20 лет.

Случайное блуждание нестационарно: его дисперсия растёт со временем, $\mathrm{Var}(p_t) = t\sigma^2$, а шок ε_t не затухает и остаётся в уровне ряда навсегда. Про такой ряд говорят, что у него есть единичный корень, или что он интегрирован первого порядка, $I(1)$. Его первая разность стационарна, $I(0)$. Цены активов обычно ведут себя как $I(1)$, доходности – как $I(0)$. Поэтому модели этой главы строят для доходностей, а не для цен.

> **Типичная ошибка: регрессия уровня на уровень**
>
> Корреляция двух цен почти ничего не говорит о связи между активами. Мы сгенерировали 1000 пар *независимых* случайных блужданий длиной 1000 шагов. У 46% пар корреляция уровней по модулю превысила 0,5, а у приращений тех же рядов – у 0%. Это явление называют ложной регрессией. Прежде чем считать

> корреляции, строить регрессии или модели ARMA, переходите от цен к доходностям. Исключение составляет коинтеграция, когда у двух нестационарных рядов есть стационарная комбинация; её мы не рассматриваем.

7.2.3 Тесты ADF и KPSS

Визуально отличить единичный корень от очень медленно затухающего стационарного ряда трудно, поэтому используют формальные тесты. Два самых распространённых построены на противоположных нулевых гипотезах.

- **Тест Дики – Фуллера (ADF, augmented Dickey – Fuller).** Нулевая гипотеза: у ряда есть единичный корень (ряд нестационарен). Малое p-значение – аргумент в пользу стационарности.
- **Тест KPSS (Kwiatkowski – Phillips – Schmidt – Shin).** Нулевая гипотеза: ряд стационарен. Малое p-значение – аргумент в пользу нестационарности.

Тесты проверяют разные гипотезы, поэтому их полезно применять вместе:

Таблица 7.2. Как читать тесты ADF и KPSS вместе

ADF	KPSS	Вывод
отвергает H_0	не отвергает H_0	ряд стационарен
не отвергает H_0	отвергает H_0	ряд нестационарен
не отвергает H_0	не отвергает H_0	данных мало, тесты не различают гипотезы
отвергает H_0	отвергает H_0	противоречие: возможен структурный сдвиг или изменение дисперсии

Применим оба теста к логарифму индекса и к его доходностям.

```
def stationarity_tests(x, trend="c"):
    """p-значения тестов ADF (H0: единичный корень) и KPSS (H0: стационарность).

    trend="c" - в уравнении теста есть константа, trend="ct" - константа и линейный тренд.
    """
    return {"ADF, p": adfuller(x, regression=trend)[1],
            "KPSS, p": kpss(x, regression=trend, nlags="auto")[1]}

# У логарифма цены есть снос вверх, поэтому для него включаем тренд
pd.DataFrame({"Логарифм цены": stationarity_tests(log_p, trend="ct"),
              "Доходность": stationarity_tests(r)}).T.style.format("{:.3f}", decimal=",")
```

Таблица 7.3. Тесты ADF и KPSS: логарифм IMOEX и дневные доходности, 2003–2026 гг.

	ADF, p	KPSS, p
Логарифм цены	0,088	0,010
Доходность	0,000	0,100

Вариант кода на R – в онлайн-версии книги.

Для логарифма цены ADF не отвергает единичный корень (p = 0,09), а KPSS отвергает стационарность: цена ведёт себя как случайное блуждание. Для доходностей всё наоборот: ADF уверенно отвергает единичный корень, KPSS не отвергает стационарность. Результат зависит от спецификации теста. У логарифма цены есть снос вверх, поэтому в уравнение теста мы включили тренд (`regression="ct"`). Если оставить только константу, ADF для периода с 2003 г. даёт p = 0,042 и формально отвергает единичный корень на уровне 5%, хотя по всей истории индекса с трендом p = 0,66. Пограничные p-значения теста на единичный корень – повод не для уверенного вывода, а для проверки его устойчивости к спецификации и периоду.

Табличные p-значения KPSS ограничены диапазоном от 0,01 до 0,1, поэтому значения 0,010 и 0,100 в таблице означают «0,01 или меньше» и «0,1 или больше».

Стационарность доходностей по этим тестам не означает, что их свойства постоянны. Средняя доходность стационарна, а волатильность, как мы видели, меняется из года в год. Тесты на единичный корень проверяют только одну форму нестационарности.

7.3 Модель AR(1): терм-спред ОФЗ

7.3.1 Устройство модели

Модель авторегрессии первого порядка AR(1) говорит, что сегодняшнее значение ряда – это доля вчерашнего плюс шок:

$$x_t = c + \varphi x_{t-1} + \varepsilon_t, \qquad |\varphi| < 1. \tag{7.4}$$

При $|\varphi| < 1$ ряд стационарен и колеблется вокруг среднего $\mu = c/(1-\varphi)$, возвращаясь к нему после каждого шока. Это свойство называют возвратом к среднему (mean reversion). Параметр φ задаёт скорость возврата: отклонение от среднего через h периодов в среднем уменьшается в φ^h раз. Удобная мера скорости – период полураспада, за который отклонение сокращается вдвое:

$$h_{1/2} = \frac{\ln 0,5}{\ln \varphi}. \tag{7.5}$$

При $\varphi = 1$ модель превращается в случайное блуждание, и возврата к среднему нет.

У модели AR(1) характерная коррелограмма: автокорреляции убывают геометрически, $\rho_k = \varphi^k$. Чтобы отличить AR(1) от процессов с более длинной памятью, используют частную автокорреляционную функцию (PACF, partial autocorrelation function). Частная автокорреляция на лаге k – это коэффициент при x_{t-k} в регрессии x_t на $x_{t-1}, \ldots, x_{t-k}$, то есть связь с k-м лагом за вычетом того, что объясняют более близкие лаги. У модели AR(1) частная автокорреляция отлична от нуля только на первом лаге: всё, что x_{t-2} говорит о x_t, уже содержится в x_{t-1}.

7.3.2 Данные: разница между длинными и короткими ставками

Доходности акций плохо подходят для иллюстрации AR(1): у них почти нет автокорреляции. Возьмём ряд, у которого память точно есть, – терм-спред, разницу между доходностями длинных и коротких государственных облигаций. Используем кривую бескупонной доходности ОФЗ, которую публикует Московская биржа, и возьмём разницу доходностей на сроках 10 лет и 3 месяца на конец каждого месяца.

```
# Кривая бескупонной доходности: строки – даты, столбцы – сроки в годах
zcyc = data.load_zcyc().pivot_table(index="date", columns="period", values="value")
# Терм-спред на последний торговый день каждого месяца
spread = (zcyc[10.0] - zcyc[0.25]).resample("ME").last().dropna()

fig, ax = plt.subplots()
ax.plot(spread.index, spread, color=plot.BLUE)
ax.axhline(0, color=plot.LIGHT, lw=0.6)
ax.set_ylabel("п. п.")
plot.source(ax, "Московская биржа, расчёты автора")
plt.show()
```

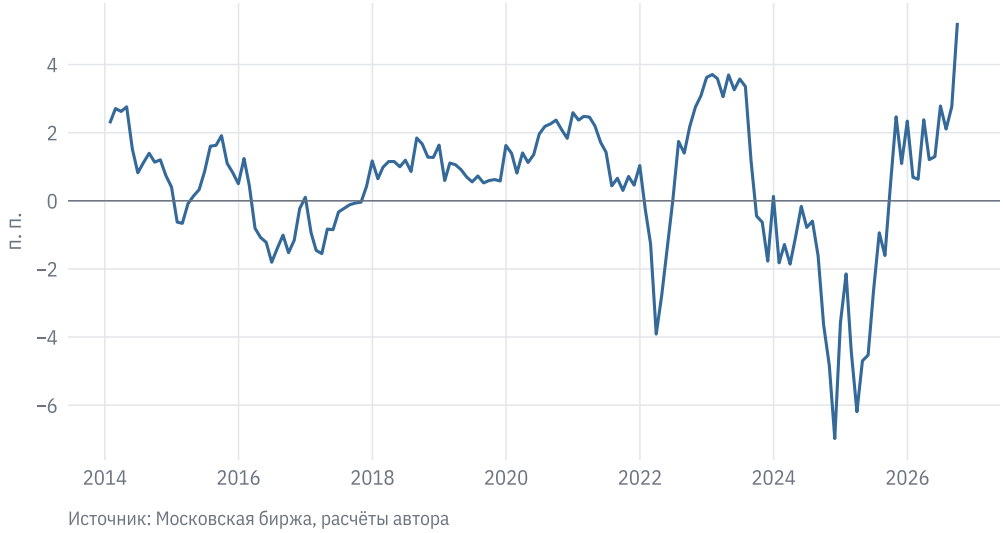
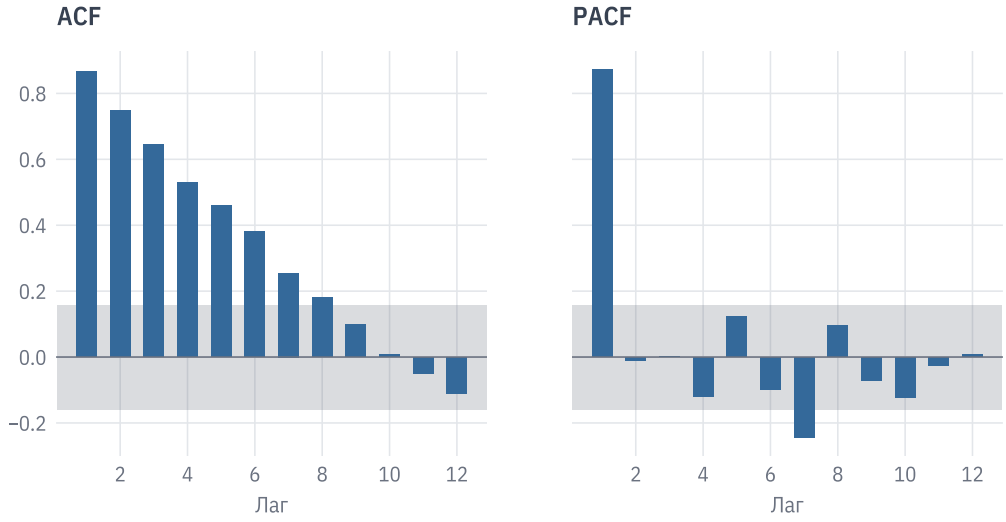


Рисунок 7.4. Терм-спред ОФЗ: разница бескупонных доходностей на сроках 10 лет и 3 месяца, конец месяца, п. п.

В нормальной ситуации длинные ставки выше коротких: инвестор требует премию за то, что вкладывает деньги надолго, и терм-спред положителен. Но в России спред часто уходил в минус – в 33% месяцев с 2014 г. Так бывает, когда Банк России резко поднимает ключевую ставку: короткие ставки идут вслед за ключевой, а длинные растут меньше, потому что рынок ожидает её снижения в будущем. Минимум, −7,0 п. п., был в ноябре 2024 г., когда ключевая ставка достигла 21%. После начала цикла снижения ставки спред вернулся в плюс.

```
fig, axes = plt.subplots(1, 2, figsize=(6.5, 2.8), sharey=True)
plot_acf_bars(axes[0], spread, nlags=12, kind="acf", title="ACF")
plot_acf_bars(axes[1], spread, nlags=12, kind="pacf", title="PACF")
plot.source(axes[0], "Московская биржа, расчёты автора", offset=32)
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 7.5. ACF и PACF месячного терм-спреда ОФЗ

Коррелограммы похожи на учебник: ACF убывает плавно, почти геометрически, а PACF после первого лага падает до уровня шума. Это «подпись» модели AR(1).

7.3.3 Оценка модели

```
# order=(p, d, q): p = 1 лаг авторегрессии, d = 0 (ряд не дифференцируем), q = 0
ar1 = ARIMA(spread, order=(1, 0, 0)).fit()
print(ar1.summary().tables[1])
#> ==============================================================================
#>                  coef    std err          z      P>|z|      [0.025      0.975]
#> ------------------------------------------------------------------------------
#> const          0.8193      0.837      0.979      0.328      -0.821       2.459
#> ar.L1          0.8992      0.027     33.205      0.000       0.846       0.952
#> sigma2         0.8269      0.074     11.129      0.000       0.681       0.972
#> ==============================================================================
```

Вариант кода на R – в онлайн-версии книги.

Оценка φ равна 0,899: терм-спред очень инерционен. Период полураспада по формуле (7.5) – около 6,5 месяца: отклонение спреда от среднего уменьшается вдвое примерно за полгода.

> **Типичная ошибка: константа – это не свободный член**
>
> В выдаче statsmodels коэффициент `const` равен 0,82, и его легко принять за c в формуле (7.4). На самом деле `ARIMA` в statsmodels (как и `Arima` в R, где он называется `mean` или `intercept`) оценивает *среднее ряда* μ. Свободный член уравнения равен

> $c = \mu(1-\varphi) = 0{,}083$. Путаница ведёт к неверным прогнозам, если строить их вручную по формуле.

А стационарен ли терм-спред вообще? Тест ADF даёт p = 0,065, то есть на уровне 5% единичный корень формально не отвергается. Причина – в малой мощности теста: у нас 153 месячных наблюдения, а φ близко к единице. На таких данных ADF плохо отличает медленный возврат к среднему от случайного блуждания. Здесь помогает экономическая логика: спред между ставками одной и той же кривой не может неограниченно уходить в сторону, как цена акции. Поэтому модель AR(1) для него разумна, но к её прогнозам на дальний горизонт стоит относиться осторожно.

7.4 Модель MA(1): неликвидная акция

7.4.1 Устройство модели

Модель скользящего среднего первого порядка MA(1) строится не из прошлых значений ряда, а из прошлых шоков:

$$r_t = \mu + \varepsilon_t + \theta\varepsilon_{t-1}. \tag{7.6}$$

Сегодняшний шок влияет на сегодняшнее значение и, с весом θ, на завтрашнее, а дальше забывается. Поэтому у MA(1) картина обратная AR(1): автокорреляция отлична от нуля только на первом лаге, $\rho_1 = \theta/(1+\theta^2)$, и обрывается после него, а частная автокорреляция затухает постепенно.

7.4.2 Эффект отскока между bid и ask

Откуда в доходностях берётся такая память на один день? Классическое объяснение предложил Roll (1984 г.). Сделки на бирже проходят то по цене продавца (ask), то по цене покупателя (bid). Даже если «справедливая» цена не меняется, цена закрытия прыгает между ними, и за ростом от bid к ask чаще следует падение обратно. Это создаёт отрицательную автокорреляцию первого порядка. Если спред между ask и bid равен s, ковариация соседних изменений цены равна $-s^2/4$. Отсюда оценка спреда по одним только ценам закрытия: $\hat{s} = 2\sqrt{-\operatorname{Cov}(r_t, r_{t-1})}$.

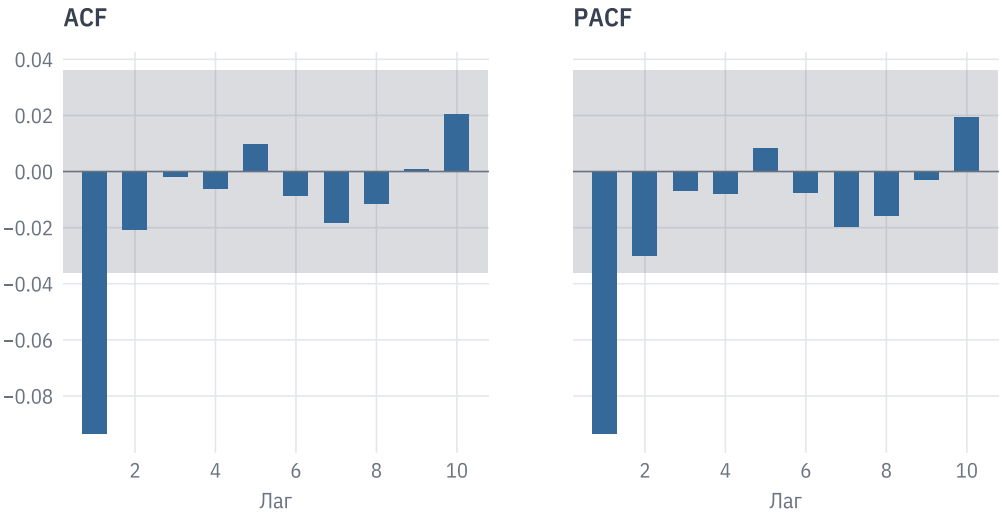
У ликвидных акций спред составляет сотые доли процента, и эффект незаметен. У неликвидных бумаг он может быть заметным. Возьмём обыкновенные акции ПАО «Нижнекамскнефтехим» (ИНН 1651000010, тикер NKNC) – крупной нефтехимической компании, большая часть акций которой не находится в свободном обращении. Для сравнения используем самую ликвидную акцию рынка – Сбербанк (SBER).

```
def load_returns(ticker, start="2015-01-01"):
    """Дневные лог-доходности по ценам с поправкой на дивиденды и медианный оборот, млн руб."""
```

```
    df = data.load_stocks(ticker, start=start).set_index("date")
    return np.log(df["adj_close"]).diff().dropna(), df["value_rub"].median() / 1e6

r_nk, turnover_nk = load_returns("NKNC")
r_sb, turnover_sb = load_returns("SBER")

fig, axes = plt.subplots(1, 2, figsize=(6.5, 2.8), sharey=True)
plot_acf_bars(axes[0], r_nk, nlags=10, kind="acf", title="ACF")
plot_acf_bars(axes[1], r_nk, nlags=10, kind="pacf", title="PACF")
plot.source(axes[0], "Московская биржа, расчёты автора", offset=32)
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 7.6. ACF и PACF дневных доходностей акций NKNC, 2015–2026 гг.

```
# Модели MA(1) и AR(1) для доходностей NKNC (в процентах — так оптимизатору проще)
ma1_nk = ARIMA(r_nk.values * 100, order=(0, 0, 1)).fit()
ar1_nk = ARIMA(r_nk.values * 100, order=(1, 0, 0)).fit()

# Оценка спреда по Роллу: s = 2 * sqrt(-Cov(r_t, r_{t-1}))
cov1 = np.cov(r_nk.values[1:], r_nk.values[:-1])[0, 1]
roll_spread = 2 * np.sqrt(-cov1) if cov1 < 0 else np.nan

pd.Series({"θ (MA(1))": ma1_nk.params[1], "BIC MA(1)": ma1_nk.bic, "BIC AR(1)": ar1_nk.bic,
           "Спред по Роллу, %": roll_spread * 100}).round(3)
#> θ (MA(1))              -0.099
#> BIC MA(1)           13300.993
#> BIC AR(1)           13302.407
#> Спред по Роллу, %       1.417
#> dtype: float64
```

Медианный дневной оборот NKNC с 2015 г. – около 2,6 млн руб., у Сбербанка – около 9,8 млрд руб., в тысячи раз больше. Первая автокорреляция доходностей NKNC равна −0,093, а на втором лаге – всего −0,021: картина обрыва после первого лага, как у MA(1). У Сбербанка первая автокорреляция −0,004 и неотличима от нуля. Оценка θ в модели MA(1) – −0,099, что соответствует $\rho_1 = -0{,}098$, близко к выборочной автокорреляции. По BIC модель MA(1) немного лучше AR(1), хотя разница невелика: при $|\theta|$ около 0,1 модели AR(1) и MA(1) почти неразличимы.

Спред по формуле Ролла получается около 1,4%. Это грубая оценка: в ковариацию попадают не только отскоки между bid и ask, но и любые другие краткосрочные эффекты. Но порядок величины разумен для бумаги, по которой в 5% торговых дней цена не менялась вовсе.

Важно, что отрицательная автокорреляция здесь не сулит прибыли. Она возникает из-за того, что цена прыгает внутри спреда, а чтобы на этом заработать, нужно покупать по bid и продавать по ask, то есть самому быть маркет-мейкером. Инвестор, торгующий по рыночным заявкам, заплатит этот спред сам. Микроструктуре рынка и измерению ликвидности посвящена отдельная глава книги.

7.5 Модели ARMA и выбор порядка

7.5.1 Общая модель

Модели AR и MA объединяются в модель ARMA(p, q) с p авторегрессионными лагами и q лагами шоков:

$$x_t = c + \sum_{i=1}^{p} \varphi_i x_{t-i} + \varepsilon_t + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j}. \tag{7.7}$$

Если ряд перед моделированием дифференцируют d раз, получается модель ARIMA(p, d, q); отсюда аргумент `order=(p, d, q)` в statsmodels. Для доходностей $d = 0$, для логарифма цены $d = 1$, и это одна и та же модель.

Классическая процедура Бокса – Дженкинса строит модель в четыре шага: идентификация (выбор p и q по коррелограммам), оценка параметров, диагностика остатков и прогноз. Для идентификации служат подписи моделей в ACF и PACF:

Таблица 7.4. Как выглядят модели AR, MA и ARMA на коррелограммах

Модель	ACF	PACF
AR(p)	затухает постепенно	обрывается после лага p
MA(q)	обрывается после лага q	затухает постепенно
ARMA(p, q)	затухает постепенно	затухает постепенно

7.5.2 Информационные критерии

На реальных данных подписи редко бывают такими чёткими, как у терм-спреда. Поэтому порядок модели чаще выбирают по информационным критериям. Они поощряют модель за правдоподобие L и штрафуют за число параметров k:

$$\mathrm{AIC} = -2\ln L + 2k, \qquad \mathrm{BIC} = -2\ln L + k\ln T. \tag{7.8}$$

Выбирают модель с наименьшим значением критерия. Лишний параметр почти всегда немного увеличивает правдоподобие, и критерии требуют, чтобы это увеличение окупало штраф. BIC при $T > 7$ штрафует сильнее AIC и выбирает более простые модели. AIC нацелен на качество прогноза, BIC – на поиск «истинной» модели, если она есть среди кандидатов.

Переберём модели ARMA(p, q) с $p, q \in \{0, 1, 2\}$ для дневных доходностей IMOEX. Оценивать будем на обучающем периоде 2003–2019 гг., а 2020–2026 гг. оставим для проверки прогноза вне выборки. Доходности умножим на 100, чтобы оценки были в процентах.

```
r100 = r * 100                                  # доходности в процентах
train, test = r100.loc[:"2019-12-31"], r100.loc["2020-01-01":]

grid = []
for p in range(3):
    for q in range(3):
        # .values: даты торговых дней идут с пропусками, частоту statsmodels не нужна
        m = ARIMA(train.values, order=(p, 0, q)).fit()
        grid.append({"p": p, "q": q, "AIC": m.aic, "BIC": m.bic})
grid = pd.DataFrame(grid)

# Разница с лучшей моделью по каждому критерию
tab = grid.assign(AIC=grid.AIC - grid.AIC.min(), BIC=grid.BIC - grid.BIC.min())
tab.pivot(index="p", columns="q", values=["AIC", "BIC"]).style.format("{:.1f}", decimal=",")
```

Таблица 7.5. Информационные критерии моделей ARMA(p, q) для дневных доходностей IMOEX, 2003–2019 гг. Значения за вычетом минимума: 0 – лучшая модель

	AIC			BIC		
q	0	1	2	0	1	2
p						
0	0,0	1,5	2,4	0,0	7,8	15,1
1	1,5	1,9	0,1	7,8	14,5	19,1
2	2,4	0,0	1,7	15,1	19,0	27,1

Вариант кода на R – в онлайн-версии книги.

BIC выбирает модель ARMA(0, 0). Это константа плюс белый шум: лучший прогноз завтрашней доходности равен средней доходности. Ни одна из более

сложных моделей не улучшает правдоподобие настолько, чтобы окупить дополнительные параметры. Например, ARMA(2, 2) по BIC хуже константы на 27,1.

AIC штрафует мягче и выбирает ARMA(2, 1), но по AIC она лучше константы всего на 0,01, то есть модели практически равноценны. Посмотрите на оценки: коэффициент авторегрессии на первом лаге 0,77, коэффициент скользящего среднего −0,76. Они почти равны по модулю и противоположны по знаку, поэтому их вклады почти взаимно сокращаются. В простейшем случае ARMA(1, 1) в записи через оператор лага L ($Lx_t = x_{t-1}$) модель имеет вид $(1 - \varphi L)x_t = (1 + \theta L)\varepsilon_t$, и при $\theta \approx -\varphi$ множители слева и справа сокращаются, оставляя белый шум. Такое «сокращение корней» – частый признак того, что модель подогнана под шум. Вне выборки она ведёт себя соответственно: R^2_{OOS} против исторического среднего равен −0,0054 (эту меру мы введём в разделе 7.7).

Этот результат согласуется с первым разделом главы: линейная память в дневных доходностях индекса слишком слаба, чтобы на ней строить модель.

7.6 Диагностика остатков

Хорошая модель должна извлечь из ряда всю линейную зависимость, и её остатки $\hat{\varepsilon}_t$ должны быть похожи на белый шум. Стандартная проверка – тест Льюнга – Бокса по остаткам. Число степеней свободы при этом уменьшают на число оценённых параметров $p + q$ (аргумент `model_df`).

```
best = ARIMA(train.values, order=(int(best_bic.p), 0, int(best_bic.q))).fit()
resid = best.resid
df_model = int(best_bic.p + best_bic.q)

# p-значения теста для 10 лагов: по остаткам и по их квадратам
pd.Series({
    "Остатки": acorr_ljungbox(resid, lags=[10], model_df=df_model)["lb_pvalue"].iloc[0],
          "Квадраты   остатков":   acorr_ljungbox(resid**2,   lags=[10],   model_df=df_model)
["lb_pvalue"].iloc[0],
}, name="p-значение Q(10)")
#> Остатки               1.789733e-03
#> Квадраты остатков    3.968749e-287
#> Name: p-значение Q(10), dtype: float64
```

По остаткам тест даёт p = 0,0018, то есть формально какая-то автокорреляция в них осталась. Мы уже знаем её природу: на длинной выборке значимыми оказываются автокорреляции порядка сотых, которые меняют знак от года к году. Информационные критерии, которые учитывают размер эффекта, а не только его значимость, сочли, что моделировать её не стоит.

Намного важнее результат для квадратов остатков. Для них статистика $Q(10)$ равна 1 365, а p-значение неотличимо от нуля. Остатки некоррелированы, но не независимы: их размах сильно зависит от прошлого. Модель ARMA описывает

условное среднее и предполагает постоянную дисперсию шока, а у доходностей она явно меняется. Такой эффект называют условной гетероскедастичностью, и для него нужны модели ARCH и GARCH из следующей главы.

7.7 Прогноз вне выборки

7.7.1 Как устроен честный прогноз

Модель, которая хорошо описывает прошлое, не обязательно хорошо предсказывает будущее. Единственная надёжная проверка – прогноз вне выборки (out-of-sample), на данных, которые не участвовали ни в выборе модели, ни в оценке её параметров. Схема такая:

1. Разделить ряд на обучающую и тестовую части по времени, без перемешивания.
2. Выбрать модель и оценить параметры только на обучающей части.
3. Для каждого дня тестовой части построить прогноз на один шаг вперёд, используя только данные, известные к этому дню.
4. Сравнить ошибки прогноза с ошибками простого эталона.

На третьем шаге есть два варианта. Можно после каждого нового наблюдения переоценивать параметры на расширяющемся окне – это честнее, но медленнее. Можно оставить параметры, оценённые на обучающей части, и только подставлять в модель новые значения ряда. Для дневных данных с тысячами точек обычно достаточно второго варианта; в statsmodels он реализован методом `apply()`.

В качестве меры качества для доходностей используют R^2 вне выборки (Campbell и Thompson 2008 г.):

$$R^2_{OOS} = 1 - \frac{\sum_t (r_t - \hat{r}_t)^2}{\sum_t (r_t - \bar{r}_t)^2}, \tag{7.9}$$

где $\hat{r}_t$ – прогноз модели, а $\bar{r}_t$ – прогноз эталона, например историческое среднее доходности на момент прогноза. Положительный R^2_{OOS} означает, что модель предсказывает лучше эталона, отрицательный – хуже.

7.7.2 Доходности индекса

Выбранная BIC модель ARMA(0, 0) прогнозирует константу. Чтобы проверка была содержательной, возьмём модель AR(1), которая использует вчерашнюю доходность, и сравним её с двумя эталонами: историческим средним и нулём.

```
# AR(1) на обучающем периоде; параметры дальше не меняем
ar1_r = ARIMA(train.values, order=(1, 0, 0)).fit()
# apply() прогоняет модель с теми же параметрами по всему ряду;
# predict() даёт прогноз на шаг вперёд для каждого дня по данным до него
pred = ar1_r.apply(r100.values).predict()[len(train):]
```

```
# Историческое среднее: для каждого дня – средняя доходность по всем предыдущим дням
hist_mean = r100.expanding().mean().shift(1).loc[test.index].values
actual = test.values

mse = lambda f: np.mean((actual - f) ** 2)
pd.Series({
    "φ в AR(1)": ar1_r.params[1],
    "R² OOS против среднего": 1 - mse(pred) / mse(hist_mean),
    "R² OOS против нуля": 1 - mse(pred) / mse(np.zeros_like(actual)),
    "Доля верно угаданных знаков": np.mean(np.sign(pred) == np.sign(actual)),
    "Доля дней роста": np.mean(actual > 0),
}).round(4)
#> φ в AR(1)                      0.0111
#> R² OOS против среднего        -0.0019
#> R² OOS против нуля            -0.0031
#> Доля верно угаданных знаков    0.5270
#> Доля дней роста                0.5317
#> dtype: float64
```

Оценка φ на обучающем периоде – 0,011, почти ноль. На 1 687 днях теста модель AR(1) проигрывает обоим эталонам: R^2_{OOS} против исторического среднего равен −0,0019, против нуля – −0,0031. Модель угадывает знак доходности в 52,7% дней, при том что доля дней роста в тестовом периоде – 53,2%. Ничего удивительного: прогноз модели положителен в 99% дней, и почти весь «успех» в угадывании знака объясняется тем, что рынок растёт чаще, чем падает.

Это типичный результат. Welch и Goyal (2008 г.) показали, что десятки переменных, которые в литературе считались предикторами доходности американского рынка, вне выборки обычно не превосходят историческое среднее. Отрицательный R^2_{OOS} простых моделей – не провал анализа, а честная характеристика рынка и планка для любой торговой стратегии, о чём подробнее пойдёт речь в главе о бэктестинге.

7.7.3 Терм-спред

Теперь повторим проверку для терм-спреда, где память в данных точно есть. Здесь рядов немного, поэтому будем переоценивать модель каждый месяц на расширяющемся окне. Эталонов три: модель AR(1), случайное блуждание (прогноз равен последнему значению) и историческое среднее.

```
test_months = spread.loc["2020-01-01":].index
fc = {"AR(1)": [], "Случайное блуждание": [], "Историческое среднее": []}
for d in test_months:
    hist = spread.loc[:d].iloc[:-1]                 # только данные до месяца прогноза
    model = ARIMA(hist, order=(1, 0, 0)).fit()      # переоцениваем на расширяющемся окне
    fc["AR(1)"].append(model.forecast(1).iloc[0])
    fc["Случайное блуждание"].append(hist.iloc[-1])
```

```
    fc["Историческое среднее"].append(hist.mean())

actual_sp = spread.loc[test_months].values
rmse = {k: np.sqrt(np.mean((actual_sp - np.array(v)) ** 2)) for k, v in fc.items()}
pd.Series(rmse, name="RMSE, п. п.").round(3)
#> AR(1)                  1.181
#> Случайное блуждание    1.183
#> Историческое среднее   2.514
#> Name: RMSE, п. п., dtype: float64
```

Модель AR(1) предсказывает терм-спред на следующий месяц вдвое точнее исторического среднего: среднеквадратичная ошибка 1,18 п. п. против 2,51. Но случайное блуждание даёт почти ту же точность (1,18 п. п.). При $\varphi \approx 0,9$ почти вся информация о следующем значении содержится в последнем, и учёт возврата к среднему уточняет прогноз на месяц вперёд совсем немного. Преимущество модели AR(1) над случайным блужданием проявляется на более длинных горизонтах, где возврат к среднему успевает накопиться.

Типичная ошибка: константный прогноз и заглядывание в будущее

Частая ошибка – обучить модель один раз и затем в цикле по тестовому периоду вызывать `forecast(steps=1)`:

```
model = ARIMA(train, order=(1, 0, 1)).fit()
preds = [model.forecast(steps=1)[0] for day in test.index]   # неправильно
```

Модель ничего не знает о новых наблюдениях, поэтому все прогнозы в списке одинаковы: это прогноз на первый день теста, повторённый много раз. Похожая ошибка – вызвать `forecast(steps=len(test))`. Это многошаговый прогноз из последней точки обучающей выборки, который быстро сходится к среднему ряда (см. ниже), а не последовательность прогнозов на день вперёд. Правильно – обновлять модель новыми данными через `apply()` или `append()` либо переоценивать её в цикле, как в примере с терм-спредом.

Противоположная ошибка – заглядывание в будущее: модель оценивают по всей выборке, включая тестовый период, а затем «проверяют» на нём. Такая проверка завышает качество, потому что параметры уже подстроены под тестовые данные. Выбор порядка модели по критериям на всей выборке – тоже заглядывание в будущее.

7.7.4 Многошаговый прогноз

Прогноз модели AR(1) на h шагов вперёд из точки T равен

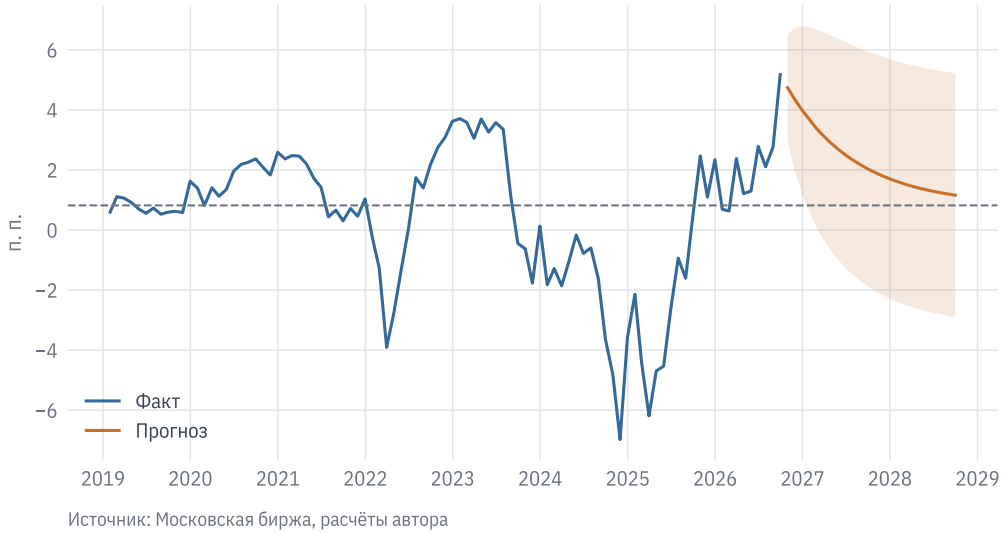
$$\hat{x}_{T+h} = \mu + \varphi^h (x_T - \mu). \tag{7.10}$$

С ростом горизонта $\varphi^h \to 0$, и прогноз сходится к среднему ряда, а доверительный интервал расширяется до безусловного разброса ряда. Для модели ARMA(0,

0) прогноз на любой горизонт равен среднему с самого начала. Построим прогноз терм-спреда на два года вперёд по модели, оценённой на всех данных.

```
# get_forecast() возвращает и точечный прогноз, и доверительные интервалы
fc24 = ar1.get_forecast(steps=24).summary_frame(alpha=0.05)

fig, ax = plt.subplots()
ax.plot(spread.loc["2019":].index, spread.loc["2019":], color=plot.BLUE, label="Факт")
ax.plot(fc24.index, fc24["mean"], color=plot.ORANGE, label="Прогноз")
ax.fill_between(fc24.index, fc24["mean_ci_lower"], fc24["mean_ci_upper"],
                color=plot.ORANGE, alpha=0.15, lw=0)
ax.axhline(ar1.params["const"], color=plot.LIGHT, lw=0.8, ls="--")   # оценка среднего μ
ax.set_ylabel("п. п.")
ax.legend(loc="lower left")
plot.source(ax, "Московская биржа, расчёты автора")
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 7.7. Прогноз терм-спреда ОФЗ по модели AR(1) на 24 месяца с 95%-м интервалом

Вариант кода на R – в онлайн-версии книги.

Последнее значение спреда, 5,2 п. п., – максимум за всю историю ряда. Модель ожидает, что через год спред снизится до 2,0 п. п., а через два – до 1,2 п. п., приближаясь к среднему. Но 95%-й интервал через два года – от −2,9 до 5,2 п. п., то есть модель допускает и сохранение высокого спреда, и его инверсию. Модель честно показывает, как быстро прогноз по прошлому ряда теряет информативность. Прогноз на дальний горизонт требует других данных: ожиданий по ключевой ставке, инфляции, бюджетной политике.

Итоги

- Автокорреляционная функция измеряет линейную зависимость ряда от своего прошлого. Граница значимости $\pm 1,96/\sqrt{T}$ сужается с ростом выборки, поэтому на длинных рядах значимыми становятся экономически ничтожные эффекты.
- Дневные доходности индекса Мосбиржи почти не автокоррелированы: первая автокорреляция мала и меняет знак от года к году. Модули и квадраты доходностей, напротив, автокоррелированы сильно и долго – это кластеризация волатильности.
- Цены ведут себя как случайное блуждание и нестационарны; доходности стационарны. Модели и корреляции строят для доходностей. Тесты ADF и KPSS имеют противоположные нулевые гипотезы, и их полезно применять вместе.
- Модель AR(1) описывает ряды с возвратом к среднему, например терм-спред ОФЗ; скорость возврата удобно выражать периодом полураспада. Модель MA(1) описывает краткую отрицательную память доходностей неликвидных акций, связанную с отскоком цены между bid и ask.
- Порядок модели ARMA выбирают по ACF и PACF и по информационным критериям. Для доходностей IMOEX BIC выбирает модель без лагов, а более сложная модель, выбранная AIC, фактически сводится к белому шуму и проигрывает вне выборки.
- Остатки проверяют тестом Льюнга – Бокса. Автокорреляция квадратов остатков указывает на изменяющуюся волатильность, которую описывают модели GARCH.
- Качество прогноза оценивают только вне выборки и относительно простого эталона. Простые модели доходностей не превосходят историческое среднее; модель терм-спреда почти не превосходит случайное блуждание на горизонте одного месяца.

7.8 Упражнения

Упражнения разделены на четыре типа. Задания **А** проверяют понимание и решаются без компьютера. Задания **Б** выполняются на данных вашей акции (тикер назначает преподаватель), поэтому готового ответа у них нет. Задания **В** учат работать с ИИ-ассистентом: не получать ответ, а проверять его. Задание **Г** – небольшое исследование, результаты которого нужно защитить устно.

7.8.1 А. Понимание

1. По 5000 дневным доходностям получена первая автокорреляция 0,04. Значима ли она на уровне 5%? Следует ли отсюда, что стратегия «покупай после роста» прибыльна? Какие данные нужны, чтобы это проверить?

2. Ряд описывается моделью AR(1) с $\varphi = 0,8$ и средним $\mu = 2$. Сегодня $x_T = 5$. Найдите прогноз на 1, 3 и 12 шагов вперёд и период полураспада. Чему равен свободный член c?
3. Для модели MA(1) с $\theta = -0,5$ найдите ρ_1 и ρ_2. Может ли у модели MA(1) первая автокорреляция быть равна $-0,7$?
4. Тест ADF даёт p = 0,40, тест KPSS – p < 0,01. Что можно сказать о ряде? А если оба теста дают p < 0,01?
5. Почему BIC обычно выбирает более простые модели, чем AIC? Для какой цели – прогноза или поиска истинной модели – вы бы предпочли каждый из критериев?
6. Аналитик обучил ARMA на 2003–2026 гг., а затем посчитал ошибку прогноза на 2020–2026 гг. и получил положительный R^2_{OOS}. Что не так с этой проверкой?

7.8.2 Б. Практика на своей акции

7. Постройте коррелограммы дневных доходностей вашей акции и их модулей с 2015 г. Проведите тест Льюнга – Бокса для 10 лагов. Сравните с результатами для IMOEX из раздела 7.1.
8. Проверьте тестами ADF и KPSS логарифм цены и доходность вашей акции. Согласуются ли выводы тестов?
9. Подберите модель ARMA(p, q) при $p, q \le 2$ по AIC и BIC на данных до конца 2019 г. (если история короче – на первых двух третях выборки). Оцените R^2_{OOS} выбранной модели и модели AR(1) против нуля и исторического среднего на оставшихся данных.
10. Для своей акции за 2015–2026 гг. посчитайте: (а) первую автокорреляцию дневных лог-доходностей; (б) оценку спреда по формуле Ролла $\hat{s} = 2\sqrt{-\operatorname{Cov}(r_t, r_{t-1})}$ (если ковариация положительна, оценка не определена – объясните, что это означает); (в) медианный дневной оборот. Сравните результаты с NKNC и SBER из раздела 7.4. На какую из двух бумаг похожа ваша? Можно ли по одной акции судить о связи ликвидности и автокорреляции? Какие эффекты, кроме отскока цены между bid и ask, могут влиять на первую автокорреляцию?

7.8.3 В. Работа с ИИ-ассистентом

11. Попросите ИИ-ассистента написать код, который обучает модель ARIMA на доходностях IMOEX до 2020 г. и строит прогноз на каждый день 2020–2026 гг. Проверьте код на две ошибки из врезки в разделе 7.7: константный прогноз и заглядывание в будущее. Если ошибок нет, найдите, как ассистент обновляет модель новыми данными, и убедитесь, что прогноз на день t не использует доходность дня t.
12. Спросите ассистента, можно ли предсказывать доходность российского рынка акций моделями ARIMA, и попросите ссылки на исследования.

Проверьте, существуют ли названные работы и говорится ли в них то, что утверждает ассистент. Сопоставьте его ответ с результатами главы.

7.8.4 Г. Мини-исследование

13. Растёт ли автокорреляция доходностей IMOEX с горизонтом? Посчитайте первую автокорреляцию дневных, недельных и месячных лог-доходностей за 2003–2026 гг. и отношение дисперсий $VR(q) = \mathrm{Var}(r_t^{(q)})/(q\,\mathrm{Var}(r_t))$ для $q = 5$ и $q = 20$ (Lo и MacKinlay 1988 г.). Оцените, сколько независимых наблюдений у вас на каждом горизонте и насколько надёжен вывод. Подготовьте короткий (3–5 минут) устный рассказ.

Вопросы для самопроверки

1. Чем отличаются ACF и PACF? Как по ним отличить модель AR(1) от MA(1)?
2. Почему некоррелированность доходностей не означает их независимости? Приведите пример из данных.
3. Какие нулевые гипотезы у тестов ADF и KPSS?
4. Почему модели ARMA строят для доходностей, а не для цен?
5. Что такое период полураспада в модели AR(1)? Чему он равен при $\varphi = 0,5$?
6. Какой вывод следует из того, что тест Льюнга – Бокса отвергает гипотезу для квадратов остатков, но почти не отвергает её для самих остатков?
7. К чему сходится многошаговый прогноз стационарной модели ARMA?

Ответы

1. ACF – полная корреляция с лагом k, PACF – корреляция с лагом k за вычетом влияния промежуточных лагов. У AR(1) ACF затухает геометрически, PACF обрывается после первого лага; у MA(1) – наоборот.
2. Некоррелированность означает отсутствие *линейной* зависимости знака и величины. Модули доходностей IMOEX имеют автокорреляцию около 0,3–0,4 на первых лагах: большие движения следуют за большими.
3. ADF: у ряда есть единичный корень. KPSS: ряд стационарен.
4. Цены нестационарны (случайное блуждание), и оценки ACF и параметров для них не имеют обычного смысла; корреляции уровней бывают ложными. Доходности стационарны.
5. Число периодов, за которое отклонение от среднего в среднем уменьшается вдвое: $h_{1/2} = \ln 0,5 / \ln \varphi$. При $\varphi = 0,5$ – один период.
6. Условное среднее описано моделью удовлетворительно, но дисперсия шока меняется во времени (условная гетероскедастичность); нужна модель волатильности, например GARCH.
7. К безусловному среднему ряда μ; доверительный интервал расширяется до безусловного разброса ряда.

Что почитать

- Tsay (2010 г.), гл. 2 – линейные модели временных рядов с примерами на финансовых данных.
- Hyndman и Athanasopoulos (2021 г.), гл. 9 – модели ARIMA, выбор порядка и прогноз; открытый учебник с кодом на R.
- Campbell и др. (1997 г.), гл. 2 – предсказуемость доходностей и тесты гипотезы случайного блуждания.
- Lo и MacKinlay (1988 г.) – тест отношения дисперсий для проверки гипотезы случайного блуждания.
- Roll (1984 г.) – оценка спреда по автокорреляции цен.
- Welch и Goyal (2008 г.) и Campbell и Thompson (2008 г.) – как оценивать прогнозы доходности вне выборки и почему их трудно сделать лучше исторического среднего.

8 Волатильность: модели ARCH и GARCH

Вечером 21 февраля 2022 г. годовая волатильность индекса Московской биржи, рассчитанная по модели этой главы, составляла 42% – выше обычного, но в пределах того, что рынок видел не раз. Через два торговых дня, после обвала 24 февраля и частичного отскока 25-го, та же модель показывала 235% годовых. Затем торги акциями остановились почти на месяц. К середине апреля оценка снизилась до 94%, но оставалась в несколько раз выше медианного уровня за два десятилетия, 21%.

Риск на финансовом рынке не постоянен, и это видно без всякой статистики. Но инвестору, риск-менеджеру банка или трейдеру опционами нужно не общее впечатление, а конкретное число: какой волатильности ждать завтра, через неделю, через месяц? Скользящее стандартное отклонение из главы «Доходность и риск» отвечает на этот вопрос плохо: оно реагирует на шок с опозданием, а потом резко «забывает» его, когда шок выходит из окна. В этой главе мы построим модели, в которых сегодняшняя волатильность зависит от вчерашних шоков по явному правилу, оценим их на данных российского рынка и проверим, насколько хорошо они предсказывают.

В этой главе

- Проверяем, что волатильность доходностей меняется во времени и зависит от прошлого: тест ARCH-LM.
- Строим модели ARCH и GARCH(1,1), интерпретируем их параметры и связь с методом EWMA.
- Учитываем тяжёлые хвосты (распределение Стьюдента) и асимметрию реакции на падения и рост (модель GJR-GARCH).
- Проверяем модель по стандартизованным остаткам и разбираем частую ошибку диагностики.
- Прогнозируем волатильность и сравниваем прогнозы вне выборки с помощью подходящих функций потерь.

> **Данные:** `indexes.parquet`, `stocks.parquet` (см. данные книги). **Нужно знать:** автокорреляция, тест Льюнга – Бокса, модели ARMA и прогноз вне выборки (глава «Свойства финансовых рядов и модели ARMA»).

Модели волатильности в Python оценивает пакет arch, в R – пакет rugarch.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from arch import arch_model
from statsmodels.stats.diagnostic import het_arch, acorr_ljungbox
from qf import data, plot
```

8.1 Кластеры волатильности

8.1.1 Что мы уже знаем

В предыдущей главе мы увидели два факта о дневных доходностях индекса IMOEX. Сами доходности почти не автокоррелированы, а их модули и квадраты автокоррелированы сильно и долго. Модель ARMA для среднего доходности оказалась бесполезной, а тест Льюнга – Бокса для квадратов её остатков отверг гипотезу об отсутствии автокорреляции с p-значением, неотличимым от нуля.

Формализуем это наблюдение. Запишем доходность как сумму ожидаемой части и шока:

$$r_t = \mu_t + \varepsilon_t, \qquad \varepsilon_t = \sigma_t z_t, \tag{8.1}$$

где μ_t – условное среднее (для доходностей индекса – практически константа), z_t – независимые одинаково распределённые величины с нулевым средним и единичной дисперсией, а σ_t – **условная волатильность**: стандартное отклонение доходности дня t при условии всего, что известно к концу дня $t-1$. Модели ARMA предполагают, что σ_t постоянна. Модели этой главы позволяют ей меняться во времени и описывают, как именно.

Слово «условная» здесь ключевое. Безусловная дисперсия доходностей за 20 лет может быть вполне постоянной, как требует стационарность, а условная – меняться каждый день. Это не противоречие: стационарный ряд может иметь периоды спокойствия и бурь, если их чередование подчиняется неизменному правилу.

8.1.2 Тест ARCH-LM

Стандартная проверка того, зависит ли дисперсия от прошлых шоков, – тест множителей Лагранжа, предложенный Engle (1982 г.). Квадраты шоков регрессируют на их собственные лаги:

$$\hat{\varepsilon}_t^2 = c_0 + c_1\hat{\varepsilon}_{t-1}^2 + \ldots + c_m\hat{\varepsilon}_{t-m}^2 + u_t. \tag{8.2}$$

Нулевая гипотеза – все $c_1, \ldots, c_m$ равны нулю, то есть условная дисперсия постоянна. Статистика TR^2 этой регрессии при нулевой гипотезе имеет распределение χ^2 с m степенями свободы. Тест ARCH-LM и тест Льюнга – Бокса для квадратов проверяют, по сути, одно и то же, и обычно дают одинаковый вывод.

```
# Дневные лог-доходности IMOEX в процентах: модели GARCH оцениваются устойчивее,
# когда значения ряда порядка единиц, а не тысячных
imoex = data.load_index("IMOEX", start="2003-01-01").set_index("date")["close"]
r = np.log(imoex).diff().dropna() * 100

# Шоки: отклонения от среднего; тест с 5 лагами (торговая неделя)
lm_stat, lm_p = het_arch(r - r.mean(), nlags=5)[:2]   # статистика и её p-значение
print(f"ARCH-LM(5) = {lm_stat:.1f}, p-значение = {lm_p:.1e}")
#> ARCH-LM(5) = 277.9, p-значение = 5.5e-58
```

Гипотеза о постоянной условной дисперсии отвергается с огромным запасом. Теперь нужно описать, как именно дисперсия зависит от прошлого.

8.2 Модели ARCH и GARCH

8.2.1 ARCH: дисперсия как функция прошлых шоков

Самая простая идея принадлежит Engle (1982 г.): пусть сегодняшняя условная дисперсия линейно зависит от квадратов вчерашних шоков. Модель ARCH(1) (autoregressive conditional heteroskedasticity – авторегрессионная условная гетероскедастичность):

$$\sigma_t^2 = \omega + \alpha\varepsilon_{t-1}^2, \qquad \omega > 0, \alpha \geq 0. \tag{8.3}$$

Вчера рынок сильно сдвинулся – сегодня ожидаем большую дисперсию; вчера было тихо – сегодня ожидаем дисперсию около ω. Модель ARCH(q) добавляет q лагов квадратов шоков. Проблема в том, что память волатильности на реальных рынках длинная: мы видели, что автокорреляция модулей доходности заметна и через 30 дней. Чтобы описать это моделью ARCH, нужны десятки лагов и столько же параметров.

8.2.2 GARCH(1,1): дисперсия с инерцией

Bollerslev (1986 г.) предложил добавить в уравнение вчерашнюю условную дисперсию. Модель GARCH(1,1) (generalized ARCH):

$$\sigma_t^2 = \omega + \alpha\varepsilon_{t-1}^2 + \beta\sigma_{t-1}^2, \qquad \omega > 0, \alpha, \beta \geq 0. \tag{8.4}$$

Сегодняшняя дисперсия складывается из трёх частей: постоянной ω, реакции на вчерашний шок $\alpha\varepsilon_{t-1}^2$ и инерции $\beta\sigma_{t-1}^2$. Подставляя σ_{t-1}^2 из того же уравнения, а затем σ_{t-2}^2 и так далее, можно увидеть, что GARCH(1,1) – это ARCH с бесконечным

числом лагов, веса которых убывают геометрически: $\alpha, \alpha\beta, \alpha\beta^2, \ldots$ Три параметра описывают длинную память, для которой ARCH понадобились бы десятки.

Из уравнения (8.4) следуют три полезные величины.

- **Персистентность** $\alpha + \beta$ показывает, какая доля отклонения дисперсии от среднего уровня сохраняется на следующий день. Модель стационарна при $\alpha + \beta < 1$.
- **Безусловная дисперсия** – уровень, вокруг которого колеблется условная:

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \beta}. \tag{8.5}$$

- **Период полураспада** шока волатильности – число дней, за которое отклонение дисперсии от $\bar{\sigma}^2$ в среднем сокращается вдвое: $h_{1/2} = \ln 0,5 / \ln(\alpha + \beta)$. Это та же формула, что для модели AR(1) в предыдущей главе. Сходство не случайно: квадраты шоков в модели GARCH(1,1) подчиняются модели ARMA(1,1) с авторегрессионным коэффициентом $\alpha + \beta$.

8.2.3 Оценка модели

Модель оценивается методом максимального правдоподобия. Для этого нужно задать модель среднего и распределение z_t. Среднее возьмём постоянным (в предыдущей главе оба информационных критерия не нашли в доходностях индекса ничего лучше константы), распределение – пока нормальным.

```
# mean="Constant": постоянное среднее; vol="GARCH", p=1, q=1: GARCH(1,1);
# p - число лагов шоков (ARCH-часть), q - число лагов дисперсии (GARCH-часть)
garch_n = arch_model(r, mean="Constant", vol="GARCH", p=1, q=1, dist="normal").fit(disp="off")
print(garch_n.summary().tables[1])   # уравнение среднего
#>                              Mean Model
#> ============================================================================
#>                  coef    std err          t      P>|t|    95.0% Conf. Int.
#> ----------------------------------------------------------------------------
#> mu             0.0834  1.884e-02      4.428  9.521e-06 [4.650e-02,  0.120]
#> ============================================================================
print(garch_n.summary().tables[2])   # уравнение волатильности
#>                               Volatility Model
#> ============================================================================
#>                  coef    std err          t      P>|t|      95.0% Conf. Int.
#> ----------------------------------------------------------------------------
#> omega          0.0382  1.555e-02      2.460  1.391e-02 [7.769e-03,6.872e-02]
#> alpha[1]       0.1227  2.515e-02      4.880  1.063e-06   [7.343e-02,  0.172]
#> beta[1]        0.8735  2.382e-02     36.668 2.386e-294     [  0.827,  0.920]
#> ============================================================================
```

Вариант кода на R – в онлайн-версии книги.

Типичная ошибка: порядок p и q

В пакете arch параметр `p` – число лагов *шоков* ε^2, а `q` – число лагов *дисперсии* σ^2. В rugarch в `garchOrder = c(1, 1)` первым идёт число лагов шоков, в большинстве учебников GARCH(p, q) пишется наоборот: p – лаги дисперсии. Для GARCH(1,1) это неважно, но при переборе моделей GARCH(1,2) и GARCH(2,1) легко перепутать одну с другой. Проверяйте по выдаче, какие коэффициенты оценены: `alpha[1], alpha[2]` или `beta[1], beta[2]`.

Оценки $\alpha = 0{,}123$ и $\beta = 0{,}873$ типичны для индексов акций. Alexander (2008 г.) приводит как ориентир для спокойных рынков α от 0,05 до 0,1 и β от 0,85 до 0,98. Российский рынок реагирует на шоки чуть сильнее, но общая картина та же: вчерашний шок входит в сегодняшнюю дисперсию с небольшим весом, а основную роль играет инерция. Персистентность 0,996, период полураспада – около 181 торгового дня.

Для сравнения: модель ARCH(1) на тех же данных даёт AIC на 2 494 единицы хуже, чем GARCH(1,1). Один лаг квадрата шока не в состоянии описать длинную память волатильности.

Обратите внимание на безусловную волатильность: по формуле (8.5) она равна 50% годовых, а выборочное стандартное отклонение доходностей за тот же период – 30%. Когда $\alpha + \beta$ близко к единице, знаменатель $1 - \alpha - \beta$ мал, и небольшая ошибка в оценке параметров сильно меняет $\bar{\sigma}^2$. К этому мы вернёмся при прогнозировании.

8.2.4 Одна модель – два пакета

Совпадают ли оценки Python и R? Сведём в таблицу коэффициенты и стандартные ошибки модели, оценённой пакетами arch и rugarch на одних и тех же данных.

Таблица 8.1. GARCH(1,1) с нормальным распределением для IMOEX, 2003–2026 гг.: оценки пакетов arch (Python) и rugarch (R)

	μ	ω	α	β
Оценка, arch	0,0834	0,0382	0,1227	0,8735
Оценка, rugarch	0,0835	0,0382	0,1228	0,8733
Робастная ст. ошибка, arch (по умолчанию)	0,0188	0,0155	0,0252	0,0238
Робастная ст. ошибка, rugarch	0,0195	0,0160	0,0251	0,0235
Обычная ст. ошибка, arch	0,0153	0,0060	0,0088	0,0084
Обычная ст. ошибка, rugarch	0,0153	0,0060	0,0088	0,0084

Оценки коэффициентов совпадают до третьего знака. Небольшие различия возникают из-за двух технических деталей. Первая – начальное значение дисперсии: уравнение (8.4) рекурсивное, и для первого дня нужно задать σ_0^2. Пакет

arch берёт взвешенное среднее квадратов первых наблюдений, rugarch – среднее квадратов шоков по всей выборке. Поэтому условная волатильность первого дня выборки у пакетов разная (1,36 и 1,90% в день), но через несколько недель различие исчезает. Вторая – разные алгоритмы оптимизации. Логарифм правдоподобия у пакетов отличается лишь в первом знаке после запятой: −10 472,15 и −10 472,02.

А вот стандартные ошибки по умолчанию различаются в разы, и это уже не техническая деталь. Метод максимального правдоподобия предполагает, что z_t распределены нормально. У доходностей IMOEX хвосты намного тяжелее, и обычные стандартные ошибки, выведенные в этом предположении, занижены. Робастные стандартные ошибки Боллерслева – Вулдриджа остаются корректными и при неверном распределении z_t. Пакет arch по умолчанию выводит робастные ошибки (`cov_type="robust"`), а rugarch в выдаче `ugarchfit` показывает оба варианта: сначала обычные, ниже, в блоке Robust Standard Errors, – робастные. Если сравнивать значимость коэффициентов по выдаче двух пакетов, легко сравнить разные вещи.

8.2.5 Условная волатильность за два десятилетия

Главный результат модели – ряд условной волатильности $\hat{\sigma}_t$. Переведём его в годовое выражение и сравним со скользящим стандартным отклонением за 63 торговых дня.

```
cond_vol = garch_n.conditional_volatility * np.sqrt(252)   # % годовых
roll_vol = r.rolling(63).std() * np.sqrt(252)

fig, ax = plt.subplots()
ax.plot(cond_vol.index, cond_vol, color=plot.BLUE, lw=0.8, label="GARCH(1,1)")
ax.plot(roll_vol.index, roll_vol, color=plot.ORANGE, lw=0.8, label="Окно 63 дня")
ax.set_ylim(0, 120)          # пик 2022 г. выходит за пределы – он показан на вставке
ax.set_ylabel("% годовых")
ax.legend(loc="upper left")

# Вставка: 2022 г. крупным планом
ins = ax.inset_axes([0.42, 0.52, 0.33, 0.42])
w = slice("2022-01-01", "2022-08-31")
ins.plot(cond_vol.loc[w].index, cond_vol.loc[w], color=plot.BLUE, lw=0.8)
ins.plot(roll_vol.loc[w].index, roll_vol.loc[w], color=plot.ORANGE, lw=0.8)
ins.set_xticks(pd.to_datetime(["2022-02-01", "2022-05-01", "2022-08-01"]))
ins.set_xticklabels(["фев", "май", "авг"])
ins.tick_params(labelsize=7)
plot.source(ax, "Московская биржа, расчёты автора")
plt.show()
```

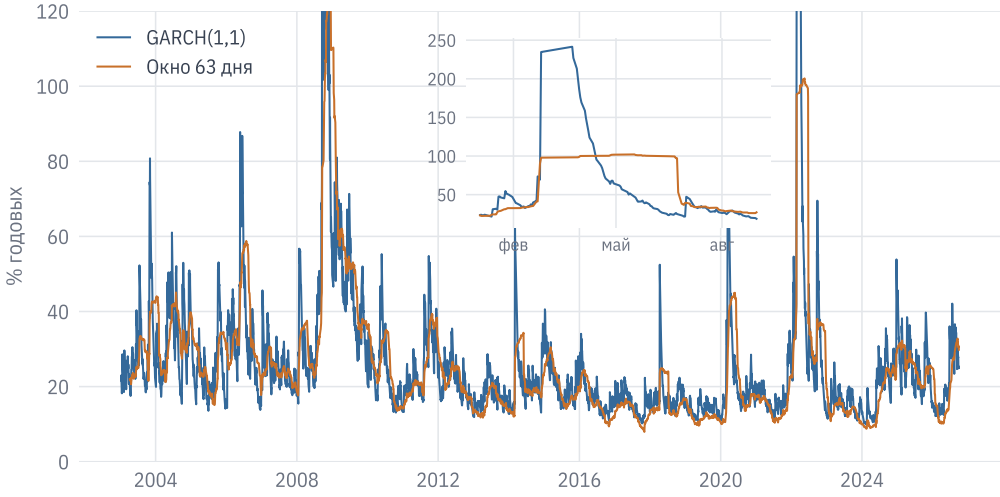


Рисунок 8.1. Условная волатильность IMOEX по модели GARCH(1,1) и скользящая волатильность за 63 дня, % годовых. Вставка – 2022 г.

Два ряда похожи в целом, но различаются в деталях, и детали важны. GARCH реагирует на шок в тот же день, а скользящее окно – постепенно: большая доходность составляет лишь 1/63 часть окна. Зато потом окно помнит шок ровно 63 дня и забывает его в один момент. Во вставке это видно на примере 2022 г.: 24.06.2022, когда доходность 24 февраля вышла из окна, скользящая волатильность за один день упала с 97% до 53%, хотя на рынке в этот день ничего особенного не произошло. Такой скачок называют эффектом призрака (ghost effect). Условная волатильность GARCH снижалась плавно, по мере затухания шока.

За весь период условная волатильность в 90% дней находилась между 13% и 52% годовых, а максимума, 242%, достигла 24.03.2022.

Особенность данных: месяц без торгов

После 25 февраля 2022 г. торги акциями не проводились до 24 марта. В ряду доходностей эти четыре недели превращаются в одно наблюдение: доходность 24 марта – это изменение индекса за месяц. Модель GARCH считает его доходностью одного дня и по уравнению (8.4) переносит в дисперсию следующего дня. Кроме того, дисперсия за время перерыва не «затухает»: для модели между 25 февраля и 24 марта прошёл один шаг. На выводах главы это сказывается мало, но при анализе именно 2022 г. такие эпизоды стоит либо исключать, либо моделировать отдельно.

8.2.6 EWMA как частный случай

До моделей GARCH и сейчас в риск-менеджменте широко используют экспоненциально взвешенную дисперсию (EWMA), популяризованную методикой RiskMetrics (J.P. Morgan/Reuters 1996 г.):

$$\sigma_t^2 = (1-\lambda)\,\varepsilon_{t-1}^2 + \lambda\,\sigma_{t-1}^2, \qquad \lambda = 0{,}94. \tag{8.6}$$

Это GARCH(1,1) с $\omega = 0$, $\alpha = 1-\lambda$ и $\beta = \lambda$, то есть с персистентностью ровно 1. У такой модели нет безусловной дисперсии, к которой возвращается прогноз: сегодняшняя оценка служит прогнозом на любой горизонт. Параметр λ не оценивают, а задают. В обмен на эти ограничения метод не требует оптимизации и одинаково работает для тысяч активов, поэтому его используют в системах расчёта рыночного риска (об этом – в главе о VaR и Expected Shortfall).

8.3 Тяжёлые хвосты и асимметрия

8.3.1 Распределение Стьюдента

В главе «Доходность и риск» мы видели, что распределение доходностей имеет тяжёлые хвосты. Модель GARCH частично их объясняет: смесь спокойных дней с малой дисперсией и бурных с большой даёт тяжёлые хвосты даже при нормальных z_t. Но только частично. Избыточный эксцесс дневных доходностей IMOEX с 2003 г. равен 52 (у нормального распределения – 0). Такое значение создают прежде всего несколько экстремальных дней, в первую очередь 24.02.2022 с доходностью $-40{,}5\%$. У стандартизованных остатков модели $\hat{z}_t = \hat{\varepsilon}_t/\hat{\sigma}_t$ эксцесс снижается до 7,7, но остаётся далёким от нуля.

Поэтому для z_t обычно берут стандартизованное распределение Стьюдента с ν степенями свободы. Чем меньше ν, тем тяжелее хвосты; при $\nu \to \infty$ оно превращается в нормальное. Параметр ν оценивается вместе с остальными.

```
garch_t = arch_model(r, mean="Constant", vol="GARCH", p=1, q=1, dist="t").fit(disp="off")

# Параметры и качество подгонки двух моделей: строки – показатели, столбцы – распределения
comp = pd.DataFrame({"Нормальное": garch_n.params, "Стьюдента": garch_t.params})
comp = comp.reindex(["mu", "omega", "alpha[1]", "beta[1]", "nu"])   # порядок как в уравнениях
comp.loc["Лог-правдоподобие"] = [garch_n.loglikelihood, garch_t.loglikelihood]
comp.loc["BIC"] = [garch_n.bic, garch_t.bic]
# Русская запись чисел: запятая, типографский минус, неразрывный пробел между разрядами
ru = lambda v: "–" if pd.isna(v) else f"{v:,.3f}".replace(",", " ").replace(".", ",").replace("-",
"-")
comp.style.format(ru)
```

	Нормальное	Стьюдента
mu	0,083	0,087
omega	0,038	0,028

	Нормальное	Стьюдента
alpha[1]	0,123	0,107
beta[1]	0,873	0,888
nu	–	5,682
Лог-правдоподобие	−10 472,154	−10 172,873
BIC	20 979,048	20 389,172

Вариант кода на R – в онлайн-версии книги.

Оценка $\nu \approx 5{,}7$: хвосты заметно тяжелее нормальных. Лог-правдоподобие выросло на 299 при одном дополнительном параметре – это огромное улучшение. Параметры уравнения дисперсии изменились мало. Распределение Стьюдента особенно важно там, где интересны именно хвосты: при расчёте VaR и Expected Shortfall нормальное распределение систематически занижает вероятность больших потерь.

```
z = garch_t.std_resid.dropna()
nu = garch_t.params["nu"]
probs = (np.arange(1, len(z) + 1) - 0.5) / len(z)       # уровни эмпирических квантилей
q_emp = np.sort(z)
q_norm = stats.norm.ppf(probs)
# Стандартизованное t: квантили t(v), делённые на его стандартное отклонение √(v/(v-2))
q_t = stats.t.ppf(probs, nu) / np.sqrt(nu / (nu - 2))

fig, axes = plt.subplots(1, 2, figsize=(6.5, 3.2), sharey=True)
for ax, q, title in [(axes[0], q_norm, "Нормальное"), (axes[1], q_t, f"Стьюдента, ν = {nu:.1f}")]:
    ax.scatter(q, q_emp, s=4, color=plot.BLUE)
    lim = [q_emp.min(), q_emp.max()]
    ax.plot(lim, lim, color=plot.LIGHT, lw=0.8, ls="--")   # линия совпадения квантилей
    ax.set_title(title)
    ax.set_xlabel("Теоретические квантили")
axes[0].set_ylabel("Квантили остатков")
plot.source(axes[0], "Московская биржа, расчёты автора", offset=32)
plt.show()
```

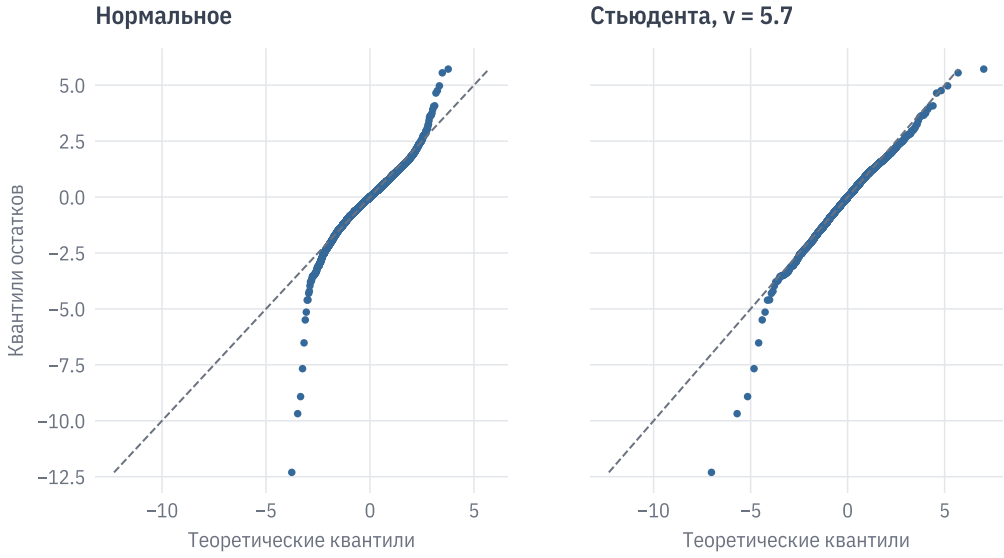


Рисунок 8.2. Квантили стандартизованных остатков GARCH(1,1) с распределением Стьюдента против квантилей нормального распределения и распределения Стьюдента

На левой панели точки на концах уходят от прямой: крупных стандартизованных шоков больше, чем допускает нормальное распределение. На правой они ложатся на прямую почти везде, кроме нескольких крайних точек. Самая левая точка – 24 февраля 2022 г.: даже с учётом повышенной волатильности накануне это событие для модели было почти невероятным.

8.3.2 Асимметрия: падения важнее роста

Модель GARCH(1,1) реагирует на квадрат шока, то есть одинаково на рост и на падение одной величины. На рынках акций это обычно не так: падения повышают волатильность сильнее, чем рост. Одно из объяснений – эффект рычага: при падении цены акции доля долга в капитале компании растёт, и акция становится рискованнее. Другое – поведение инвесторов, которые распродают активы в панике, но не скупают их с тем же азартом.

Модель GJR-GARCH (Glosten и др. 1993 г.) добавляет слагаемое, которое включается только при отрицательном шоке:

$$\sigma_t^2 = \omega + \alpha\varepsilon_{t-1}^2 + \gamma\,\varepsilon_{t-1}^2\,\mathbb{1}[\varepsilon_{t-1} < 0] + \beta\sigma_{t-1}^2. \tag{8.7}$$

После роста дисперсия реагирует с коэффициентом α, после падения – с коэффициентом $\alpha + \gamma$. Если $\gamma > 0$ и значим, асимметрия есть. Персистентность этой модели при симметричном распределении z_t равна $\alpha + \gamma/2 + \beta$.

```
# o=1: одно асимметричное слагаемое (GJR)
gjr_t = arch_model(r, mean="Constant", vol="GARCH", p=1, o=1, q=1, dist="t").fit(disp="off")
```

```
print(gjr_t.summary().tables[2])
#>                              Volatility Model
#> ==============================================================================
#>                  coef    std err          t      P>|t|      95.0% Conf. Int.
#> ------------------------------------------------------------------------------
#> omega          0.0310  6.416e-03      4.834  1.339e-06 [1.844e-02,4.359e-02]
#> alpha[1]       0.0738  9.383e-03      7.870  3.549e-15 [5.545e-02,9.223e-02]
#> gamma[1]       0.0623  1.467e-02      4.244  2.200e-05 [3.351e-02,9.103e-02]
#> beta[1]        0.8869  9.978e-03     88.890      0.000     [  0.867,  0.906]
#> ==============================================================================
```

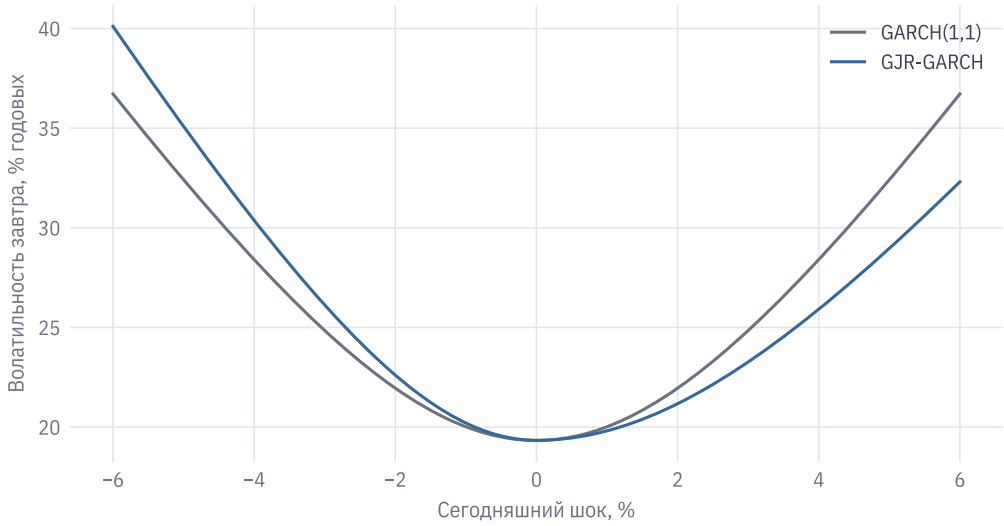
Вариант кода на R – в онлайн-версии книги.

Коэффициент $\gamma = 0{,}062$ значим, и BIC модели GJR лучше, чем у симметричной, на 13. После падения индекса дисперсия реагирует на шок в 1,8 раза сильнее, чем после роста той же величины. Наглядно это показывает кривая влияния новостей (news impact curve): условная дисперсия завтрашнего дня как функция сегодняшнего шока при прочих равных.

```
shock = np.linspace(-6, 6, 241)                          # сегодняшний шок, %
# Прочие равные: вчерашняя дисперсия – на уровне медианы условной дисперсии
sig2_prev = np.median(gjr_t.conditional_volatility**2)
pg, pt = gjr_t.params, garch_t.params

sig2_garch = pt["omega"] + pt["alpha[1]"] * shock**2 + pt["beta[1]"] * sig2_prev
sig2_gjr = (pg["omega"] + (pg["alpha[1]"] + pg["gamma[1]"] * (shock < 0)) * shock**2
            + pg["beta[1]"] * sig2_prev)

fig, ax = plt.subplots()
ax.plot(shock, np.sqrt(sig2_garch * 252), color=plot.GREY, label="GARCH(1,1)")
ax.plot(shock, np.sqrt(sig2_gjr * 252), color=plot.BLUE, label="GJR-GARCH")
ax.set_xlabel("Сегодняшний шок, %")
ax.set_ylabel("Волатильность завтра, % годовых")
ax.legend()
plot.source(ax, "Московская биржа, расчёты автора", offset=32)
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 8.3. Кривая влияния новостей: условная волатильность следующего дня в зависимости от сегодняшнего шока, % годовых

8.3.3 Параметры для разных активов

Посмотрим, насколько эти результаты общие. Оценим GJR-GARCH с распределением Стьюдента для нескольких ликвидных акций, индекса IMOEX и индекса государственных облигаций RGBI. Возьмём период с 2010 г., чтобы история была у всех бумаг.

```
def fit_gjr(prices):
    """GJR-GARCH(1,1) с t-распределением по ряду цен; возвращает ключевые характеристики."""
    ret = np.log(prices.dropna()).diff().dropna() * 100
    res = arch_model(ret, mean="Constant", vol="GARCH", p=1, o=1, q=1, dist="t").fit(disp="off")
    p = res.params
    pers = p["alpha[1]"] + p["gamma[1]"] / 2 + p["beta[1]"]
    return {"α": p["alpha[1]"], "γ": p["gamma[1]"], "β": p["beta[1]"],
            "α + γ/2 + β": pers,
            "Полураспад, дней": np.log(0.5) / np.log(pers) if pers < 0.999 else np.nan,
            "ν": p["nu"]}

ix = data.load_index(["IMOEX", "RGBI"], start="2010-01-01").pivot(index="date", columns="ticker",
values="close")
stocks = data.load_prices(["SBER", "GAZP", "LKOH", "GMKN", "MGNT", "MTSS"], start="2010-01-01")
prices = pd.concat([ix, stocks], axis=1)

tab = pd.DataFrame({t: fit_gjr(prices[t]) for t in prices.columns}).T
# Формат для каждого столбца: три знака у коэффициентов, целые дни полураспада
fmt = {c: "{:.3f}" for c in tab.columns} | {"Полураспад, дней": "{:.0f}", "ν": "{:.1f}"}
tab.style.format(fmt, decimal=",", na_rep="–")
```

Таблица 8.2. Параметры модели GJR-GARCH(1,1) с распределением Стьюдента, дневные доходности 2010–2026 гг.

	α	γ	β	α + γ/2 + β	Полураспад, дней	ν
IMOEX	0,043	0,084	0,892	0,977	30	6,2
RGBI	0,206	0,170	0,709	1,000	–	4,0
GAZP	0,062	0,107	0,858	0,974	26	4,8
GMKN	0,061	0,052	0,889	0,976	29	5,0
LKOH	0,044	0,102	0,878	0,973	25	5,6
MGNT	0,065	0,127	0,814	0,943	12	4,6
MTSS	0,046	0,082	0,903	0,990	70	4,4
SBER	0,065	0,090	0,880	0,990	70	4,8

У всех акций γ положителен (от 0,05 до 0,13): асимметрия – общее свойство рынка, а не особенность индекса. Хвосты у отдельных акций тяжелее, чем у индекса (ν меньше), потому что в индексе специфические шоки компаний частично гасят друг друга.

Сравните персистентность IMOEX в этой таблице с оценкой по периоду с 2003 г. С 2010 г. $\alpha + \gamma/2 + \beta = 0{,}977$ и период полураспада около 30 дней, а с 2003 г. персистентность была заметно ближе к единице. Кризис 2008 г., когда высокая волатильность держалась месяцами, добавляет в оценку «длинную память». Параметры GARCH не являются константами природы: они зависят от выборки, и чем ближе персистентность к единице, тем сильнее.

Индекс облигаций RGBI устроен иначе. Персистентность упирается в единицу (1,000), и период полураспада не определён. Волатильность облигаций определяется режимами денежно-кредитной политики: в период высокой и меняющейся ключевой ставки колебания цен ОФЗ велики, в период стабильной ставки – малы, и такие режимы длятся годами. Модель GARCH(1,1) описывает это как почти бесконечную память. Это сигнал, что для таких рядов нужны другие модели, например с явной сменой режимов.

8.4 Диагностика модели

Если модель верна, стандартизованные остатки $\hat{z}_t = \hat{\varepsilon}_t / \hat{\sigma}_t$ должны быть независимыми и одинаково распределёнными. Значит, ни в них, ни в их квадратах не должно остаться автокорреляции. Проверяем тестом Льюнга – Бокса и тестом ARCH-LM.

```
z = gjr_t.std_resid.dropna()      # стандартизованные остатки: ε / σ
raw = gjr_t.resid.dropna()        # «сырые» остатки: ε = r - μ

pd.DataFrame({
    "Ljung-Box, z":  [acorr_ljungbox(z, lags=[10])["lb_pvalue"].iloc[0]],
```

```
    "Ljung-Box, z²": [acorr_ljungbox(z**2, lags=[10])["lb_pvalue"].iloc[0]],
    "ARCH-LM, z":    [het_arch(z, nlags=5)[1]],
    "ARCH-LM, ε (неверно)": [het_arch(raw, nlags=5)[1]],
}, index=["p-значение"]).T.round(4)
#>                       p-значение
#> Ljung-Box, z                0.1507
#> Ljung-Box, z²               0.5656
#> ARCH-LM, z                  0.1494
#> ARCH-LM, ε (неверно)        0.0000
```

По стандартизованным остаткам оба теста не находят оставшейся зависимости в дисперсии: p-значение ARCH-LM равно 0,15. Модель забрала из квадратов доходностей всю автокорреляцию, которую тесты способны обнаружить.

> **Типичная ошибка: диагностика по «сырым» остаткам**
> Атрибут `res.resid` в пакете arch – это остатки уравнения среднего $\hat{\varepsilon}_t = r_t - \hat{\mu}$, а не стандартизованные остатки. В них по-прежнему есть кластеры волатильности: модель ничего не делает с самими шоками, она описывает их дисперсию. Тест ARCH-LM по `res.resid` даёт p-значение меньше 0,001 и приводит к ложному выводу, что модель «не справилась» и нужно увеличивать число лагов. Проверять нужно `res.std_resid`, то есть $\hat{\varepsilon}_t / \hat{\sigma}_t$. В R стандартизованные остатки возвращает `residuals(fit, standardize = TRUE)`.

8.5 Прогноз волатильности

8.5.1 Прогноз на несколько дней вперёд

Прогноз условной дисперсии GARCH(1,1) на h дней вперёд из момента T строится рекурсивно по уравнению (8.4), в котором будущие ε^2 заменяются их ожиданием σ^2:

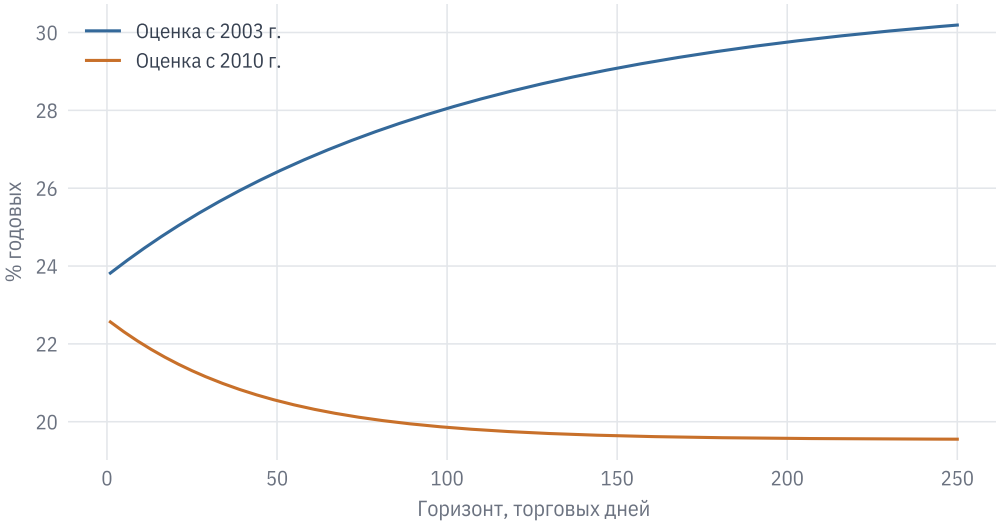
$$\hat{\sigma}^2_{T+h} = \bar{\sigma}^2 + (\alpha + \beta)^{h-1}\left(\hat{\sigma}^2_{T+1} - \bar{\sigma}^2\right). \tag{8.8}$$

Прогноз сходится к безусловной дисперсии $\bar{\sigma}^2$ со скоростью, заданной персистентностью. Если сейчас волатильность ниже среднего уровня, модель ожидает её роста, если выше – снижения. Прогноз волатильности за период из H дней – корень из суммы прогнозов дисперсии на каждый из этих дней.

```
r10 = r.loc["2010-01-01":]
gjr_10 = arch_model(r10, mean="Constant", vol="GARCH", p=1, o=1, q=1, dist="t").fit(disp="off")

fig, ax = plt.subplots()
for res, label, color in [(gjr_t, "Оценка с 2003 г.", plot.BLUE), (gjr_10, "Оценка с 2010 г.",
plot.ORANGE)]:
    # Прогноз из последней даты: дисперсии на 1, 2, ..., 250 дней вперёд
```

```
    var_path = res.forecast(horizon=250, reindex=False).variance.iloc[-1].values
    ax.plot(np.arange(1, 251), np.sqrt(var_path * 252), color=color, label=label)
ax.set_xlabel("Горизонт, торговых дней")
ax.set_ylabel("% годовых")
ax.legend()
plot.source(ax, "Московская биржа, расчёты автора", offset=32)
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 8.4. Прогноз волатильности IMOEX по моделям GJR-GARCH на 250 торговых дней вперёд, % годовых. Оценки по периодам с 2003 и с 2010 г.

Вариант кода на R – в онлайн-версии книги.

На 29.09.2026 условная волатильность индекса – около 24% годовых. Краткосрочные прогнозы двух моделей почти совпадают, потому что определяются текущим состоянием рынка. Долгосрочные расходятся даже по направлению. Модель, оценённая с 2003 г., ожидает роста волатильности: через год – до 30%. Модель, оценённая с 2010 г., ожидает снижения до 20%. Долгосрочный прогноз определяется безусловной дисперсией, то есть тем, сколько кризисов попало в выборку: с 2003 г. в неё входит 2008 г., с 2010 г. – нет. Прогноз GARCH на год вперёд – это по сути ответ на вопрос «какой была средняя волатильность в истории», и к нему нужно относиться как к допущению аналитика, а не как к свойству данных.

8.5.2 Как сравнивать прогнозы волатильности

Проверим прогнозы вне выборки по той же схеме, что в предыдущей главе. Модели оценим на 2003–2019 гг., параметры зафиксируем и будем каждый день 2020–2026 гг. строить прогноз по данным, известным накануне. Сравним четыре

метода: GJR-GARCH с распределением Стьюдента, EWMA с $\lambda = 0,94$, скользящее окно в 63 дня и историческую дисперсию по всем прошлым данным.

Сложность в том, что истинная волатильность ненаблюдаема. Сравнивать прогноз приходится с её зашумлённой оценкой: для прогноза на день – с квадратом доходности r_t^2, для прогноза на месяц – с реализованной дисперсией, суммой квадратов доходностей за следующие 21 торговый день. Patton (2011 г.) показал, что при зашумлённой оценке не всякая функция потерь ранжирует прогнозы правильно. Две функции потерь, которые ранжируют их верно, – среднеквадратичная ошибка по дисперсии и функция QLIKE:

$$L(\hat{\sigma}^2, \mathrm{RV}) = \ln \hat{\sigma}^2 + \frac{\mathrm{RV}}{\hat{\sigma}^2}, \tag{8.9}$$

где $\hat{\sigma}^2$ – прогноз дисперсии, а RV – её реализованная оценка. QLIKE штрафует за занижение волатильности сильнее, чем за завышение, и слабо реагирует на отдельные экстремальные наблюдения. Среднеквадратичная ошибка, напротив, почти целиком определяется несколькими днями с наибольшими доходностями.

```
split, H = "2019-12-31", 21
test_idx = r.loc["2020-01-01":].index

# GJR-GARCH: параметры по данным до split (last_obs), прогнозы из каждой даты после split.
# Строка с датой t содержит прогноз на t+1, ..., t+H по данным до t включительно.
gjr_is = arch_model(r, mean="Constant", vol="GARCH", p=1, o=1, q=1, dist="t").fit(
    last_obs=split, disp="off")
fvar = gjr_is.forecast(start=split, horizon=H, reindex=False).variance

fc_1 = {"GJR-GARCH": fvar["h.01"]}                # прогноз дисперсии на день
fc_H = {"GJR-GARCH": fvar.sum(axis=1)}            # прогноз суммарной дисперсии за H дней

# EWMA (RiskMetrics): оценка на t – прогноз на t+1 и на каждый из следующих дней
eps2 = (r - r.loc[:split].mean()) ** 2
ewma = eps2.ewm(alpha=1 - 0.94, adjust=False).mean()
roll = r.rolling(63).var()                         # скользящее окно 63 дня
hist = r.expanding().var()                         # вся доступная история
for name, v in {"EWMA": ewma, "Окно 63 дня": roll, "Историческая": hist}.items():
    fc_1[name], fc_H[name] = v, v * H

# Реализованные значения: r² дня t и сумма r² за дни t, ..., t+H-1
r2 = r**2
rv_H = r2.rolling(H).sum().shift(-(H - 1))

def evaluate(mask):
    rows = {}
    for name in fc_1:
        # прогноз для дня t сделан накануне: сдвигаем на один торговый день
        f1 = fc_1[name].shift(1).reindex(test_idx)[mask]
        fH = fc_H[name].shift(1).reindex(test_idx)[mask]
```

```
        real1, realH = r2.reindex(test_idx)[mask], rv_H.reindex(test_idx)[mask]
        ok = realH.notna()
        rows[name] = {
            "QLIKE, 1 день": np.mean(np.log(f1) + real1 / f1),
            "QLIKE, 21 день": np.mean(np.log(fH[ok]) + realH[ok] / fH[ok]),
             "RMSE, 21 день": np.sqrt(np.mean((np.sqrt(realH[ok] / H * 252) - np.sqrt(fH[ok] / H *
252)) ** 2)),
        }
    return pd.DataFrame(rows).T

all_days = np.ones(len(test_idx), dtype=bool)
no_2022 = test_idx.year != 2022
oos = pd.concat({"2020–2026": evaluate(all_days), "без 2022 г.": evaluate(no_2022)}, axis=1)
oos.style.format("{:.2f}", decimal=",")
```

Таблица 8.3. Прогноз волатильности IMOEX вне выборки, 2020–2026 гг. QLIKE – средние потери (меньше – лучше), RMSE – среднеквадратичная ошибка прогноза волатильности на 21 день, п. п. годовых

	2020–2026			без 2022 г.		
	QLIKE, 1 день	QLIKE, 21 день	RMSE, 21 день	QLIKE, 1 день	QLIKE, 21 день	RMSE, 21 день
GJR-GARCH	1,52	4,85	21,03	1,26	4,48	8,38
EWMA	1,59	5,01	20,98	1,30	4,64	8,81
Окно 63 дня	1,80	5,22	22,09	1,43	4,78	9,86
Историче-ская	2,23	5,29	21,09	1,78	4,84	15,10

По QLIKE на обоих горизонтах лучше всех прогнозирует модель GJR-GARCH, за ней EWMA, затем скользящее окно, а историческая дисперсия – хуже всех. Модели, которые быстро реагируют на новую информацию, выигрывают у тех, кто усредняет длинную историю.

Среднеквадратичная ошибка за весь период рисует другую картину: все методы оказываются примерно одинаковыми, и даже историческая дисперсия почти не уступает GARCH (21,1 против 21,0 п. п.). Причина – февраль и март 2022 г. Реализованная волатильность этих недель в разы превышает любой прогноз, и квадраты этих ошибок заглушают всё остальное. Без 2022 г. порядок восстанавливается: GJR-GARCH – 8,4 п. п., EWMA – 8,8, окно – 9,9, историческая дисперсия – 15,1.

Вывод согласуется с большим исследованием Hansen и Lunde (2005 г.), где сотни моделей волатильности сравнивались на валютных курсах и акциях. Модели сложнее GARCH(1,1) редко дают заметное преимущество, но и сама GARCH(1,1) заметно лучше простых правил, которые не учитывают, как быстро затухают шоки.

Типичная ошибка: прогноз из одной точки вместо прогноза на каждый день
Та же ловушка, что в предыдущей главе, встречается и здесь. Вызов `res.forecast(horizon=len(test))` у модели, оценённой на обучающей выборке, даёт *один* прогноз из последней обучающей даты на все дни теста вперёд. Он плавно сходится к безусловной дисперсии и не содержит информации о том, что происходило в тестовом периоде. Для проверки нужны прогнозы на день вперёд из каждой даты: в пакете arch – `fit(last_obs=...)` и `forecast(start=...)`, как в коде выше.

Вторая ошибка связана с интерпретацией: полоса $\hat{\mu} \pm \hat{\sigma}$ вокруг прогноза доходности – это не 95%-й доверительный интервал. При нормальном распределении она накрывает около 68% дней, а для 95% нужно $\pm 1,96\hat{\sigma}$. При распределении Стьюдента множитель зависит от ν.

Итоги

- Дисперсия доходностей меняется во времени и зависит от прошлых шоков. Тест ARCH-LM и тест Льюнга – Бокса для квадратов доходностей это подтверждают.
- В модели GARCH(1,1) сегодняшняя дисперсия складывается из константы, реакции на вчерашний шок (α) и инерции (β). Персистентность $\alpha + \beta$ определяет, как быстро затухают шоки волатильности; для IMOEX она близка к единице.
- EWMA – частный случай GARCH(1,1) без возврата к среднему уровню.
- Распределение Стьюдента для стандартизованных шоков заметно лучше нормального. Модель GJR-GARCH учитывает, что падения повышают волатильность сильнее роста; для российских акций асимметрия значима.
- Параметры GARCH, особенно безусловная дисперсия, зависят от выборки, и долгосрочный прогноз волатильности отражает прежде всего число кризисов в истории.
- Диагностику проводят по стандартизованным остаткам $\hat{\varepsilon}_t/\hat{\sigma}_t$, а не по остаткам уравнения среднего.
- Прогнозы волатильности сравнивают вне выборки с устойчивыми к шуму функциями потерь, например QLIKE. GARCH превосходит EWMA, скользящее окно и историческую дисперсию, но экстремальные эпизоды вроде 2022 г. не предсказывает ни одна модель.

8.6 Упражнения

Упражнения разделены на четыре типа. Задания **А** проверяют понимание и решаются без компьютера. Задания **Б** выполняются на данных вашей акции (тикер назначает преподаватель), поэтому готового ответа у них нет. Задания **В** учат работать с ИИ-ассистентом: не получать ответ, а проверять его. Задание **Г** – небольшое исследование, результаты которого нужно защитить устно.

8.6.1 А. Понимание

1. В модели GARCH(1,1) $\omega = 0,02$, $\alpha = 0,08$, $\beta = 0,90$ (доходности в процентах). Найдите безусловную дневную и годовую волатильность и период полураспада. Вчера условная дисперсия была 1,5, а доходность отклонилась от среднего на -3%. Чему равна условная дисперсия сегодня?
2. Как изменится ответ на вопрос 1 для модели GJR-GARCH с теми же параметрами и $\gamma = 0,06$? А если вчерашний шок был $+3\%$?
3. Почему EWMA можно считать моделью GARCH(1,1)? Какие ограничения на параметры это накладывает и к чему сходится прогноз EWMA на длинном горизонте?
4. Объясните, почему тест ARCH-LM по остаткам уравнения среднего почти всегда отвергает нулевую гипотезу даже для хорошей модели GARCH.
5. Модель GARCH оценили на двух выборках и получили персистентность 0,97 и 0,995. Во сколько раз различаются периоды полураспада? Как это скажется на прогнозе волатильности на год вперёд?

8.6.2 Б. Практика на своей акции

6. Проверьте наличие ARCH-эффекта в доходностях своей акции тестом ARCH-LM. Оцените GARCH(1,1) с нормальным распределением и распределением Стьюдента. Сравните модели по BIC и сравните оценку ν с оценкой для IMOEX.
7. Оцените GJR-GARCH для своей акции. Значим ли γ? Постройте кривую влияния новостей и сравните её с кривой для IMOEX.
8. Постройте график условной волатильности своей акции и скользящей волатильности за 63 дня. Найдите эпизод, где заметен эффект призрака, и объясните его.
9. Повторите сравнение прогнозов вне выборки из раздела 8.5 для своей акции. Совпадает ли ранжирование методов с результатом для индекса?

8.6.3 В. Работа с ИИ-ассистентом

10. Попросите ИИ-ассистента написать код, который оценивает GARCH(1,1) на доходностях IMOEX, проверяет модель и строит прогноз волатильности на 2020–2026 гг. Проверьте: по каким остаткам проводится диагностика; в каких единицах доходности; как построен прогноз – из одной точки или на каждый день; как переводится дневная волатильность в годовую. Запишите найденные ошибки.
11. Спросите ассистента, какие значения α и β «типичны» для модели GARCH(1,1) на рынке акций и откуда эти цифры. Сравните с оценками главы для российских акций. Если ассистент ссылается на источник, проверьте, существует ли он и говорится ли там это.

8.6.4 Г. Мини-исследование

12. Предсказывает ли условная волатильность GARCH будущую доходность? Разбейте дни 2010–2026 гг. на пять групп по условной волатильности IMOEX, известной накануне, и посчитайте среднюю доходность и коэффициент Шарпа индекса в следующем месяце для каждой группы. Есть ли премия за риск в периоды высокой волатильности? Сколько независимых эпизодов высокой волатильности у вас на самом деле? Подготовьте короткий (3–5 минут) устный рассказ.

Вопросы для самопроверки

1. Чем условная дисперсия отличается от безусловной? Может ли стационарный ряд иметь меняющуюся условную дисперсию?
2. Что измеряют параметры α и β в модели GARCH(1,1)? Что такое персистентность?
3. Почему для моделей GARCH обычно используют распределение Стьюдента, а не нормальное?
4. Что показывает коэффициент γ в модели GJR-GARCH?
5. Какие остатки нужно проверять после оценки модели GARCH и какими тестами?
6. К чему сходится прогноз волатильности GARCH на длинном горизонте и почему он чувствителен к выбору выборки?
7. Почему для сравнения прогнозов волатильности используют функцию QLIKE, а не только среднеквадратичную ошибку?

Ответы

1. Условная дисперсия – дисперсия доходности при условии информации к предыдущему дню, безусловная – средний уровень за всё время. Да: ряд GARCH с $\alpha + \beta < 1$ стационарен, хотя его условная дисперсия меняется каждый день.
2. α – реакция дисперсии на вчерашний шок, β – инерция дисперсии. Персистентность $\alpha + \beta$ – доля отклонения дисперсии от среднего уровня, сохраняющаяся на следующий день.
3. Даже после учёта меняющейся волатильности стандартизованные шоки имеют тяжёлые хвосты; нормальное распределение занижает вероятность крупных движений, что особенно важно для VaR и ES.
4. Насколько сильнее дисперсия реагирует на отрицательный шок, чем на положительный той же величины (асимметрия, эффект рычага).
5. Стандартизованные остатки $\hat{\varepsilon}_t/\hat{\sigma}_t$ и их квадраты: тест Льюнга – Бокса, тест ARCH-LM, QQ-график.
6. К безусловной дисперсии $\omega/(1-\alpha-\beta)$. При персистентности, близкой к единице, знаменатель мал, и небольшие изменения оценок (например, из-за включения кризисного года) сильно меняют уровень.

7. Истинная волатильность ненаблюдаема, её заменяют зашумлённой оценкой (квадратом доходности или реализованной дисперсией). QLIKE ранжирует прогнозы правильно даже при таком шуме и слабо зависит от отдельных экстремальных наблюдений, тогда как среднеквадратичную ошибку определяют несколько крупнейших дней.

Что почитать

- Engle (1982 г.) и Bollerslev (1986 г.) – исходные работы о моделях ARCH и GARCH.
- Tsay (2010 г.), гл. 3 – модели условной гетероскедастичности с примерами на финансовых данных.
- Alexander (2008 г.), гл. II.4 – модели GARCH с точки зрения практика риск-менеджмента.
- Glosten и др. (1993 г.) – модель с асимметричной реакцией волатильности на падения и рост.
- Hansen и Lunde (2005 г.) – сравнение сотен моделей волатильности с GARCH(1,1).
- Patton (2011 г.) – как сравнивать прогнозы волатильности, когда истинная волатильность ненаблюдаема.

9 Прогнозирование и машинное обучение

Представьте, что студент приносит на семинар модель градиентного бустинга для дневной доходности индекса Мосбиржи. 15 признаков: лаги доходности, скользящие средние, волатильность, индекс относительной силы, оборот торгов, день недели. Модель проверена честно, на кросс-валидации для временных рядов, где каждый тестовый период идёт после обучающего. Результат на 4 910 днях вне выборки: $R^2 = 0{,}59$, знак доходности угадан в 86% дней. Если бы это было правдой, автор модели очень быстро стал бы очень богатым человеком.

Это неправда. В предыдущих главах мы видели, что линейная автокорреляция дневных доходностей индекса близка к нулю, а простые модели вне выборки не превосходят историческое среднее. Нелинейная модель могла бы найти то, что пропустили линейные, но не в таком масштабе. Ошибка в данных: несколько признаков посчитаны по окну, которое включает тот самый день, доходность которого модель должна предсказать. Модель не прогнозирует, а вычисляет ответ из подсказки.

Эта глава о том, как применять машинное обучение к финансовым рядам и не обмануть себя. Методы машинного обучения сами по себе не сложнее эконометрики, и современные библиотеки делают их применение делом нескольких строк. Трудность в другом: финансовые данные упорядочены во времени, сигнала в них мало, а шума много. Поэтому главная часть работы – правильно поставить задачу, построить признаки без заглядывания в будущее и проверить модель так, чтобы проверка имитировала реальное использование.

В этой главе

- Ставим задачу прогноза: что известно в момент прогноза, что прогнозируем, с чем сравниваем.
- Разбираем утечку данных на примере модели, которая «угадывает» 86% дней.
- Строим валидацию во времени: расширяющееся и скользящее окно, очистка обучения при пересекающихся целях.

- Сравниваем линейную модель и градиентный бустинг в прогнозе доходности и волатильности.
- Выясняем, почему стратегия на прогнозах модели может обыгрывать рынок на бумаге и не обыгрывать в реальности.

Данные: `indexes.parquet` (см. данные книги). **Нужно знать:** прогноз вне выборки и R^2_{OOS} (глава «Свойства финансовых рядов и модели ARMA»), модели волатильности (глава «Волатильность: модели ARCH и GARCH»), основы регрессии.

В главе используются пакеты scikit-learn и LightGBM.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import lightgbm as lgb
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.model_selection import TimeSeriesSplit, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from qf import data, plot

# Числа в таблицах: запятая, типографский минус
ru = lambda v, d=3: f"{v:.{d}f}".replace(".", ",").replace("-", "−")
```

9.1 Постановка задачи прогноза

9.1.1 Что известно и что прогнозируем

Любую задачу прогноза нужно начинать с временной оси. Пусть торги дня t закончились, и мы знаем цены закрытия, обороты и всё, что произошло до этого момента. Это информационное множество $\mathcal{F}_t$. Прогноз доходности следующего дня r_{t+1} может использовать только то, что входит в $\mathcal{F}_t$:

$$\hat{r}_{t+1} = f(x_t), \qquad x_t \in \mathcal{F}_t. \tag{9.1}$$

В таблице данных это означает, что в строке с датой t стоят признаки x_t, посчитанные по данным до конца дня t включительно, а целевая переменная – доходность *следующего* дня. В pandas цель получают сдвигом назад: `y = r.shift(-1)`. Если вместо этого в строке t стоит доходность дня t, то любой признак, использующий цену закрытия дня t, уже содержит ответ.

Сначала предскажите

Скользящее среднее доходности за 5 дней `r.rolling(5).mean()` в строке t – это среднее каких дней? Можно ли использовать его как признак для прогноза r_t? А для прогноза r_{t+1}?

9.1.2 С чем сравнивать

Качество прогноза имеет смысл только по сравнению с простым эталоном. Для дневных доходностей естественные эталоны – нулевой прогноз и историческое среднее. Меры качества те же, что в главе о моделях ARMA:

- R^2 вне выборки относительно эталона: положительный – модель лучше эталона, отрицательный – хуже;
- доля верно угаданных знаков. Её нужно сравнивать не с 50%, а с долей дней роста: модель, которая всегда предсказывает рост, угадает знак во всех растущих днях.

Для прогноза волатильности эталоном служит последнее наблюдённое значение или модель GARCH.

9.2 Утечка данных

9.2.1 Как модель «угадала» 85% дней

Вернёмся к модели из начала главы. Она воспроизводит признаки из прошлогоднего учебного ноутбука курса: цель – доходность дня t, признаки – лаги доходности, скользящие средние `ma5`, `ma10`, `ma20`, скользящие стандартные отклонения, индекс относительной силы (RSI), оборот торгов, день недели и месяц. Лаги сдвинуты правильно, а скользящие статистики – нет: `r.rolling(5).mean()` в строке t – это среднее доходностей дней с $t-4$ по t, то есть включает цель.

Насколько серьёзна такая подсказка, показывает обычная линейная регрессия. Скользящее среднее за 5 дней и четыре лага содержат цель точно:

$$r_t = 5 \cdot \mathrm{ma5}_t - r_{t-1} - r_{t-2} - r_{t-3} - r_{t-4}. \tag{9.2}$$

```
# Признаки с утечкой: скользящее среднее включает день t, цель – доходность дня t
imoex = data.load_index("IMOEX", start="2003-01-01").set_index("date")
r = np.log(imoex["close"]).diff()

leaky = pd.DataFrame({"y": r, "ma5": r.rolling(5).mean()})
for k in range(1, 5):
    leaky[f"lag{k}"] = r.shift(k)
leaky = leaky.dropna()

ols = LinearRegression().fit(leaky.drop(columns="y"), leaky["y"])
print("R² =", round(ols.score(leaky.drop(columns="y"), leaky["y"]), 6))
#> R² = 1.0
print("Коэффициенты:", ols.coef_.round(3))
#> Коэффициенты: [ 5. -1. -1. -1. -1.]
```

Вариант кода на R – в онлайн-версии книги.

R^2 равен единице, коэффициенты – ровно 5 и -1. Бустинг не знает этого тождества, но быстро находит его приближённо, как и зависимость цели от

RSI и скользящей волатильности, посчитанных по тем же окнам. Отсюда и 86% угаданных знаков.

Типичная ошибка: признак, который знает ответ

Утечка данных (data leakage) – использование при обучении или проверке информации, которой не было бы в момент прогноза. Самые частые формы в финансовых задачах:

1. **Скользящее окно включает цель.** `rolling(...)` в строке t включает наблюдение t. Если цель – r_t, признак нужно сдвинуть: `r.rolling(5).mean().shift(1)`. Если цель – r_{t+1} (как в формуле (9.1)), сдвигать признаки не нужно, сдвигается цель.
2. **Данные того же дня.** Оборот, максимум и минимум дня t становятся известны только после закрытия. Для прогноза r_t их использовать нельзя.
3. **Масштабирование по всей выборке.** `StandardScaler` или `MinMaxScaler`, обученный на всех данных до разбиения, передаёт в обучающую часть среднее и разброс тестовой. Масштабирование – часть модели, и его параметры оцениваются только на обучающих данных (в scikit-learn это делает `Pipeline`).
4. **Подбор модели на тесте.** Ранняя остановка бустинга по ошибке на тестовом фолде, выбор гиперпараметров или признаков по результату на тесте превращают тест в ещё одну обучающую выборку.
5. **Случайное разбиение.** `train_test_split` по умолчанию перемешивает наблюдения, и модель учится на будущем, чтобы «предсказать» прошлое (см. раздел 9.3).

Главный признак утечки – результат, который слишком хорош. Прежде чем радоваться $R^2 = 0,6$ на дневных доходностях, ищите ошибку.

9.3 Валидация во времени

9.3.1 Почему не подходит обычная кросс-валидация

Обычная k-кратная кросс-валидация делит данные на k частей случайным образом, обучает модель на $k-1$ частях и проверяет на оставшейся. Для независимых наблюдений это разумно. Для временного ряда – нет, по двум причинам. Во-первых, модель обучается на данных, которые идут *после* тестовых, а в реальности будущее недоступно. Во-вторых, соседние наблюдения временного ряда похожи, и если одно попало в обучение, а соседнее – в тест, модель проверяется почти на тех же данных, на которых училась.

Вместо этого используют схемы, в которых тест всегда идёт после обучения.

- **Расширяющееся окно** (expanding window): модель обучается на всех данных до начала теста; с каждым шагом обучающая выборка растёт. Так устроен `TimeSeriesSplit` в scikit-learn.

- **Скользящее окно** (sliding window): обучающая выборка фиксированной длины сдвигается вперёд. Это уместно, если старые данные устаревают, например после смены рыночного режима.
- **Разрыв перед тестом** (`gap`): из конца обучения удаляют наблюдения, цели которых ещё не известны на момент начала теста или пересекаются с его целями. Это очистка (purging). Эмбарго в более общей схеме дополнительно исключает наблюдения после тестового блока, если обучение включает более поздние даты (López de Prado 2018 г.).

```
n, k = 100, 4
schemes = {}
rng = np.random.default_rng(3)
folds = rng.permutation(n) % k
schemes["Случайные\nфолды"] = [(np.where(folds ≠ i)[0], np.where(folds == i)[0], []) for i in
range(k)]
test_len = 16
starts = [36, 52, 68, 84]
schemes["Расширяющееся\nокно"] = [(np.arange(0, s), np.arange(s, s + test_len), []) for s in starts]
schemes["Скользящее\nокно"] = [(np.arange(s - 36, s), np.arange(s, s + test_len), []) for s in starts]
schemes["С разрывом"] = [(np.arange(0, s - 5), np.arange(s, s + test_len), np.arange(s - 5, s)) for
s in starts]

fig, axes = plt.subplots(len(schemes), 1, figsize=(6.5, 4.6), sharex=True)
for ax, (name, splits) in zip(axes, schemes.items()):
    for row, (tr_i, te_i, gap_i) in enumerate(splits):
        ax.scatter(tr_i, [row] * len(tr_i), marker="s", s=6, color=plot.BLUE, lw=0)
        ax.scatter(te_i, [row] * len(te_i), marker="s", s=6, color=plot.ORANGE, lw=0)
        ax.scatter(gap_i, [row] * len(gap_i), marker="s", s=6, color=plot.GREY, lw=0)
    ax.set_yticks([])
    ax.set_ylabel(name, rotation=0, ha="right", va="center", fontsize=8)
    ax.grid(False)
axes[-1].set_xlabel("Время →")
plt.show()
```

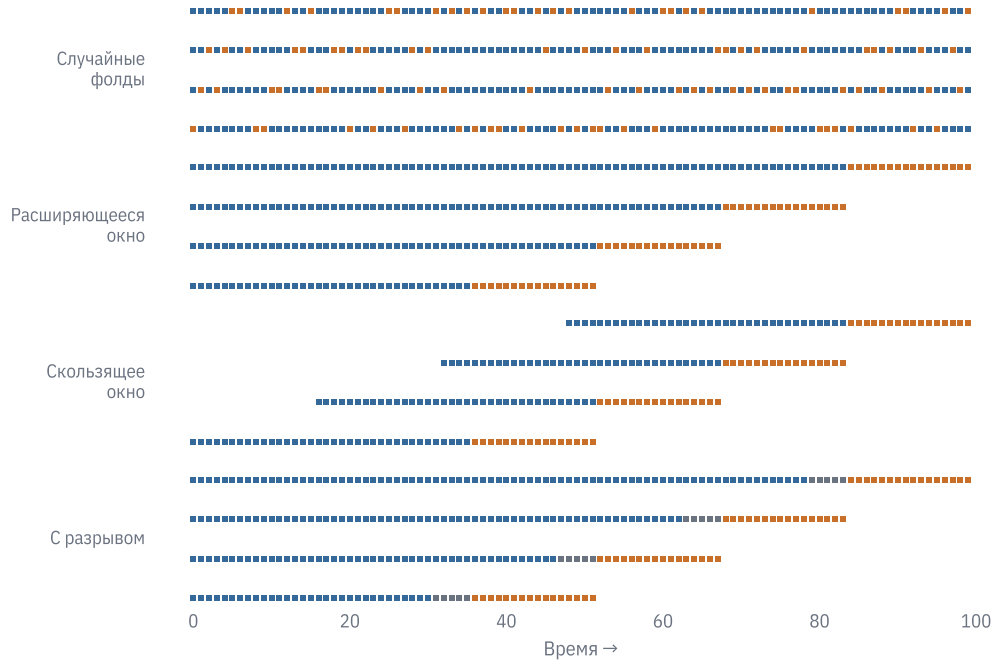


Рисунок 9.1. Схемы валидации: случайное разбиение, расширяющееся окно, скользящее окно и расширяющееся окно с разрывом. Синий – обучение, оранжевый – тест, серый – исключённые наблюдения

9.3.2 Насколько обманывает перемешивание

Разница между схемами становится драматической, когда цели перекрываются. Возьмём в качестве цели доходность индекса за следующие 20 торговых дней. Цели двух соседних дней совпадают на 19 дней из 20. При случайном разбиении у каждого тестового наблюдения есть почти точная копия его цели в обучающей выборке, и модель, запомнившая обучающие данные, «предскажет» тест.

```
# Признаки по данным до конца дня t (включительно)
def make_features(close, value):
    r = np.log(close).diff()
    f = pd.DataFrame(index=close.index)
    for k in range(5):
        f[f"r_lag{k}"] = r.shift(k)                    # r_t, r_{t-1}, ..., r_{t-4}
    for w in (5, 10, 20, 60):
        f[f"mom{w}"] = r.rolling(w).sum()              # доходность за w дней
        f[f"vol{w}"] = r.rolling(w).std()              # волатильность за w дней
    f["rsi"] = rsi_of(close)                           # индекс относительной силы по ценам
    log_value = np.log(value.replace(0, np.nan))
    f["dvalue"] = log_value - log_value.rolling(20).mean()   # оборот относительно среднего
    f["dow"] = f.index.dayofweek
    return f
```

```
def rsi_of(x, n=14):
    """Индекс относительной силы: средний рост к среднему падению за n дней, шкала 0–100."""
    d = x.diff()
    up, down = d.clip(lower=0).rolling(n).mean(), (-d.clip(upper=0)).rolling(n).mean()
    return 100 - 100 / (1 + up / down)

feats = make_features(imoex["close"], imoex["value_rub"])
r = np.log(imoex["close"]).diff()
# Цель: доходность за 20 следующих дней, r_{t+1} + ... + r_{t+20}
fwd20 = r[::-1].rolling(20).sum()[::-1].shift(-1).rename("y")
d20 = pd.concat([feats, fwd20], axis=1).dropna().loc["2005":]
X20, y20 = d20.drop(columns="y"), d20["y"]

def cv_r2(splitter, embargo=0):
    """R² прогноза, собранного по тестовым фолдам; embargo – разрыв в днях до и после теста."""
    pred = pd.Series(np.nan, index=y20.index)
    for tr_i, te_i in splitter.split(X20):
        if embargo:
            tr_i = tr_i[(tr_i < te_i.min() - embargo) | (tr_i > te_i.max() + embargo)]
        m = lgb.LGBMRegressor(**gb_params).fit(X20.iloc[tr_i], y20.iloc[tr_i])
        pred.iloc[te_i] = m.predict(X20.iloc[te_i])
    ok = pred.notna()
    return 1 - ((y20[ok] - pred[ok]) ** 2).sum() / ((y20[ok] - y20[ok].mean()) ** 2).sum()

cv_tab = pd.Series({
    "Случайные фолды (перемешивание)": cv_r2(KFold(5, shuffle=True, random_state=0)),
    "Расширяющееся окно": cv_r2(TimeSeriesSplit(5)),
    "Расширяющееся окно, разрыв 20 дней": cv_r2(TimeSeriesSplit(5), embargo=20),
}, name="R² вне выборки")
cv_tab.to_frame().style.format(lambda v: ru(v, 2))
```

	R² вне выборки
Случайные фолды (перемешивание)	0,31
Расширяющееся окно	−0,29
Расширяющееся окно, разрыв 20 дней	−0,30

При перемешивании модель показывает $R^2 = 0{,}31$: как будто месячная доходность индекса предсказуема на треть. При проверке во времени $R^2 = -0{,}29$, то есть модель хуже простого среднего. Ни одна из этих цифр не говорит, что признаки бесполезны или полезны, – только первая говорит неправду. Разрыв здесь почти не меняет результат, потому что в `TimeSeriesSplit` обучающая выборка и так целиком предшествует тесту; перекрываются лишь последние 20 целей перед тестом. При схемах с обучающими данными до и после теста нужно отдельно исключать пересекающиеся цели (purging) и учитывать зависимость признаков после теста (embargo).

9.4 Прогноз доходности: линейная модель и бустинг

9.4.1 Модели

Сравним две модели разной природы.

Гребневая регрессия (ridge regression) – линейная регрессия со штрафом за размер коэффициентов:

$$\hat{\beta} = \arg\min_{\beta} \sum_{t} (y_t - x_t'\beta)^2 + \lambda \|\beta\|^2. \tag{9.3}$$

Штраф λ стягивает коэффициенты к нулю. Когда сигнал слаб, а признаков много, это защищает от переобучения: модель не подстраивается под случайные совпадения. Признаки перед оценкой приводят к одному масштабу.

Градиентный бустинг строит прогноз как сумму сотен небольших деревьев решений. Каждое новое дерево обучается на ошибках суммы предыдущих, а вклад каждого дерева умножается на малый шаг обучения. Деревья сами находят нелинейности и взаимодействия признаков, например что после сильного падения при высокой волатильности рынок ведёт себя иначе, чем при низкой. Библиотека LightGBM – одна из самых быстрых реализаций. Главные гиперпараметры: шаг обучения (`learning_rate`), число деревьев (`n_estimators`), размер дерева (`num_leaves`) и минимальное число наблюдений в листе (`min_data_in_leaf`). Чем крупнее деревья и чем меньше наблюдений в листе, тем гибче модель и тем легче она запоминает шум.

9.4.2 Проверка на 2012–2026 гг.

Цель – доходность индекса на следующий день. Признаки – те же, что в предыдущем разделе, но теперь все они посчитаны по данным до конца дня t, а цель сдвинута на день вперёд. Каждый год с 2012 по 2026 г. модели заново обучаются на всех предыдущих годах (расширяющееся окно) и прогнозируют доходности текущего года.

```
d1 = pd.concat([feats, r.shift(-1).rename("y")], axis=1).dropna().loc["2005":]

preds = {"Историческое среднее": [], "Гребневая регрессия": [], "LightGBM": []}
actual = []
for year in range(2012, 2027):
    train = d1.loc[: f"{year - 1}-12-31"]
    test = d1.loc[f"{year}-01-01": f"{year}-12-31"]
    X_tr, y_tr, X_te = train.drop(columns="y"), train["y"], test.drop(columns="y")
    # Масштабирование внутри Pipeline: среднее и разброс – только по обучающим данным
    ridge = make_pipeline(StandardScaler(), Ridge(alpha=100)).fit(X_tr, y_tr)
    booster = lgb.LGBMRegressor(**gb_params).fit(X_tr, y_tr)
    preds["Историческое среднее"].append(pd.Series(y_tr.mean(), index=test.index))
    preds["Гребневая регрессия"].append(pd.Series(ridge.predict(X_te), index=test.index))
    preds["LightGBM"].append(pd.Series(booster.predict(X_te), index=test.index))
```

```
    actual.append(test["y"])

y_oos = pd.concat(actual)
pred_oos = {name: pd.concat(p) for name, p in preds.items()}
sse = lambda p: ((y_oos - p) ** 2).sum()
ret_tab = pd.DataFrame({
    name: {"R² против нуля": 1 - sse(p) / (y_oos ** 2).sum(),
           "R² против среднего": 1 - sse(p) / sse(pred_oos["Историческое среднее"]),
           "Доля угаданных знаков": (np.sign(p) == np.sign(y_oos)).mean()}
    for name, p in pred_oos.items()}).T
ret_tab.style.format(ru)
```

Таблица 9.1. Прогноз дневной доходности IMOEX вне выборки, 2012–2026 гг., переобучение раз в год

	R² против нуля	R² против среднего	Доля угаданных знаков
Историческое среднее	−0,001	0,000	0,520
Гребневая регрессия	−0,015	−0,014	0,498
LightGBM	−0,044	−0,044	0,519

Обе модели проигрывают даже нулевому прогнозу: R^2 гребневой регрессии – −0,015, бустинга – −0,044. Бустинг проигрывает сильнее: гибкая модель находит в обучающих данных больше закономерностей, которые оказываются шумом. Доля угаданных знаков у бустинга, 51,9%, почти совпадает с долей дней роста, 52,0%, – того же добился бы прогноз «завтра рост».

Это ожидаемый результат, а не неудача метода. Gu и др. (2020 г.) показали, что машинное обучение действительно помогает прогнозировать доходности американских акций. Но это прогноз в *поперечном* разрезе: какие из тысяч акций в следующем месяце вырастут сильнее других. Предсказать, куда пойдёт весь рынок завтра, гораздо труднее, и даже в лучших работах R^2 вне выборки измеряется долями процента.

9.5 Где сигнал есть: прогноз волатильности

В главе о GARCH мы видели, что волатильность, в отличие от доходности, хорошо предсказуема. Здесь у машинного обучения больше шансов. Будем прогнозировать среднюю дисперсию доходности на следующие 5 торговых дней. Её зашумлённая оценка – средний квадрат дневной доходности за эти дни.

Стандартный эталон – модель HAR (heterogeneous autoregressive) (Corsi 2009 г.). В оригинале HAR использует реализованную волатильность из внутридневных данных. Здесь мы проверяем упрощённую модель с той же структурой горизонтов: вместо неё используем шумный дневной показатель на основе квадратов доходностей. Модель объясняет будущую дисперсию тремя средними квадратов прошлых доходностей: за день, неделю и месяц:

$$\ln \mathrm{RV}_{t+1:t+5} = b_0 + b_d \ln \mathrm{RV}_t + b_w \ln \mathrm{RV}_{t-4:t} + b_m \ln \mathrm{RV}_{t-21:t} + u_t. \qquad (9.4)$$

Идея модели в том, что на рынке действуют участники с разными горизонтами: внутридневные спекулянты, недельные трейдеры, месячные инвесторы. Каждый реагирует на волатильность своего горизонта. Три средних с разными окнами дают длинную память, как у GARCH, но оцениваются обычной регрессией. Сравним HAR с бустингом на тех же трёх признаках и на расширенном наборе, куда добавлены доходность за неделю и квадраты отрицательных доходностей (асимметрия, как в модели GJR).

```
r100 = r.dropna() * 100
sq = r100 ** 2
H = 5
rv = pd.DataFrame({
    "d": np.log(sq + 1e-4),                          # последний день
    "w": np.log(sq.rolling(5).mean() + 1e-4),        # неделя
    "m": np.log(sq.rolling(22).mean() + 1e-4),       # месяц
    "ret_w": r100.rolling(5).sum(),                  # доходность за неделю
    "neg": np.log((r100 < 0) * sq + 1e-4),           # квадрат отрицательной доходности
})
# Цель: логарифм среднего квадрата доходности за дни t+1, ..., t+5
rv["y"] = np.log(sq[::-1].rolling(H).mean()[::-1].shift(-1))
rv = rv.dropna().loc["2004":]
har_cols = ["d", "w", "m"]

vol_preds = {"Последняя неделя": [], "HAR": [], "LightGBM (d, w, m)": [], "LightGBM (все признаки)":
[]}
vol_actual = []
vol_params  =  dict(learning_rate=0.03,  num_leaves=15,  min_data_in_leaf=100,  n_estimators=300,
verbose=-1)
for year in range(2012, 2027):
    # Эмбарго: последние H целей обучающей выборки заходят в тестовый год
    train = rv.loc[: f"{year - 1}-12-31"].iloc[:-H]
    test = rv.loc[f"{year}-01-01": f"{year}-12-31"]
    har = LinearRegression().fit(train[har_cols], train["y"])
    gb3 = lgb.LGBMRegressor(**vol_params).fit(train[har_cols], train["y"])
    gb_all = lgb.LGBMRegressor(**vol_params).fit(train.drop(columns="y"), train["y"])
    vol_preds["Последняя неделя"].append(test["w"])
    vol_preds["HAR"].append(pd.Series(har.predict(test[har_cols]), index=test.index))
   vol_preds["LightGBM (d, w, m)"].append(pd.Series(gb3.predict(test[har_cols]), index=test.index))
    vol_preds["LightGBM (все признаки)"].append(
        pd.Series(gb_all.predict(test.drop(columns="y")), index=test.index))
    vol_actual.append(test["y"])

yv = pd.concat(vol_actual)
vol_tab = pd.DataFrame({
    name: {"RMSE": np.sqrt(((yv - pd.concat(p)) ** 2).mean()),
           "R²": 1 - ((yv - pd.concat(p)) ** 2).sum() / ((yv - yv.mean()) ** 2).sum()}
    for name, p in vol_preds.items()}).T
vol_tab.style.format(ru)
```

Таблица 9.2. Прогноз средней дисперсии на 5 дней вперёд вне выборки, 2012–2026 гг. Ошибки – в логарифмах дисперсии

	RMSE	R^2
Последняя неделя	1,086	−0,000
HAR	0,897	0,317
LightGBM (d, w, m)	0,925	0,274
LightGBM (все признаки)	0,913	0,293

Вариант кода на R – в онлайн-версии книги.

Картина противоположна прогнозу доходности. Волатильность предсказуема: модель HAR объясняет вне выборки 32% дисперсии логарифма будущей волатильности. Бустинг на тех же трёх признаках даёт 27%, на расширенном наборе – 29%. Три коэффициента линейной регрессии оказываются не хуже сотен деревьев. В последней оценке HAR наибольший вес у месячной компоненты (0,61), затем у недельной (0,18), а у дневной он почти нулевой: один день – слишком шумная оценка волатильности.

Отсюда общий урок. Машинное обучение выигрывает там, где зависимость сложная и нелинейная, а данных много. Когда главная зависимость проста и хорошо известна, как долгая память волатильности, правильно выбранные признаки важнее выбора алгоритма. Модели вроде HAR служат эталоном, который сложная модель должна превзойти, чтобы оправдать свою сложность.

9.6 От прогноза к стратегии

Прогноз сам по себе не приносит денег, деньги приносит основанное на нём решение. Простейшее решение – быть в рынке, когда модель прогнозирует рост, и вне рынка, когда падение. Посчитаем среднегодовую лог-доходность такой стратегии для бустинга из раздела 9.4 и сравним с покупкой и удержанием индекса.

Здесь это предварительная оценка решений по прогнозу, ещё не исполнимый бэктест: умножение знака прогноза на следующую доходность предполагает вход уже по цене закрытия, на которой рассчитаны признаки. Доступность такого исполнения отдельно не доказана. В разделе 15.2 зададим явную задержку сделки; до этой проверки числа ниже нельзя считать достижимой торговой доходностью.

Результат удивляет. При R^2 ниже нуля стратегия на прогнозах LightGBM приносит 7,0% в год против 3,6% у индекса. Мы перебрали 18 сочетаний гиперпараметров, и в 16 из них стратегия обгоняет индекс; годовая доходность колеблется от 2,9% до 10,1%. Неужели модель всё-таки что-то нашла?

Разберём, откуда преимущество. Из 3,3 п. п. годового преимущества 2,8 п. п. даёт один-единственный день: накануне 24 февраля 2022 г. модель оказалась вне

рынка, и стратегия избежала падения индекса на 33% (лог-доходность −0,40). После сильного падения и всплеска волатильности 21–22 февраля модель прогнозировала отрицательную доходность, и это совпало с одним из крупнейших обвалов в истории рынка. Совпадение осмысленное (при высокой волатильности модель осторожнее), но построено на одном эпизоде, который не повторится в той же форме.

Кроме того, стратегия меняет позицию около 99 раз в год. Если каждая смена позиции стоит 0,1% (комиссии и спред), годовая доходность стратегии падает до −2,9%, ниже, чем у индекса. И наконец, сравнение неполное: индекс IMOEX не учитывает дивиденды, которые получает держатель акций, а деньги вне рынка в нашем расчёте не приносят дохода, хотя на самом деле их можно разместить по ставке денежного рынка. Как правильно учитывать бенчмарк, издержки и множественность перебранных вариантов, разбирается в главе о бэктестинге и переобучении.

Итоги

- Задача прогноза начинается с временной оси: признаки строки t используют только данные до конца дня t, цель – величина из будущего, например `r.shift(-1)`.
- Утечка данных – главная опасность машинного обучения в финансах. Скользящие окна, включающие цель, данные того же дня, масштабирование по всей выборке и подбор модели на тесте дают правдоподобные, но ложные результаты. Слишком хороший результат – повод искать ошибку.
- Для временных рядов нужна валидация во времени: расширяющееся или скользящее окно, очистка обучения при пересекающихся целях. Случайное разбиение при 20-дневной цели превращает отрицательный R^2 в положительный.
- В прогнозе дневной доходности индекса ни гребневая регрессия, ни бустинг не превосходят нулевой прогноз. Бустинг проигрывает сильнее: гибкость без сигнала – это переобучение.
- Волатильность предсказуема, но простая модель HAR на трёх признаках не уступает бустингу. Правильные признаки важнее сложного алгоритма.
- Стратегия на прогнозах может обыграть рынок на бумаге за счёт одного эпизода и без учёта издержек. Прежде чем верить таким результатам, их нужно проверить методами бэктестинга.

9.7 Упражнения

Упражнения разделены на четыре типа. Задания **А** проверяют понимание и решаются без компьютера. Задания **Б** выполняются на данных вашей акции (тикер назначает преподаватель), поэтому готового ответа у них нет. Задания **В**

учат работать с ИИ-ассистентом: не получать ответ, а проверять его. Задание **Г** – небольшое исследование, результаты которого нужно защитить устно.

9.7.1 А. Понимание

1. Для каждого признака определите, можно ли использовать его в строке t для прогноза r_{t+1} и для прогноза r_t: (а) `r.shift(1)`; (б) `r.rolling(20).std()`; (в) объём торгов дня t; (г) цена открытия дня $t+1$; (д) день недели дня $t+1$.
2. Почему модель, которая всегда прогнозирует рост, может угадывать знак доходности в 53% дней? Какой эталон нужен для доли угаданных знаков?
3. Цель – доходность за следующие 10 дней, признаки считаются ежедневно. Почему случайное разбиение на обучающую и тестовую выборки завышает качество модели? Какой разрыв перед тестом достаточен?
4. Что произойдёт, если стандартизовать признаки по всей выборке до разбиения? Почему это утечка, хотя целевая переменная не используется?
5. В чём разница между гребневой регрессией и градиентным бустингом с точки зрения переобучения? Почему при слабом сигнале бустинг часто проигрывает линейной модели?

9.7.2 Б. Практика на своей акции

6. Постройте признаки из раздела 9.3 для своей акции. Специально внесите утечку, заменив цель на доходность того же дня, и оцените R^2 бустинга. Затем исправьте постановку и сравните.
7. Повторите сравнение гребневой регрессии и LightGBM из раздела 9.4 для своей акции. Добавьте в признаки доходность индекса IMOEX за последние дни. Улучшился ли прогноз?
8. Оцените модель HAR для волатильности своей акции и сравните её с бустингом. Совпадают ли выводы с результатами для индекса?
9. Постройте важность признаков бустинга (`feature_importance`) в модели с утечкой и в честной модели. Какой признак «самый важный» в первом случае и почему этому нельзя верить?

9.7.3 В. Работа с ИИ-ассистентом

10. Попросите ИИ-ассистента написать код прогноза дневной доходности акции с помощью LightGBM с набором технических индикаторов. Проверьте код по списку из врезки «Признак, который знает ответ»: как сдвинута цель, какие окна включают день t, как разбиты данные, где обучается масштабирование, по какой выборке делается ранняя остановка. Запишите найденные ошибки.
11. Попросите ассистента объяснить, почему модель на ваших данных дала $R^2 = 0,6$ для дневных доходностей. Предложил ли он проверить утечку или начал объяснять «силу модели»?

9.7.4 Г. Мини-исследование

12. Стратегия из раздела 9.6 опережает индекс в большинстве сочетаний гиперпараметров. Проверьте устойчивость результата: (а) исключите 2022 г.; (б) учтите издержки 0,05% и 0,1% на смену позиции; (в) сравните с индексом полной доходности MCFTR и начислите деньгам вне рынка ставку RUONIA; (г) замените бустинг на правило «вне рынка, если волатильность за 20 дней выше медианы». Что остаётся от преимущества? Подготовьте короткий (3–5 минут) устный рассказ.

Вопросы для самопроверки

1. Что такое утечка данных? Приведите три примера из финансовых задач.
2. Почему обычная k-кратная кросс-валидация не подходит для временных рядов?
3. Чем расширяющееся окно отличается от скользящего и когда уместно каждое?
4. Зачем исключать последние обучающие наблюдения перед тестом?
5. Что делает штраф в гребневой регрессии?
6. Почему модель HAR при трёх параметрах не уступает градиентному бустингу в прогнозе волатильности?
7. Почему положительная доходность стратегии на прогнозах модели ещё не доказывает, что модель что-то предсказывает?

Ответы

1. Использование при обучении или проверке информации, недоступной в момент прогноза: скользящее окно, включающее цель; данные того же дня (объём, максимум, цена закрытия); масштабирование по всей выборке; подбор гиперпараметров или ранняя остановка по тесту; случайное перемешивание временного ряда.
2. Модель обучается на данных, которые идут после тестовых, а соседние, похожие друг на друга наблюдения попадают одновременно в обучение и в тест.
3. Расширяющееся окно использует всю доступную историю, скользящее – только последние наблюдения фиксированной длины. Скользящее окно уместно, если старые данные устарели из-за смены режима; расширяющееся – если закономерность стабильна и важен объём данных.
4. Когда цели соседних наблюдений перекрываются (например, доходность за 20 дней), обучающие наблюдения рядом с тестом содержат почти ту же цель. В нашей схеме консервативный разрыв равен горизонту цели. В общем случае проверяют даты начала и окончания каждой цели и момент доступности ответа.
5. Стягивает коэффициенты к нулю, уменьшая дисперсию оценок и подгонку под шум ценой небольшого смещения.

6. Главная зависимость в волатильности – долгая память – почти линейна в логарифмах и хорошо описывается средними за день, неделю и месяц. Сложной модели нечего добавить, а гибкость увеличивает ошибку.
7. Результат может определяться одним-двумя эпизодами, исчезать после учёта издержек, дивидендов и дохода от денег вне рынка и быть лучшим из многих перебранных вариантов.

Что почитать

- López de Prado (2018 г.), гл. 7 – кросс-валидация в финансах: очистка (purging) и эмбарго.
- Gu и др. (2020 г.) – сравнение методов машинного обучения в прогнозе доходностей акций в поперечном разрезе.
- Corsi (2009 г.) – модель HAR для реализованной волатильности.
- Welch и Goyal (2008 г.) – почему прогнозы доходности рынка вне выборки редко превосходят историческое среднее.

ЧАСТЬ IV

Риск и портфели

10 VaR и Expected Shortfall

Глава готовится

Эта глава входит во второй этап работы над книгой. Планируемое содержание:

- Меры хвостового риска: VaR и ES
- Историческая, параметрическая и Монте-Карло оценки
- VaR на основе GARCH
- Бэктест VaR: тесты Купика и Кристофферсена
- Кейс: риск российского рынка в 2022 г.

11 Портфельная теория

В сентябре 2026 г. аналитик решает собрать портфель из 15 крупных российских акций по всем правилам учебника. Он берёт месячные доходности за последние пять лет, оценивает ожидаемые доходности и ковариации и находит портфель с максимальным коэффициентом Шарпа. Ответ оптимизатора: 100% капитала вложить в одну акцию, TATN (ПАО «Татнефть» им. В. Д. Шашина, ИНН 1644003838). Годом раньше та же процедура давала портфель из 3 акций: LKOH – 70%, TATN – 27%, PLZL – 3%.

Теория, созданная, чтобы обосновать диверсификацию, советует сложить все яйца в одну корзину. Это не ошибка в коде, а известное свойство оптимизации по Марковицу: она максимально доверяет входным оценкам и усиливает их ошибки. Портфельная теория при этом остаётся одной из самых важных идей в финансах: она объясняет, почему диверсификация снижает риск, как соотносить рискованные активы с безрисковым и почему риск отдельной бумаги важен не сам по себе, а через её вклад в риск портфеля. В этой главе мы разберём и идеи, и то, что происходит, когда их применяют к реальным данным.

В этой главе

- Считаем доходность и риск портфеля; выясняем, как ковариации определяют выгоду диверсификации.
- Строим эффективную границу и портфель с максимальным коэффициентом Шарпа с учётом высокой безрисковой ставки.
- Разбираем, почему оптимальные веса неустойчивы, и как с этим бороться: сжатие ковариаций, ограничения, простые правила.
- Проверяем стратегии вне выборки: равные веса, веса по капитализации, минимальная дисперсия, максимальный Шарп.
- Сравниваем распределение капитала между акциями, облигациями и деньгами, включая правило 60/40.

Данные: `stocks.parquet`, `indexes.parquet`, `ruonia.parquet` (см. данные книги). **Нужно знать:** доходность, волатильность и коэффициент Шарпа (глава «Доходность и риск»).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from sklearn.covariance import LedoitWolf
from qf import data, plot
```

11.1 Доходность и риск портфеля

11.1.1 Формулы

Портфель – это набор активов с весами $w_1, \ldots, w_n$, которые в сумме дают единицу. Доходность портфеля за период – взвешенная сумма доходностей активов (для простых, а не логарифмических доходностей):

$$r_p = \sum_{i=1}^{n} w_i r_i = w' r. \tag{11.1}$$

Ожидаемая доходность портфеля – такое же взвешенное среднее: $\mu_p = w'\mu$. С риском иначе. Дисперсия портфеля зависит не только от дисперсий активов, но и от их ковариаций:

$$\sigma_p^2 = \sum_{i=1}^{n}\sum_{j=1}^{n} w_i w_j \sigma_{ij} = w'\Sigma w, \tag{11.2}$$

где Σ – ковариационная матрица доходностей, $\sigma_{ij} = \rho_{ij}\sigma_i\sigma_j$. Для двух активов:

$$\sigma_p^2 = w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2w_1 w_2 \rho_{12}\sigma_1\sigma_2. \tag{11.3}$$

Если $\rho_{12} < 1$, волатильность портфеля меньше взвешенного среднего волатильностей. В этом и состоит выгода диверсификации: доходность усредняется, а риск – меньше чем усредняется.

11.1.2 Акции, облигации и деньги

Возьмём три базовых класса российских активов: индекс акций с учётом дивидендов MCFTR, индекс государственных облигаций с учётом купонов RGBITR и денежный рынок – вложения под ставку RUONIA, которая начисляется за каждый календарный день. Посмотрим, какие портфели из акций и облигаций были доступны в 2012–2026 гг.

```
# Месячные доходности индексов полной доходности
ix = data.load_index(["MCFTR", "RGBITR"], start="2011-12-01").pivot(
    index="date", columns="ticker", values="close")
M = ix.resample("ME").last().pct_change().dropna()

# Денежный рынок: RUONIA начисляется за календарные дни, в том числе выходные
```

```
ruonia = data.load_ruonia(start="2011-01-01").set_index("date")["rate"] / 100
daily = ruonia.reindex(pd.date_range(ruonia.index.min(), ix.index.max()), method="ffill")
rf = ((1 + daily / 365).resample("ME").prod() - 1).reindex(M.index)

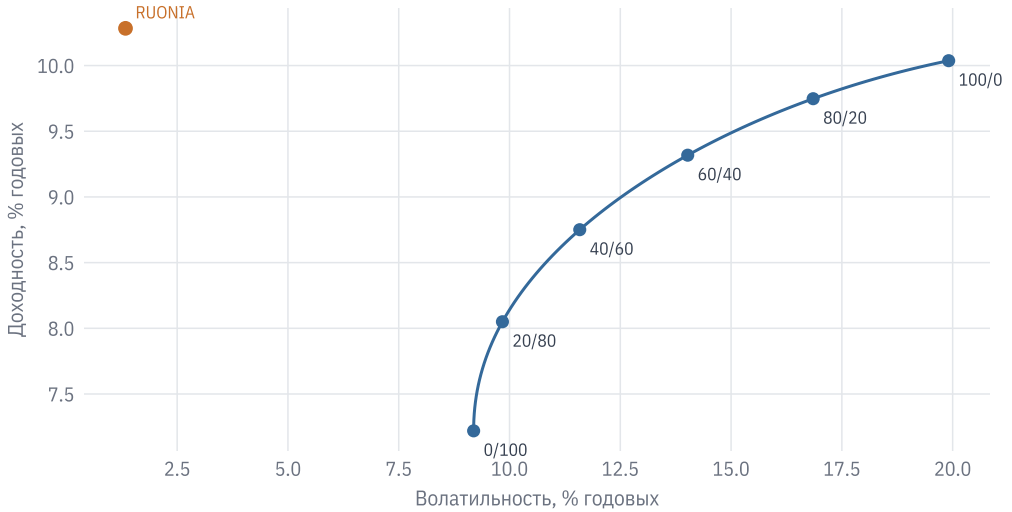
def ann_return(x):
    """Среднегодовая (геометрическая) доходность по месячным доходностям."""
    return (1 + x).prod() ** (12 / len(x)) - 1

def perf(x, rf):
    """Доходность, волатильность, коэффициент Шарпа (к RUONIA) и максимальная просадка."""
    excess = x - rf
    wealth = (1 + x).cumprod()
    return {"Доходность": ann_return(x), "Волатильность": x.std() * np.sqrt(12),
            "Шарп": excess.mean() / excess.std() * np.sqrt(12),
            "Макс. просадка": (wealth / wealth.cummax() - 1).min()}

# Портфели с разной долей акций, ребалансировка раз в месяц
mixes = {f"{int(w * 100)}/{int(100 - w * 100)}": w * M["MCFTR"] + (1 - w) * M["RGBITR"]
         for w in [1, 0.8, 0.6, 0.4, 0.2, 0]}
```

```
w_grid = np.linspace(0, 1, 101)
curve = pd.DataFrame([perf(w * M["MCFTR"] + (1 - w) * M["RGBITR"], rf) for w in w_grid], index=w_grid)

fig, ax = plt.subplots()
ax.plot(curve["Волатильность"] * 100, curve["Доходность"] * 100, color=plot.BLUE)
for name, x in mixes.items():
    p = perf(x, rf)
    ax.scatter(p["Волатильность"] * 100, p["Доходность"] * 100, color=plot.BLUE, s=18, zorder=3)
    ax.annotate(name, (p["Волатильность"] * 100, p["Доходность"] * 100),
                xytext=(4, -10), textcoords="offset points", fontsize=7)
cash = perf(rf, rf)
ax.scatter(cash["Волатильность"] * 100, cash["Доходность"] * 100, color=plot.ORANGE, s=24, zorder=3)
ax.annotate("RUONIA", (cash["Волатильность"] * 100, cash["Доходность"] * 100),
            xytext=(4, 4), textcoords="offset points", fontsize=7, color=plot.ORANGE)
ax.set_xlabel("Волатильность, % годовых")
ax.set_ylabel("Доходность, % годовых")
plot.source(ax, "Московская биржа, Банк России, расчёты автора", offset=32)
plt.show()
```



Источник: Московская биржа, Банк России, расчёты автора

Рисунок 11.1. Портфели из акций (MCFTR) и облигаций (RGBITR) с разной долей акций и денежный рынок (RUONIA), 2012–2026 гг., месячная ребалансировка

Кривая показывает, как меняются риск и доходность при переходе от облигаций к акциям. Выгода диверсификации здесь скромная: корреляция месячных доходностей акций и облигаций равна 0,46, причём с 2020 г. она выше (0,51), чем в 2012–2019 гг. (0,34). На американском рынке в последние десятилетия корреляция акций и облигаций была в основном отрицательной, и облигации страховали портфель от падения акций. В России главный шок для обоих классов активов – резкое повышение ключевой ставки: оно одновременно снижает цены облигаций и оценки акций.

Ещё важнее оранжевая точка. Денежный рынок за 2012–2026 гг. принёс 10,3% годовых при почти нулевой волатильности. Акции с дивидендами дали 10,0% при волатильности 20% и просадке до 51%, облигации – 7,2%. За полтора десятилетия инвестор в российские акции не получил премии за риск по отношению к деньгам. Это не закон природы, а итог конкретного периода с двумя тяжёлыми кризисами и долгими эпизодами высокой ключевой ставки. Но он напоминает, что ожидаемая премия за риск акций – оценка, а не гарантия, и что безрисковый актив в России – серьёзная альтернатива.

11.2 Диверсификация

Сколько акций нужно, чтобы диверсифицировать портфель? Для портфеля из N акций с равными весами дисперсия равна

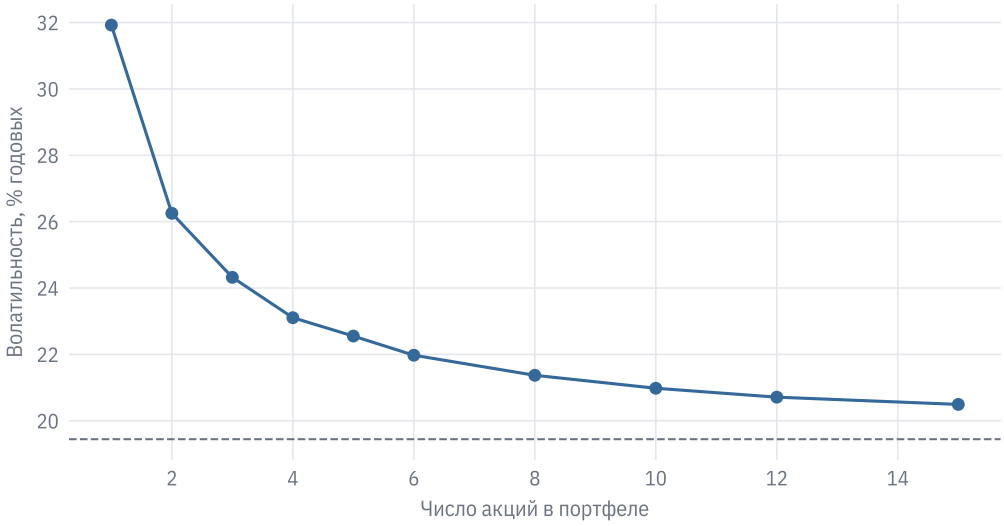
$$\sigma_p^2 = \frac{1}{N}\,\bar{\sigma}^2 + \left(1 - \frac{1}{N}\right)\overline{\sigma_{ij}}, \tag{11.4}$$

где $\bar{\sigma}^2$ – средняя дисперсия акций, а $\overline{\sigma_{ij}}$ – средняя ковариация пар. При росте N первое слагаемое исчезает, второе остаётся. Индивидуальный (специфический) риск компаний диверсифицируется, общий для всех рыночный риск – нет. Проверим это на 15 крупных российских акциях: для каждого N возьмём 300 случайных наборов по N акций и посчитаем среднюю волатильность портфеля с равными весами.

```
TICKERS = ["SBER", "GAZP", "LKOH", "GMKN", "ROSN", "NVTK", "TATN", "MTSS",
           "MGNT", "SNGS", "CHMF", "NLMK", "PLZL", "ALRS", "VTBR"]
prices = data.load_prices(TICKERS, start="2011-12-01").loc[:, TICKERS]   # adj_close: с дивидендами
R = prices.resample("ME").last().pct_change().dropna()

rng = np.random.default_rng(0)
sizes = [1, 2, 3, 4, 5, 6, 8, 10, 12, 15]
avg_vol = {n: np.mean([R[rng.choice(TICKERS, n, replace=False)].mean(axis=1).std() * np.sqrt(12)
                       for draw in range(300)]) for n in sizes}

C = R.cov().values * 12
avg_cov = C[np.triu_indices(len(TICKERS), 1)].mean()      # средняя ковариация пар
fig, ax = plt.subplots()
ax.plot(sizes, np.array(list(avg_vol.values())) * 100, marker="o", color=plot.BLUE)
ax.axhline(np.sqrt(avg_cov) * 100, color=plot.LIGHT, ls="--", lw=0.8)
ax.set_xlabel("Число акций в портфеле")
ax.set_ylabel("Волатильность, % годовых")
plot.source(ax, "Московская биржа, расчёты автора", offset=32)
plt.show()
```



Источник: Московская биржа, расчёты автора

Рисунок 11.2. Средняя волатильность портфеля из N случайно выбранных акций с равными весами, месячные доходности 2012–2026 гг. Пунктир – предел при бесконечном N

Одна акция в среднем имеет волатильность 32% годовых, портфель из пяти – 23%, из всех пятнадцати – 20%. Кривая быстро выходит на плато около 19%: средняя корреляция российских акций между собой – 0,40, и общий рыночный риск составляет большую часть риска каждой бумаги. Первые пять акций снижают риск сильнее, чем следующие десять. Но из этого не следует, что пяти акций достаточно: средняя волатильность скрывает разброс, и конкретный портфель из пяти бумаг может оказаться сильно сконцентрирован в одной отрасли.

11.3 Эффективная граница

11.3.1 Задача Марковица

Markowitz (1952 г.) предложил формализовать выбор портфеля как задачу оптимизации. Для каждого уровня ожидаемой доходности m найдём портфель с наименьшей дисперсией:

$$\min_{w} w'\Sigma w \quad \text{при} \quad w'\mu = m,\ \sum_i w_i = 1,\ w_i \geq 0. \tag{11.5}$$

Условие $w_i \geq 0$ запрещает короткие продажи; для частного инвестора и большинства фондов это реалистично. Множество решений для разных m образует **эффективную границу**: портфели, для которых нельзя повысить ожидаемую доходность, не увеличив риск. Крайняя левая точка – **портфель минимальной дисперсии**.

11.3.2 Безрисковый актив и касательный портфель

Если можно вложиться в безрисковый актив со ставкой r_f, то инвестор комбинирует его с одним рискованным портфелем. Все такие комбинации лежат на прямой, выходящей из точки $(0, r_f)$; наклон прямой – коэффициент Шарпа рискованного портфеля. Лучшая прямая касается эффективной границы, и точка касания – **касательный портфель** с максимальным коэффициентом Шарпа:

$$\max_{w} \frac{w'\mu - r_f}{\sqrt{w'\Sigma w}}. \tag{11.6}$$

Выбор между риском и доходностью сводится к выбору доли касательного портфеля: осторожный инвестор держит больше денег, смелый – меньше. Это теорема о разделении (two-fund separation): состав рискованной части не зависит от отношения инвестора к риску.

> **Типичная ошибка: нулевая безрисковая ставка**
>
> Во многих учебных примерах и в настройках библиотек по умолчанию $r_f = 0$ (например, в прошлогоднем учебном ноутбуке курса коэффициент Шарпа портфеля считался «assuming zero interest free rate»). Для американского рынка

> 2010-х годов, когда ставки были близки к нулю, это безобидно. Для России, где ставка денежного рынка в 2012–2026 гг. в среднем превышала 10% годовых, – нет. Касательный портфель, коэффициенты Шарпа и сам вывод о том, стоит ли вообще держать акции, зависят от r_f. Ставку нужно брать того же периода и той же частоты, что и доходности, и начислять за календарные, а не торговые дни.

```
def optimize(mu, S, kind, rf=0.0, target=None):
    """Веса без коротких продаж: 'minvar' - минимум дисперсии (при заданной доходности target),
    'sharpe' - максимум коэффициента Шарпа."""
    n = len(mu)
    cons = [{"type": "eq", "fun": lambda w: w.sum() - 1}]
    if target is not None:
        cons.append({"type": "eq", "fun": lambda w: w @ mu - target})
    if kind == "minvar":
        objective = lambda w: w @ S @ w
    else:
        objective = lambda w: -(w @ mu - rf) / np.sqrt(w @ S @ w)
   res = minimize(objective, np.ones(n) / n, bounds=[(0, 1)] * n, constraints=cons, method="SLSQP")
    return res.x

mu = R.mean().values * 12                # оценки ожидаемых доходностей, годовые
S = R.cov().values * 12                  # оценка ковариационной матрицы, годовая
rf_ann = rf.reindex(R.index).mean() * 12  # средняя ставка RUONIA за тот же период

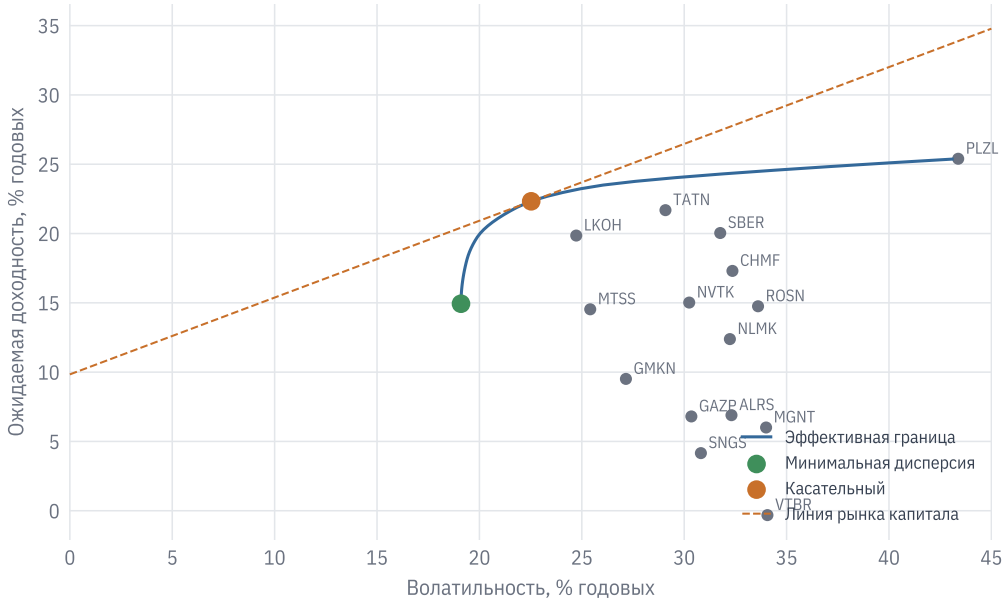
w_minvar = optimize(mu, S, "minvar")
w_tan = optimize(mu, S, "sharpe", rf=rf_ann)
targets = np.linspace(w_minvar @ mu, mu.max() * 0.999, 40)
frontier = [(np.sqrt(w @ S @ w), w @ mu) for w in (optimize(mu, S, "minvar", target=m) for m
in targets)]
```

Вариант кода на R – в онлайн-версии книги.

```
fvol, fret = np.array(frontier).T
vol_i, mu_i = np.sqrt(np.diag(S)), mu

fig, ax = plt.subplots(figsize=(6.5, 3.8))
ax.scatter(vol_i * 100, mu_i * 100, s=14, color=plot.GREY)
for t, x, y in zip(TICKERS, vol_i, mu_i):
     ax.annotate(t, (x * 100, y * 100), xytext=(3, 2), textcoords="offset points", fontsize=6.5,
color=plot.LIGHT)
ax.plot(fvol * 100, fret * 100, color=plot.BLUE, label="Эффективная граница")
for w, name, color in [(w_minvar, "Минимальная дисперсия", plot.GREEN), (w_tan, "Касательный",
plot.ORANGE)]:
    ax.scatter(np.sqrt(w @ S @ w) * 100, w @ mu * 100, s=40, color=color, zorder=3, label=name)
tan_vol, tan_ret = np.sqrt(w_tan @ S @ w_tan), w_tan @ mu
x_line = np.linspace(0, 0.45, 50)
ax.plot(x_line * 100, (rf_ann + (tan_ret - rf_ann) / tan_vol * x_line) * 100,
        color=plot.ORANGE, ls="--", lw=0.8, label="Линия рынка капитала")
ax.set_xlim(0, 45)
```

```
ax.set_xlabel("Волатильность, % годовых")
ax.set_ylabel("Ожидаемая доходность, % годовых")
ax.legend(loc="lower right", fontsize=7)
plot.source(ax, "Московская биржа, Банк России, расчёты автора", offset=32)
plt.show()
```



Источник: Московская биржа, Банк России, расчёты автора

Рисунок 11.3. Эффективная граница 15 крупных российских акций без коротких продаж по данным 2012–2026 гг., портфель минимальной дисперсии, касательный портфель и линия рынка капитала от ставки RUONIA

По данным всего периода касательный портфель состоит из 3 акций: TATN – 40%, PLZL – 31%, LKOH – 29%. Портфель минимальной дисперсии разнообразнее (10 акций с заметным весом), его волатильность 19% против 20% у портфеля с равными весами. Но всё это оценки *в выборке*: оптимизатор знал доходности, которые ещё предстояло получить. Настоящий вопрос – как такие портфели работают, если строить их только по прошлым данным.

11.4 Почему оптимальные веса неустойчивы

11.4.1 Ошибка в оценке средних

Для оптимизации нужны ожидаемые доходности μ и ковариации Σ. Истинных значений никто не знает, и их заменяют выборочными оценками. С ковариациями это работает терпимо, со средними – плохо. Стандартная ошибка оценки среднегодовой доходности по T годам наблюдений равна $\sigma/\sqrt{T}$ и не зависит от

частоты данных: месячные или дневные наблюдения не помогают (Merton 1980 г.).

Для типичной российской акции с волатильностью около 30% годовых пятилетняя история даёт стандартную ошибку среднегодовой доходности около 14% в год. При оценке «18% годовых» интервал плюс-минус одна стандартная ошибка простирается от 4% до 32%. При независимых наблюдениях и нормальном приближении такие интервалы покрывали бы истинное среднее примерно в двух третях повторных выборок; это не вероятность для уже фиксированного параметра. Оптимизатор этого не знает. Он принимает оценки за точные и вкладывает максимум в бумаги, которым повезло в выборке. Поэтому Michaud (1989 г.) назвал оптимизацию по Марковицу «максимизатором ошибок оценки».

Посмотрим, как меняется касательный портфель, если каждый год пересчитывать его по последним пяти годам данных.

```
def tangency(window):
    """Касательный портфель по окну месячных доходностей (ставка – средняя RUONIA за окно)."""
    return optimize(window.mean().values * 12, window.cov().values * 12, "sharpe",
                    rf=rf.reindex(window.index).mean() * 12)

rolling_w = pd.DataFrame({year: tangency(R.loc[f"{year - 5}-01-01": f"{year - 1}-12-31"])
                          for year in range(2017, 2027)}, index=TICKERS).T
shown = rolling_w.loc[:, rolling_w.max() > 0.05]          # акции, хоть раз получавшие заметный вес

fig, ax = plt.subplots()
bottom = np.zeros(len(shown))
colors = plt.cm.tab20(np.linspace(0, 1, shown.shape[1]))
for (t, col), c in zip(shown.items(), colors):
    ax.bar(shown.index, col.values * 100, bottom=bottom, color=c, label=t, width=0.8)
    bottom += col.values * 100
ax.set_ylabel("Вес, %")
ax.legend(ncol=5, fontsize=6.5, loc="upper center", bbox_to_anchor=(0.5, -0.12), frameon=False)
ax.set_xticks(shown.index)
plt.show()
```

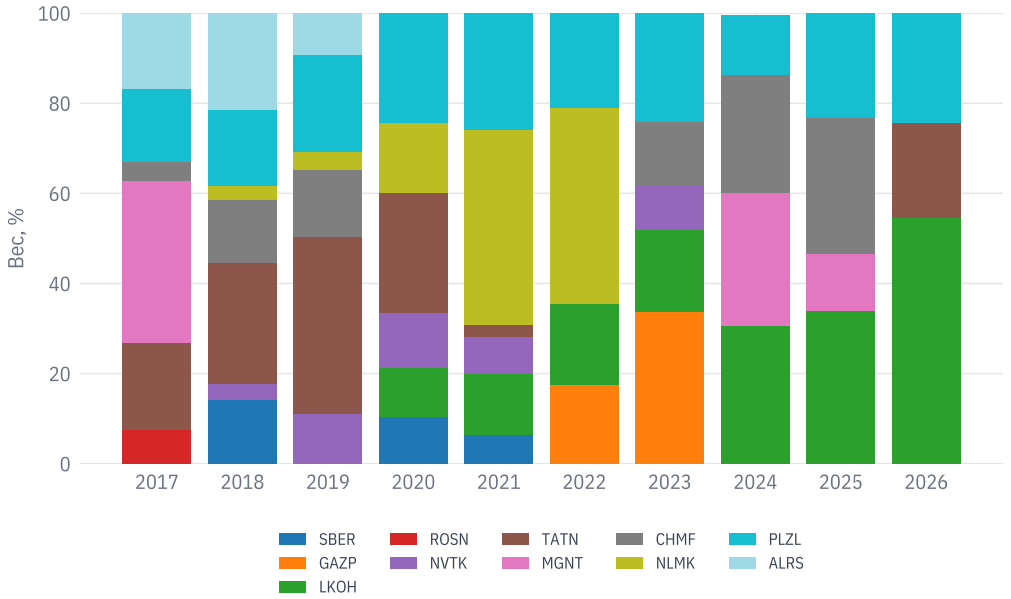


Рисунок 11.4. Веса касательного портфеля, оценённого по месячным доходностям за предыдущие 5 лет, на начало каждого года

От года к году портфель меняется почти до неузнаваемости. В нём от 3 до 7 бумаг, в среднем за год перекладывается 35% капитала, а 4 акции из пятнадцати ни разу не получили заметного веса. Экономические характеристики компаний так быстро не меняются. Меняются оценки средних доходностей, в которые попадают и выпадают удачные и неудачные годы.

11.4.2 Что с этим делают

Практика выработала несколько способов обуздать ошибки оценки.

- **Не оценивать средние.** Портфель минимальной дисперсии использует только ковариации, которые оцениваются точнее.
- **Сжимать оценки.** Метод Ledoit – Wolf (Ledoit и Wolf 2004 г.) сдвигает выборочную ковариационную матрицу к простой структуре (например, одинаковые корреляции), уменьшая влияние случайных экстремальных оценок. Похожая идея для средних – сжатие к общему среднему.
- **Ограничивать веса.** Потолок веса одной бумаги, например 20%, не даёт оптимизатору уйти в угол.
- **Отказаться от оптимизации.** Равные веса ($1/N$) или веса по капитализации не используют оценок вовсе.

Какой способ лучше, решает только проверка вне выборки.

11.5 Проверка вне выборки

Сравним пять правил. Каждый год с 2017 по 2026 г. веса пересчитываются по месячным доходностям предыдущих пяти лет и держатся весь год (с ежемесячной ребалансировкой к этим весам). Бенчмарк – индекс MCFTR, представляющий корзину крупных ликвидных акций. Результаты рассчитаны до последней доступной даты: заключительный месяц и 2026 год неполные. Издержки здесь не вычитаются; оборот показывает, насколько чувствительным может быть результат к их добавлению.

```
cap = data.load_stocks(TICKERS, start="2011-12-01", columns=["market_cap"]).pivot(
    index="date", columns="ticker", values="market_cap").loc[:, TICKERS].resample("ME").last()

rules = ["1/N", "Капитализация", "Мин. дисперсия", "Мин. дисперсия (Ledoit – Wolf)", "Макс. Шарп"]
oos = {k: [] for k in rules}
weights_by_year = {k: [] for k in rules}
turns = {k: [] for k in rules}
drifted = {k: None for k in rules}
for year in range(2017, 2027):
    est = R.loc[f"{year - 5}-01-01": f"{year - 1}-12-31"]            # только прошлые данные
    mu_e, S_e = est.mean().values * 12, est.cov().values * 12
    S_lw = LedoitWolf().fit(est.values).covariance_ * 12
    last_cap = cap.loc[f"{year - 1}-12-31"]
    w = {"1/N": np.ones(len(TICKERS)) / len(TICKERS),
         "Капитализация": (last_cap / last_cap.sum()).values,
         "Мин. дисперсия": optimize(mu_e, S_e, "minvar"),
         "Мин. дисперсия (Ledoit – Wolf)": optimize(mu_e, S_lw, "minvar"),
         "Макс. Шарп": optimize(mu_e, S_e, "sharpe", rf=rf.reindex(est.index).mean() * 12)}
    test = R.loc[f"{year}-01-01": f"{year}-12-31"]
    for k in rules:
        oos[k].append(test @ w[k])
        weights_by_year[k].append(w[k])
        for date, asset_return in test.iterrows():
            if drifted[k] is not None:
                turns[k].append(float(np.abs(w[k] - drifted[k]).sum() / 2))
            gross = w[k] * (1 + asset_return.values)
            drifted[k] = gross / gross.sum()

rows = {}
for k in rules:
    x = pd.concat(oos[k])
    rows[k] = perf(x, rf.reindex(x.index)) | {"Оборот": sum(turns[k]) * 12 / len(x)}
mc = M["MCFTR"].loc["2017":]
rows["Индекс MCFTR"] = perf(mc, rf.reindex(mc.index)) | {"Оборот": np.nan}
rows["RUONIA"] = perf(rf.loc["2017":], rf.loc["2017":]) | {"Оборот": np.nan}

ru_fmt = lambda v: "–" if pd.isna(v) else f"{v:.2f}".replace(".", ",").replace("-", "–")
pct_fmt = lambda v: "–" if pd.isna(v) else f"{v:.1%}".replace(".", ",").replace("-", "–")
tab = pd.DataFrame(rows).T
tab.style.format(pct_fmt).format(ru_fmt, subset=["Шарп"])
```

Таблица 11.1. Портфели из 15 акций вне выборки, 2017–2026 гг. Шарп – к ставке RUONIA; оборот – односторонний оборот за год при ежемесячной ребалансировке, без первоначальной покупки

	Доходность	Волатильность	Шарп	Макс. просадка	Оборот
1/N	5,6%	22,0%	−0,11	−47,4%	29,6%
Капитализация	7,2%	23,4%	−0,02	−47,8%	33,4%
Мин. дисперсия	5,1%	22,1%	−0,13	−48,7%	54,7%
Мин. дисперсия (Ledoit – Wolf)	6,0%	21,5%	−0,10	−47,4%	45,8%
Макс. Шарп	5,1%	22,5%	−0,12	−53,1%	60,9%
Индекс MCFTR	7,7%	21,3%	−0,03	−51,3%	–
RUONIA	10,9%	1,5%	–	0,0%	–

Результат типичен для таких сравнений. Портфель с максимальным коэффициентом Шарпа, лучший *в выборке* по определению, вне выборки оказывается среди худших: 5,1% годовых при наибольшей просадке (53%) и обороте 61% капитала в год. Равные веса дают 5,6%, веса по капитализации – 7,2%. Сжатие ковариационной матрицы улучшает портфель минимальной дисперсии, но не превращает его в победителя. DeMiguel и др. (2009 г.) сравнили 14 оптимизационных моделей с правилом $1/N$ на данных развитых рынков и пришли к тому же выводу: при реальной длине истории выигрыш от оптимизации съедается ошибками оценки.

Второй вывод касается рынка в целом. За 2017–2026 гг. ни одно правило, ни индекс не обогнали денежный рынок: коэффициенты Шарпа относительно RUONIA отрицательны у всех. Это тот же урок, что в разделе 11.1: результат определяется не столько тем, как распределить деньги между акциями, сколько тем, сколько вложить в акции вообще.

> **Особенность данных: ошибка выжившего**
>
> Мы выбрали 15 акций, которые торгуются с 2011 г. *и до сих пор*. Компании, которые за это время ушли с биржи, обанкротились или были выкуплены, в выборку не попали. Это ошибка выжившего: выборка систематически лучше, чем были возможности инвестора в прошлом. Общий набор бумаг не устраняет смещение: оно может менять и абсолютные результаты, и ранжирование правил. Направление и величину искажения именно этой выборки без истории выбывших компаний установить нельзя. Подробнее – в главе о бэктестинге.

11.6 Распределение капитала: правило 60/40

Для частного инвестора главный вопрос – не какие акции купить, а какую долю сбережений держать в акциях, облигациях и деньгах. Популярные правила не используют оптимизацию. Правило 60/40 предлагает держать 60% в акциях и

40% в облигациях. Правило «доля облигаций равна возрасту» делает портфель консервативнее с годами: 30-летнему – 70% акций, 60-летнему – 40%.

```
alloc = {f"Акции {k.split('/')[0]}%": perf(x, rf) for k, x in mixes.items()}
alloc["Денежный рынок (RUONIA)"] = perf(rf, rf)
alloc_tab = pd.DataFrame(alloc).T
alloc_tab.style.format(pct_fmt).format(ru_fmt, subset=["Шарп"])
```

Таблица 11.2. Портфели из индексов акций (MCFTR) и облигаций (RGBITR) с ежемесячной ребалансировкой и денежный рынок, 2012–2026 гг.

	Доходность	Волатильность	Шарп	Макс. просадка
Акции 100%	10,0%	19,9%	0,09	−51,3%
Акции 80%	9,7%	16,9%	0,06	−43,3%
Акции 60%	9,3%	14,0%	0,01	−34,7%
Акции 40%	8,8%	11,6%	−0,06	−28,3%
Акции 20%	8,1%	9,8%	−0,16	−24,8%
Акции 0%	7,2%	9,2%	−0,27	−22,8%
Денежный рынок (RUONIA)	10,3%	1,3%	–	0,0%

За 2012–2026 гг. все смеси акций и облигаций проиграли денежному рынку по доходности. По подпериодам картина разная: в 2012–2016 гг. акции принесли 14,8% годовых, а деньги – 9,0%; в 2017–2021 гг. – 18,1% и 7,0%; в 2022–2026 гг. – −2,3% и 15,2%. Отставание на всём интервале связано с результатом последнего подпериода: в первые два подпериода акции обгоняли деньги на 6 и 11 п. п. в год, и вывод о «ненужности» акций, сделанный в конце 2021 г., был бы противоположным. Это ещё одна иллюстрация того, как сильно оценки средних доходностей зависят от выбранного периода.

Правило 60/40 выросло на американском рынке, где облигации долго были хорошей страховкой для акций, а ставка денежного рынка – низкой. В России обе предпосылки не выполнялись, и механический перенос правила требует осторожности. Это не значит, что акции «не нужны»: на длинных горизонтах премия за риск акций в большинстве стран положительна. Но её величина и надёжность в российских данных – вопрос открытый, и честный анализ должен это показывать.

Итоги

- Доходность портфеля – взвешенное среднее доходностей активов, а риск зависит от ковариаций: при корреляции ниже единицы волатильность портфеля меньше средней волатильности активов.

- Диверсификация устраняет специфический риск компаний, но не рыночный. Для российских акций со средней корреляцией около 0,4 волатильность портфеля быстро выходит на плато около 20% годовых.
- Эффективная граница Марковица и касательный портфель – основа теории выбора портфеля. Касательный портфель зависит от безрисковой ставки, и в России её нельзя считать нулевой.
- Оптимальные веса крайне чувствительны к оценкам средних доходностей, которые по пятилетней истории имеют ошибку порядка 14% годовых. Оптимизация усиливает эти ошибки: портфели концентрируются в немногих бумагах и резко меняются год от года.
- Вне выборки портфель с максимальным коэффициентом Шарпа проигрывает равным весам и весам по капитализации. Сжатие ковариаций, ограничения и отказ от оценки средних делают оптимизацию полезнее.
- Акции и облигации в России положительно коррелированы, а денежный рынок в 2012–2026 гг. обогнал и акции, и облигации. Правила вроде 60/40 нужно проверять на местных данных, а не переносить механически.

11.7 Упражнения

Упражнения разделены на четыре типа. Задания **А** проверяют понимание и решаются без компьютера. Задания **Б** выполняются на данных вашей акции (тикер назначает преподаватель), поэтому готового ответа у них нет. Задания **В** учат работать с ИИ-ассистентом: не получать ответ, а проверять его. Задание **Г** – небольшое исследование, результаты которого нужно защитить устно.

11.7.1 А. Понимание

1. Волатильности двух акций – 30% и 20%, корреляция – 0,4. Найдите волатильность портфеля 50/50. При какой корреляции она равна 25%? А 5%?
2. Средняя волатильность акций – 30%, средняя корреляция – 0,4. Оцените по формуле (11.4) волатильность портфеля из 1, 5, 20 и бесконечного числа акций с равными весами.
3. Безрисковая ставка – 15%, ожидаемая доходность рыночного портфеля – 18%, волатильность – 25%. Чему равен коэффициент Шарпа? Какую долю рыночного портфеля выберет инвестор, желающий волатильность 10%, и какую ожидаемую доходность получит?
4. Почему стандартная ошибка оценки средней доходности не уменьшается при переходе от месячных данных к дневным, а ошибка оценки волатильности уменьшается?
5. Почему портфель минимальной дисперсии вне выборки обычно устойчивее касательного?

11.7.2 Б. Практика на своей акции

6. Добавьте свою акцию к 15 акциям главы (если она уже там, замените её соседней по капитализации). Как изменились эффективная граница и касательный портфель? Получила ли ваша акция вес и почему?
7. Постройте кривую портфелей «ваша акция – индекс MCFTR» по данным последних пяти лет. Какой вес вашей акции даёт минимальную дисперсию? Совпадает ли он с весом акции в индексе?
8. Повторите проверку вне выборки из раздела 11.5 с ограничением: вес одной акции не больше 15%. Как изменились результаты портфелей минимальной дисперсии и максимального Шарпа?

11.7.3 В. Работа с ИИ-ассистентом

9. Попросите ИИ-ассистента построить с помощью PyPortfolioOpt портфель с максимальным коэффициентом Шарпа из пяти российских акций. Проверьте: какая безрисковая ставка использована по умолчанию; по ценам с дивидендами или без; разрешены ли короткие продажи (`weight_bounds`); оценён ли результат вне выборки. Запишите, что нужно изменить.
10. Спросите ассистента, работает ли правило 60/40 для российского инвестора. Сравните его ответ с результатами раздела 11.6. На каких данных основаны его утверждения?

11.7.4 Г. Мини-исследование

11. Сравните вне выборки три способа распределить капитал между MCFTR, RGBITR и денежным рынком: фиксированные 60/30/10, портфель минимальной дисперсии этих трёх активов и правило «держать долю акций тем выше, чем ниже доходность акций за прошлый год». Что выигрывает и насколько результат зависит от периода? Подготовьте короткий (3–5 минут) устный рассказ.

Вопросы для самопроверки

1. Почему дисперсия портфеля не равна взвешенной сумме дисперсий активов?
2. Какой риск устраняется диверсификацией, а какой нет?
3. Что такое эффективная граница и портфель минимальной дисперсии?
4. Что такое касательный портфель и как он зависит от безрисковой ставки?
5. Почему оптимизацию по Марковицу называют «максимизатором ошибок оценки»?
6. Что делает метод Ledoit – Wolf?
7. Почему правило $1/N$ трудно обыграть вне выборки?

Ответы

1. Потому что в неё входят ковариации: $\sigma_p^2 = w'\Sigma w$. При корреляциях ниже единицы колебания активов частично гасят друг друга.
2. Устраняется специфический риск отдельных компаний; остаётся систематический (рыночный) риск, общий для всех бумаг.
3. Эффективная граница – портфели с минимальной дисперсией при каждом уровне ожидаемой доходности (и максимальной доходностью при каждом уровне риска). Портфель минимальной дисперсии – её крайняя левая точка.
4. Портфель рискованных активов с максимальным коэффициентом Шарпа; точка касания эффективной границы и прямой из точки $(0, r_f)$. При росте r_f он смещается к более рискованным и доходным портфелям.
5. Оптимизатор принимает оценки средних и ковариаций за точные и вкладывает больше всего в бумаги, которые в выборке выглядели лучше всего, в том числе из-за ошибки оценки.
6. Сжимает выборочную ковариационную матрицу к простой целевой структуре с весом, который оценивается по данным, уменьшая влияние шумных оценок.
7. Оно не использует оценок, поэтому не подвержено ошибкам оценки; выигрыш от оптимизации при реальной длине истории обычно меньше, чем потери от этих ошибок.

Что почитать

- Markowitz (1952 г.) – исходная статья о выборе портфеля.
- Bodie и др. (2021 г.), гл. 6–8 – портфельная теория в учебном изложении.
- Michaud (1989 г.) – почему оптимизация усиливает ошибки оценки.
- Ledoit и Wolf (2004 г.) – сжатие ковариационной матрицы.
- DeMiguel и др. (2009 г.) – сравнение оптимизационных моделей с правилом $1/N$.
- Merton (1980 г.) – почему ожидаемую доходность так трудно оценить.

12 CAPM и факторные модели

Глава готовится

Эта глава входит в первую версию книги. Планируемое содержание:

- Модель CAPM: линия рынка капитала и линия рынка ценных бумаг
- Оценка беты и её устойчивость
- Событийный анализ (event study)
- Факторы Фамы – Френча на российском рынке: размер, стоимость, моментум
- Проверка факторных моделей

ЧАСТЬ V

Долговой рынок

13 Облигации

Облигация обещает выплаты, но не обещает неизменную рыночную цену. Одна и та же последовательность купонов может стоить существенно по-разному при разных ставках дисконтирования. Покупателю поэтому нужны как минимум три ответа: сколько денег он платит сейчас, какие выплаты получает в будущем и сколько может потерять при продаже до погашения.

В этой главе мы построим небольшой практикум на четырёх ОФЗ с постоянным купонным доходом — ОФЗ-ПД. Начнём с биржевой котировки, восстановим денежные потоки и посчитаем доходность. Затем проверим процентный риск двумя способами: приблизительно через дюрацию и точно через повторное дисконтирование выплат.

В этом практикуме

- Переводим чистую цену в процентах номинала в сумму с НКД в рублях.
- Считаем эффективную доходность по реальным датам и суммам выплат.
- Измеряем дюрацию, выпуклость и рублёвый риск одного базисного пункта.
- Сопоставляем доходность купонной бумаги с бескупонной кривой.
- Проверяем сценарии изменения ставок и обсуждаем применение к собственной позиции.

Данные: отдельный снимок `data/bonds/` на 30.09.2026: ОФЗ 26232, 26236, 26243 и 26238. Источник — Московская биржа. Доступ к хранилищу для выполнения примеров не нужен.

Предпосылки: известные купоны и погашение всего номинала одним платежом; годовая эффективная ставка, ACT/365F. Дата оценки совпадает с датой котировки — учебная модель T+0. Налоги и комиссии здесь не включены. Флоатеры, линкеры и облигации с офертами требуют расширения модели.

13.1 Чистая и грязная цена

Котировка облигации обычно выражена в процентах от непогашенного номинала. При этом накопленный купонный доход — НКД — задаётся в рублях на одну бумагу. Их нельзя складывать без приведения к одной единице измерения.

$$P_{\text{dirty}} = \frac{Q_{\text{clean}}}{100} F + A,$$

где Q_{clean} — котировка, F — номинал, A — НКД. Грязная цена — стоимость облигации вместе с накопленным купонным доходом. Комиссия брокера в эту формулу ещё не входит.

```
ticker = "SU26238RMFS4"
quote = data.load_bonds(
    ticker, start=valuation_date, end=valuation_date
).iloc[0]

dirty = bonds.dirty_price_from_quote(
    quote.clean_pct, quote.face_value, quote.accrued
)
print(f"Чистая цена: {quote.clean_pct:.3f}% номинала")
#> Чистая цена: 50.740% номинала
print(f"НКД: {quote.accrued:.2f} руб.")
#> НКД: 23.15 руб.
print(f"Грязная цена: {dirty:.2f} руб.")
#> Грязная цена: 530.55 руб.
```

Для выбранного выпуска получаем 50,740% от номинала 1 000 руб. плюс НКД 23,15 руб. Итого — 530,55 руб. на одну облигацию.

НКД компенсирует продавцу часть купона, накопленную за время владения после предыдущей выплаты. Внутри обычного купонного периода его можно приблизить пропорционально прошедшим дням:

$$A_{\text{linear}} = C \frac{d - d_0}{d_1 - d_0},$$

где C — сумма купона, d_0, d_1 — границы периода. Это полезная проверка, но условия начисления и округления определяются выпуском. Для оценки мы используем наблюдаемое поле `ACCINT`, а не заменяем его приближением.

```
coupons = data.load_bond_coupons(ticker)
next_coupon = coupons.loc[
    coupons.payment_date > valuation_date
].iloc[0]
accrued_linear = bonds.accrued_interest(
    valuation_date, next_coupon.period_start,
    next_coupon.payment_date, next_coupon.amount
)
print(f"Линейная проверка НКД: {accrued_linear:.4f} руб.")
#> Линейная проверка НКД: 23.1462 руб.
```

В день выплаты наш расчёт исключает платёж, уже пришедшийся на дату оценки, и рассматривает только будущие поступления. Для реальной сделки

отдельно проверяют дату расчётов и право на ближайший купон: простого сравнения с датой выплаты может быть недостаточно.

13.2 Восстановление денежных потоков

ОФЗ-ПД платит купоны, а в конце срока возвращает номинал. Последняя выплата поэтому состоит из двух частей. Номинал следует прибавить один раз — в дату погашения, а не к каждому купону.

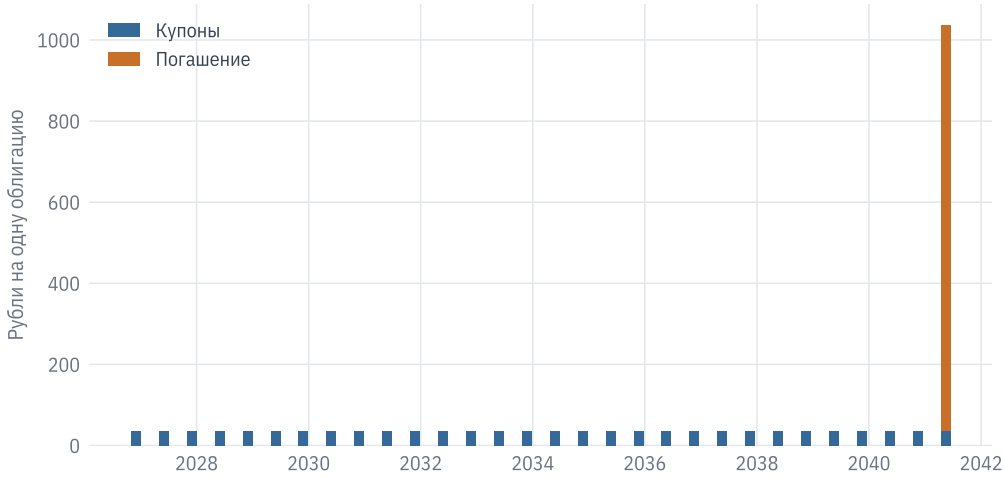
```
def cashflows_for(ticker):
    c = data.load_bond_coupons(ticker)
    p = data.load_bond_principal(ticker)
    c = c.loc[c.payment_date > valuation_date]
    p = p.loc[p.payment_date > valuation_date]
    if c.empty or p.empty or c.amount.isna().any() or p.amount.isna().any():
        raise ValueError("Неизвестные или отсутствующие выплаты")
    payments = (
        c[["payment_date", "amount"]].rename(columns={"amount": "coupon"})
        .merge(
            p[["payment_date", "amount"]].rename(columns={"amount": "principal"}),
            on="payment_date", how="left", validate="one_to_one"
        )
        .fillna({"principal": 0.0})
        .sort_values("payment_date")
    )
    if payments.principal.sum() != p.amount.sum():
        raise ValueError("Не все погашения попали в график")
    payments["amount"] = payments.coupon + payments.principal
    flows = [
        bonds.CashFlow(row.payment_date.date(), row.amount)
        for row in payments.itertuples()
    ]
    return flows, payments

flows, payments = cashflows_for(ticker)
print(payments.tail(4).to_string(index=False))
#> payment_date  coupon  principal  amount
#>   2039-11-16    35.4        0.0    35.4
#>   2040-05-16    35.4        0.0    35.4
#>   2040-11-14    35.4        0.0    35.4
#>   2041-05-15    35.4     1000.0  1035.4
```

Снимок при выгрузке проверяется на полноту купонных периодов, известность сумм и единственное погашение номинала. Пропуск суммы не означает нулевой купон. В таком случае цена и доходность не должны рассчитываться как будто выплаты нет.

```
fig, ax = plt.subplots()
ax.bar(payments.payment_date, payments.coupon, width=65, label="Купоны")
```

```
ax.bar(payments.payment_date, payments.principal, bottom=payments.coupon,
       width=65, label="Погашение")
ax.set_ylabel("Рубли на одну облигацию")
ax.legend()
plot.source(ax, "Московская биржа; расчёты автора", offset=35)
plt.show()
```



Источник: Московская биржа; расчёты автора

Рисунок 13.1. Будущие выплаты ОФЗ 26238: купоны и возврат номинала

13.3 Доходность как решение уравнения

Доходность к погашению y — ставка, при которой приведённая стоимость выплат совпадает с грязной ценой:

$$P_{\text{dirty}} = \sum_{i=1}^{n} \frac{CF_i}{(1+y)^{t_i}}, \qquad t_i = \frac{d_i - d}{365}.$$

Здесь y записана в долях: 0,15 означает 15% годовых. Мы используем фактическое число дней и постоянный знаменатель 365 — соглашение ACT/365F. Это явно выбранная модель, которую можно независимо проверить.

Для известной ставки цену получить просто. Обратную задачу решаем численно: при положительных будущих поступлениях цена монотонно убывает с ростом ставки, поэтому корень единственен. В модуле `qf/bonds.py` используется метод Брента.

```
ytm = bonds.yield_to_maturity(flows, valuation_date, dirty)
reconstructed_price = bonds.price_from_yield(flows, valuation_date, ytm)
```

```
assert np.isclose(reconstructed_price, dirty, atol=1e-7)
print(f"YTM модели: {ytm:.4%}")
#> YTM модели: 16.4617%
print(f"YIELDCLOSE биржи: {quote.iss_ytm_pct:.4f}%")
#> YIELDCLOSE биржи: 16.4600%
```

Полученная доходность — 16,46%. Она отличается от ставки купона: купон начисляется на номинал, а покупатель платит рыночную цену. Кроме купонов, в доходности учитывается разница между покупной ценой и погашением.

Доходность к погашению не равна гарантированной доходности продажи через год. Для такой продажи важна будущая рыночная цена. Чтобы интерпретировать YTM как сложный темп роста итогового капитала до погашения, потребуется ещё предположение о реинвестировании промежуточных купонов.

Почему модель может отличаться от биржи

Наш расчёт использует дату котировки как дату оценки. Биржевая доходность может рассчитываться на другую дату расчётов и по другому соглашению. В частности, с 23 июня 2025 года биржа применяет простую доходность в последнем купонном периоде до погашения. Поэтому сравнивать числа следует вместе с определениями, датами и потоками, а не требовать механического совпадения.

Источники: методика расчёта НКД и доходности, изменение подхода в 2025 году.

13.4 Дюрация, выпуклость и DV01

Срок погашения учитывает только последний платёж. Дюрация Маколея усредняет сроки всех выплат с весами по их приведённой стоимости:

$$D_M = \frac{\sum_i t_i \, PV(CF_i)}{P}, \qquad D_{\text{mod}} = \frac{D_M}{1+y}.$$

В нашей модели с годовой эффективной ставкой модифицированная дюрация связывает небольшое изменение доходности с относительным изменением цены:

$$\frac{\Delta P}{P} \approx -D_{\text{mod}} \Delta y.$$

При переходе от 15% к 16% величина Δy равна 0,01, а не 1. Для более крупных изменений полезен второй член разложения — выпуклость:

$$C = \frac{1}{P} \sum_i \frac{t_i(t_i+1)CF_i}{(1+y)^{t_i+2}}, \qquad \frac{\Delta P}{P} \approx -D_{\text{mod}} \Delta y + \frac{1}{2} C (\Delta y)^2.$$

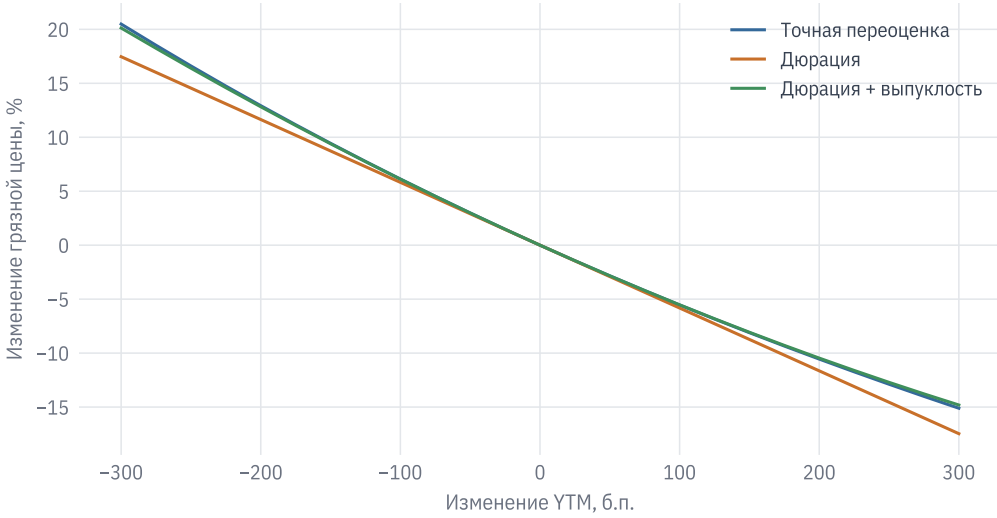
```
risk = bonds.risk_measures(flows, valuation_date, ytm)
print(f"Дюрация Маколея: {risk['macaulay_years']:.3f} года")
#> Дюрация Маколея: 6.780 года
print(f"Модифицированная дюрация: {risk['modified_years']:.3f} года")
#> Модифицированная дюрация: 5.821 года
print(f"DV01: {risk['dv01']:.4f} руб./б.п.")
#> DV01: 0.3089 руб./б.п.
```

DV01 переводит чувствительность в деньги: $DV01 = PD_{\text{mod}} 10^{-4}$. Здесь это положительная величина, описывающая приблизительную потерю цены при росте доходности на один базисный пункт. Для позиции из N одинаковых облигаций её умножают на N.

```
scenarios = pd.DataFrame(bonds.yield_scenarios(
    flows, valuation_date, ytm, np.arange(-300, 301, 25)
))
scenario_100 = scenarios.loc[scenarios.shift_bps == 100].iloc[0]
position_dv01 = 100 * risk["dv01"]
position_loss_100bp = 100 * (scenario_100.dirty_price - dirty)
print(f"DV01 позиции из 100 бумаг: {position_dv01:.2f} руб./б.п.")
#> DV01 позиции из 100 бумаг: 30.89 руб./б.п.
print(f"Точная переоценка при +100 б.п.: {position_loss_100bp:.2f} руб.")
#> Точная переоценка при +100 б.п.: -2938.19 руб.
```

Это **мгновенная переоценка**: дата остаётся прежней, купоны не выплачиваются, НКД не изменяется. Доходность владения за несколько месяцев — другая задача, в которой надо учесть прошедшее время и полученные выплаты.

```
fig, ax = plt.subplots()
for column, label in [
    ("change_pct", "Точная переоценка"),
    ("duration_pct", "Дюрация"),
    ("duration_convexity_pct", "Дюрация + выпуклость"),
]:
    ax.plot(scenarios.shift_bps, scenarios[column], label=label)
ax.set_xlabel("Изменение YTM, б.п.")
ax.set_ylabel("Изменение грязной цены, %")
ax.legend()
plot.source(ax, "Московская биржа; расчёты автора", offset=35)
plt.show()
```



Источник: Московская биржа; расчёты автора

Рисунок 13.2. Переоценка ОФЗ 26238: точный расчёт, дюрация и поправка на выпуклость

13.5 Сравнение выпусков и бескупонная кривая

Купонная облигация состоит из выплат на разные даты. Поэтому единая YTM такой бумаги и ставка бескупонной облигации на срок её погашения — разные показатели. Чтобы учесть всю временную структуру ставок, каждую выплату дисконтируют по ставке своего срока:

$$P(z) = \sum_i \frac{CF_i}{[1 + r(t_i) + z]^{t_i}}.$$

В нашем расчёте $r(t)$ — эффективная годовая спот-ставка. Между опубликованными узлами КБД линейно интерполируется логарифм дисконт-фактора. В начале добавляется точка $D(0) = 1$, что даёт постоянную спот-ставку до первого узла. За последним узлом экстраполяция запрещена. Это прозрачное приближение по опубликованной сетке, а не повторная оценка параметрической модели биржи.

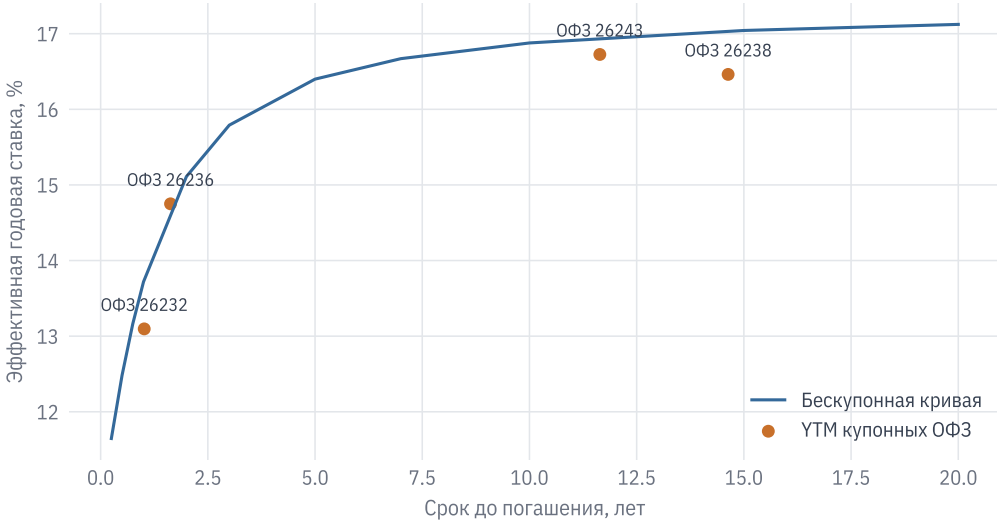
Величина z, при которой $P(z)$ совпадает с наблюдаемой грязной ценой, — **модельный Z-спред**. Для сравнения бумаг он полезнее механической разности между YTM и спот-ставкой на срок погашения. Но он также отражает особенности цен и выбранной модели; отрицательный спред отдельной ОФЗ не означает отрицательный кредитный риск.

```
rows = []
for sid in meta["secids"]:
    q = data.load_bonds(sid, start=valuation_date, end=valuation_date).iloc[0]
```

```
    cf, _ = cashflows_for(sid)
    p = bonds.dirty_price_from_quote(q.clean_pct, q.face_value, q.accrued)
    y = bonds.yield_to_maturity(cf, valuation_date, p)
    m = bonds.risk_measures(cf, valuation_date, y)
    z = bonds.z_spread(
        cf, valuation_date, p, curve.years, curve.yield_pct / 100
    )
    rows.append({
        "Выпуск": q["name"], "Цена, руб.": p, "YTM, %": y * 100,
        "Дюрация, лет": m["modified_years"], "DV01, руб.": m["dv01"],
        "Z-спред, б.п.": z * 10000,
        "Срок, лет": (q.maturity - valuation_date).days / 365,
    })
comparison = pd.DataFrame(rows).sort_values("Срок, лет")
```

Выпуск	Цена, руб.	YTM, %	Дюрация, лет	DV01, руб.	Z-спред, б.п.	Срок, лет
ОФЗ 26232	966.74	13.10	0.86	0.08	−64.73	1.02
ОФЗ 26236	900.07	14.75	1.34	0.12	0.34	1.63
ОФЗ 26243	705.64	16.73	5.07	0.36	10.19	11.64
ОФЗ 26238	530.55	16.46	5.82	0.31	−25.34	14.63

```
fig, ax = plt.subplots()
ax.plot(curve.years, curve.yield_pct, label="Бескупонная кривая")
ax.scatter(comparison["Срок, лет"], comparison["YTM, %"], color=plot.ORANGE,
           label="YTM купонных ОФЗ")
for _, row in comparison.iterrows():
    ax.annotate(row["Выпуск"], (row["Срок, лет"], row["YTM, %"]),
                xytext=(0, 7), textcoords="offset points", ha="center", fontsize=7)
ax.set_xlabel("Срок до погашения, лет")
ax.set_ylabel("Эффективная годовая ставка, %")
ax.legend()
plot.source(ax, "Московская биржа; расчёты автора", offset=35)
plt.show()
```



Источник: Московская биржа; расчёты автора

Рисунок 13.3. КБД на дату практикума и модельные YTM четырёх купонных ОФЗ

Для сценария параллельного сдвига КБД можно сохранить найденный спред и изменить все спот-ставки на одну величину. Это не то же самое, что сдвинуть YTM одной бумаги: купонные платежи дисконтируются по разным точкам кривой.

```
z = bonds.z_spread(flows, valuation_date, dirty, curve.years, curve.yield_pct / 100)
curve_up_price = bonds.price_from_curve(
    flows, valuation_date, curve.years, curve.yield_pct / 100 + 0.01, spread=z
)
print(f"Изменение цены при +100 б.п. ко всей КБД: {curve_up_price / dirty - 1:.2%}")
#> Изменение цены при +100 б.п. ко всей КБД: -5.44%
```

13.6 Как использовать расчёт для своей позиции

Практический вопрос удобно формулировать через изменение позиции: «Если заменить часть коротких ОФЗ длинными, сколько дополнительного процентного риска я беру?» Сначала следует сравнить дюрации и рублёвые DV01 позиций, затем проверить точную переоценку при нескольких сценариях. Для портфеля DV01 складываются, если речь идёт об одинаково определённом сдвиге доходностей.

Перед использованием результата для покупки нужны дополнительные входные данные:

- **Исполнимая цена и объём.** Цена закрытия из снимка — исторический ориентир; покупка оценивается по доступной цене продавца и глубине рынка.

- **Дата расчётов и право на выплату.** НКД и перечень причитающихся купонов должны соответствовать конкретной сделке.
- **Горизонт владения.** При продаже до погашения нужен сценарий будущей цены, а не только сегодняшняя YTM.
- **Издержки и ограничения.** Комиссии, налоги, размер позиции и потребность в ликвидности меняют итог решения.

Для корпоративной облигации к этому добавляется кредитный риск эмитента. Более высокая доходность не заменяет анализ способности платить и условий оферты. Эти вопросы относятся к следующей главе.

Интерактивная версия практикума находится в проекте `moex-analytics`: `marimo/bond-analysis.py`. Там можно менять выпуск и величину шока, а также переключаться между учебным снимком и хранилищем на выбранную дату. Для текущей торговли потребуется отдельный слой котировок и расчёта сделки.

Итоги

Грязная цена связывает котировку с денежными потоками. YTM сводит весь график к одной ставке, дюрация и выпуклость описывают чувствительность цены, а DV01 переводит её в рубли. КБД позволяет дисконтировать выплаты с учётом разных сроков. Все эти показатели зависят от даты, соглашений и полноты графика платежей.

13.7 Упражнения

13.7.1 А. Понимание

1. Почему облигация с купоном 7,1% может иметь доходность около 16%? Какую роль играет цена ниже номинала?
2. Почему срок до погашения и дюрация различаются? Как изменится дюрация при более раннем получении части выплат?
3. Почему разность между YTM и КБД на срок погашения не равна Z-спреду?

13.7.2 Б. Практика

4. Повторите расчёт для ОФЗ 26236. Проверьте обратную подстановку найденной доходности в цену.
5. Постройте таблицу переоценки всех четырёх выпусков при сдвигах −200, −100, +100 и +200 б.п. Сравните потери на одну бумагу и на одинаковую вложенную сумму.
6. Для позиции из 100 бумаг ОФЗ 26238 сравните оценку потери по DV01 и точный результат при +100 б.п. Объясните знак расхождения.

13.7.3 В. Работа с ИИ-ассистентом

7. Попросите ассистента написать расчёт доходности, не показывая модуль книги. Проверьте его на бескупонной облигации: 1000 руб. через 730 дней, цена $1000/1,1^2$. Затем добавьте промежуточный купон и неравные интервалы. Найдите расхождения в единицах ставки и обработке дат.

13.7.4 Г. Мини-исследование

8. Сравните два портфеля с одинаковой начальной стоимостью: только ОФЗ 26236 и смесь ОФЗ 26232/26238. Подберите доли смеси с близкой дюрацией. Сравните точную переоценку при параллельных сдвигах КБД и при росте только длинных ставок. Объясните, почему близкая дюрация не делает портфели одинаковыми.

Вопросы для самопроверки

1. В каких единицах заданы чистая котировка и НКД?
2. Какие будущие потоки входят в расчёт на дату оценки?
3. Что означает ACT/365F и чем 100 б.п. отличаются от 100%?
4. Можно ли интерпретировать YTM как доходность продажи через три месяца?
5. Чем отличается сценарий сдвига ставок от доходности владения?

Ответы

1. Чистая котировка — в процентах номинала, НКД — в рублях на бумагу. Сначала переводим цену в рубли.
2. Только платежи после даты оценки, на которые имеет право держатель. В практикуме сложные случаи купонной отсечки исключены при проверке снимка.
3. Фактическое число дней делится на 365; 100 б.п. — один процентный пункт, или 0,01 в долях ставки.
4. Нет. Для продажи нужна будущая цена, а также полученные купоны и издержки.
5. В мгновенном сценарии меняются ставки, но не дата. Доходность владения учитывает течение времени, выплаты и цену выхода.

Что почитать

- Московская биржа: методика расчёта НКД и доходности.
- Московская биржа: методика кривой бескупонной доходности.
- Банк России: КБД на дату практикума.

14 Кредитный риск

Глава готовится

Эта глава входит во второй этап работы над книгой. Планируемое содержание:

- Кредитные спреды и их связь с рейтингами
- Долговая нагрузка эмитента: ND/EBITDA по МСФО, долг/EBIT и покрытие процентов по РСБУ
- Дефолты на рынке высокодоходных облигаций
- Событийный анализ: понижение рейтинга и доходность облигаций

ЧАСТЬ VI

Торговые стратегии

15 Бэктестинг и переобучение

Представьте два отчёта об одной стратегии. В обоих случаях инвестор держит акции, пока индекс выше своей средней за 200 торговых дней, и выходит в деньги, когда индекс опускается ниже неё. Первый отчёт показывает плавный рост капитала, второй – заметные просадки и гораздо более скромную доходность. Окно средней одинаковое, данные одинаковые. Разница в том, когда программа разрешает купить и продать.

Такая ошибка может занимать одну строку. Если решение по цене закрытия сегодняшнего дня приносит доход от вчерашнего закрытия до сегодняшнего, инвестор получает прибыль за время, когда он ещё не знал о сделке. Компьютер послушно считает результат, но этот результат нельзя было получить на рынке.

Бэктест – историческая симуляция инвестиционного правила. Он соединяет прогноз, решение о позиции, исполнение сделки и учёт денег. В этой главе мы построим небольшой бэктест, разберём его по частям и увидим, почему даже аккуратно рассчитанная кривая капитала ещё не доказывает, что найдена работающая стратегия.

В этой главе

- Разделим момент появления сигнала, момент сделки и период получения дохода.
- Учтём дивиденды, денежную ставку и издержки, проверим арифметику на маленьком примере.
- Сравним правило скользящей средней с владением индексом и денежным рынком.
- Проверим, что происходит при переборе параметров и последовательном выборе стратегии.
- Обсудим диагностику переобучения и состав воспроизводимого отчёта.

15.1 От гипотезы к торговому правилу

Фраза «покупать растущие акции» не задаёт стратегию. Какие акции доступны? За какой период измеряется рост? Когда принимается решение? Что делать с деньгами после продажи? Пока эти вопросы не решены, одна идея допускает десятки реализаций, а исследователь легко выбирает ту, которая лучше выглядит задним числом.

Для учебного эксперимента зафиксируем правило: после закрытия сравниваем индекс полной доходности MCFTR с его средней за 200 наблюдений. Если индекс выше средней, желаемая позиция равна единице; иначе она равна нулю. Решение исполняется **на следующем доступном закрытии**, поэтому новая позиция начинает приносить доход ещё через один интервал между закрытиями. Коротких продаж и заёмных денег нет. Свободный капитал получает доход денежного рынка.

Экономическая гипотеза состоит в том, что устойчивые движения рынка иногда продолжаются достаточно долго, чтобы трендовый фильтр позволял участвовать в росте и пережидать часть падений. Её слабое место тоже понятно заранее: при частых разворотах стратегия будет покупать после роста и продавать после снижения. Поэтому разумный вопрос исследования – компенсирует ли защита от отдельных затяжных падений пропущенный рост, ложные переключения и издержки?

Мы не объявляем окно 200 оптимальным. Это исходное правило для разбора. Позже сравним несколько окон, но будем отдельно учитывать сам факт такого поиска.

> **Особенность данных: индекс нельзя купить**
> MCFTR – расчётный индекс полной доходности, а не цена торгуемого инструмента. Мы моделируем условный портфель, точно воспроизводящий индекс. В действительности понадобятся корзина акций или подходящий фонд; возникнут отклонение от индекса, расходы на управление и ограничения исполнения. Налоги и фактические даты поступления дивидендов здесь не моделируются. Поэтому результат главы – исследовательский ориентир, а не доходность доступного инвестиционного продукта.

15.2 Время в бэктесте

Пусть P_t – значение индекса на закрытии дня t, а $r_t = P_t/P_{t-1} - 1$ – доходность, записанная в строке t. Она относится к интервалу, который **закончился** на этой дате. Сигнал s_t появляется после получения P_t:

$$s_t = \mathbb{1}\left(P_t > \frac{1}{n}\sum_{j=0}^{n-1} P_{t-j}\right). \tag{15.1}$$

После закрытия понедельника мы узнаём сигнал. Во вторник на закрытии совершаем сделку. Первая доходность новой позиции – изменение индекса от вторника до среды. В строке среды хранится сигнал понедельника, то есть позиция за интервал t равна s_{t-2}. В данном соглашении нужен `shift(2)`.

Момент	Что уже известно	Что происходит
Закрытие понедельника	Цена понедельника и вся прошлая история	Рассчитываем сигнал
Закрытие вторника	Сигнал понедельника	Исполняем решение и платим издержки
Закрытие среды	Изменение цены после сделки	Записываем первый доход новой позиции

Почему не `shift(1)`? Он приписал бы сигналу понедельника доход от понедельника до вторника. Это допустимо лишь при отдельном обосновании исполнения уже по цене закрытия понедельника. Например, решение должно быть сформировано до аукциона закрытия на информации, доступной к тому моменту. Окончательную цену закрытия заранее знать нельзя.

Если сделка исполняется на открытии вторника, нужны цены открытия и отдельный расчёт доходов от закрытия до открытия и от открытия до закрытия. Сдвиг массива не заменяет модель исполнения. Здесь мы выбираем более простое соглашение со следующим закрытием, которое можно проверить на дневных данных.

15.2.1 Данные и денежный счёт

```
ix = data.load_index(["MCFTR", "IMOEX"], start="2009-01-01").pivot(
    index="date", columns="ticker", values="close").dropna()
px = ix["MCFTR"]
market = px.pct_change()

# Денежный индекс: начисление за календарные дни между закрытиями.
rate = data.load_ruonia(start="2010-01-01").set_index("date")["rate"] / 100
calendar = pd.date_range(rate.index.min(), px.index.max(), freq="D")
daily_rate = rate.reindex(calendar, method="ffill")
cash_index = (1 + daily_rate.shift(1).fillna(0) / 365).cumprod()
cash = cash_index.reindex(px.index).pct_change()

START = "2011-01-01"  # предыдущая история нужна для прогрева средних
market = market.loc[START:]
cash = cash.reindex(market.index)
assert market.notna().all() and cash.notna().all()
assert px.rolling(300).mean().shift(1).reindex(market.index).notna().all()
end_date = str(market.index[-1].date())
```

Деньги работают и в выходные. Поэтому нельзя разделить RUONIA на 252 и начислять её только в биржевые дни. Мы строим календарный денежный индекс,

затем берём его изменения между датами наблюдений рынка. Это приближённая модель капитализации по RUONIA, а не точная реконструкция банковского счёта или фонда денежного рынка. Ставка применяется только к денежной позиции; торговый сигнал от неё не зависит.

Последнее наблюдение в эксперименте – 29.09.2026. 2026 год неполный. Через перерывы в торгах индексная доходность связывает последние доступные закрытия, тогда как денежный счёт продолжает начисляться по календарю. Мы не предполагаем, что в отсутствие котировок можно было совершить сделку.

15.2.2 Минимальный движок

Вектор `target` ниже – позиция **после** сделки на закрытии текущего дня. Вектор `held` – позиция, с которой инвестор пришёл к этому закрытию. Сначала считаем доход старой позиции, затем списываем расходы на переключение и устанавливаем новую.

```
def sma_signal(prices, window=200):
    return (prices > prices.rolling(window).mean()).astype(float)

def run_target(target, risky, money, cost=0.001):
    """Позиции 0/1 после закрытия; cost – доля капитала за переключение."""
    target = target.reindex(risky.index).fillna(0).astype(float)
    assert target.isin([0, 1]).all()
    held = target.shift(1, fill_value=0)
    traded = (target - held).abs()
    gross = held * risky + (1 - held) * money
    net = (1 + gross) * (1 - cost * traded) - 1
    return pd.DataFrame({"return": net, "held": held,
                         "target": target, "traded": traded})

def backtest(signal, cost=0.001):
    # В строке t исполняем сигнал t-1; доход t получает старая позиция.
    target = signal.shift(1).reindex(market.index)
    return run_target(target, market, cash, cost)

signal = sma_signal(px)
bt = backtest(signal)
```

При каждом переходе 0⟶1 или 1⟶0 списывается 0,1% капитала после рыночного изменения. Это условная совокупная цена исполнения, а не тариф конкретного брокера. Переключение туда и обратно требует двух списаний. Для бинарной позиции такая запись особенно прозрачна: в начале теста капитал находится в деньгах, первый вход тоже оплачивается, в конце открытая позиция оценивается по рынку без принудительной продажи.

У дробных весов всё сложнее: они изменяются вместе с ценами, даже если инвестор ничего не делает. Оборот нужно считать относительно фактических весов

перед сделкой, а не относительно прошлых целевых весов. Мы уже учитывали это в портфельном примере главы 11.5.

15.2.3 Проверка на трёх датах

Большая таблица метрик плохо помогает найти ошибку в одной сделке. Гораздо полезнее искусственный ряд, для которого результат легко посчитать вручную. Пусть в первый день мы покупаем актив, во второй он дорожает на 10%, а на закрытии третьего дня мы его продаём. Денежная ставка нулевая.

```
toy = pd.Series([0.0, 0.10, 0.0])
check = run_target(pd.Series([1, 1, 0]), toy, toy * 0, cost=0.01)
expected = 0.99 * 1.10 * 0.99
assert np.isclose((1 + check["return"]).prod(), expected)
check.style.format(pct_fmt,    subset=["return"]).format("{:.0f}",    subset=["held",    "target",
"traded"])
```

	return	held	target	traded
0	−1,0%	0	1	1
1	10,0%	1	1	0
2	−1,0%	1	0	1

Вариант кода на R – в онлайн-версии книги.

Первый день приносит только расход на вход, второй – доход уже купленного актива, третий – расход на выход. Если прибыль 10% появилась до покупки или комиссия взята один раз вместо двух, дальнейшие графики анализировать рано.

15.3 Как меняется результат при исправлении ошибок

Теперь сравним корректный по нашей временной схеме расчёт с заведомо неправильным. Во втором варианте сигнал по сегодняшнему закрытию умножается на доходность, которая уже закончилась. Отдельно отключим издержки и доход на деньги, чтобы видеть вклад каждого допущения.

```
def metrics(ret, money):
    wealth = np.r_[1.0, (1 + ret).cumprod().values]
    years = (ret.index[-1] - ret.index[0]).days / 365.25
    excess = ret - money.reindex(ret.index)
    sd = excess.std()
    return {"CAGR": wealth[-1] ** (1 / years) - 1,
            "Волатильность": ret.std() * np.sqrt(252),
            "Шарп": excess.mean() / sd * np.sqrt(252) if sd > 1e-12 else np.nan,
            "Просадка": (wealth / np.maximum.accumulate(wealth) - 1).min()}

today = signal.reindex(market.index)
# Намеренно ошибочный вариант: решение сегодняшнего закрытия знает r_t.
```

```
bad_turn = today.diff().abs()
bad_turn.iloc[0] = today.iloc[0]
bad = (1 + today * market + (1 - today) * cash) * (1 - 0.001 * bad_turn) - 1
no_cost = backtest(signal, cost=0)["return"]
no_cash = run_target(signal.shift(1), market, cash * 0)["return"]
paths = {"MCFTR": market, "RUONIA": cash,
         "SMA200": bt["return"], "SMA200 без издержек": no_cost,
         "SMA200 без дохода на деньги": no_cash,
         "Ошибка времени": bad}
comparison = pd.DataFrame({k: metrics(v, cash) for k, v in paths.items()}).T
```

```
comparison.style.format(pct_fmt).format(ru_fmt, subset=["Шарп"])
```

Таблица 15.1. Одна стратегия при разных допущениях, с 2011 г. до конца снимка. Шарп рассчитан относительно RUONIA

	CAGR	Волатильность	Шарп	Просадка
MCFTR	8,3%	23,1%	0,06	−53,5%
RUONIA	9,9%	0,6%	–	0,0%
SMA200	8,5%	14,4%	−0,02	−26,9%
SMA200 без издержек	9,4%	14,4%	0,04	−26,0%
SMA200 без дохода на деньги	5,3%	14,4%	−0,23	−32,1%
Ошибка времени	24,4%	13,4%	1,00	−14,5%

Ошибка времени даёт 24,4% годовых, тогда как исполнение на следующем закрытии с издержками – 8,5%. Разница возникла без новой информации и без улучшения прогноза. Именно поэтому проверка временной схемы важнее выбора красивой метрики.

Стратегия находится в акциях в 70% наблюдений и переключается примерно 8,5 раза в год. При такой частоте даже небольшая цена сделки имеет накопленный эффект. Но бесплатное исполнение само по себе не доказывает наличие преимущества: доходность нужно сравнить ещё и с денежным рынком.

В этом сравнении SMA200 уменьшает максимальную просадку, но её CAGR ниже денежной альтернативы, которая даёт 9,9% годовых. Защита капитала от части падений и высокая доходность – разные свойства. Кроме того, один дополнительный день задержки иногда случайно помогает, иногда мешает: более позднее исполнение не обязано давать худший результат на каждой конкретной истории.

```
fig, ax = plt.subplots()
for label, color in [("MCFTR", plot.BLUE), ("SMA200", plot.ORANGE), ("RUONIA", plot.GREEN)]:
    ax.plot((1 + paths[label]).cumprod(), label=label, color=color)
ax.set(ylabel="Капитал", yscale="log")
ax.legend(frameon=False, ncol=3)
```

```
plot.source(ax, "Московская биржа, Банк России; расчёты автора")
plt.show()
```

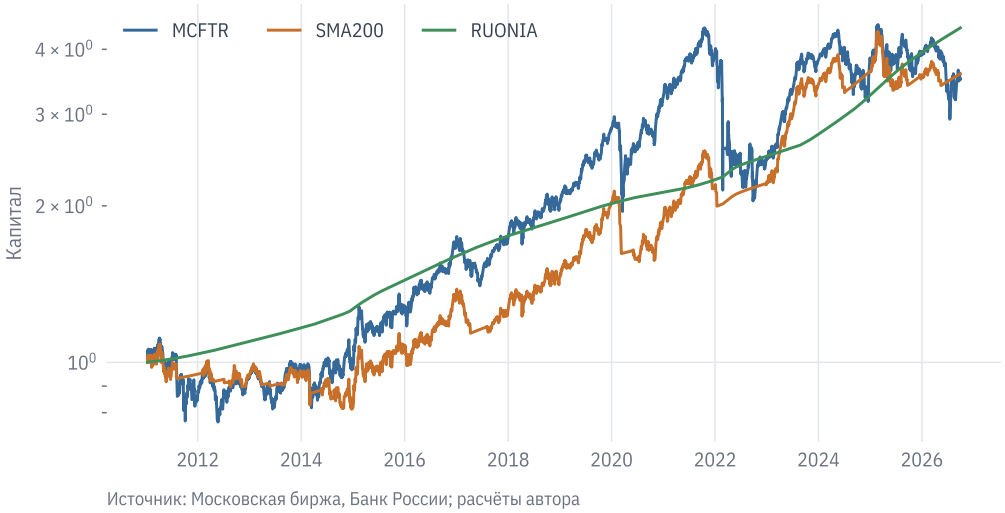


Рисунок 15.1. Капитал при владении индексом, следовании SMA200 и размещении денег под RUONIA. Начальный капитал равен единице

15.3.1 Какой бенчмарк нужен

У стратегии с денежной позицией два естественных вопроса. Первый: стоило ли вообще принимать рыночный риск вместо размещения денег под ставку? Второй: окупились ли переключения по сравнению с постоянным владением акциями? Для первого сравниваем с RUONIA, для второго – с MCFTR. Ни один бенчмарк не заменяет другой.

Индекс IMOEX не учитывает дивиденды. Если стратегия получает дивиденды, а её бенчмарк – нет, сравнение систематически благоприятствует стратегии. Это легко увидеть на тех же датах.

Ценовой индекс приносит 1,9% годовых против 8,3% у индекса полной доходности. Это разные экономические величины, хотя обе серии описывают один рынок.

Коэффициент Шарпа здесь рассчитан по **избыточным** доходностям: сначала вычитаем доход денег за тот же интервал, затем делим среднее на стандартное отклонение полученной серии. Для самого денежного счёта избыточная доходность тождественно нулевая, поэтому его Шарп не определён. Деление средней денежной доходности на небольшой разброс ставок дало бы впечатляющее число, но не измеряло бы премию за риск.

Годовая шкала волатильности и Шарпа через $\sqrt{252}$ – обычное приближение. При автокорреляции и нерегулярных интервалах оно не является точным преобразо-

ванием. CAGR мы считаем по календарной длине периода; он учитывает накопление капитала, но ничего не говорит о неопределённости будущего результата.

15.3.2 Проверка издержек

```
cost_rows = {}
for basis_points in (0, 5, 10, 25, 50):
    net = backtest(signal, cost=basis_points / 10000)["return"]
    cost_rows[f"{basis_points} б. п."] = metrics(net, cash)
pd.DataFrame(cost_rows).T.style.format(pct_fmt).format(ru_fmt, subset=["Шарп"])
```

Таблица 15.2. Чувствительность SMA200 к условной стоимости одного переключения

	CAGR	Волатильность	Шарп	Просадка
0 б. п.	9,4%	14,4%	0,04	−26,0%
5 б. п.	8,9%	14,4%	0,01	−26,5%
10 б. п.	8,5%	14,4%	−0,02	−26,9%
25 б. п.	7,1%	14,4%	−0,11	−31,8%
50 б. п.	4,8%	14,5%	−0,25	−41,6%

Один базисный пункт – 0,01 процентного пункта. Таблица – сценарный расчёт, а не оценка реальных расходов. Реальные издержки зависят от комиссии брокера и биржи, разницы цен покупки и продажи, размера заявки и её влияния на рынок. Одинаковые 10 б. п. для небольшой заявки в ликвидной акции и крупной заявки в редкоторгуемой бумаге – сильное допущение.

Для стратегии по отдельным акциям полезно дополнительно ограничить размер заявки долей **прошлого** дневного оборота. Если для ограничения используется полный оборот дня исполнения, доступный только вечером, снова возникает заглядывание в будущее. Нельзя также считать все заявки исполненными по цене закрытия при остановке торгов или отсутствии встречных заявок.

15.4 Проверка данных и причинности

15.4.1 Тест усечения истории

Сигнал, рассчитанный на данных до некоторой даты, должен совпадать с сигналом на ту же дату после добавления будущих наблюдений. Это простая проверка причинности.

```
cut = px.index[len(px) // 2]
pd.testing.assert_series_equal(
    sma_signal(px).loc[:cut],
    sma_signal(px.loc[:cut])
)
```

Проверка ловит, например, нормировку по среднему всей истории, центрированное скользящее окно и заполнение пропусков будущими значениями. Но она не доказывает, что исходная база была доступна инвестору: в сегодняшней выгрузке могут находиться пересмотренные данные, а состав бумаг мог быть выбран задним числом. Проверять нужно и код, и происхождение данных.

15.4.2 Бумаги, которые исчезли

Исследование на сегодняшнем списке акций отвечает на вопрос «что было бы с компаниями, дожившими до сегодня». Исторический инвестор не знал этот список. Для полноценного теста состав доступных бумаг нужно восстанавливать на каждую дату, включая выбывшие компании, изменения тикеров, сделки выкупа и приостановки торгов.

Исчезновение котировки не означает автоматически потерю 100% капитала: могла произойти смена тикера, реорганизация или выплата при выкупе. Но и переносить последнюю цену вперёд бесконечно, делая актив безрисковым, нельзя. Для каждого типа события нужно правило оценки и выхода. Справочники `renames` и `delisted` в `qf.data` помогают искать такие события; сами по себе они не восстанавливают все денежные потоки.

В этой главе индекс сокращает объём работы с отдельными корпоративными событиями, но не устраняет вопросы исполнимости. Если перейти от индексного примера к собственному портфелю, понадобятся история состава, доступные тогда данные, реальные дивиденды и правила обработки пропусков. Учебный снимок избранных акций не является полной исторической вселенной рынка.

> **Сначала предскажите**
>
> Программа выбирает десять акций с наибольшим средним оборотом за **весь** период 2011–2026 гг., а затем торгует ими начиная с 2011 г. Все сигналы сдвинуты на два дня. Осталось ли заглядывание в будущее?
>
> Да: в выборе списка бумаг. Корректный сдвиг сигнала не исправляет отбор активов по будущей ликвидности.

15.5 Когда лучший параметр оказывается случайным

До сих пор мы проверяли одно правило. Обычно исследователь пробует окна 50, 100, 150 и 200 дней, пересечения двух средних, ограничения волатильности, разные наборы бумаг. Даже если в каждом расчёте нет утечки, лучшая конфигурация получает преимущество от самого соревнования.

Это похоже на выбор самого удачливого из участников, бросающих монету. В большой группе почти наверняка найдётся длинная серия успехов. Чтобы оценить её необычность, нужно знать, сколько участников пытались её получить.

15.5.1 Сетка правил

```
signals = {}
for window in range(10, 310, 10):
    signals[f"SMA{window}"] = sma_signal(px, window)
for fast, slow in itertools.product([5, 10, 20, 50], [50, 100, 150, 200, 250]):
    if fast < slow:
        signals[f"MA{fast}/{slow}"] = (
            px.rolling(fast).mean() > px.rolling(slow).mean()).astype(float)

returns_grid = pd.DataFrame({k: backtest(s)["return"] for k, s in signals.items()})
excess_grid = returns_grid.sub(cash, axis=0)
sharpes = excess_grid.mean() / excess_grid.std() * np.sqrt(252)
best_name = sharpes.idxmax()
best_sharpe = float(sharpes.max())
```

```
sma_scores = sharpes.reindex([f"SMA{n}" for n in range(10, 310, 10)])
fig, ax = plt.subplots()
ax.plot(range(10, 310, 10), sma_scores, marker="o", markersize=3, color=plot.BLUE)
ax.axhline(0, color=plot.GREY, lw=0.8)
ax.set(xlabel="Окно средней, торговых дней", ylabel="Шарп к RUONIA")
plot.source(ax, "Московская биржа, Банк России; расчёты автора", offset=34)
plt.show()
```

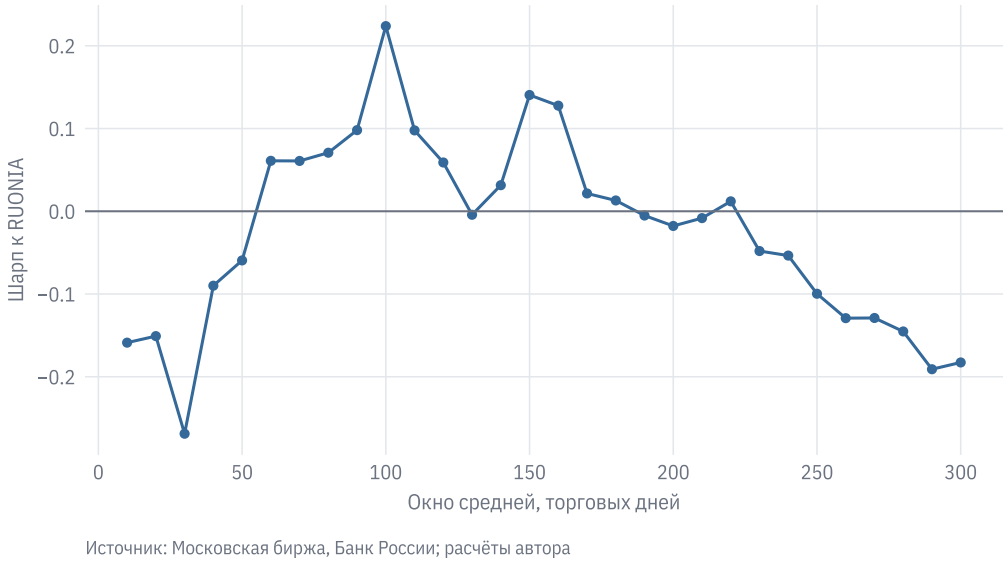


Рисунок 15.2. Коэффициент Шарпа относительно RUONIA при переборе окна средней. Все точки рассчитаны на одной истории

Из 49 конфигураций победила SMA100, её Шарп равен 0,22. Это описание результата поиска **на этой истории**. Оно не является оценкой качества заранее выбранного правила. Более того, настоящих попыток могло быть больше: смена

начальной даты, метрики победы или набора параметров тоже использует информацию о результатах.

Соседние окна помогают оценить хрупкость вывода. Один высокий пик среди слабых соседей настораживает сильнее широкого участка похожих результатов. Однако выбор «красивого плато» после просмотра графика тоже является подбором. Он должен быть записан в журнале экспериментов и затем проверен на других данных.

15.5.2 Последовательный выбор без знания будущего

Вместо единственного победителя за всю историю можно каждый год выбирать правило по предыдущим годам, фиксировать его и применять в следующем году. Это проверяет уже **процедуру выбора**, а не только одну среднюю.

Сделаем такую проверку с 2014 г. Сетка правил заранее фиксирована. На начало каждого года выбираем максимальный Шарп за всю доступную прошлую историю. При смене правила учитываем фактическую позицию, чтобы не потерять расходы на переход между стратегиями. Новый выбор впервые исполняется на первом закрытии нового года; до него остаётся прежняя позиция.

```
target_grid = pd.DataFrame({k: s.shift(1) for k, s in signals.items()})
selected_target = pd.Series(0.0, index=market.loc["2014":].index)
choices = {}
for year in sorted(selected_target.index.year.unique()):
    past = excess_grid.loc[:f"{year - 1}-12-31"]
    score = past.mean() / past.std()
    chosen = score.idxmax()
    choices[int(year)] = chosen
    dates = selected_target.index[selected_target.index.year == year]
    selected_target.loc[dates] = target_grid.loc[dates, chosen]

wf = run_target(selected_target, market.reindex(selected_target.index),
                cash.reindex(selected_target.index))
# Тот же старт и та же оплата первого входа для фиксированного правила.
fixed = run_target(target_grid[best_name], market.reindex(selected_target.index),
                   cash.reindex(selected_target.index))
wf_comparison = pd.DataFrame({
    "Ежегодный выбор": metrics(wf["return"], cash),
    "Победитель всей истории": metrics(fixed["return"], cash),
    "MCFTR": metrics(market.reindex(selected_target.index), cash),
    "RUONIA": metrics(cash.reindex(selected_target.index), cash)
}).T
```

```
wf_comparison.style.format(pct_fmt).format(ru_fmt, subset=["Шарп"])
```

Таблица 15.3. Последовательный выбор и ретроспективный победитель на общем периоде с 2014 г.

	CAGR	Волатильность	Шарп	Просадка
Ежегодный выбор	12,9%	13,3%	0,19	−22,6%
Победитель всей истории	15,4%	13,7%	0,35	−18,8%
MCFTR	10,5%	23,5%	0,10	−53,5%
RUONIA	11,0%	0,7%	–	0,0%

У ежегодного выбора нет доступа к доходностям будущего года. У строки «Победитель всей истории» такой доступ был на этапе выбора имени стратегии. Это разные эксперименты, даже если их кривые капитала построены одной функцией.

И всё же ежегодная процедура не превращает уже просмотренную историю в новый независимый тест. Мы выбрали саму сетку и способ ранжирования, зная этот рынок. Результат полезен как историческая проверка устойчивости. Для следующего шага нужно зафиксировать правило и дождаться новых данных либо использовать заранее отложенный период, который не участвовал в разработке. После изменения правила по результатам этого периода он перестаёт быть окончательным тестом.

15.6 Диагностика переобучения

15.6.1 PBO: как часто победитель теряет своё место

Метод CSCV делит историю на чётное число блоков. Для каждого способа выбрать половину блоков определяем лучшую конфигурацию на них и её ранг на остальных. Доля разбиений, в которых победитель оказывается ниже медианы остальных результатов, оценивает PBO (Bailey и др. 2017 г.).

```
def estimate_pbo(excess, n_blocks=8):
    blocks = np.array_split(excess.to_numpy(), n_blocks)
    ranks = []
    for train_ids in itertools.combinations(range(n_blocks), n_blocks // 2):
        test_ids = [i for i in range(n_blocks) if i not in train_ids]
        train = np.concatenate([blocks[i] for i in train_ids])
        test = np.concatenate([blocks[i] for i in test_ids])
        score_train = train.mean(0) / train.std(0, ddof=1)
        score_test = test.mean(0) / test.std(0, ddof=1)
        winner = score_train.argmax()
        rank = stats.rankdata(score_test, method="average")[winner]
        ranks.append(rank / (excess.shape[1] + 1))
    return float(np.mean(np.asarray(ranks) < 0.5)), len(ranks)

pbo8, splits8 = estimate_pbo(excess_grid, 8)
pbo10, splits10 = estimate_pbo(excess_grid, 10)
```

В нашем эксперименте PBO при восьми блоках равно 61% (70 разбиений), при десяти – 63%. Это частота неудачного **ранжирования**, а не вероятность будущего денежного убытка. Блоки здесь переставляются между обучением и проверкой, поэтому CSCV не заменяет последовательный тест во времени. Результат зависит от набора кандидатов и разбиения; малое PBO само по себе не гарантирует прибыль.

15.6.2 Дефлированный коэффициент Шарпа

DSR сравнивает наблюдаемый Шарп с порогом, повышенным из-за перебора, и учитывает длину выборки, асимметрию и четвёртый момент доходностей (Bailey и López de Prado 2014 г.). Условно: насколько убедителен победитель с учётом числа попыток, в которых он мог случайно появиться?

Число независимых попыток здесь не равно автоматически 49: соседние средние дают похожие позиции. Незарегистрированные неудачные идеи, напротив, увеличивают реальный объём поиска. Поэтому одна цифра DSR без журнала попыток и допущений о зависимости конфигураций создаёт ложную точность. Для учебного проекта полезнее честно показать всю сетку, последовательную проверку и чувствительность результата. Подробный расчёт DSR оставим дополнительным заданием; его нельзя трактовать как вероятность будущей прибыльности стратегии.

15.7 Почему прогноз ARIMA не равен торговой прибыли

В прошлогоднем ноутбуке курса стратегия строилась вокруг ARIMA: оценить модель, получить прогноз, купить при положительном прогнозе. Между этими шагами скрываются два разных вопроса. Первый – правильно ли прогноз привязан ко времени. Второй – достаточно ли его величины, чтобы оправдать отказ от денежной доходности и оплатить сделки.

Если модель обучена до $t-1$, вызов одношагового прогноза предсказывает t, а не $t+1$. Нельзя просто записать число в следующую строку и тем самым продлить горизонт. Между переоценками параметров состояние модели должно обновляться новыми наблюдениями; повторный вызов прогноза без обновления повторяет старый прогноз.

Для прозрачности возьмём AR(1), то есть ARIMA(1,0,0), для простых дневных доходностей MCFTR:

$$r_t = c + \phi r_{t-1} + \varepsilon_t.$$

В нашем соглашении решение после закрытия t исполняется на закрытии $t+1$. Поэтому первая доходность позиции – r_{t+2}, и сравнивать с денежной альтернативой нужно **двухшаговый** прогноз $c + \phi(c + \phi r_t)$. Параметры переоцениваем раз в год только на прошлой истории. Для денежной альтернативы используем

последнюю ранее наблюдавшуюся ставку, а для простоты – однодневное начисление; это приближение особенно грубо перед длинными выходными.

```
full_r = px.pct_change()
ar_signal = pd.Series(0.0, index=px.index)
ar_parameters = []
for year in sorted(market.index.year.unique()):
    train = pd.DataFrame({"y": full_r, "lag": full_r.shift(1)}).loc[
        :f"{year - 1}-12-31"].dropna()
    design = np.column_stack([np.ones(len(train)), train["lag"]])
    intercept, phi = np.linalg.lstsq(design, train["y"], rcond=None)[0]
    dates = px.index[px.index.year == year]
    forecast2 = intercept + phi * (intercept + phi * full_r.reindex(dates))
    known_rate = rate.reindex(calendar, method="ffill").shift(1).reindex(dates)
    ar_signal.loc[dates] = (forecast2 > known_rate / 365).astype(float)
    ar_parameters.append({"year": int(year), "phi": float(phi)})

ar_bt = backtest(ar_signal)
ar_comparison = pd.DataFrame({
    "AR(1)": metrics(ar_bt["return"], cash),
    "SMA200": metrics(bt["return"], cash),
    "MCFTR": metrics(market, cash), "RUONIA": metrics(cash, cash)
}).T
```

```
ar_comparison.style.format(pct_fmt).format(ru_fmt, subset=["Шарп"])
```

Таблица 15.4. Прогноз AR(1), преобразованный в позицию: тот же календарь исполнения и те же издержки

	CAGR	Волатильность	Шарп	Просадка
AR(1)	7,9%	23,1%	0,04	−53,5%
SMA200	8,5%	14,4%	−0,02	−26,9%
MCFTR	8,3%	23,1%	0,06	−53,5%
RUONIA	9,9%	0,6%	–	0,0%

В этой реализации AR(1) даёт 7,9% годовых и Шарп к денежному рынку 0,04. По накопленному капиталу правило уступает деньгам. Небольшой положительный Шарп этому не противоречит: он использует среднюю арифметическую избыточную доходность, тогда как CAGR учитывает геометрическое накопление и потери от колебаний. Вывод относится к нашему правилу, горизонту и периоду; он не является доказательством бесполезности всего класса ARIMA.

Это намеренно простое торговое правило. Условие «прогноз выше ставки» не учитывает риск ошибки, вероятную продолжительность позиции и будущие расходы на выход. Небольшое изменение прогноза возле порога может вызывать частые сделки без существенного экономического преимущества. Добавление

зоны бездействия способно снизить оборот, но её ширина станет ещё одним параметром поиска.

Сравнение моделей по AIC отвечает на вопрос о статистическом описании ряда. Оно не выбирает автоматически торговую стратегию с лучшей доходностью после издержек. Положительный прогноз тоже не равен предложению купить: он может быть ниже дохода денег. Поэтому отчёт о прогнозной модели и отчёт о стратегии должны содержать разные проверки.

15.8 Как оформить результат исследования

Хороший отчёт позволяет другому человеку восстановить, что именно было известно инвестору перед каждой сделкой. Для проекта достаточно краткого протокола и воспроизводимого ноутбука.

1. **Гипотеза.** Какой механизм предполагается и какой результат будет считаться её опровержением.
2. **Данные.** Источник, дата снимка, период, исторический состав бумаг, корректировки, пропуски и корпоративные события.
3. **Временная схема.** Когда доступны признаки, когда формируется сигнал, по какой цене и когда исполняется заявка.
4. **Учёт капитала.** Денежная ставка, дивиденды, комиссии, проскальзывание, ограничения объёма, способ оценки открытых позиций.
5. **История поиска.** Все испытанные варианты, включая отрицательные; отдельно период разработки и проверки.
6. **Результат.** Капитал, просадка, доходность, избыточный Шарп, оборот и чувствительность к издержкам рядом с бенчмарками на одинаковых датах.
7. **Ограничения.** Какие допущения особенно влияют на вывод и какие новые данные могли бы его изменить.

ИИ-ассистент может помочь написать движок или найти несогласованные даты. Но он способен столь же быстро сгенерировать сотню новых конфигураций, увеличив объём поиска. Записывать нужно не число запросов к ассистенту, а число реально просмотренных вариантов и причины изменений. Отрицательный результат при корректной методике – полноценный результат исследования.

Итоги

- Бэктест начинается с временной схемы. При сигнале после закрытия и исполнении на следующем закрытии новая позиция получает доход только со следующего интервала.
- Дивиденды, доход свободных денег и цена переключений могут существенно изменить сравнение. Для стратегии с выходом в деньги нужны и рыночный, и денежный бенчмарки.

- Маленький пример с ручным расчётом и тест усечения истории находят ошибки, которые легко скрываются в длинной кривой капитала.
- Перебор параметров создаёт преимущество у победителя даже без утечки в каждом отдельном расчёте. Проверять нужно всю процедуру выбора.
- PBO и DSR помогают обсуждать переобучение, но не заменяют корректные данные, последовательную проверку и новые наблюдения.
- Прогноз доходности превращается в стратегию только после задания горизонта, правила позиции и модели исполнения.

15.9 Упражнения

15.9.1 А. Понимание

1. Сигнал рассчитан после закрытия понедельника, сделка исполняется на открытии вторника. Какие цены нужны для расчёта первой доходности? Почему недостаточно умножить `signal.shift(1)` на доходность между закрытиями?
2. За год стратегия заработала 12%, индекс полной доходности – 9%, денежный рынок – 15%. Какие из утверждений верны: стратегия обогнала рынок; стратегия принесла положительную избыточную доходность к деньгам; торговые сигналы создали ценность? Какой дополнительный анализ нужен для последнего утверждения?
3. Целевые веса двух акций остаются 50/50. Одна выросла на 20%, другая не изменилась. Найдите фактические веса перед ребалансировкой и долю капитала, которую нужно переложить. Почему неизменность целевых весов не означает нулевой оборот?
4. PBO равно 40%. Можно ли сказать, что с вероятностью 40% инвестор потеряет деньги? Объясните, какое событие оценивается.

15.9.2 Б. Практика на данных

5. Повторите SMA200 для своей акции, используя `adj_close`. Сравните с владением этой акцией и денежным рынком на одинаковых датах. Укажите, какие допущения заложены в скорректированную цену.
6. Проверьте стоимость переключения 0, 10, 25 и 50 б. п. Посчитайте число переключений и долю времени в акции. Объясните чувствительность результата через эти величины.
7. Проведите тест усечения истории на трёх датах. Затем намеренно замените обычную среднюю на центрированную и покажите, что тест обнаруживает ошибку.
8. Разделите историю на несколько заранее указанных подпериодов. Покажите, зависит ли преимущество правила от одного кризиса. Не выбирайте новый параметр отдельно для каждого периода после просмотра результата.

15.9.3 В. Работа с ИИ-ассистентом

9. Попросите ассистента написать стратегию по скользящей средней. До запуска отметьте в коде момент доступности сигнала, момент сделки, период доходности и списание издержек. Проверьте код на трёх датах с известным ответом. Приложите исходный вариант, обнаруженные ошибки и исправления.
10. Передайте ассистенту таблицу всех вариантов из сетки и попросите назвать лучший. Затем попросите объяснить, как выбор победителя меняет интерпретацию его Шарпа. Проверьте, не выдаёт ли ответ исторический максимум за независимую оценку качества.

15.9.4 Г. Мини-исследование

11. Зафиксируйте одну гипотезу, две-три конфигурации, бенчмарки и критерий отказа от гипотезы. Проведите последовательную проверку, стресс по издержкам и аудит доступности данных. Сдайте журнал всех попыток и краткий вывод, включая отрицательный результат, если преимущество не подтверждается.
12. Дополнительно: реализуйте DSR по статье Bailey и López de Prado (2014 г.). Укажите единицы Шарпа, способ оценки моментов, предполагаемое число независимых попыток и чувствительность результата к этому числу. Объясните, почему для коррелированных доходностей стандартная формула требует дополнительной осторожности.

Вопросы для самопроверки

1. Что именно сдвигает `shift(2)` в нашем примере?
2. Почему RUONIA начисляется по календарю, а не по числу биржевых строк?
3. Почему тест усечения истории не выявляет все формы заглядывания в будущее?
4. Чем последовательный выбор отличается от применения лучшей конфигурации за всю историю?
5. Когда ранее отложенный тест перестаёт быть независимым?
6. Почему для торгового правила AR(1) здесь используется двухшаговый прогноз?

Ответы

1. Он сопоставляет интервалу доходности сигнал, сформированный двумя наблюдениями ранее: после сигнала проходит одно закрытие до исполнения и ещё одно до записи первого дохода. Это следствие конкретной модели исполнения, а не универсальное правило.
2. Между соседними торгами могут пройти выходные или длительный перерыв. Деньги при этом продолжают приносить доход.

3. Код может быть причинным относительно предоставленной таблицы, но сама таблица могла быть пересмотрена, а бумаги – отобраны по будущим результатам.
4. Последовательный выбор использует для каждого решения только прошлую часть истории. Ретроспективный победитель выбран с использованием будущих результатов относительно начала торговли.
5. Когда его результат используют, чтобы изменить правило, параметры, признаки или способ оценки. Он становится частью разработки.
6. Сигнал после закрытия t исполняется на закрытии $t + 1$, поэтому первая доходность новой позиции относится к $t + 2$.

Что почитать

- Bailey и др. (2017 г.) – вероятность переобучения бэктеста и метод CSCV.
- Bailey и López de Prado (2014 г.) – дефлированный коэффициент Шарпа и поправка на множественный поиск.
- López de Prado (2018 г.) – организация финансовых экспериментов и ограничения обычных схем проверки моделей.

16 Моментум и управление волатильностью

Глава готовится

Эта глава входит во второй этап работы над книгой. Планируемое содержание:

- Кросс-секционный моментум на российском рынке
- Фильтр ликвидности, делистинги, издержки
- Трендовый фильтр и размещение денег на денежном рынке
- Таргетирование волатильности
- Проверка на новых данных

17 Микроструктура рынка и как её измеряют

Глава готовится

Эта глава входит во второй этап работы над книгой. Планируемое содержание:

- Ликвидность и эффективность: спред, глубина, ценовое воздействие; модели Кайла, Ролла, Глостена – Милгрома
- Меры ликвидности по дневным данным: Амихуд, Ролл, Корвин – Шульц, доля нулевых доходностей
- Эффективность по дневным данным: автокорреляция и коэффициент дисперсий Ло – Маккинли
- Насколько точны дневные меры: сравнение с котируемым спредом по внутридневным данным
- Анатомия журнала заявок Мосбиржи и восстановление стакана
- Один день в стакане Сбербанка: микроцена, спред, глубина, дисбаланс потока заявок
- Российский рынок после 2022 г.: больше розничных инвесторов – меньше ли эффективность?
- Предсказуемость не означает прибыль: транзакционные издержки

Литература

Список включает учебники, на которые опирается курс, классические работы по темам глав и источники данных книги. Работы, упомянутые в тексте, приводятся также в разделах «Что почитать» в конце глав.

Alexander, Carol. 2008 г. *Market Risk Analysis. Volume II: Practical Financial Econometrics*. Wiley.

Bailey, David H., Jonathan M. Borwein, Marcos López de Prado, и Qiji Jim Zhu. 2017 г. «The Probability of Backtest Overfitting». *The Journal of Computational Finance* 20 (4): 39–69. https://www.davidhbailey.com/dhbpapers/backtest-prob.pdf.

Bailey, David H., и Marcos López de Prado. 2014 г. «The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality». *The Journal of Portfolio Management* 40 (5): 94–107.

Bodie, Zvi, Alex Kane, и Alan J. Marcus. 2021 г. *Investments*. 12-я изд. McGraw-Hill.

Bollerslev, Tim. 1986 г. «Generalized Autoregressive Conditional Heteroskedasticity». *Journal of Econometrics* 31 (3): 307–27.

Bondarkov, Sergey, Victor Ledenev, и Dmitriy Skougarevskiy. 2025 г. «Russian Financial Statements Database». *Scientific Data* 12. https://doi.org/10.1038/s41597-025-05150-1.

Campbell, John Y., Andrew W. Lo, и A. Craig MacKinlay. 1997 г. *The Econometrics of Financial Markets*. Princeton University Press.

Campbell, John Y., и Samuel B. Thompson. 2008 г. «Predicting Excess Stock Returns Out of Sample: Can Anything Beat the Historical Average?» *The Review of Financial Studies* 21 (4): 1509–31.

Cont, Rama. 2001 г. «Empirical Properties of Asset Returns: Stylized Facts and Statistical Issues». *Quantitative Finance* 1 (2): 223–36. https://doi.org/10.1080/713665670.

Corsi, Fulvio. 2009 г. «A Simple Approximate Long-Memory Model of Realized Volatility». *Journal of Financial Econometrics* 7 (2): 174–96.

DeMiguel, Victor, Lorenzo Garlappi, и Raman Uppal. 2009 г. «Optimal Versus Naive Diversification: How Inefficient is the 1/N Portfolio Strategy?» *The Review of Financial Studies* 22 (5): 1915–53.

Engle, Robert F. 1982 г. «Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation». *Econometrica* 50 (4): 987–1007.

Glosten, Lawrence R., Ravi Jagannathan, и David E. Runkle. 1993 г. «On the Relation between the Expected Value and the Volatility of the Nominal Excess Return on Stocks». *The Journal of Finance* 48 (5): 1779–801.

Gu, Shihao, Bryan Kelly, и Dacheng Xiu. 2020 г. «Empirical Asset Pricing via Machine Learning». *The Review of Financial Studies* 33 (5): 2223–73.

Hansen, Peter R., и Asger Lunde. 2005 г. «A Forecast Comparison of Volatility Models: Does Anything Beat a GARCH(1,1)?» *Journal of Applied Econometrics* 20 (7): 873–89.

Herndon, Thomas, Michael Ash, и Robert Pollin. 2014 г. «Does High Public Debt Consistently Stifle Economic Growth? A Critique of Reinhart and Rogoff». *Cambridge Journal of Economics* 38 (2): 257–79.

Hilpisch, Yves. 2018 г. *Python for Finance: Mastering Data-Driven Finance*. 2-я изд. O'Reilly.

Hyndman, Rob J., и George Athanasopoulos. 2021 г. *Forecasting: Principles and Practice*. 3-я изд. OTexts. https://otexts.com/fpp3/.

J.P. Morgan/Reuters. 1996 г. *RiskMetrics – Technical Document*. 4-я изд. J.P. Morgan.

Ledoit, Olivier, и Michael Wolf. 2004 г. «Honey, I Shrunk the Sample Covariance Matrix». *The Journal of Portfolio Management* 30 (4): 110–19.

Lo, Andrew W., и A. Craig MacKinlay. 1988 г. «Stock Market Prices Do Not Follow Random Walks: Evidence from a Simple Specification Test». *The Review of Financial Studies* 1 (1): 41–66.

López de Prado, Marcos. 2018 г. *Advances in Financial Machine Learning*. Wiley.

Markowitz, Harry. 1952 г. «Portfolio Selection». *The Journal of Finance* 7 (1): 77–91.

McKinney, Wes. 2022 г. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. 3-я изд. O'Reilly. https://wesmckinney.com/book/.

Merton, Robert C. 1980 г. «On Estimating the Expected Return on the Market: An Exploratory Investigation». *Journal of Financial Economics* 8 (4): 323–61.

Michaud, Richard O. 1989 г. «The Markowitz Optimization Enigma: Is „Optimized« Optimal?» *Financial Analysts Journal* 45 (1): 31–42.

Patton, Andrew J. 2011 г. «Volatility Forecast Comparison Using Imperfect Volatility Proxies». *Journal of Econometrics* 160 (1): 246–56.

Reinhart, Carmen M., и Kenneth S. Rogoff. 2010 г. «Growth in a Time of Debt». *American Economic Review* 100 (2): 573–78.

Roll, Richard. 1984 г. «A Simple Implicit Measure of the Effective Bid-Ask Spread in an Efficient Market». *The Journal of Finance* 39 (4): 1127–39.

Sargent, Thomas J., и John Stachurski. 2024 г. *Python Programming for Economics and Finance*. QuantEcon. https://python-programming.quantecon.org.

Scheuch, Christoph, Stefan Voigt, Patrick Weiss, и Christoph Frey. 2024 г. *Tidy Finance with Python*. Chapman; Hall/CRC. https://www.tidy-finance.org.

Sharpe, William F. 1966 г. «Mutual Fund Performance». *The Journal of Business* 39 (1): 119–38.

Tsay, Ruey S. 2010 г. *Analysis of Financial Time Series*. 3-я изд. Wiley.

VanderPlas, Jake. 2023 г. *Python Data Science Handbook: Essential Tools for Working with Data*. 2-я изд. O'Reilly.

Welch, Ivo, и Amit Goyal. 2008 г. «A Comprehensive Look at the Empirical Performance of Equity Premium Prediction». *The Review of Financial Studies* 21 (4): 1455–508.

Приложения

A Основы Python для финансиста

Код в этой книге короткий: обычно это 5–15 строк, которые загружают данные, что-то считают и строят график. Чтобы читать и менять такой код, не нужно быть программистом. Достаточно понимать несколько десятков конструкций языка и трёх библиотек: NumPy для вычислений, pandas для таблиц и matplotlib для графиков. Это приложение собирает их в одном месте.

Приложение не заменяет учебник по Python. Оно отвечает на практический вопрос: что нужно знать, чтобы уверенно работать с кодом глав и ноутбуков курса. Каждый пример стоит запустить у себя и немного изменить: поменять тикер, дату, ставку. Перед запуском полезно предсказать результат. Если предсказание не совпало с выводом, вы нашли место, которое пока понимаете неточно, и это самая ценная информация при обучении.

В этом приложении

- Где запускать код и почему важен порядок выполнения ячеек.
- Числа, строки, логические значения; списки, словари и другие контейнеры.
- Условия, циклы и функции на финансовых примерах: налог, сложный процент, NPV и IRR.
- Как читать сообщения об ошибках.
- NumPy, pandas и matplotlib: от массива чисел до графика цен акций.

Данные: `stocks.parquet`, `sectors.csv` (см. данные книги). **Нужно знать:** ничего; приложение рассчитано на тех, кто начинает с нуля.

A.1 Где запускать код

A.1.1 Ноутбук и ячейки

Код курса удобнее всего запускать в **ноутбуке** (notebook) – документе, который состоит из ячеек. Ячейка бывает двух видов: с кодом и с текстом в формате

Markdown. Ноутбуки открываются в Jupyter на своём компьютере или в Google Colab в браузере. Расширение файла – `.ipynb`.

Ячейку с кодом запускает сочетание **Shift + Enter**. Под ячейкой появляется результат: всё, что напечатано функцией `print()`, и значение последней строки ячейки. Предыдущие строки ячейки тоже выполняются, но их значения не выводятся:

```
price = 280.0
quantity = 100
price * quantity
#> 28000.0
```

За выполнением кода стоит **ядро** (kernel) – запущенный процесс Python, который хранит в памяти все созданные переменные. Переменная `price` из ячейки выше останется доступной во всех следующих ячейках, пока ядро работает.

A.1.2 Порядок выполнения и скрытое состояние

Ядро помнит не то, что написано в ноутбуке сейчас, а то, что было выполнено. Это главный источник путаницы у начинающих. Допустим, вы выполнили ячейку `price = 280.0`, затем исправили её на `price = 300.0`, но не запустили заново. В памяти осталось 280, и все последующие расчёты будут использовать старое число, хотя на экране написано 300. Бывает и обратное: вы удалили ячейку, в которой создавалась переменная, а ноутбук продолжает работать – до первого перезапуска.

> **Типичная ошибка**
>
> Ноутбук работает у автора, но падает у проверяющего. Почти всегда причина в том, что ячейки выполнялись не по порядку или какая-то переменная осталась в памяти от удалённого кода. Перед сдачей работы перезапустите ядро и выполните все ячейки сверху вниз: в Jupyter – **Kernel → Restart Kernel and Run All Cells**, в Colab – **Среда выполнения → Перезапустить сеанс и выполнить все**. Если ноутбук дошёл до конца без ошибок, его результат воспроизводим.

A.1.3 Установка

В **Google Colab** ничего устанавливать не нужно: Python и основные библиотеки уже есть, достаточно Google-аккаунта и браузера. Это самый быстрый способ начать. Минус в том, что сеанс Colab временный: после перезапуска все переменные и скачанные файлы пропадают, и ноутбук нужно выполнить заново.

На своём компьютере нужен Python 3.11 или новее. Проще всего установить дистрибутив Miniconda и создать для курса отдельное **окружение** – изолированную копию Python со своим набором пакетов. Тогда пакеты курса не будут конфликтовать с другими программами:

```
conda create -n qf python=3.12
conda activate qf
pip install -r requirements.txt
jupyter lab
```

Файл `requirements.txt` лежит в корне репозитория книги и перечисляет все нужные пакеты. Как скачать репозиторий и данные, описано в разделе 1.3. Кроме Jupyter, ноутбуки открывают редакторы VS Code и Positron; на результат расчётов выбор редактора не влияет.

A.2 Значения, переменные и типы

A.2.1 Переменные

Переменная – это имя, связанное со значением. Знак `=` в Python означает не равенство, а присваивание: вычисли выражение справа и запомни результат под именем слева. Поэтому запись, бессмысленная в математике, в коде совершенно нормальна:

```
cash = 100_000          # подчёркивание разделяет разряды и не влияет на число
cash = cash - 28_000    # купили акций на 28 тыс. руб.
cash
#> 72000
```

Имена переменных пишут строчными буквами, слова разделяют подчёркиванием: `start_date`, `risk_free_rate`. Хорошее имя объясняет, что лежит в переменной, и избавляет от комментариев. Сравните `x = 0.16` и `key_rate = 0.16`.

A.2.2 Числа и арифметика

У каждого значения есть **тип**. Числа бывают целыми (`int`) и дробными (`float`, от *floating point* – число с плавающей точкой). Тип можно узнать функцией `type()`:

```
print(type(100), type(280.5), type("SBER"), type(True))
#> <class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

Арифметические операции привычны, кроме двух: возведение в степень записывается как `**`, а не `^`, и есть отдельное целочисленное деление.

Таблица А.1. Арифметические операции

Операция	Запись	Пример	Результат
сложение, вычитание	`+`, `-`	`280 - 275`	`5`
умножение, деление	`*`, `/`	`7 / 2`	`3.5`
целочисленное деление, остаток	`//`, `%`	`7 // 2`, `7 % 2`	`3`, `1`
степень	`**`	`1.16 ** 3`	`1.560896`

Например, вклад 100 тыс. руб. под 16% годовых с ежегодной капитализацией через три года вырастет до:

```
100_000 * (1 + 0.16) ** 3
#> 156089.59999999995
```

> **Сначала предскажите**
> Что вернёт сравнение `0.1 + 0.2 == 0.3`: `True` или `False`? Запишите ответ, потом запустите код.

Дробные числа хранятся в компьютере в двоичном виде, поэтому многие десятичные дроби представлены не точно. Отсюда известный результат:

```
0.1 + 0.2 == 0.3
#> False
```

Для финансовых расчётов это почти никогда не важно: погрешность составляет около 10^{-16}. Но сравнивать дробные числа через `==` не стоит. Для сравнения есть функция `math.isclose()`, а в pandas и NumPy – `np.isclose()`.

А.2.3 Строки и форматирование

Строка (`str`) – текст в кавычках: `"SBER"`, `'2026-01-01'`. Строки можно складывать (`"SB" + "ER"`), узнавать их длину функцией `len()` и менять методами вроде `.upper()` и `.lower()`.

На практике со строками чаще всего приходится выводить числа в читаемом виде. Для этого служат **f-строки**: перед кавычкой ставится буква `f`, а внутри фигурных скобок пишется выражение и, через двоеточие, формат:

```
value = 28_045.5
ret = 0.0842
print(f"Стоимость пакета: {value:,.0f} руб.")
#> Стоимость пакета: 28,046 руб.
print(f"Доходность: {ret:.1%}")
#> Доходность: 8.4%
```

Формат `:,.0f` означает «разделить тысячи запятой, без знаков после точки», `:.1%` – «умножить на 100 и показать один знак после точки со знаком процента». Формат меняет только вид числа: в переменной `ret` по-прежнему хранится 0.0842, а не 8,4.

A.2.4 Логические значения и сравнения

Сравнение возвращает **логическое значение** (`bool`): `True` или `False`. Операторы сравнения – `==` (равно), `!=` (не равно), `<`, `<=`, `>`, `>=`. Условия объединяют словами `and`, `or`, `not`:

```
price = 280.0
price > 250 and price < 300
#> True
```

Не путайте `=` и `==`. Первое присваивает значение, второе сравнивает два значения. Ошибка `if price = 250:` вызовет `SyntaxError`.

Особое значение `None` означает «ничего»: например, функцию, которая ничего не возвращает, или ещё не заданный параметр. С `None` нельзя производить вычисления.

A.3 Контейнеры

Отдельные числа встречаются редко. Обычно у нас набор тикеров, ряд цен, состав портфеля. Для хранения нескольких значений в Python есть **контейнеры**. В финансовых расчётах чаще всего нужны четыре.

A.3.1 Списки

Список (`list`) – упорядоченный набор значений в квадратных скобках:

```
tickers = ["SBER", "GAZP", "LKOH", "MGNT"]
len(tickers)
#> 4
```

> **Сначала предскажите**
> Какой тикер вернёт `tickers[1]`? Сколько элементов и какие вернёт `tickers[1:3]`? Ответьте до того, как прочитаете следующий абзац.

Элементы списка нумеруются **с нуля**: первый элемент – `tickers[0]`, второй – `tickers[1]`. Отрицательный номер считает с конца: `tickers[-1]` – последний элемент. Срез `tickers[1:3]` возвращает элементы с номерами 1 и 2: левая граница включается, правая – нет.

```
print(tickers[0], tickers[-1], tickers[1:3])
#> SBER MGNT ['GAZP', 'LKOH']
```

Нумерация с нуля и правило «правая граница не включается» сначала кажутся неудобными, но к ним быстро привыкают. Они согласованы: `tickers[:2]` и `tickers[2:]` вместе дают весь список без повторов и пропусков.

Список можно изменять: менять элементы, добавлять новые методом `.append()`, сортировать функцией `sorted()`. Функции `sum()`, `min()`, `max()` работают со списками чисел:

```
closes = [280.74, 279.33, 276.35, 272.45]   # SBER, закрытие 23–28.09.2026
closes.append(274.65)                       # добавили 29.09.2026
sum(closes) / len(closes)
#> 276.704
```

А.3.2 Кортежи

Кортеж (`tuple`) похож на список, но записывается в круглых скобках и не может быть изменён после создания. Кортеж удобен для небольшой группы связанных значений: тикер и цена, начальная и конечная дата периода. Кортеж можно **распаковать** в несколько переменных:

```
period = ("2021-01-04", "2026-11-03")
start, end = period
start
#> '2021-01-04'
```

А.3.3 Словари

Словарь (`dict`) хранит пары «ключ – значение» и позволяет найти значение по ключу. Это естественный способ записать портфель: тикер → число акций.

```
portfolio = {"SBER": 100, "LKOH": 5, "GAZP": 300}
prices = {"SBER": 274.65, "LKOH": 5_369.0, "GAZP": 97.58}   # закрытие 29.09.2026
portfolio["LKOH"]
#> 5
```

Ключи в словаре уникальны. Если присвоить значение существующему ключу, старое значение будет заменено; если ключа нет, он будет добавлен: `portfolio["MGNT"] = 2`. Обращение к отсутствующему ключу вызывает ошибку `KeyError`. Методы `.keys()`, `.values()` и `.items()` возвращают ключи, значения и пары; мы воспользуемся ими в разделе о циклах.

A.3.4 Множества

Множество (`set`) хранит только уникальные значения, без порядка. Оно полезно, когда нужно убрать повторы или сравнить два набора. Например, какие бумаги портфеля отсутствуют в списке тикеров, по которым у нас есть данные?

```
set(portfolio) - set(tickers)
#> set()
```

Ответ пуст: все три бумаги портфеля есть в списке. Если добавить в портфель бумагу, которой нет в `tickers`, она появится в результате. Такая проверка часто нужна перед расчётом: она ловит опечатки в тикерах и бумаги, которые выпали из данных.

Таблица A.2. Основные контейнеры Python

Контейнер	Запись	Изменяемый	Когда использовать
список	`[1, 2, 3]`	да	упорядоченный ряд значений
кортеж	`(1, 2)`	нет	несколько связанных значений, распаковка
словарь	`{"SBER": 100}`	да	поиск значения по ключу
множество	`{1, 2, 3}`	да	уникальные значения, сравнение наборов

A.4 Условия и циклы

A.4.1 Условия: if, elif, else

Код не всегда должен выполняться одинаково. Конструкция `if` выполняет блок строк, только если условие истинно. Блок выделяется **отступом** в четыре пробела – в Python отступы являются частью синтаксиса, а не оформлением.

Упрощённый пример: налог с прибыли от продажи акций по ставке 13%; при убытке налога нет.

```
profit = -15_000
if profit > 0:
    tax = 0.13 * profit
else:
    tax = 0
tax
#> 0
```

Если вариантов больше двух, добавляют ветки `elif` («иначе если»). Python проверяет условия сверху вниз и выполняет первую подходящую ветку:

```
ret = -0.12
if ret > 0.10:
    label = "сильный рост"
elif ret > 0:
    label = "рост"
elif ret > -0.10:
    label = "снижение"
else:
    label = "сильное снижение"
label
#> 'сильное снижение'
```

A.4.2 Цикл for

Цикл `for` повторяет блок кода для каждого элемента контейнера. Посчитаем стоимость портфеля: для каждой бумаги умножим число акций на цену и накопим сумму.

```
total = 0
for ticker, quantity in portfolio.items():
    total = total + quantity * prices[ticker]
total
#> 83584.0
```

Метод `.items()` возвращает пары «ключ – значение», и на каждом шаге цикла пара распаковывается в переменные `ticker` и `quantity`. Если нужен номер шага, используют `enumerate()`, а для повторения заданное число раз – `range()`: `range(5)` даёт числа 0, 1, 2, 3, 4.

Цикл естественно описывает рост капитала по годам. Пусть годовые доходности были +10%, −5% и +20%:

```
wealth = 100
for year, r in enumerate([0.10, -0.05, 0.20], start=1):
    wealth = wealth * (1 + r)
    print(f"Год {year}: {wealth:.1f}")
#> Год 1: 110.0
#> Год 2: 104.5
#> Год 3: 125.4
```

Итог – 125,4, а не 125, как дала бы сумма доходностей. Доходность каждого года начисляется на уже изменившийся капитал. Подробнее об этом – в главе о доходности и риске (раздел 4.1).

А.4.3 Списковые включения

Частая задача – построить новый список, применив операцию к каждому элементу старого. Это можно сделать циклом, но в Python есть более короткая запись – **списковое включение** (list comprehension):

```
returns = [0.10, -0.05, 0.20, -0.15, 0.08]
growth = [1 + r for r in returns]
losses = [r for r in returns if r < 0]
print(growth)
#> [1.1, 0.95, 1.2, 0.85, 1.08]
print(losses)
#> [-0.05, -0.15]
```

Читать эту запись удобно справа налево: «для каждого `r` из `returns`, если `r` меньше нуля, взять `r`».

А.4.4 Цикл while

Цикл `while` повторяет блок, пока условие истинно. Он нужен, когда число повторений заранее неизвестно. За сколько лет вклад под 16% годовых удвоится?

```
wealth, years = 1.0, 0
while wealth < 2:
    wealth = wealth * 1.16
    years = years + 1
years
#> 5
```

Если условие никогда не станет ложным, цикл будет выполняться бесконечно. В Jupyter его останавливает кнопка **Interrupt** (■).

А.5 Функции

А.5.1 Зачем нужны функции

Функция – именованный фрагмент кода, который принимает входные значения (**аргументы**) и возвращает результат. Встроенными функциями вы уже пользовались: `len()`, `sum()`, `print()`. Свою функцию стоит написать, когда один и тот же расчёт повторяется несколько раз: код становится короче, а ошибку достаточно исправить в одном месте.

Функция объявляется словом `def`, результат возвращается словом `return`. Строка в тройных кавычках сразу после объявления – **документация** функции; её показывает `help()`.

```
def future_value(amount, rate, years, periods_per_year=1):
    """Будущая стоимость вклада при капитализации periods_per_year раз в год."""
```

```
    n = periods_per_year
    return amount * (1 + rate / n) ** (n * years)

print(future_value(100_000, 0.16, 3))
#> 156089.59999999995
print(future_value(100_000, 0.16, 3, periods_per_year=12))
#> 161095.6598320968
```

Аргумент `periods_per_year` имеет **значение по умолчанию**: если его не указать, капитализация ежегодная. При ежемесячной капитализации вклад за три года вырастет сильнее – на 5 006 руб. Аргументы можно передавать по порядку или по имени. Именованная запись длиннее, но защищает от перепутанных местами ставки и срока.

A.5.2 Пример: NPV и IRR

Чистая приведённая стоимость (net present value, NPV) проекта – сумма его денежных потоков, дисконтированных по ставке r:

$$\mathrm{NPV} = \sum_{t=0}^{T} \frac{CF_t}{(1+r)^t}. \tag{A.1}$$

Запишем формулу функцией. Номер периода t даёт `enumerate()`:

```
def npv(rate, cashflows):
    """NPV потоков cashflows[0], cashflows[1], ... при ставке rate."""
    return sum(cf / (1 + rate) ** t for t, cf in enumerate(cashflows))

cashflows = [-100, 40, 40, 0, 40]
npv(0.12, cashflows)
#> -6.977236047480236
```

Проект с вложением 100 и тремя поступлениями по 40 при ставке 12% имеет отрицательную NPV: дисконтированные поступления не покрывают вложение.

Внутренняя норма доходности (IRR) – ставка, при которой NPV равна нулю. Решить уравнение явно нельзя, но его можно решить численно. Функция `brentq` из библиотеки SciPy ищет корень функции на отрезке, где она меняет знак. Ей передаётся сама функция – в Python функцию можно передать как аргумент, как любое другое значение:

```
from scipy.optimize import brentq

def irr(cashflows):
    return brentq(lambda r: npv(r, cashflows), -0.99, 1.0)

print(f"IRR = {irr(cashflows):.2%}")
#> IRR = 8.36%
```

Запись `lambda r: npv(r, cashflows)` создаёт короткую безымянную функцию одного аргумента *r*. IRR ниже 12%, что согласуется с отрицательной NPV при этой ставке.

A.5.3 Проверка функций

Функцию полезно проверить на примере, ответ которого известен заранее. Для этого есть оператор `assert`: он ничего не делает, если условие истинно, и останавливает программу с ошибкой, если ложно.

```
assert abs(npv(0.0, [-100, 50, 50])) < 1e-12          # без дисконтирования NPV = сумма потоков
assert abs(npv(irr(cashflows), cashflows)) < 1e-9     # в точке IRR NPV равна нулю
```

Такие проверки занимают одну строку, а ловят ошибки, которые иначе прошли бы незамеченными: перепутанный знак, индекс, начинающийся с единицы, процент вместо доли.

A.6 Ошибки и как их читать

Ошибки – нормальная часть работы с кодом. Когда Python не может выполнить строку, он останавливается и печатает **трассировку** (traceback). Читать её нужно снизу вверх: последняя строка сообщает тип ошибки и её описание, выше указано место, где она произошла.

```
portfolio["MOEX"]
#> KeyError: 'MOEX'
```

В книге показана только последняя строка сообщения; в Jupyter над ней будет трассировка с номером строки и фрагментом кода. `KeyError: 'MOEX'` означает, что в словаре `portfolio` нет ключа `"MOEX"`. Причина почти всегда в опечатке или в том, что значение не было добавлено.

Таблица А.3. Частые ошибки и их причины

Ошибка	Что означает	Типичная причина в ноутбуке
`SyntaxError`	Python не может разобрать строку	пропущена скобка, кавычка или двоеточие; `=` вместо `==`
`IndentationError`	неверный отступ	смешаны пробелы и табуляция, лишний отступ
`NameError`	имя не найдено	не выполнена ячейка, где создаётся переменная; опечатка
`TypeError`	операция не подходит для типа	сложение строки с числом; вызов функции с лишним аргументом
`KeyError`	нет такого ключа	опечатка в тикере или названии столбца, регистр букв
`IndexError`	нет элемента с таким номером	список короче, чем ожидалось; пустая выборка
`ValueError`	недопустимое значение	неверный формат даты; пустые данные
`ModuleNotFoundError`	пакет не установлен	не выполнен `pip install`; ноутбук открыт не в той папке

Если причина непонятна, помогает простая последовательность. Прочитайте последнюю строку сообщения. Проверьте, выполнены ли все предыдущие ячейки. Напечатайте значения переменных из строки с ошибкой: часто оказывается, что в переменной лежит не то, что вы ожидали. Скопировать сообщение об ошибке в поисковик или ИИ-ассистенту тоже полезно, но объяснение нужно проверить на своём коде, а не принимать на веру.

А.7 Модули и пакеты

Базовый Python умеет немного. Основная сила языка – в **пакетах**: готовых библиотеках, которые устанавливаются отдельно. Пакет устанавливается один раз командой `pip install` в терминале (в ноутбуке – `!pip install`), а в каждом ноутбуке подключается командой `import`:

```
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

math.log(2)
#> 0.6931471805599453
```

Сокращения `np`, `pd` и `plt` – общепринятое соглашение; их используют в документации, книгах и ответах ИИ-ассистентов. Конструкция `from ... import ...` подключает отдельную часть пакета. Так в книге подключается модуль `qf`, который лежит в папке книги: `from qf import data`.

Чтобы узнать, что делает функция, используйте `help(функция)` или, в Jupyter, вопросительный знак: `np.log?`. Список методов объекта показывает `dir()`. Документация основных библиотек подробная и содержит примеры; привычка заглядывать в неё экономит много времени.

A.8 NumPy: вычисления над массивами

A.8.1 Массивы

Со списками неудобно считать. Прибавить 1 к каждому элементу списка нельзя: `[1, 2, 3] + 1` вызовет `TypeError`, а умножение списка на число повторяет список. Для вычислений служит **массив** NumPy (`np.array`): операции с ним применяются сразу ко всем элементам.

```
prices_arr = np.array(closes)      # пять цен SBER из раздела о списках
prices_arr[1:] / prices_arr[:-1] - 1
#> array([-0.00502244, -0.01066839, -0.01411254,  0.00807488])
```

Мы посчитали дневные доходности одной строкой, без цикла: каждый элемент, начиная со второго, разделили на предыдущий. Такой стиль называют **векторизацией**. Векторизованный код короче и работает в десятки раз быстрее цикла, что важно на рядах в тысячи наблюдений.

NumPy содержит математические функции, которые тоже работают поэлементно (`np.log`, `np.exp`, `np.sqrt`), и статистические: `np.mean`, `np.std`, `np.percentile`. Функцию NPV из предыдущего раздела можно записать без цикла:

```
cf = np.array(cashflows)
t = np.arange(len(cf))          # 0, 1, 2, 3, 4
np.sum(cf / (1 + 0.12) ** t)
#> -6.977236047480236
```

A.8.2 Логические маски

Сравнение массива с числом даёт массив логических значений – **маску**. По маске можно отобрать элементы:

```
rets = prices_arr[1:] / prices_arr[:-1] - 1
print(rets < 0)
#> [ True  True  True False]
print(rets[rets < 0])
#> [-0.00502244 -0.01066839 -0.01411254]
```

Тот же приём работает в pandas и постоянно встречается в книге: отобрать дни с падением, бумаги с высокой капитализацией, годы с отрицательной доходностью.

A.8.3 Случайные числа

Для моделирования нужны случайные числа. Генератор создаётся функцией `np.random.default_rng()`; аргумент `seed` фиксирует последовательность, чтобы результат повторялся при каждом запуске. Смоделируем год торговли акцией с дневной доходностью в среднем 0,05% и волатильностью 2%:

```
rng = np.random.default_rng(seed=42)
daily = rng.normal(loc=0.0005, scale=0.02, size=252)
path = 100 * np.cumprod(1 + daily)
print(f"Конец года: {path[-1]:.1f}, минимум: {path.min():.1f}")
#> Конец года: 85.1, минимум: 79.3
```

Функция `np.cumprod` считает накопленное произведение: каждый элемент `path` – стоимость вложения в соответствующий день. Запустите пример с другим `seed`, и путь окажется совсем другим при тех же параметрах. Это полезная интуиция: одна реализация случайного процесса мало говорит о его свойствах.

A.9 pandas: таблицы с данными

A.9.1 Series и DataFrame

pandas – основная библиотека для работы с таблицами. В ней два главных объекта. **Series** – одномерный ряд значений с подписями (индексом), например цены одной акции по датам. **DataFrame** – таблица, каждый столбец которой является Series, а все столбцы имеют общий индекс.

Загрузим данные двух акций за 2025–2026 гг. Функции модуля `qf` возвращают обычные таблицы pandas:

```
stocks = data.load_stocks(["SBER", "LKOH"], start="2025-01-01")
stocks.shape
#> (884, 12)
```

Атрибут `.shape` показывает размер таблицы: число строк и столбцов. С новой таблицей всегда стоит начинать одинаково: посмотреть первые строки, названия и типы столбцов.

```
stocks.head(3)
#>         date ticker    open  ...    adj_close       shares    market_cap
#> 0 2025-01-03   LKOH  7255.0  ...  5807.169973  692865762.0  4.934590e+12
#> 1 2025-01-06   LKOH  7114.5  ...  5779.446892  692865762.0  4.911033e+12
#> 2 2025-01-08   LKOH  7090.0  ...  5780.262277  692865762.0  4.911725e+12
```

```
#>
#> [3 rows x 12 columns]
```

```
stocks.dtypes
#> date          datetime64[ms]
#> ticker                object
#> open                 float64
#> low                  float64
#> high                 float64
#> close                float64
#> waprice              float64
#> volume               float64
#> value_rub            float64
#> adj_close            float64
#> shares               float64
#> market_cap           float64
#> dtype: object
```

Таблица хранится в **длинном формате**: каждая строка – одна бумага в один день, тикер записан в отдельном столбце. Метод `.describe()` выводит основные статистики по числовым столбцам, а `.isna().sum()` – число пропусков в каждом столбце.

A.9.2 Выбор столбцов и строк

Столбец выбирается по имени в квадратных скобках, несколько столбцов – списком имён. Строки отбирают по маске, как в NumPy:

```
sber = stocks[stocks["ticker"] == "SBER"]
sber[["date", "close", "volume"]].tail(3)
#>           date   close      volume
#> 881 2026-09-25  276.35  17573507.0
#> 882 2026-09-28  272.45  25688835.0
#> 883 2026-09-29  274.65  18505319.0
```

Условия объединяют операторами `&` (и) и `|` (или), каждое условие берётся в скобки: `stocks[(stocks["ticker"] == "SBER") & (stocks["close"] > 300)]`. Слова `and` и `or` с таблицами не работают.

Для доступа по подписям индекса служит `.loc`, по номерам строк – `.iloc`. Запись `sber.iloc[0]` – первая строка, `sber.iloc[-1]` – последняя; `sber.loc[:, "close"]` – столбец `close` целиком.

A.9.3 Широкий формат и индекс дат

Для расчётов по нескольким бумагам удобнее **широкий формат**: даты по строкам, тикеры по столбцам. Метод `.pivot()` переворачивает длинную таблицу в широкую:

```
prices_df = stocks.pivot(index="date", columns="ticker", values="adj_close")
prices_df.tail(3)
#> ticker        LKOH    SBER
#> date
#> 2026-09-25  5374.0  276.35
#> 2026-09-28  5299.5  272.45
#> 2026-09-29  5369.0  274.65
```

Теперь индекс таблицы – даты. Это открывает удобный отбор по периодам: `prices_df.loc["2026-03"]` вернёт все дни марта 2026 г., `prices_df.loc["2025-06-01":"2025-12-31"]` – второе полугодие 2025 г. Функция `data.load_prices()` сразу возвращает таблицу в широком формате; здесь мы проделали этот шаг сами, чтобы увидеть, как он устроен.

A.9.4 Вычисления по столбцам

Арифметика с таблицами работает так же, как с массивами NumPy: поэлементно, с выравниванием по индексу. Метод `.pct_change()` считает изменение к предыдущей строке, то есть простую доходность:

```
returns_df = prices_df.pct_change().dropna()
returns_df.describe().round(4)
#> ticker      LKOH      SBER
#> count   441.0000  441.0000
#> mean      0.0000    0.0007
#> std       0.0191    0.0131
#> min      -0.0773   -0.0554
#> 25%      -0.0099   -0.0065
#> 50%      -0.0012    0.0000
#> 75%       0.0088    0.0070
#> max       0.0795    0.0772
```

Первая строка доходностей пуста – для первого дня нет предыдущего, – поэтому её удаляет `.dropna()`. Накопленный рост вложения даёт `(1 + returns_df).cumprod()`, а нормированные цены с начальным значением 100 – `100 * prices_df / prices_df.iloc[0]`.

A.9.5 Группировка и смена частоты

Многие вопросы требуют свести дневные данные к месяцам или годам. Для смены частоты временного ряда есть `.resample()`. Код `"ME"` означает конец месяца, `"YE"` – конец года. Цена на конец каждого месяца и месячные доходности:

```
monthly = prices_df.resample("ME").last()
monthly.pct_change().tail(3).round(3)
#> ticker       LKOH   SBER
#> date
#> 2026-07-31  0.017  0.037
```

```
#> 2026-08-31  0.017 -0.002
#> 2026-09-30  0.140 -0.004
```

Метод `.groupby()` делит таблицу на группы по значению столбца и применяет к каждой группе одну и ту же операцию. Например, средний дневной оборот в рублях по каждой бумаге:

```
stocks.groupby("ticker")["value_rub"].mean() / 1e9
#> ticker
#> LKOH     5.692325
#> SBER    10.571860
#> Name: value_rub, dtype: float64
```

Результат – в миллиардах рублей. Оборот Сбербанка почти вдвое выше, чем у ЛУКОЙЛа: эта разница в ликвидности важна для торговых стратегий в главе о бэктестинге.

A.9.6 Объединение таблиц

Данные из разных источников приходится объединять. Метод `.merge()` соединяет две таблицы по общему столбцу – так же, как функция ВПР в Excel, но сразу для всех строк. Добавим к бумагам отрасль из справочника книги:

```
sectors = data.load_reference("sectors")
last_day = stocks[stocks["date"] == stocks["date"].max()]
last_day[["ticker", "close", "market_cap"]].merge(sectors, on="ticker", how="left")
#>   ticker    close    market_cap           sector
#> 0   LKOH  5369.00  3.719996e+12      Нефть и газ
#> 1   SBER   274.65  5.928855e+12  Банки и финансы
```

Аргумент `how="left"` сохраняет все строки левой таблицы, даже если для тикера не нашлось отрасли: в этом случае в столбце `sector` будет пропуск. Проверка числа строк до и после объединения – хорошая привычка: если оно выросло, в правой таблице есть повторяющиеся ключи.

A.9.7 Пропуски

Пропущенные значения в pandas обозначаются `NaN` (not a number). Метод `.isna()` находит их, `.dropna()` удаляет строки с пропусками, `.fillna()` и `.ffill()` заполняют их – последний переносит вперёд предыдущее известное значение. С заполнением нужно быть осторожным. Перенос вперёд использует только прошлое и обычно безопасен; заполнение средним по всей выборке или перенос назад (`.bfill()`) приносит в прошлое информацию из будущего. Почему это опасно, разбирается в разделе 3.6.

A.10 matplotlib: графики

Графики в книге строятся библиотекой matplotlib. Стандартный способ: создать **рисунок** (figure) и **оси** (axes) функцией `plt.subplots()`, нарисовать на осях данные и подписать их.

```
plot.use_book_style()
normalized = 100 * prices_df / prices_df.iloc[0]

fig, ax = plt.subplots()
ax.plot(normalized.index, normalized["SBER"], label="SBER")
ax.plot(normalized.index, normalized["LKOH"], label="LKOH")
ax.axhline(100, color="grey", linewidth=0.8)
ax.set_ylabel("Начальная величина = 100")
ax.legend()
plot.source(ax, "Московская биржа, расчёт автора")
plt.show()
```

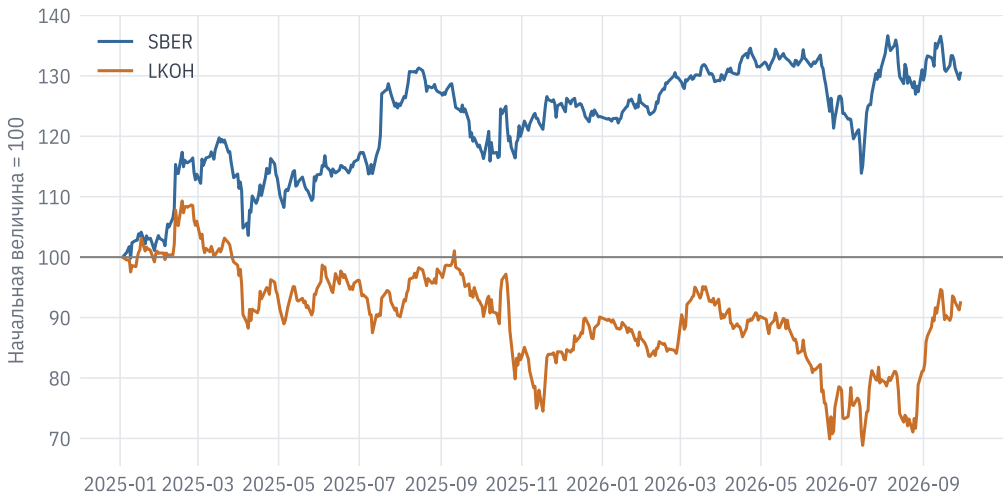


Рисунок A.1. Рост 100 руб., вложенных в акции SBER и LKOH в начале 2025 г. (с учётом дивидендов)

У таблиц pandas есть метод `.plot()`, который делает то же короче: `normalized.plot()` рисует все столбцы с легендой. Его удобно использовать для быстрого взгляда на данные, а явное построение через `ax` – когда нужно настроить детали. Функции модуля `qf.plot` задают единый стиль графиков книги и добавляют подпись источника; без них графики получатся в стандартном стиле matplotlib.

Распределение величины показывает гистограмма:

```
fig, ax = plt.subplots()
ax.hist(returns_df["SBER"] * 100, bins=50)
```

```
ax.set_xlabel("Дневная доходность, %")
ax.set_ylabel("Число дней")
plt.show()
```

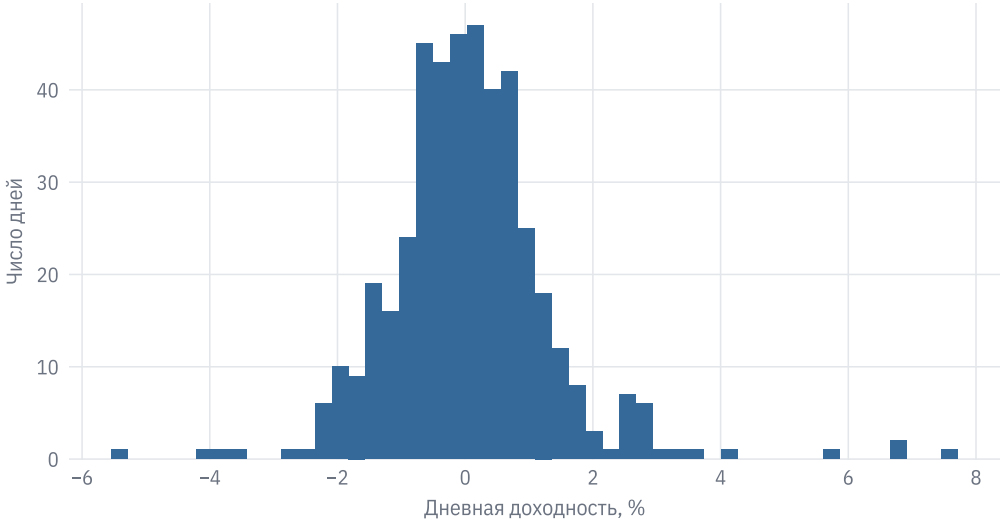


Рисунок А.2. Распределение дневных доходностей SBER, 2025–2026 гг.

Сохранить рисунок в файл можно методом `fig.savefig("chart.png", dpi=200)`. Для вставки в текст работы лучше подходит векторный формат: `fig.savefig("chart.svg")` или `.pdf`.

A.11 Как учиться дальше

Чтобы читать код книги, описанного в приложении достаточно. Дальше навык растёт от практики, а не от чтения. Несколько привычек делают её эффективнее.

- **Сначала предсказать, потом запустить.** Перед выполнением ячейки ответьте себе, что она выведет. Расхождение показывает, что именно вы поняли неверно.
- **Менять по одному.** Изменили одну вещь – запустили – посмотрели результат. Если изменить сразу пять строк и получить ошибку, найти причину трудно.
- **Проверять на маленьком примере.** Прежде чем применять функцию к 20 годам данных, проверьте её на трёх числах, ответ для которых можно посчитать вручную.
- **Работать с ИИ-ассистентом как с консультантом.** Просите объяснить код построчно, найти ошибку, предложить способ. Но результат проверяйте сами: ассистенты уверенно ошибаются в деталях, например в формулах годового

пересчёта или в работе с датами. Принципы работы с ассистентом в курсе описаны в разделе 1.5.

Итоги

- Ноутбук выполняет код в ядре, которое помнит выполненное, а не написанное. Перед сдачей работы ноутбук нужно перезапустить и выполнить сверху вниз.
- Основные типы значений – числа (`int`, `float`), строки, логические значения и `None`. Дробные числа нельзя сравнивать через `==`.
- Списки хранят упорядоченные значения и нумеруются с нуля; словари находят значение по ключу; множества помогают сравнивать наборы.
- Условия и циклы управляют порядком выполнения; блоки выделяются отступом. Функции избавляют от повторения кода, а `assert` проверяет их на известных примерах.
- Трассировку ошибки читают снизу вверх.
- NumPy выполняет вычисления сразу над всеми элементами массива; pandas добавляет к массивам подписи и даёт операции с таблицами: отбор, смену формата и частоты, группировку и объединение; matplotlib строит графики.

A.12 Упражнения

Задания **А** решаются без компьютера: предскажите результат, затем проверьте себя запуском. Задания **Б** выполняются на данных вашей акции. Задания **В** учат проверять ответы ИИ-ассистента. Задание **Г** – небольшой самостоятельный расчёт.

A.12.1 А. Понимание

1. Что выведет каждая строка?

```
xs = [10, 20, 30, 40, 50]
print(xs[1], xs[-2], xs[1:3], len(xs[2:]))
```

2. Чему равна переменная `x` после выполнения кода? Почему?

```
x = 100
for r in [0.5, -0.5]:
    x = x * (1 + r)
```

3. Почему код `if price = 250:` вызывает ошибку, а `if price == 250:` – нет?
4. В ячейке 1 создаётся `rate = 0.16`, в ячейке 2 считается `npv(rate, cashflows)`. Студент исправил ячейку 1 на `rate = 0.12` и запустил только ячейку 2. Какую ставку использовал расчёт?
5. Что вернёт `prices_df.loc["2026-03"]` и чем это отличается от `prices_df.iloc[3]`?

A.12.2 Б. Практика на своей акции

6. Загрузите данные своей акции с помощью `data.load_stocks()`. Сколько в таблице строк, какие первая и последняя даты, есть ли пропуски в столбцах `close` и `adj_close`?
7. Посчитайте месячные доходности по `adj_close` и найдите три худших месяца. С какими событиями они связаны?
8. Напишите функцию `period_return(prices, start, end)`, которая возвращает изменение цены между двумя датами. Проверьте её через `assert` на примере, ответ которого известен.
9. Постройте на одном графике нормированные цены своей акции и индекса IMOEX (функция `data.load_index()`) за последние три года.

A.12.3 В. Работа с ИИ-ассистентом

10. Попросите ИИ-ассистента написать функцию IRR без сторонних библиотек. Проверьте её на потоках `[-100, 40, 40, 0, 40]` и сравните с результатом `brentq` из раздела A.5. Как функция ведёт себя на потоках `[-100, 230, -132]`, у которых две IRR (10% и 20%)? Запишите, какие допущения сделал ассистент.
11. Ассистент написал код для средней дневной доходности каждой бумаги в 2026 г. Найдите в нём три ошибки, не запуская его, а затем проверьте себя запуском. Подсказка: посмотрите, в каком порядке идут строки таблицы `df`.

```
df = data.load_stocks(["SBER", "LKOH"], start="2026-01-01")
df["ret"] = df["close"].pct_change()
df.groupby("date")["ret"].mean()
```

A.12.4 Г. Мини-исследование

12. Смоделируйте 1000 путей цены по образцу раздела A.8 с теми же параметрами. Какая доля путей заканчивается ниже 100? Какова медиана максимальной просадки? Сравните с просадками реальных акций из главы о доходности и риске.

Вопросы для самопроверки

1. Чем отличается результат `7 / 2` от `7 // 2`?
2. Что выведет `f"{0.0842:.1%}"`?
3. Какой контейнер подходит для хранения портфеля «тикер → число акций» и почему?
4. Почему `[1, 2, 3] + 1` вызывает ошибку, а `np.array([1, 2, 3]) + 1` – нет?
5. Чем длинный формат таблицы отличается от широкого? Какой метод pandas переводит первый во второй?
6. Почему заполнение пропусков методом `.bfill()` опасно для финансовых данных?

Ответы

1. `7 / 2 = 3.5` – обычное деление; `7 // 2 = 3` – целая часть от деления.
2. `8.4%`: формат умножает на 100 и добавляет знак процента.
3. Словарь: значение (число акций) находится по ключу (тикеру), а тикеры не повторяются.
4. Список – контейнер общего назначения, сложение для него означает склеивание двух списков, а число не является списком. Массив NumPy выполняет арифметику поэлементно.
5. В длинном формате одна строка – одно наблюдение (бумага в день), тикер – отдельный столбец; в широком – даты по строкам, тикеры по столбцам. Перевод выполняет `.pivot()`.
6. Он подставляет в прошлую дату значение из будущего. В бэктесте это создаёт заглядывание вперёд: стратегия «знает» цену, которой ещё не было.

Что почитать

- Sargent и Stachurski (2024 г.) – открытый онлайн-курс QuantEcon: Python для экономистов от основ языка до NumPy, pandas и matplotlib.
- McKinney (2022 г.) – книга автора pandas; подробно о работе с таблицами и временными рядами. Полный текст открыт на сайте автора.
- VanderPlas (2023 г.) – справочник по NumPy, pandas, matplotlib и scikit-learn с примерами.
- Hilpisch (2018 г.) – Python в прикладных финансовых задачах.

B Справочник по данным

Глава готовится

Эта глава входит в первую версию книги. Планируемое содержание:

- Состав снимка данных книги и полной истории рынка
- Описание таблиц и полей
- Функции модулей `qf` (Python) и `R/qf.R`
- Как обновить данные

С Проекты, экзамен и оценивание

Это приложение описывает, как устроен контроль в курсе «Количественные финансы»: что входит в итоговую оценку, какие проекты выполняют студенты, по каким критериям они оцениваются и как проходит экзамен. Если вы читаете книгу самостоятельно, проекты можно использовать как большие упражнения, а критерии – как список вопросов для самопроверки своей работы.

С.1 Итоговая оценка

Таблица С.1. Структура итоговой оценки

Компонент	Вес
Проект 1. Фундаментальный анализ отрасли	20%
Проект 2. Модель и стратегия для акции	25%
Работа на семинарах	10%
Письменный экзамен	35%
Посещаемость	10%

Каждый компонент оценивается по 10-балльной шкале. Итоговая оценка – взвешенная сумма:

$$О = 0,20\,П_1 + 0,25\,П_2 + 0,10\,С + 0,35\,Э + 0,10\,П_{пос},$$

которая округляется по правилам НИУ ВШЭ.

Структура оценки построена вокруг простой идеи. ИИ-ассистенты уже умеют писать код для большинства задач курса, поэтому оценивается прежде всего то, что ассистент за студента сделать не может: поставить вопрос, проверить данные, объяснить результат и защитить свои решения. Больше половины оценки (экзамен и семинарские задания) студент получает за работу, которую выполняет сам и при преподавателе, а проекты подкрепляются выборочной устной защитой.

С.2 Общие правила для проектов

С.2.1 Индивидуальные задания

Каждый студент получает своё задание: отрасль (код ОКВЭД) для первого проекта и акцию Московской биржи для второго. Задания распределяет преподаватель. Одинаковых заданий у двух студентов нет, поэтому готовых ответов не существует.

С.2.2 Что и как сдаётся

Все работы сдаются через Google-формы, ссылки на которые публикуются в чате курса. В форме указывается адрес электронной почты – по нему работы, рецензии и оценки связываются между собой, поэтому используйте один и тот же адрес весь курс. Срок сдачи – время, до которого форма принимает ответы; после него форма закрывается.

- **Ноутбук** (Jupyter или Quarto) с полным анализом. Ноутбук должен запускаться «с нуля» – сверху вниз, в чистом окружении, на данных книги или на данных, загрузка которых есть в самом ноутбуке, – и давать те же числа, что в тексте.
- **Журнал работы с ИИ** (см. ниже). Без журнала из оценки за проект вычитается 1 балл.
- Для проекта 2 – **файл предварительной фиксации** (см. раздел С.4).

Выводы пишутся в ноутбуке обычным текстом, а не в комментариях к коду. Каждый вывод должен опираться на конкретные числа из вашего анализа.

С.2.3 ИИ-ассистенты

Пользоваться ИИ-ассистентами можно и нужно – это часть современной работы аналитика. Условия два.

1. **Вы отвечаете за каждую строку.** Если на защите (раздел «Устная защита» ниже) вы не можете объяснить, что делает фрагмент кода и почему он написан именно так, баллы за соответствующий раздел не засчитываются.
2. **Вы ведёте журнал.** В нём – основные запросы к ассистенту и то, что вы сделали с ответом.

Таблица С.2. Образец журнала работы с ИИ

№	Что я спросил	Что ответил ассистент (кратко)	Что я сделал с ответом и почему
1	Код загрузки истории акции из ISS	Функция с запросом к `/history/...`	Принял, но добавил постраничную загрузку: ассистент получал только первые 100 строк
2	Как выбрать порядок ARMA	Перебор по AIC	Принял; дополнительно проверил остатки тестом Льюнга – Бокса

Журнал оценивается не по числу запросов. Если в нём задокументированы ошибки ассистента, которые вы нашли и исправили, к оценке за проект добавляется до 0,5 балла: это и есть навык, которому учит курс.

С.2.4 Устная защита

Устная защита проводится **выборочно**: так на семинарах остаётся время на материал курса, а каждый студент всё равно знает, что может быть вызван на защиту.

Кто защищается. По каждому проекту на защиту вызываются:

- около трети студентов, выбранных случайно после срока сдачи;
- студенты, в работах которых есть признаки того, что автор не до конца понимает свой анализ: выводы расходятся с числами, код не совпадает с описанием, в журнале работы с ИИ нет записей при очевидно сгенерированном коде;
- студенты, которые сами просят о защите, – например, чтобы повысить оценку или оспорить её.

Список вызванных публикуется в чате курса вместе с оценками за письменную часть проекта.

Как проходит. Защита идёт 5 минут на консультации, вне семинаров; время студент выбирает сам в электронном расписании преподавателя. Студент открывает свой ноутбук, а преподаватель задаёт 2–3 вопроса по конкретным местам работы:

- почему выбран этот период, показатель, модель или параметр;
- что изменится, если поменять одно из решений; одно изменение нужно сделать прямо на защите и объяснить результат;
- где вы опирались на ИИ-ассистента и как проверяли его ответы.

Как учитывается. Для тех, кто защищался, защита – отдельный критерий (2 балла из 10), и, кроме того, она **подтверждает авторство**: баллы за раздел, который студент не может объяснить, не засчитываются. Для тех, кто не защи-

щался, оценка за проект считается по остальным критериям и пересчитывается в 10-балльную шкалу: например, 7 баллов из 8 возможных дают $7/8 \\times 10 = 8,75$.

С.2.5 Академическая честность

Работы, совпадающие по содержанию с работами других студентов или прошлых лет, оцениваются в 0 баллов. Обсуждать проекты друг с другом можно; сдавать чужой анализ – нельзя.

С.3 Проект 1. Фундаментальный анализ отрасли

С.3.1 Задание

По данным бухгалтерской отчётности российских компаний (набор RFSD, главы 5 и 6) проведите сравнительный анализ компаний своей отрасли и ответьте на вопрос: **за счёт чего лидеры отрасли эффективнее остальных?**

Работа включает:

1. **Постановку.** До начала расчётов запишите гипотезу – например: «Крупные компании отрасли более рентабельны за счёт оборачиваемости активов, а не маржи». Опишите, какие компании войдут в анализ и по какому правилу.
2. **Данные и их проверку.** Отберите компании по коду ОКВЭД, очистите данные, проверьте связь форм отчётности (например, что сходится баланс), найдите и объясните выбросы.
3. **Расчёты.** Коэффициенты рентабельности, оборачиваемости, ликвидности и долговой нагрузки; их распределения в отрасли; разложение DuPont.
4. **Анализ.** Сравнение лидеров и остальных, динамика за несколько лет, проверка гипотезы.
5. **Выводы и ограничения.** Что можно и что нельзя утверждать по этим данным.

С.3.2 Критерии оценки

Таблица С.3. Критерии оценки проекта 1 (10 баллов). * Только для тех, кто вызван на защиту; для остальных оценка пересчитывается по остальным критериям

Критерий	Баллы	0 баллов	1 балл	2 балла
Постановка	1	нет гипотезы или правила отбора	гипотеза и правило отбора сформулированы до анализа	–
Данные и их проверка	2	данные использованы как есть	есть очистка, но нет проверок	отбор объяснён, формы сверены, выбросы найдены и объяснены
Расчёты и визуализация	1	ошибки в формулах	коэффициенты верны, графики читаются	–
Интерпретация	2	общие фразы без чисел	выводы с числами, но без объяснения механизма	выводы конкретны для отрасли, механизм объяснён (маржа, оборачиваемость, леверидж)
Выводы и ограничения	1	ограничения не обсуждаются	названы ограничения данных и метода (РСБУ, выжившие, консолидация)	–
Взаимная проверка	1	замечание формальное	найдено содержательное замечание к работе соседа и дан ответ на замечания к своей	–
Устная защита*	2	не может объяснить свои решения	объясняет решения, но затрудняется с изменением на месте	объясняет решения и уверенно интерпретирует изменение

С.3.3 Взаимная проверка

Взаимная проверка учит читать чужой анализ критически – это навык, который нужен при проверке любого результата, в том числе полученного от ИИ-ассистента. Она проходит в три шага и занимает одну неделю после срока сдачи проекта.

1. **Распределение.** На следующий день после срока сдачи каждый студент получает письмо со ссылкой на работу однокурсника и на форму рецензии. Рецензенты распределяются случайно; свою работу и работу того, кто рецензирует вас, вы проверять не будете.
2. **Рецензия** (срок – 5 дней). В форме нужно заполнить три поля:
   - одно содержательное замечание: ошибка в данных или расчётах, необоснованный вывод, пропущенная проверка – с указанием места в ноутбуке;
   - один вопрос к автору;
   - что в работе сделано хорошо и чему можно научиться.
3. **Ответ автора** (срок – 2 дня после получения рецензии). Автор получает рецензию по почте и в форме ответа отвечает на замечание и вопрос, а также отмечает, была ли рецензия полезной: *содержательная* (указывает на реальную проблему или важный вопрос) или *формальная*.

Как оценивается (1 балл в проекте 1): по 0,5 балла за рецензию, которую автор отметил как содержательную, и за ответ на полученную рецензию. Если вы не согласны с отметкой автора, напишите преподавателю – спорные случаи он разбирает сам. Пользоваться ИИ-ассистентом при рецензировании можно, но замечание должно касаться конкретного места работы: общие советы вроде «стоит добавить больше графиков» содержательными не считаются.

С.4 Проект 2. Модель и стратегия для акции

С.4.1 Задание

Для назначенной акции Московской биржи постройте модель временного ряда (главы 7–9), проверьте, есть ли у неё прогнозная сила, и на её основе разработайте и честно протестируйте торговую стратегию (глава 15). Главный вопрос: **даёт ли модель что-то сверх простого бенчмарка после учёта издержек?** Отрицательный ответ, полученный корректно, – полноценный результат.

С.4.2 Предварительная фиксация

До расчётов на тестовом периоде сдайте короткий файл фиксации (одна страница):

- гипотеза: что именно модель должна предсказывать и почему;
- обучающий и тестовый периоды;
- метрики качества прогноза и стратегии;
- бенчмарки: наивный прогноз, индекс MCFTR, вложение по ставке RUONIA;
- размер издержек на сделку.

После сдачи фиксации эти решения менять нельзя. Если вы всё же что-то изменили, об этом нужно прямо написать в работе и объяснить причину, – такое изменение не штрафуется, скрытое изменение оценивается как ошибка.

С.4.3 Проверка на новых данных

Модель и правила стратегии фиксируются к дедлайну. Через несколько недель после дедлайна преподаватель запускает ноутбук на данных, которых на момент сдачи ещё не было. Оценивается не доходность на этом периоде – она во многом случайна, – а то, что ноутбук воспроизводится без правок, и то, насколько результаты на новых данных согласуются с выводами работы. Большое расхождение, которое работа не объясняет, – повод для вызова на устную защиту.

С.4.4 Критерии оценки

Таблица С.4. Критерии оценки проекта 2 (10 баллов). * Только для тех, кто вызван на защиту; для остальных оценка пересчитывается по остальным критериям

Критерий	Баллы	0 баллов	1 балл	1,5–2 балла
Предварительная фиксация	1	нет или составлена задним числом	гипотеза, периоды, метрики и бенчмарки зафиксированы до теста	–
Данные и разведочный анализ	1	нескорректированные цены, разрывы не проверены	скорректированные цены, проверка разрывов и стационарности	–
Модель и диагностика	1,5	модель не обоснована	выбор модели обоснован, остатки проверены	1,5: понятно, что модель улавливает и чего нет
Честная оценка	2	нет проверки вне выборки или есть утечка данных	прогноз вне выборки и сравнение с наивным прогнозом	2: сравнение со всеми бенчмарками с учётом издержек, утечек нет
Интерпретация и ограничения	1,5	результат не объяснён	объяснено, почему результат такой	1,5: обсуждены ограничения; отрицательный результат не спрятан
Проверка на новых данных	1	ноутбук не запускается	запускается; расхождения с выводами объяснены	–
Устная защита*	2	не может объяснить свои решения	объясняет решения, но затрудняется с изменением на месте	2: объясняет решения и уверенно интерпретирует изменение

Обратите внимание, чего нет в критериях: баллов за число моделей, за «продвинутые» методы и за доходность стратегии. Нейросеть без честной проверки оценивается ниже, чем простая модель AR(1) с корректным сравнением, издержками и проверкой на новых данных.

C.5 Работа на семинарах

Почти каждый семинар начинается с короткого письменного задания по материалу прошлой лекции и соответствующей главы. Задание выполняется **на бумаге, без компьютера и телефона** – так проверяется ваше собственное понимание, а не ответ ассистента.

Как проходит.

1. **Задание (5 минут).** В начале семинара на экране появляется одно задание, устроенное как упражнения типов A и B в книге: предсказать результат фрагмента кода, объяснить график или таблицу, найти ошибку в коде, ответить на вопрос для самопроверки. Ответ – несколько предложений, короткий расчёт или исправленная строка кода. В начале листка укажите свой адрес электронной почты.
2. **Обмен и разбор (5 минут).** Листки передаются соседу. Преподаватель разбирает правильный ответ и показывает критерии; сосед оценивает ваш ответ: 2 балла – верно и объяснено, 1 – частично верно, 0 – неверно или нет ответа.
3. **Ввод оценки (1 минута).** Проверяющий вносит оценку в Google-форму с телефона: чей листок он проверил и какой балл поставил. Листки сдаются преподавателю.

Преподаватель просматривает выборку листков и сверяет оценки. Если проверяющий заметно ошибся в оценке, исправляется и оценка автора, и у проверяющего снимается балл за это задание – поэтому к проверке стоит относиться так же серьёзно, как к ответу. Если вы не согласны с оценкой, напишите преподавателю в течение недели: листки хранятся до конца курса.

Оценка. Оценка за работу на семинарах – доля набранных баллов от максимума по 80% лучших заданий, переведённая в 10-балльную шкалу. Пропущенное задание получает 0 баллов; 20% худших результатов не учитываются, поэтому один-два пропуска по уважительной причине на оценку не влияют. Опоздавшие к началу семинара задание не выполняют.

C.6 Посещаемость

Оценка за посещаемость – доля посещённых занятий, переведённая в 10-балльную шкалу.

С.7 Экзамен

Экзамен письменный, проходит в аудитории, длится 80 минут (одна пара). Пользоваться компьютером, телефоном и другими устройствами нельзя. Экзамен проверяет понимание, а не умение писать код: весь код на экзамене нужно прочитать и понять, но не написать. Максимум – 100 баллов, которые переводятся в 10-балльную шкалу. Время в таблице ориентировочное: порядок и распределение времени между частями студент выбирает сам.

Таблица С.5. Структура экзамена

Часть	Баллы	Время	Что проверяется
1. Тест	20	~10 мин	10 вопросов с выбором ответа по всем темам курса
2. Интерпретация результата	30	~25 мин	дан один готовый результат – график, таблица оценок модели или расчёт риска; нужно объяснить, что он означает, и назвать ограничения
3. Найдите ошибку	25	~20 мин	дан фрагмент кода с логической или концептуальной ошибкой (не синтаксической); нужно найти её, объяснить, почему это ошибка с точки зрения финансов или статистики, и предложить исправление
4. Кейс	25	~20 мин	описана практическая ситуация; нужно применить понятия курса и дать обоснованную рекомендацию

С.7.1 Примеры заданий

Тест. Дневные лог-доходности акции за пять дней: 0,01; −0,02; 0,005; 0,01; −0,005. Чему равна лог-доходность за пять дней? (а) 0; (б) 0,01; (в) −0,01; (г) нельзя определить без цен.

Интерпретация. Для дневных доходностей индекса оценена модель GARCH(1,1): $\omega = 0,00001$, $\alpha = 0,09$, $\beta = 0,89$. Что означают α и β? Что можно сказать об устойчивости шоков волатильности? Какую безусловную годовую во-

латильность подразумевает модель? Какие допущения модели вы бы проверили в первую очередь?

Найдите ошибку. Студент протестировал стратегию «держать акцию, когда цена выше 20-дневной средней»:

```
signal = (price > price.rolling(20).mean()).astype(int)
strategy = signal * price.pct_change()
sharpe = strategy.mean() / strategy.std() * np.sqrt(252)
```

Коэффициент Шарпа получился 1,4. В чём ошибка и как она влияет на результат? *(Сигнал, рассчитанный по цене закрытия дня t, применяется к доходности того же дня – это заглядывание в будущее; нужен* `signal.shift(1)`*. Кроме того, не учтены издержки и безрисковая ставка.)*

Кейс. Инвестор в начале 2024 г. выбирает между фондом денежного рынка и индексным фондом на акции Мосбиржи. Ключевая ставка – 16%, её дальнейшее направление неизвестно. Какие показатели риска и доходности вы бы сравнили? Как на выбор влияет горизонт инвестора? Что показали данные за 2024 г. и можно ли по одному году судить о правильности решения?

С.7.2 Как готовиться

Лучшая подготовка – вопросы для самопроверки и упражнения типов A и B в конце каждой главы: экзаменационные задания устроены так же. Полезно также пересмотреть свои проекты: в части «Интерпретация» часто встречаются результаты, похожие на те, что вы получали сами.